AI Customer Support Platform

Scrum Artifacts

Product Backlog, Sprint Plan, Definitions of Ready and Done, and Design Diagrams

Team:	Amine Essid
Product Owner:	Amine Essid
Scrum Master:	Amine Essid
Developers:	Amine Essid
Organization:	Amine Essid
Version:	1.0.1
Date:	October 23, 2026

Scrum Guide terminology is used throughout: Product Backlog, Sprint Backlog, Increment, Product Goal and Sprint Goal.

About this document

This document collects the Scrum artifacts for the **AI Customer Support Platform**: an AI-native customer support platform that handles full conversations over text and voice, answers from approved organizational knowledge, performs explicitly authorized business actions, and hands over to human agents with context.

How it is organized.

- **Part I – Product and Scrum Framework:** vision and scope, the Scrum charter and events, and an overview of the Product Backlog.
- **Part II – Epics and User Stories:** the 12 epics and 38 user stories, each with its acceptance criteria and the diagrams that explain the behavior.
- **Part III – Release and Sprint Planning:** release goals and risks, the provisional Sprint plan with one diagram per Sprint, and the Definitions of Ready and Done.
- **Part IV – Architecture and Design Reference:** system architecture, module catalog, communication contracts, domain model, AI engine, voice, integrations, security and observability.
- **Part V – Technology Report:** the proposed technology stack, the technologies for every epic, user story and Sprint, the role and usage rule of each technology, and the decisions still to validate.
- **Part VI – UML Models:** use case, class, sequence, activity and structural (component, package, deployment, object) diagrams.

Status of the content. The backlog, sprint sequence and estimates are a *planning hypothesis*, not a commitment. Team size, capacity and calendar dates are unknown; story points are team-specific and are left as "TBD" in the backlog table. Dates in the Gantt chart are placeholders. Names on the title page are placeholders to be replaced.

Conventions. `MUST` means required for correctness or security; `SHOULD` means recommended unless a documented reason exists. AI output is treated as untrusted input: authorization and business invariants are enforced by application code.

Diagram sources. All diagrams are Mermaid sources kept in `diagrams/mermaid/mmd/` and rendered to PDF by the Mermaid CLI. The Markdown files in `diagrams/mermaid/` are the editable originals.

Contents

About this document **1**

I Product and Scrum Framework **20**

1 Product Vision and Scope **21**

1.1 Vision . . . 21
1.2 Primary users . . . 21
1.3 Product outcomes . . . 21
1.4 Product principles . . . 21
1.5 MVP scope . . . 21
1.5.1 Included . . . 22
1.5.2 Later phases . . . 22
1.6 Explicit non-goals for the first milestone . . . 22
1.7 Success measures . . . 22

2 Scrum Charter **25**

2.1 Product goal . . . 25
2.2 Scrum setup . . . 25
2.3 Product Goal . . . 27
2.4 Proposed release goals . . . 27
2.5 Sprint approach . . . 27
2.6 Working assumptions . . . 29
2.7 Scrum Events, Artifacts and Working Agreements . . . 30
2.7.1 Sprint Planning . . . 30
2.7.2 Daily Scrum . . . 30
2.7.3 Sprint Review . . . 30
2.7.4 Sprint Retrospective . . . 30
2.7.5 Backlog refinement . . . 30
2.7.6 Scrum artifacts and commitments . . . 30
2.7.7 Suggested working agreements . . . 31

3 Product Backlog Overview **32**

3.1 Epics . . . 32
3.2 Suggested order of work . . . 32
3.3 Full Product Backlog . . . 38

II Epics and User Stories **41**

4 E01 — Tenant and Access Foundation **42**
4.1 Epic overview . . . 42
4.2 US-001 — Create and configure an organization . . . 43
4.3 US-002 — Invite users and assign roles . . . 44
4.3.1 Invitation lifecycle . . . 44
4.3.2 Role assignment and revocation . . . 45
4.4 US-003 — Test tenant isolation . . . 46
4.4.1 Where tenant filtering is enforced . . . 47

5 E02 — Conversation and Message Core **48**
5.1 Epic overview . . . 49
5.2 Conversation lifecycle (used by all stories) . . . 50
5.3 US-010 — Start a conversation through website chat . . . 51
5.4 US-011 — Send and receive text messages . . . 52
5.4.1 Ordering and persistence . . . 52
5.4.2 Message delivery status as seen by the customer . . . 53
5.5 US-012 — Support agent inspects the timeline . . . 54
5.6 US-013 — Deduplicate inbound events . . . 55

6 E03 — Website Chat Channel **56**
6.1 Epic overview . . . 56
6.2 US-020 — Administrator configures the website widget . . . 57
6.3 US-021 — Visitor opens chat and receives a session . . . 58
6.3.1 Token claim checks . . . 59
6.4 US-022 — Reconnect after temporary network failure . . . 60

7 E04 — Knowledge Ingestion and RAG **62**
7.1 Epic overview . . . 63
7.2 US-030 — Register and upload a knowledge source . . . 64
7.3 US-031 — Parse, chunk, embed and index asynchronously . . . 65
7.3.1 Pipeline . . . 65
7.3.2 Ingestion state machine . . . 66
7.3.3 Deletion and freshness propagation . . . 67
7.4 US-032 — Retrieve relevant chunks . . . 68
7.5 US-033 — Grounded answer or clear fallback . . . 69

8 E05 — AI Orchestration **70**
8.1 Epic overview . . . 70
8.2 US-040 — Process a turn through a versioned agent configuration . . . 71
8.3 US-041 — Follow-up question using conversation context . . . 72

8.4 US-042 — Handle model timeout or provider failure . . . 73
8.5 US-043 — Publish and roll back an agent version . . . 74

9 E06 — Safe Business Tools 75
9.1 Epic overview . . . 75
9.2 Three-layer connector architecture . . . 76
9.3 US-050 — Enable a versioned tool for an agent . . . 77
9.4 US-051 — Customer asks for order status . . . 78
9.5 US-052 — Handle tool timeout and duplicate requests . . . 79
9.6 US-053 — Audit tool calls . . . 80

10 E07 — Human Inbox and Handoff 81
10.1 Epic overview . . . 82
10.2 Ownership model . . . 83
10.3 US-060 — Support agent views the unassigned queue . . . 84
10.4 US-061 — Customer requests a human . . . 85
10.5 US-062 — Handoff summary for the agent . . . 86
10.6 US-063 — Support agent resolves the conversation . . . 87

11 E08 — Tickets and Case Tracking 88
11.1 Epic overview . . . 88
11.2 Conversation vs ticket . . . 88
11.3 Ticket status lifecycle . . . 89
11.4 US-070 — Create a ticket from a conversation . . . 90
11.5 US-071 — Support lead filters tickets . . . 92

12 E09 — Voice MVP 93
12.1 Epic overview . . . 94
12.2 Reuse of the text engine . . . 95
12.3 US-080 — Start and end a voice session . . . 96
12.4 US-081 — Speak and receive a spoken response . . . 97
12.5 US-082 — Interrupt the AI while it speaks . . . 98
12.6 US-083 — Switch between text and voice . . . 99
12.7 US-084 — Agent takes over a voice conversation or receives its context . . . 100

13 E10 — Workflow, Routing and SLA 101
13.1 Epic overview . . . 102
13.2 Principle: deterministic procedure, AI proposes . . . 103
13.3 US-090 — Define workflow trigger and steps . . . 104
13.3.1 Authoring and versioning . . . 104
13.3.2 Run execution . . . 105
13.4 US-091 — Routing rules and SLA targets . . . 106

13.4.1 SLA timers . . . 107
13.5 US-092 — Retry a failed workflow step safely . . . 108

14 E11 — Integrations and Additional Channels 109
14.1 Epic overview . . . 110
14.2 Adapter boundary . . . 111
14.3 US-100 — Configure an integration securely . . . 112
14.3.1 Integration health states . . . 113
14.4 US-101 — Normalize provider webhook into canonical event . . . 114
14.5 US-102 — Continue support through a second channel . . . 115

15 E12 — Observability, Evaluation and Usage 116
15.1 Epic overview . . . 116
15.2 US-110 — Trace one request across modules . . . 117
15.2.1 Correlation identifiers . . . 118
15.3 US-111 — Resolution and handoff metrics . . . 119
15.3.1 Platform signal groups . . . 120
15.4 US-112 — Run AI regression evaluations . . . 121
15.4.1 Improvement loop . . . 122
15.5 US-113 — Inspect usage by agent and channel . . . 123

III Release and Sprint Planning 124

16 Release Goals, Risks and Decision Log 125
16.1 Release goals . . . 125
16.1.1 R1 — Text support MVP . . . 125
16.1.2 R2 — Voice MVP . . . 126
16.1.3 R3 — Operational expansion . . . 126
16.2 Initial risks . . . 126
16.3 Architecture decision records to create . . . 129
16.4 Decisions not to make prematurely . . . 131

17 Provisional Sprint Plan 134
17.1 Sprint 0 / Enablement (only if needed) . . . 137
17.2 Sprint 1 — Secure conversation foundation . . . 139
17.3 Sprint 2 — Website chat vertical slice . . . 139
17.4 Sprint 3 — Knowledge ingestion and retrieval . . . 140
17.5 Sprint 4 — Grounded AI answers . . . 141
17.6 Sprint 5 — Safe business tool and human handoff . . . 142
17.7 Sprint 6 — Tickets and release hardening . . . 143
17.8 Sprint 7 — Voice technical spike / MVP slice . . . 145

17.9 Sprint 8 — Voice synchronization and handoff . . . 145
17.10 Later Sprints . . . 146
17.11 Planning caveats . . . 146

18 Definition of Ready and Definition of Done 149
18.1 Definition of Ready (team working agreement) . . . 149
18.2 Definition of Done . . . 150
18.3 AI-specific Done checks . . . 150
18.4 Voice-specific Done checks . . . 150

IV Architecture and Design Reference 152

19 System Architecture 153
19.1 Recommended starting point . . . 153
19.2 Logical layers . . . 153
19.3 Architecture constraints . . . 153
19.4 Suggested deployable components at MVP . . . 153
19.5 System context . . . 155
19.6 Logical layers . . . 156
19.7 MVP deployable components . . . 157
19.8 Module map — split by domain . . . 158
19.8.1 A. Customer interaction and conversation (M01–M03) . . . 158
19.8.2 B. AI conversation and voice (M04–M09) . . . 159
19.8.3 C. Support operations (M10–M14) . . . 160
19.8.4 D. Integrations and insights (M15–M17) . . . 161
19.8.5 E. Platform and governance (M18–M20) . . . 162
19.9 Combined module dependency view . . . 163
19.10 Event catalog by lifecycle . . . 164

20 Full Module Catalog 165
20.1 A. Customer interaction and conversation . . . 165
20.1.1 M01 — Conversation Management . . . 165
20.1.2 M02 — Channel Management . . . 165
20.1.3 M03 — Customer Identity and Profiles . . . 165
20.2 B. AI conversation and voice . . . 165
20.2.1 M04 — AI Conversation Engine . . . 165
20.2.2 M05 — Knowledge Base and Retrieval . . . 165
20.2.3 M06 — AI Tools and Business Actions . . . 166
20.2.4 M07 — Real-Time Voice Sessions . . . 166
20.2.5 M08 — Speech Processing . . . 166
20.2.6 M09 — Transcript and Multimodal Synchronization . . . 166

20.3 C. Support operations and automation . . . 166
20.3.1 M10 — Human Support Inbox and Handoff . . . 166
20.3.2 M11 — Ticket and Case Management . . . 166
20.3.3 M12 — Workflow Automation . . . 167
20.3.4 M13 — Routing, Policies and SLAs . . . 167
20.3.5 M14 — Notifications and Follow-Ups . . . 167
20.4 D. Business integrations and insights . . . 167
20.4.1 M15 — Integration Hub . . . 167
20.4.2 M16 — Analytics and Reporting . . . 167
20.4.3 M17 — AI Quality, Evaluation and Improvement . . . 167
20.5 E. Platform administration and governance . . . 167
20.5.1 M18 — Organizations, Users and Access Control . . . 167
20.5.2 M19 — Agent Configuration and Administration . . . 168
20.5.3 M20 — Security, Audit, Observability and Usage . . . 168
20.6 Shared technical capabilities . . . 168

21 Module Communication and Contracts 169
21.1 Communication principles . . . 169
21.2 Synchronous interactions . . . 169
21.3 Asynchronous events/jobs . . . 169
21.4 Canonical inbound message envelope . . . 170
21.5 Tool invocation envelope . . . 170
21.6 Delivery state machine . . . 171
21.7 Reliability patterns . . . 171
21.8 Error contract . . . 171
21.9 Cross-cutting flow diagrams . . . 171
21.9.1 Inbound webhook handling . . . 172
21.9.2 Transactional outbox and inbox . . . 173
21.9.3 Outbound delivery state machine . . . 174
21.9.4 Retry decision . . . 175
21.9.5 Tool execution — full pipeline (split in two parts) . . . 176
21.9.6 Tool execution state machine . . . 178
21.9.7 Tenant resolution and isolation . . . 179
21.9.8 Secrets and credential flow . . . 180
21.9.9 Graceful degradation . . . 181
21.9.10 Data lifecycle and retention . . . 182
21.9.11 Security test matrix . . . 183

22 Domain Model 184
22.1 Core aggregate boundaries . . . 184
22.2 Relationship overview . . . 184

22.3 Data modeling rules . . . 184
22.4 Suggested message kinds . . . 184
22.5 Data model diagrams . . . 185
22.5.1 Aggregate map . . . 186
22.5.2 Organizations, users and access . . . 187
22.5.3 Customer and channel . . . 188
22.5.4 Conversation . . . 189
22.5.5 Agent configuration and execution . . . 190
22.5.6 Knowledge . . . 191
22.5.7 Tools and integrations . . . 192
22.5.8 Handoff and tickets . . . 193
22.5.9 Voice . . . 194
22.5.10 Workflow, routing and SLA . . . 195
22.5.11 Audit, usage and evaluation . . . 196

23 AI Conversation Engine **197**
23.1 Responsibilities . . . 197
23.2 Recommended turn pipeline . . . 197
23.3 State machine . . . 197
23.4 Prompt/context construction . . . 197
23.5 Tool execution policy . . . 198
23.6 Response policy . . . 198
23.7 Model-provider interface . . . 198
23.8 AI engine diagrams . . . 198
23.8.1 Turn pipeline overview . . . 198
23.8.2 Turn state machine . . . 203
23.8.3 Context sections and trust levels . . . 204
23.8.4 Escalation decision tree . . . 205
23.8.5 Response policy decision . . . 206
23.8.6 Model-provider adapter . . . 207
23.8.7 Agent version lifecycle (publish, rollback) . . . 208
23.8.8 Failure handling inside a turn . . . 209

24 Voice and Multimodal Conversations **210**
24.1 Goal . . . 210
24.2 Voice subsystem responsibilities . . . 210
24.2.1 Real-time voice session . . . 210
24.2.2 Speech processing . . . 210
24.3 Turn and interruption semantics . . . 210
24.4 Switching modalities . . . 210
24.5 Human handoff . . . 211

24.6 Voice-specific operational metrics . . . 211
24.7 Privacy . . . 211
24.8 Voice and multimodal diagrams . . . 211
24.8.1 Voice session lifecycle . . . 212
24.8.2 Two speech strategies . . . 213
24.8.3 One voice turn (Option A) with persistence boundaries . . . 216
24.8.4 Interruption (barge-in) . . . 217
24.8.5 Partial vs final transcript handling . . . 218
24.8.6 Modality switching in one conversation . . . 219
24.8.7 Voice handoff bundle . . . 220
24.8.8 Voice privacy and consent . . . 221
24.8.9 Voice metrics pipeline . . . 222

25 Integrations and Tool Contracts 223
25.1 Connector architecture . . . 223
25.2 Tool definition fields . . . 223
25.3 Example: read-only order lookup . . . 223
25.4 Error mapping . . . 224
25.5 Webhook handling . . . 224

26 Security, Privacy and Reliability 226
26.1 Tenant isolation . . . 226
26.2 Authentication and authorization . . . 226
26.3 Secret management . . . 226
26.4 Prompt injection and untrusted content . . . 226
26.5 Privacy and retention . . . 226
26.6 Reliability . . . 227
26.7 Upload safety . . . 227
26.8 Security testing checklist . . . 227

27 Observability, Analytics and AI Evaluation 228
27.1 Correlation model . . . 228
27.2 Operational signals . . . 228
27.2.1 API and platform . . . 228
27.2.2 Conversation and delivery . . . 228
27.2.3 AI and RAG . . . 228
27.2.4 Voice . . . 229
27.3 Evaluation plan . . . 229
27.4 Release gates . . . 229

V Technology Report 230

28 Technology Stack and Choices 231

28.1 Status and how to read this . . . 231
28.2 Selection criteria . . . 231
28.3 Assumptions . . . 231
28.4 Stack at a glance . . . 231
28.5 How a request flows through the stack . . . 235
28.6 Voice stack . . . 237

29 Technology by Task 239

29.1 E01 — Tenant and access foundation . . . 239
29.2 E02 — Conversation and message core . . . 239
29.3 E03 — Website chat channel . . . 239
29.4 E04 — Knowledge ingestion and RAG . . . 240
29.5 E05 — AI orchestration . . . 240
29.6 E06 — Safe business tools . . . 240
29.7 E07 — Human inbox and handoff . . . 241
29.8 E08 — Tickets and case tracking . . . 241
29.9 E09 — Voice MVP . . . 241
29.10E10 — Workflow, routing and SLA . . . 242
29.11E11 — Integrations and additional channels . . . 242
29.12E12 — Observability, evaluation and usage . . . 242
29.13Technology by Sprint . . . 242

30 Technology Roles, Rules and Decisions 244

30.1 Roles and usage rules . . . 244
30.2 Cross-cutting technology . . . 246
30.2.1 Security . . . 247
30.2.2 Observability . . . 247
30.2.3 Testing and quality . . . 247
30.2.4 Delivery . . . 247
30.3 Decisions to validate . . . 247
30.4 Lock-in and risk assessment . . . 248

VI UML Models 249

31 UML Use Case Diagrams 250

31.1 System-level use cases . . . 250
31.2 Customer use cases . . . 251
31.3 Support staff use cases . . . 252

31.4 Administration and governance use cases . . . 253

32 UML Class Diagrams 254

32.1 Access and tenancy (M18) . . . 254
32.2 Conversation core (M01, M03) . . . 255
32.3 Channel management and delivery (M02) . . . 256
32.4 AI Conversation Engine: turn orchestration (M04) . . . 257
32.5 U2.4b Agent configuration and versions (M19) . . . 258
32.6 Knowledge and retrieval (M05) . . . 258
32.7 Tools: registry and execution (M06) . . . 259
32.8 U2.6b Integrations and connectors (M15) . . . 260
32.9 Human handoff and tickets (M10, M11, M13) . . . 261
32.10Voice and speech (M07, M08, M09) . . . 262
32.11Workflow automation (M12) and notifications (M14) . . . 263
32.12Security, audit, usage and evaluation (M16, M17, M20) . . . 264
32.13Design patterns applied . . . 265

33 UML Sequence Diagrams 266

33.1 Text MVP end to end: intake . . . 267
33.2 U3.1b Text MVP end to end: AI turn and reply . . . 268
33.3 Duplicate webhook delivery . . . 269
33.4 Prompt injection blocked . . . 270
33.5 Sensitive action with human approval . . . 271
33.6 Human handoff with context . . . 272
33.7 Knowledge document ingestion . . . 273
33.8 Agent version publish with release gate . . . 274
33.9 Customer data deletion request . . . 275
33.10Voice barge-in (condensed) . . . 276
33.11Circuit breaker around a provider . . . 277

34 UML Activity Diagrams (with swimlanes) 278

34.1 Customer support conversation — end to end . . . 279
34.2 Document ingestion . . . 280
34.3 Human handoff . . . 281
34.4 Tool call authorization . . . 282
34.5 Release process . . . 283
34.6 Incident response for provider outage . . . 284

35 UML Structural Diagrams: Component, Package, Deployment, Object 285

35.1 Component diagram: provided and required interfaces . . . 285
35.2 U5.1b Component diagram: outbound ports and adapters . . . 287

35.3 Package diagram (code organization and allowed dependencies) 288
35.4 Deployment diagram — MVP 289
35.5 Deployment diagram — later: independent real-time voice component 290
35.6 Object diagram — one live conversation (instance specification) 291
35.7 Composite structure — tool execution internals 292

List of Figures

1.1 System context . . . 23
1.2 Product roadmap . . . 24

2.1 Scrum roles and accountabilities . . . 26
2.2 Scrum events within a Sprint . . . 28
2.3 Scrum workflow . . . 29

3.1 Epic dependency graph . . . 34
3.2 Epic delivery map by release . . . 35
3.3 Module-to-epic coverage . . . 36
3.4 User story lifecycle . . . 37

4.1 E01: Epic overview . . . 42
4.2 E01: US-001 — Create and configure an organization . . . 43
4.3 E01: US-002 — Invite users and assign roles: Invitation lifecycle . . . 44
4.4 E01: US-002 — Invite users and assign roles: Role assignment and revocation . . . 45
4.5 E01: US-003 — Test tenant isolation . . . 46
4.6 E01: US-003 — Test tenant isolation: Where tenant filtering is enforced . . . 47

5.1 E02: Epic overview . . . 49
5.2 E02: Conversation lifecycle (used by all stories) . . . 50
5.3 E02: US-010 — Start a conversation through website chat . . . 51
5.4 E02: US-011 — Send and receive text messages: Ordering and persistence . . . 52
5.5 E02: US-011 — Send and receive text messages: Message delivery status as seen by the customer . . . 53
5.6 E02: US-012 — Support agent inspects the timeline . . . 54
5.7 E02: US-013 — Deduplicate inbound events . . . 55

6.1 E03: Epic overview . . . 56
6.2 E03: US-020 — Administrator configures the website widget . . . 57
6.3 E03: US-021 — Visitor opens chat and receives a session . . . 58
6.4 E03: US-021 — Visitor opens chat and receives a session: Token claim checks . . . 59
6.5 E03: US-022 — Reconnect after temporary network failure . . . 60
6.6 E03: US-022 — Reconnect after temporary network failure . . . 61

7.1 E04: Epic overview . . . 63
7.2 E04: US-030 — Register and upload a knowledge source . . . 64
7.3 E04: US-031 — Parse, chunk, embed and index asynchronously: Pipeline . . . 65
7.4 E04: US-031 — Parse, chunk, embed and index asynchronously: Ingestion state machine . . . 66

7.5 E04: US-031 — Parse, chunk, embed and index asynchronously: Deletion and freshness propagation . . . 67
7.6 E04: US-032 — Retrieve relevant chunks . . . 68
7.7 E04: US-033 — Grounded answer or clear fallback . . . 69

8.1 E05: Epic overview . . . 70
8.2 E05: US-040 — Process a turn through a versioned agent configuration . . . 71
8.3 E05: US-041 — Follow-up question using conversation context . . . 72
8.4 E05: US-042 — Handle model timeout or provider failure . . . 73
8.5 E05: US-043 — Publish and roll back an agent version . . . 74

9.1 E06: Epic overview . . . 75
9.2 E06: Three-layer connector architecture . . . 76
9.3 E06: US-050 — Enable a versioned tool for an agent . . . 77
9.4 E06: US-051 — Customer asks for order status . . . 78
9.5 E06: US-052 — Handle tool timeout and duplicate requests . . . 79
9.6 E06: US-053 — Audit tool calls . . . 80

10.1 E07: Epic overview . . . 82
10.2 E07: Ownership model . . . 83
10.3 E07: US-060 — Support agent views the unassigned queue . . . 84
10.4 E07: US-061 — Customer requests a human . . . 85
10.5 E07: US-062 — Handoff summary for the agent . . . 86
10.6 E07: US-063 — Support agent resolves the conversation . . . 87

11.1 E08: Epic overview . . . 88
11.2 E08: Conversation vs ticket . . . 88
11.3 E08: Ticket status lifecycle . . . 89
11.4 E08: US-070 — Create a ticket from a conversation . . . 90
11.5 E08: US-070 — Create a ticket from a conversation . . . 91
11.6 E08: US-071 — Support lead filters tickets . . . 92

12.1 E09: Epic overview . . . 94
12.2 E09: Reuse of the text engine . . . 95
12.3 E09: US-080 — Start and end a voice session . . . 96
12.4 E09: US-081 — Speak and receive a spoken response . . . 97
12.5 E09: US-082 — Interrupt the AI while it speaks . . . 98
12.6 E09: US-083 — Switch between text and voice . . . 99
12.7 E09: US-084 — Agent takes over a voice conversation or receives its context . . . 100

13.1 E10: Epic overview . . . 102
13.2 E10: Principle: deterministic procedure, AI proposes . . . 103
13.3 E10: US-090 — Define workflow trigger and steps: Authoring and versioning . . . 104

13.4 E10: US-090 — Define workflow trigger and steps: Run execution . . . 105
13.5 E10: US-091 — Routing rules and SLA targets . . . 106
13.6 E10: US-091 — Routing rules and SLA targets: SLA timers . . . 107
13.7 E10: US-092 — Retry a failed workflow step safely . . . 108

14.1 E11: Epic overview . . . 110
14.2 E11: Adapter boundary . . . 111
14.3 E11: US-100 — Configure an integration securely . . . 112
14.4 E11: US-100 — Configure an integration securely: Integration health states . . . 113
14.5 E11: US-101 — Normalize provider webhook into canonical event . . . 114
14.6 E11: US-102 — Continue support through a second channel . . . 115

15.1 E12: Epic overview . . . 116
15.2 E12: US-110 — Trace one request across modules . . . 117
15.3 E12: US-110 — Trace one request across modules: Correlation identifiers . . . 118
15.4 E12: US-111 — Resolution and handoff metrics . . . 119
15.5 E12: US-111 — Resolution and handoff metrics: Platform signal groups . . . 120
15.6 E12: US-112 — Run AI regression evaluations . . . 121
15.7 E12: US-112 — Run AI regression evaluations: Improvement loop . . . 122
15.8 E12: US-113 — Inspect usage by agent and channel . . . 123

16.1 Release 1 exit criteria . . . 125
16.2 Release 2 exit criteria . . . 126
16.3 Risk map . . . 128
16.4 ADR dependency order . . . 130
16.5 Decisions to defer . . . 131
16.6 Rollout and rollback . . . 132
16.7 Release acceptance flow . . . 133

17.1 Sprint sequence and goals . . . 135
17.2 What the demo can show after each Sprint . . . 136
17.3 Provisional timeline (placeholder dates) . . . 137
17.4 Sprint 0 detail . . . 138
17.5 Sprint 1 detail . . . 139
17.6 Sprint 2 detail . . . 140
17.7 Sprint 3 detail . . . 141
17.8 Sprint 4 detail . . . 142
17.9 Sprint 5 detail . . . 143
17.10Sprint 6 detail . . . 144
17.11Sprint 7 detail . . . 145
17.12Sprint 8 detail . . . 146
17.13Story-to-Sprint traceability . . . 147

17.14Sprint Review demo path for the Text MVP 148

18.1 Definition of Ready check 149
18.2 Definition of Done layers 151

19.1 System context 155
19.2 Logical layers 156
19.3 MVP deployable components 157
19.4 Module map — split by domain: A. Customer interaction and conversation (M01–M03) . 158
19.5 Module map — split by domain: B. AI conversation and voice (M04–M09) 159
19.6 Module map — split by domain: C. Support operations (M10–M14) 160
19.7 Module map — split by domain: D. Integrations and insights (M15–M17) 161
19.8 Module map — split by domain: E. Platform and governance (M18–M20) 162
19.9 Combined module dependency view 163
19.10Event catalog by lifecycle 164

21.1 Inbound webhook handling 172
21.2 Transactional outbox and inbox 173
21.3 Outbound delivery state machine 174
21.4 Retry decision 175
21.5 Tool execution — full pipeline (split in two parts): Part A: validation and authorization . 176
21.6 Tool execution — full pipeline (split in two parts): Part B: execution and result handling 177
21.7 Tool execution state machine 178
21.8 Tenant resolution and isolation 179
21.9 Secrets and credential flow 180
21.10Graceful degradation 181
21.11Data lifecycle and retention 182
21.12Security test matrix 183

22.1 Aggregate map 186
22.2 Organizations, users and access 187
22.3 Customer and channel 188
22.4 Conversation 189
22.5 Agent configuration and execution 190
22.6 Knowledge 191
22.7 Tools and integrations 192
22.8 Handoff and tickets 193
22.9 Voice 194
22.10Workflow, routing and SLA 195
22.11Audit, usage and evaluation 196

23.1 Turn pipeline overview 198

23.2 Turn pipeline overview: Phase 1 — Prepare (resolve trust, check state, build context) . . 199
23.3 Turn pipeline overview: Phase 2 — Understand and retrieve 200
23.4 Turn pipeline overview: Phase 3 — Plan and act (tool loop) 201
23.5 Turn pipeline overview: Phase 4 — Respond and persist 202
23.6 Turn state machine 203
23.7 Context sections and trust levels 204
23.8 Escalation decision tree 205
23.9 Response policy decision 206
23.10Model-provider adapter 207
23.11Agent version lifecycle (publish, rollback) 208
23.12Failure handling inside a turn 209

24.1 Voice session lifecycle 212
24.2 Two speech strategies: Option A — modular pipeline 213
24.3 Two speech strategies: Option B — speech-to-speech 214
24.4 Two speech strategies: Decision: which to start with 215
24.5 One voice turn (Option A) with persistence boundaries 216
24.6 Interruption (barge-in) 217
24.7 Partial vs final transcript handling 218
24.8 Modality switching in one conversation 219
24.9 Voice handoff bundle 220
24.10Voice privacy and consent 221
24.11Voice metrics pipeline 222

25.1 Three-layer connector architecture 225

28.1 Technology stack map 234
28.2 A customer message through the stack 236
28.3 Voice stack 238

29.1 When each technology arrives 243

30.1 Delivery toolchain 246

31.1 System-level use cases 250
31.2 Customer use cases 251
31.3 Support staff use cases 252
31.4 Administration and governance use cases 253

32.1 Access and tenancy (M18) 254
32.2 Conversation core (M01, M03) 255
32.3 Channel management and delivery (M02) 256
32.4 AI Conversation Engine: turn orchestration (M04) 257

32.5 U2.4b Agent configuration and versions (M19) . . . 258
32.6 Knowledge and retrieval (M05) . . . 258
32.7 Tools: registry and execution (M06) . . . 259
32.8 U2.6b Integrations and connectors (M15) . . . 260
32.9 Human handoff and tickets (M10, M11, M13) . . . 261
32.10Voice and speech (M07, M08, M09) . . . 262
32.11Workflow automation (M12) and notifications (M14) . . . 263
32.12Security, audit, usage and evaluation (M16, M17, M20) . . . 264
32.13Design patterns applied . . . 265

33.1 Text MVP end to end: intake . . . 267
33.2 U3.1b Text MVP end to end: AI turn and reply . . . 268
33.3 Duplicate webhook delivery . . . 269
33.4 Prompt injection blocked . . . 270
33.5 Sensitive action with human approval . . . 271
33.6 Human handoff with context . . . 272
33.7 Knowledge document ingestion . . . 273
33.8 Agent version publish with release gate . . . 274
33.9 Customer data deletion request . . . 275
33.10Voice barge-in (condensed) . . . 276
33.11Circuit breaker around a provider . . . 277

34.1 Customer support conversation — end to end . . . 279
34.2 Document ingestion . . . 280
34.3 Human handoff . . . 281
34.4 Tool call authorization . . . 282
34.5 Release process . . . 283
34.6 Incident response for provider outage . . . 284

35.1 Component diagram: provided and required interfaces . . . 286
35.2 U5.1b Component diagram: outbound ports and adapters . . . 287
35.3 Package diagram (code organization and allowed dependencies) . . . 288
35.4 Deployment diagram — MVP . . . 289
35.5 Deployment diagram — later: independent real-time voice component . . . 290
35.6 Object diagram — one live conversation (instance specification) . . . 291
35.7 Composite structure — tool execution internals . . . 292

List of Tables

Part I

Product and Scrum Framework

Chapter 1

Product Vision and Scope

1.1 Vision

Build an AI-native customer support platform that can conduct complete, context-aware customer conversations over text and voice, answer using trusted business knowledge, perform explicitly authorized actions, and transfer to human agents when appropriate.

1.2 Primary users

- **Customer:** asks questions or requests an action using a supported channel.
- **Support agent:** reviews, takes over, and resolves conversations or tickets.
- **Support lead:** manages queues, policies, reporting, and quality.
- **Organization administrator:** configures channels, knowledge, AI agents, integrations, and access.
- **Platform operator:** monitors reliability, cost, abuse, and platform health.

1.3 Product outcomes

- Reduce time to a correct answer.
- Resolve repetitive requests without removing access to human support.
- Preserve context across text/voice and channel changes where identity can be safely linked.
- Make AI actions auditable and permission-controlled.
- Let organizations configure knowledge, tools, policies, and escalation behavior.
- Measure resolution quality, not only response speed.

1.4 Product principles

1. **Conversation-first:** a conversation is the durable interaction record; channel sessions are delivery mechanisms.
2. **Modality-independent intelligence:** text and voice share the same orchestration and authorization model.
3. **Grounded answers:** use approved knowledge and business data; be transparent about uncertainty.
4. **Safe action execution:** models propose tool calls; application services authorize and execute them.
5. **Human escape hatch:** customers can request human support and policy can escalate automatically.
6. **Tenant isolation by default:** organization boundaries apply to data, search, tools, events, and analytics.
7. **Observable and testable:** every important AI decision, tool execution, handoff, and failure is measurable.
8. **Provider portability:** model, speech, channel, and integration providers are accessed through adapters.

1.5 MVP scope

1.5.1 Included

- Organization and user access foundation
- Website text chat
- Conversation lifecycle and message persistence
- Basic customer identity
- Knowledge source ingestion and retrieval
- AI response orchestration
- One read-only business tool
- Human handoff/inbox
- Basic ticket creation
- Audit, logs, metrics, traces, and usage tracking

1.5.2 Later phases

- Real-time browser voice
- STT/TTS and speech-to-speech provider options
- Email and external messaging channels
- Workflow builder and advanced routing/SLA
- More connectors and write-capable tools
- Advanced analytics, evaluation, billing, and enterprise governance

1.6 Explicit non-goals for the first milestone

- Replacing all existing CRM/helpdesk systems
- Arbitrary autonomous changes to customer accounts
- Unreviewed financial transactions
- Supporting every channel/provider at launch
- Training a foundation model
- Splitting every module into a microservice

1.7 Success measures

- Correct answer rate on a curated evaluation set
- Groundedness/source support rate
- Successful task completion rate
- Human escalation rate, segmented by reason
- Median and p95 response latency
- Tool success/failure rate
- Customer satisfaction
- Cost per resolved conversation
- Availability and message-delivery success

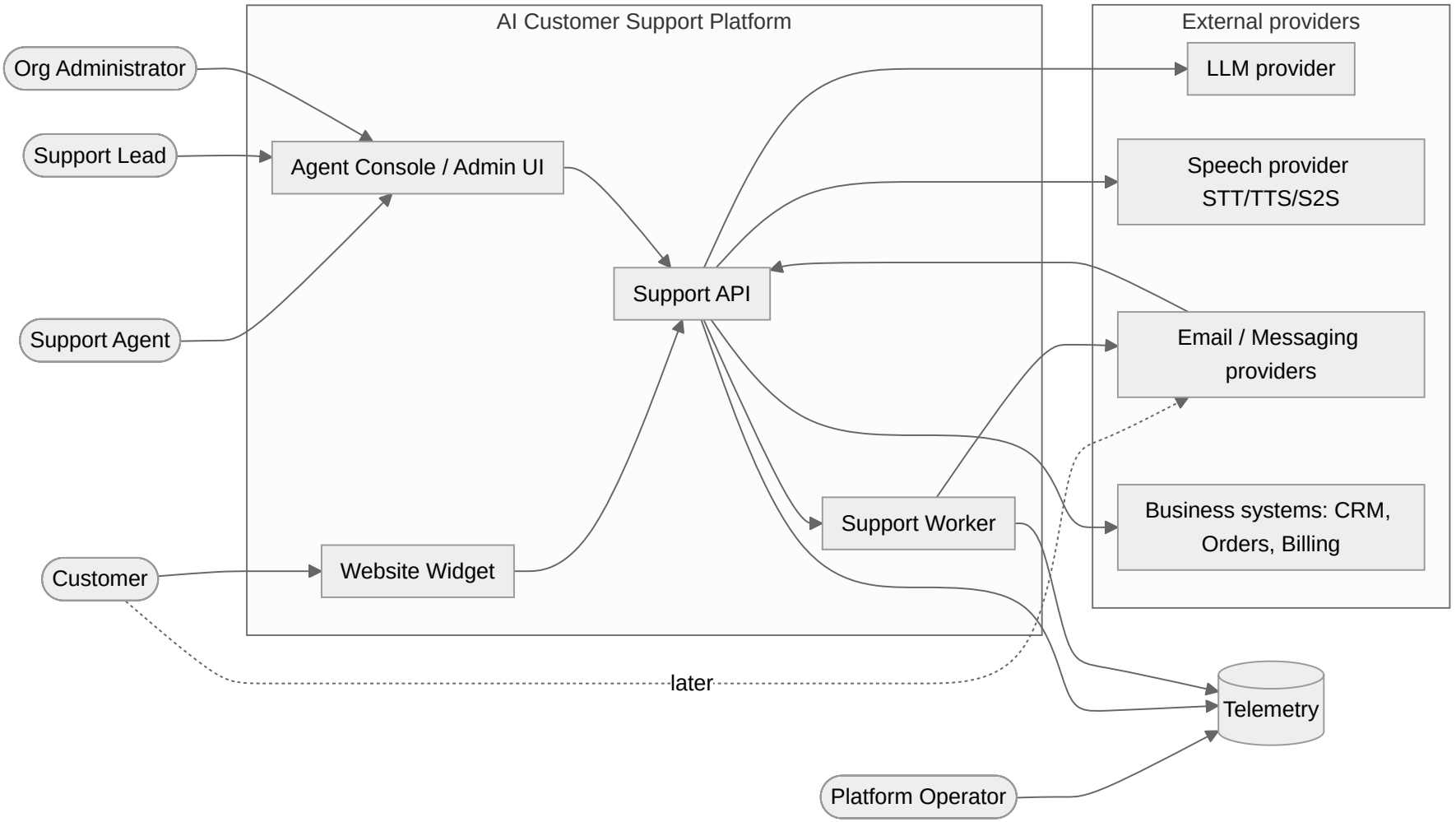


Figure 1.1: System context

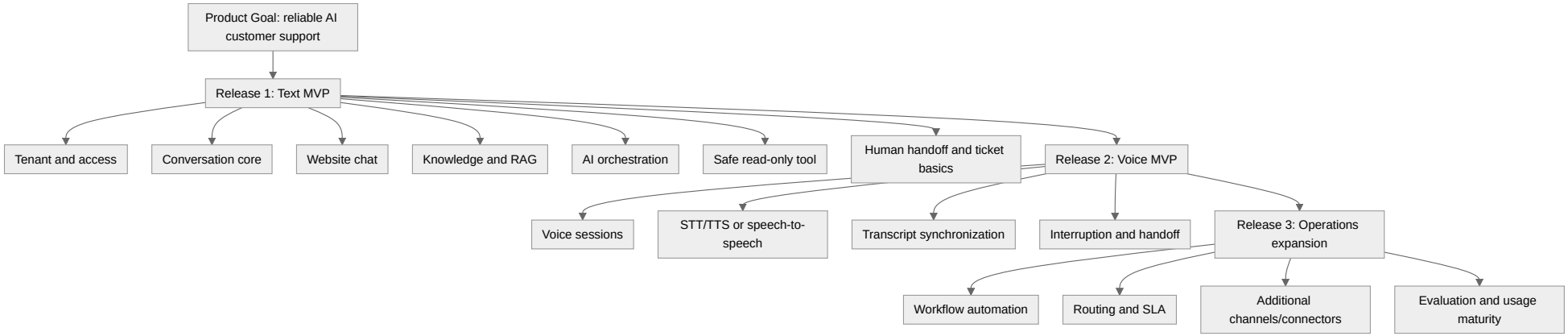


Figure 1.2: Product roadmap

Chapter 2

Scrum Charter

2.1 Product goal

Deliver an AI-native customer support platform that handles complete text conversations first, provides grounded answers and safe tool use, supports human handoff, and then expands to real-time voice and additional channels.

2.2 Scrum setup

- **Product Owner:** accountable for product value, Product Goal, and Product Backlog ordering.
- **Scrum Master:** coaches Scrum, removes impediments, and helps the team improve.
- **Developers:** cross-functional team that designs, builds, tests, integrates, secures, and operates a usable Increment.
- **Stakeholders:** support teams, organization administrators, customers, security/privacy, and business-system owners.

Roles can be held by a small team, but accountabilities must remain clear.

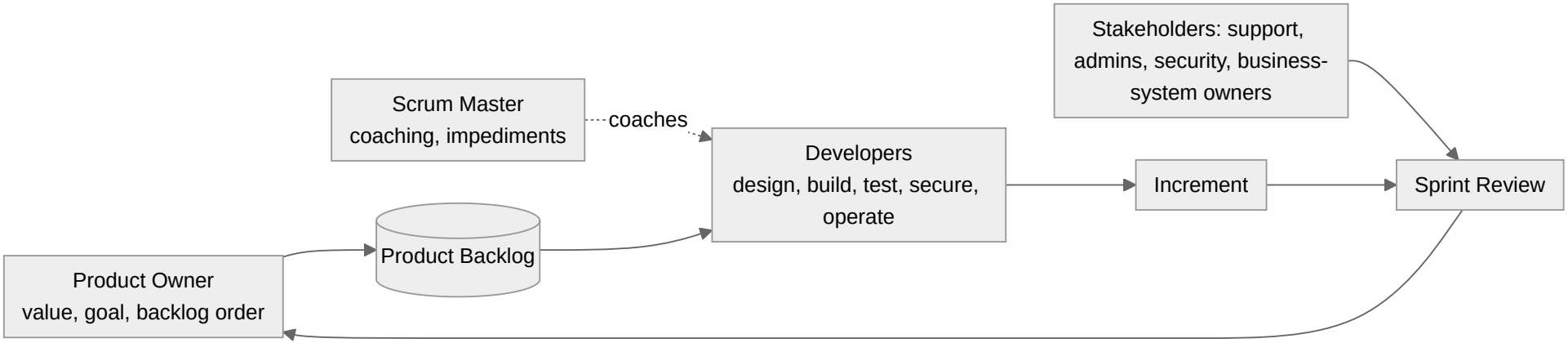


Figure 2.1: Scrum roles and accountabilities

2.3 Product Goal

A customer can contact an organization through website chat, receive accurate knowledge-grounded answers, complete at least one authorized read-only business lookup, and reach a human without losing conversation context. The architecture must be ready to add voice without duplicating business reasoning.

2.4 Proposed release goals

- **Release 1 — Text support MVP:** tenant foundation, website chat, conversation lifecycle, RAG, one read-only tool, handoff, basic analytics and audit.
- **Release 2 — Voice MVP:** real-time browser voice, speech processing, transcript synchronization, interruption handling, voice handoff and voice observability.
- **Release 3 — Operations expansion:** additional channels, workflow automation, routing/SLA, connectors, evaluation, and commercial usage controls.

2.5 Sprint approach

- Suggested Sprint length: 2 weeks.
- Each Sprint has one Sprint Goal and a potentially releasable Increment.
- Sprint Planning selects work based on Product Goal, ordered backlog, capacity, and Definition of Done.
- Daily Scrum is for Developers to inspect progress toward the Sprint Goal and adapt the plan.
- Sprint Review inspects the Increment with stakeholders and adapts the Product Backlog.
- Sprint Retrospective improves team effectiveness and quality.
- Refinement is ongoing; it is not a formal Scrum event.

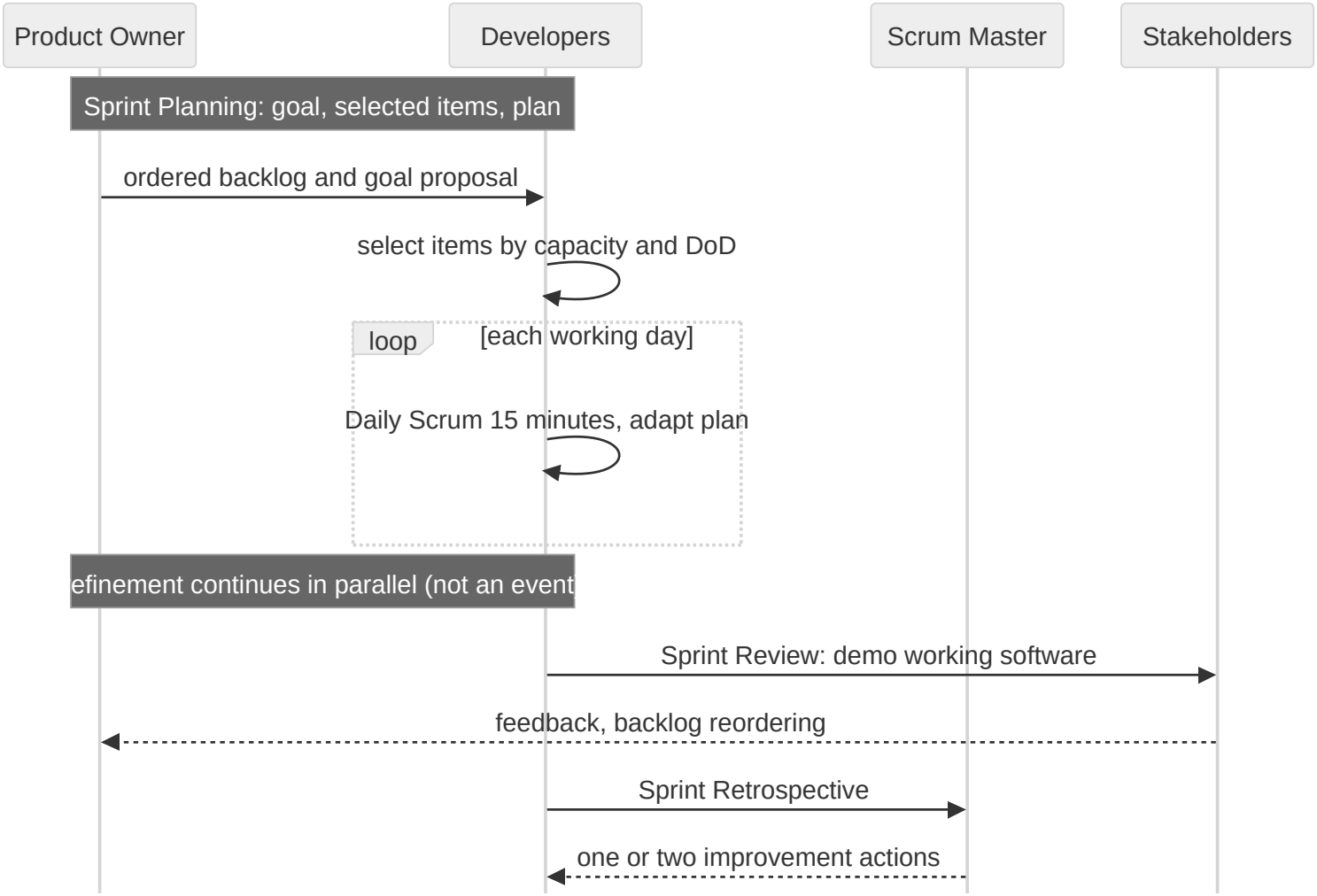


Figure 2.2: Scrum events within a Sprint

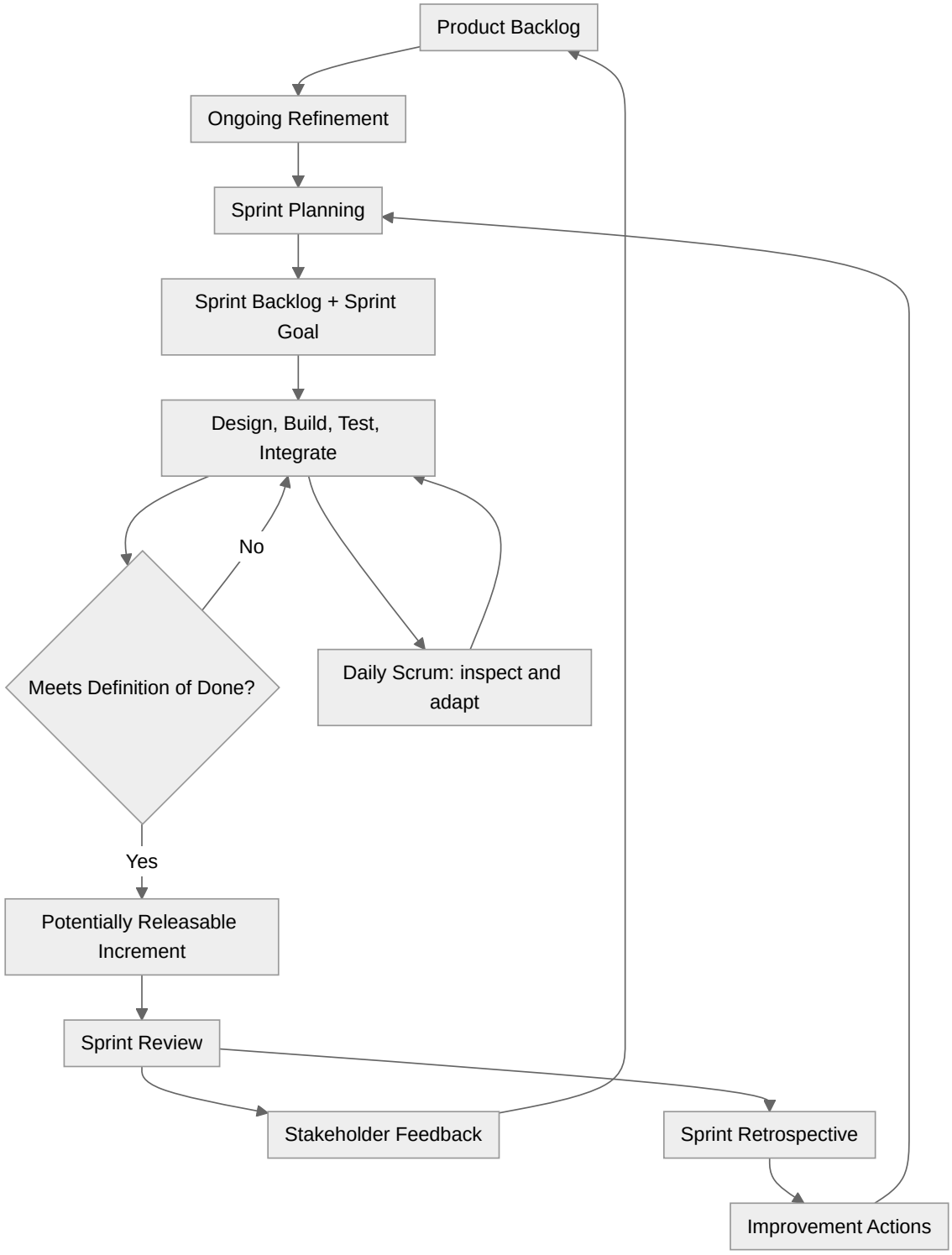


Figure 2.3: Scrum workflow

2.6 Working assumptions

- A small team is building a modular monolith initially.
- Product design, infrastructure, security and automated tests are part of delivery—not separate downstream phases.
- Story points are team-specific estimates, not time conversions.
- Sprint sequence and estimates are provisional until capacity and dependencies are confirmed.

2.7 Scrum Events, Artifacts and Working Agreements

2.7.1 Sprint Planning

Inputs:

- Product Goal and ordered Product Backlog
- latest Increment and feedback
- team capacity, leave, and known operational work

Outputs:

- Sprint Goal
- selected Product Backlog Items
- initial delivery plan

2.7.2 Daily Scrum

- 15 minutes for Developers.
- Inspect progress toward the Sprint Goal.
- Adapt the plan for the next day.
- Use follow-up conversations for detailed problem solving.

2.7.3 Sprint Review

- Demonstrate working software, not slides alone.
- Inspect outcomes against the Sprint Goal.
- Invite support staff/admins to test realistic scenarios.
- Capture feedback and reorder backlog based on evidence.

2.7.4 Sprint Retrospective

Inspect:

- delivery flow and collaboration;
- defect/rework causes;
- CI/test speed;
- architectural friction;
- security/privacy and operational issues;
- one or two actionable improvements for the next Sprint.

2.7.5 Backlog refinement

Ongoing activity to split, clarify, reorder, and estimate upcoming work. Refinement is not a formal Scrum event.

2.7.6 Scrum artifacts and commitments

- **Product Backlog / Product Goal:** ordered work and desired future state.
- **Sprint Backlog / Sprint Goal:** selected work and plan to achieve the Sprint Goal.
- **Increment / Definition of Done:** integrated usable result meeting quality requirements.

2.7.7 Suggested working agreements

- Every story has observable acceptance criteria.
- Security, observability, and automated tests are included in feature work.
- Merge small changes frequently.
- No direct production secrets in source control.
- Do not let model output directly bypass application permissions.
- Review incidents and failed evaluations without blaming individuals.
- Keep story points team-specific; never treat them as hours.

Chapter 3

Product Backlog Overview

The Product Backlog is an ordered list of everything needed to reach the Product Goal. It is emergent: this is a starter backlog, not a commitment. IDs are stable references for planning. Story points are team-specific estimates and are shown as TBD until the team estimates them.

3.1 Epics

Epic	Title	Goal	Stories	Release
E01	Tenant and access foundation	establish secure organization boundaries.	3	R1
E02	Conversation and message core	persist and retrieve a reliable conversation timeline.	4	R1
E03	Website chat channel	provide the first customer-facing channel.	3	R1
E04	Knowledge ingestion and RAG	ground answers in organization-approved sources.	4	R1
E05	AI orchestration	coordinate each conversation turn safely.	4	R1
E06	Safe business tools	let the AI invoke a small allowlisted set of business operations.	4	R1
E07	Human inbox and handoff	ensure a human can take over with context.	4	R1
E08	Tickets and case tracking	track support work beyond one interaction.	2	R1
E09	Voice MVP	support real-time browser voice using the same conversation engine.	5	R2
E10	Workflow, routing and SLA	automate repeatable support operations.	3	R3
E11	Integrations and additional channels	add provider connectors without contaminating core domain logic.	3	R3
E12	Observability, evaluation and usage	operate and improve the platform using evidence.	4	R1/R3

3.2 Suggested order of work

1. E01 — Tenant and access foundation
2. E02 — Conversation and message core
3. E03 — Website chat channel
4. E12 — Observability, evaluation and usage
5. E04 — Knowledge ingestion and RAG
6. E05 — AI orchestration
7. E06 — Safe business tools
8. E07 — Human inbox and handoff
9. E08 — Tickets and case tracking
10. E09 — Voice MVP
11. E10 — Workflow, routing and SLA

12. E11 — Integrations and additional channels
13. E12 — Observability, evaluation and usage

Reorder based on user research, risk, technical dependencies and Sprint Review feedback.

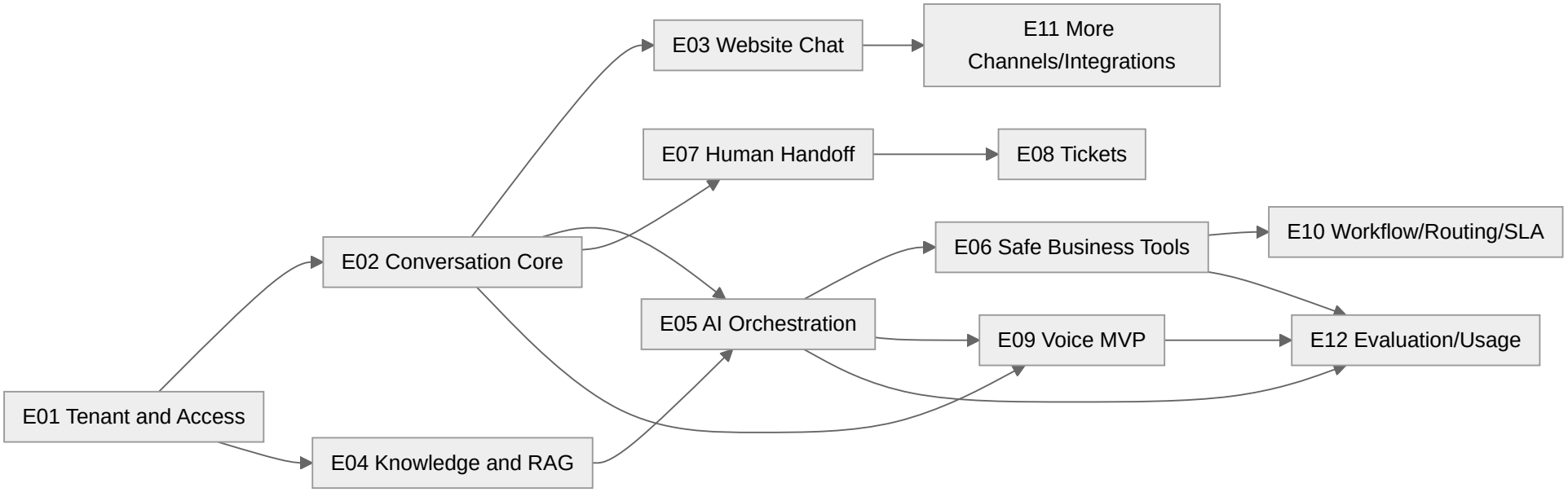


Figure 3.1: Epic dependency graph

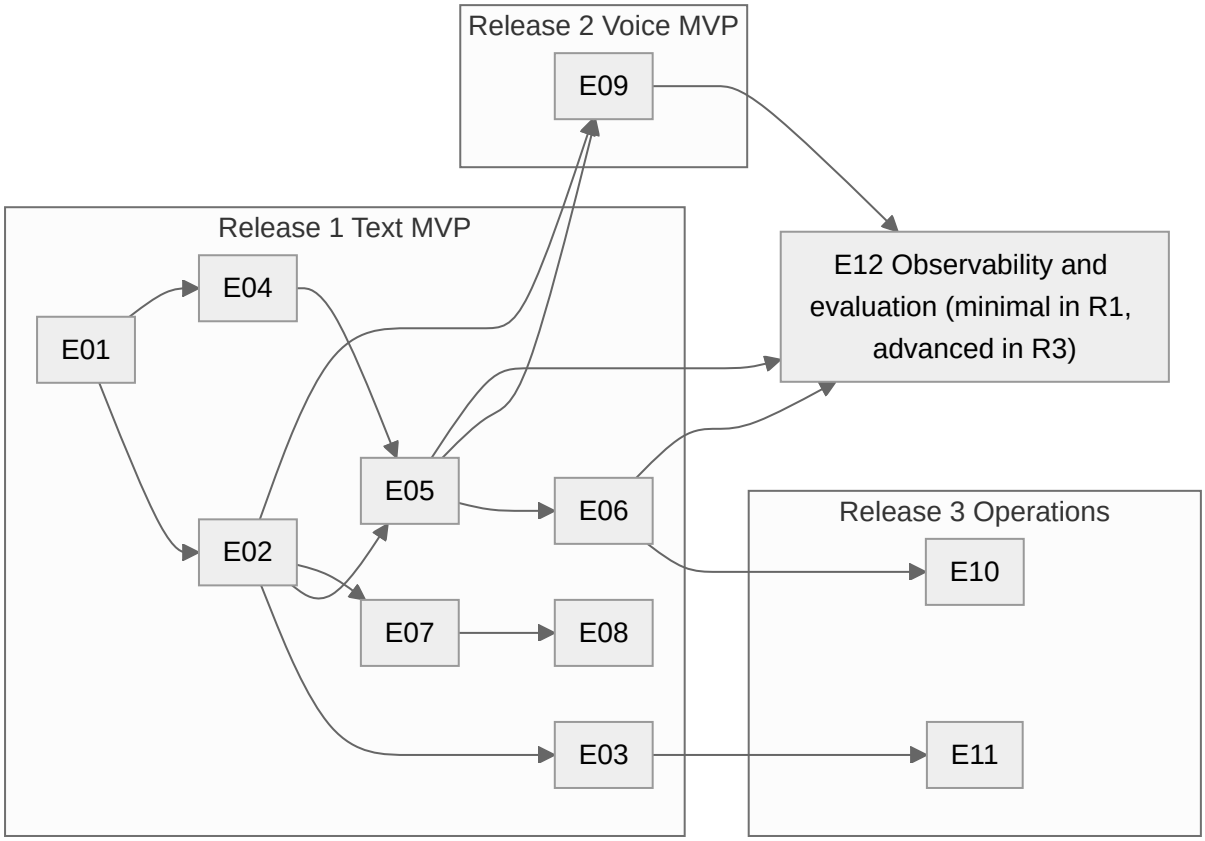


Figure 3.2: Epic delivery map by release

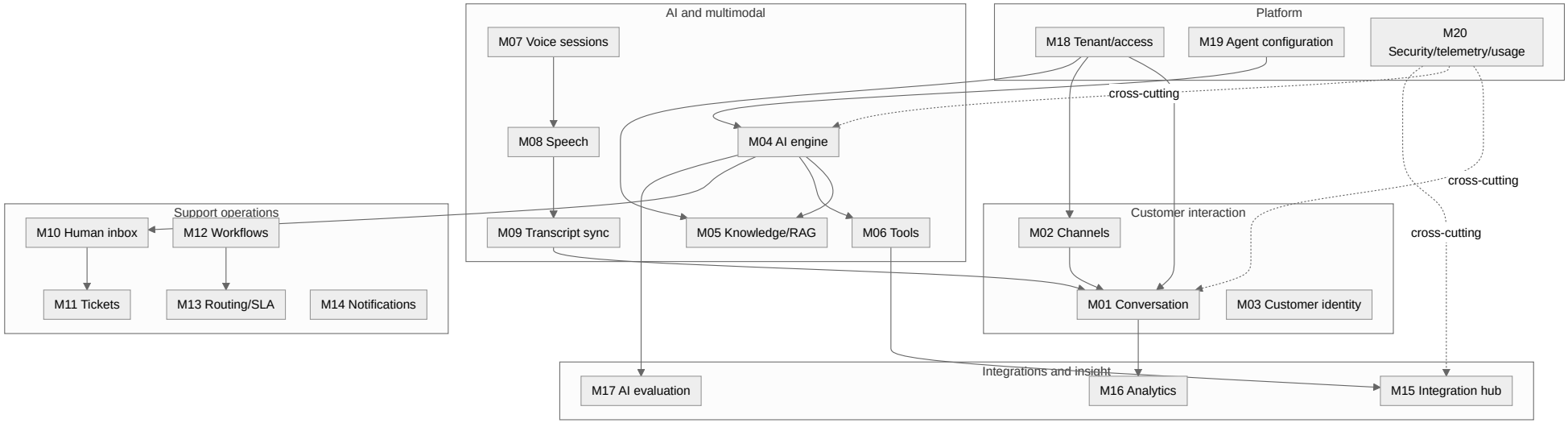


Figure 3.3: Module-to-epic coverage

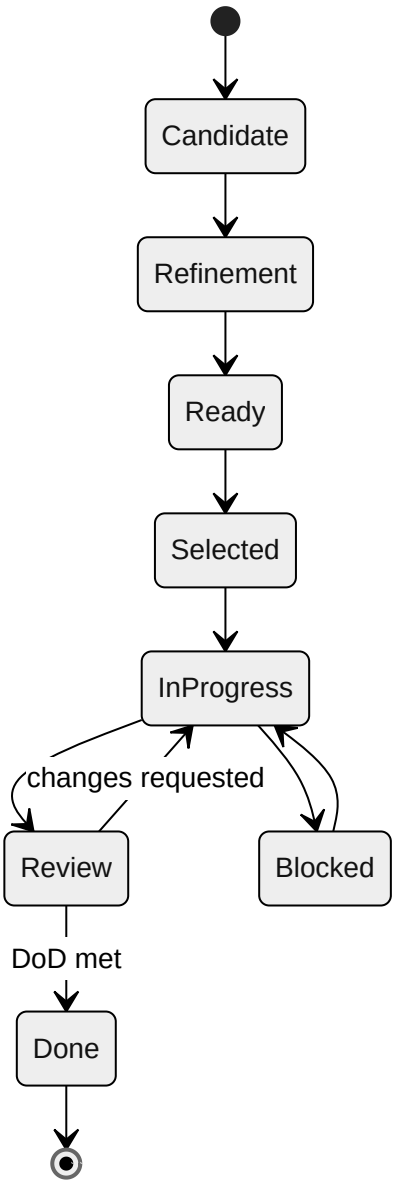


Figure 3.4: User story lifecycle

3.3 Full Product Backlog

Sprint column shows the provisional assignment from the Sprint plan; “Backlog” means not yet scheduled.

ID	Epic	User story	Acceptance criteria	Sprint	Points
US-001	E01	As an organization administrator, I can create/configure an organization so that resources have an owner.	organization ID is generated; default settings exist; audit event is recorded.	S1	TBD
US-002	E01	As an administrator, I can invite users and assign roles.	invitation expires; role is validated server-side; revoked users lose access.	Backlog	TBD
US-003	E01	As a developer, I can test tenant isolation.	cross-tenant read/write attempts are denied in automated tests.	S1	TBD
US-010	E02	As a customer, I can start a conversation through website chat.	conversation is tenant/channel scoped; initial event is persisted.	S1	TBD
US-011	E02	As a customer, I can send and receive text messages.	message order is stable; delivery status is represented; errors are recoverable.	S1	TBD
US-012	E02	As a support agent, I can inspect the conversation timeline.	messages, system events, and visibility levels are distinguished.	Backlog	TBD
US-013	E02	As a system, I can deduplicate inbound events.	replayed provider event does not create duplicate messages.	S2	TBD
US-020	E03	As an administrator, I can configure a website widget.	widget is associated with the correct organization and agent configuration.	S2	TBD
US-021	E03	As a visitor, I can open chat and receive a session.	session token is scoped, expires, and cannot access other tenants.	S2	TBD
US-022	E03	As a visitor, I can reconnect after a temporary network failure.	existing conversation can be resumed under a valid session policy.	S2	TBD
US-030	E04	As an administrator, I can register and upload a knowledge source.	file limits and type checks are enforced; ingestion state is visible.	S3	TBD
US-031	E04	As a system, I can parse, chunk, embed, and index a document asynchronously.	failed ingestion can be retried; document version is traceable.	S3	TBD
US-032	E04	As an AI agent, I can retrieve relevant chunks for a question.	results are tenant-filtered and include source metadata.	S3	TBD
US-033	E04	As a customer, I receive a grounded answer or a clear fallback.	answer is not represented as sourced when retrieval returns no useful evidence.	S4	TBD
US-040	E05	As a system, I can process a turn through a versioned agent configuration.	model/config version, latency, and outcome are recorded.	S4	TBD

ID	Epic	User story	Acceptance criteria	Sprint	Points
US-041	E05	As a customer, I can ask a follow-up question using conversation context.	relevant recent context is used without exposing another customer's data.	S4	TBD
US-042	E05	As a system, I can handle model timeout or provider failure.	bounded retry/fallback occurs; no false success message is delivered.	S4	TBD
US-043	E05	As an administrator, I can publish and roll back an agent version.	published versions are immutable and audit logged.	S6	TBD
US-050	E06	As an administrator, I can enable a versioned tool for an agent.	tool must be allowlisted and schema-valid.	S5	TBD
US-051	E06	As a customer, I can ask for an order status lookup.	customer verification and tenant scope are checked; result is typed and minimal.	S5	TBD
US-052	E06	As a system, I can safely handle tool timeout and duplicate requests.	idempotency behavior is defined; unknown write outcome is not blindly retried.	Backlog	TBD
US-053	E06	As a security reviewer, I can audit tool calls.	actor, tool/version, outcome, timestamp and correlation ID are available with sensitive values redacted.	S5	TBD
US-060	E07	As a support agent, I can view an unassigned queue.	queue is tenant scoped and supports assignment.	S5	TBD
US-061	E07	As a customer, I can request a human.	AI stops responding when handoff policy requires; the customer receives a clear status.	S5	TBD
US-062	E07	As a support agent, I receive a handoff summary.	summary includes transcript, unresolved request and prior tool outcomes permitted by policy.	S5	TBD
US-063	E07	As a support agent, I can resolve the conversation.	resolution and ownership changes are recorded.	Backlog	TBD
US-070	E08	As a support agent, I can create a ticket from a conversation.	ticket links to source conversation and has a status history.	S6	TBD
US-071	E08	As a support lead, I can filter tickets by status, priority and assignment.	filters respect permissions and tenant boundaries.	S6	TBD
US-080	E09	As a customer, I can start and end a voice session.	microphone permission is explicit; session state and failures are visible.	S7	TBD
US-081	E09	As a customer, I can speak and receive a spoken response.	speech provider is configurable; failures have a fallback path.	S7	TBD
US-082	E09	As a customer, I can interrupt the AI while it speaks.	obsolete audio is canceled/truncated where supported and not recorded as fully delivered.	S8	TBD
US-083	E09	As a customer, I can switch between text and voice in one conversation.	finalized transcript and message order remain coherent.	S8	TBD
US-084	E09	As a support agent, I can take over a voice conversation or receive its context.	handoff behavior is explicit for the chosen telephony/media provider.	S8	TBD
US-090	E10	As an administrator, I can define a workflow trigger and steps.	workflow version, run state, and failures are visible.	Backlog	TBD

ID	Epic	User story	Acceptance criteria	Sprint	Points
US-091	E10	As a support lead, I can define routing rules and SLA targets.	rule priority and fallback behavior are deterministic.	Backlog	TBD
US-092	E10	As a system, I can retry a failed workflow step safely.	retry policy is bounded; idempotency is defined per step.	Backlog	TBD
US-100	E11	As an administrator, I can configure an integration securely.	credentials are secret references; test status and health are visible.	Backlog	TBD
US-101	E11	As a system, I can normalize a provider webhook into a canonical event.	signature verified; replay deduplicated; event processing observable.	Backlog	TBD
US-102	E11	As a customer, I can continue support through a second channel.	identity linking follows verification policy; duplicate conversations are avoided where possible.	Backlog	TBD
US-110	E12	As an operator, I can trace one request across modules.	correlation IDs connect API, worker, model, retrieval and tool activity.	S6	TBD
US-111	E12	As a product owner, I can see resolution and handoff metrics.	definitions are documented and metrics are tenant-safe.	Backlog	TBD
US-112	E12	As a team, we can run AI regression evaluations.	dataset and agent/model version are recorded; critical policy tests gate release.	S6	TBD
US-113	E12	As an administrator, I can inspect usage by agent and channel.	provider usage is reconciled and sensitive data is not exposed in metrics.	Backlog	TBD

Part II

Epics and User Stories

Chapter 4

E01 — Tenant and Access Foundation

Goal: establish secure organization boundaries. Modules: M18, M20.

Stories in this epic

ID	User story	Sprint
US-001	As an organization administrator, I can create/configure an organization so that resources have an owner.	S1
US-002	As an administrator, I can invite users and assign roles.	Backlog
US-003	As a developer, I can test tenant isolation.	S1

4.1 Epic overview

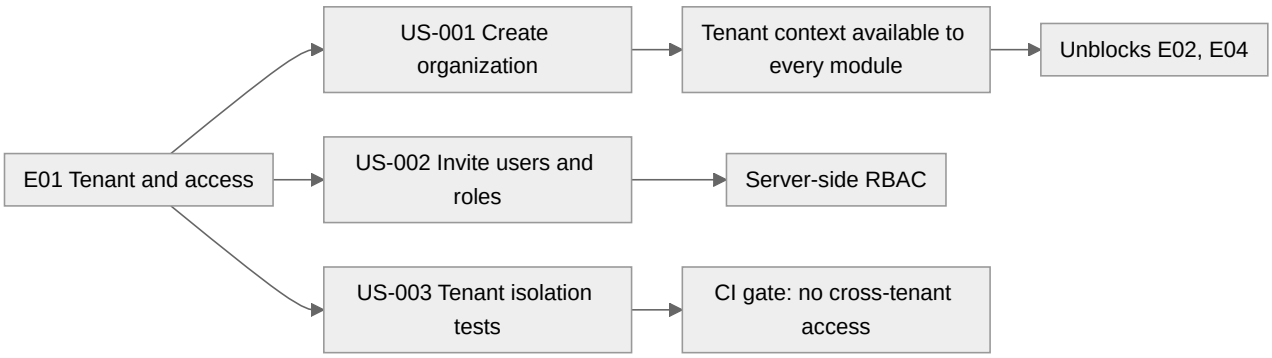


Figure 4.1: E01: Epic overview

4.2 US-001 — Create and configure an organization

User story US-001

Story. As an organization administrator, I can create/configure an organization so that resources have an owner.
Acceptance. organization ID is generated; default settings exist; audit event is recorded.

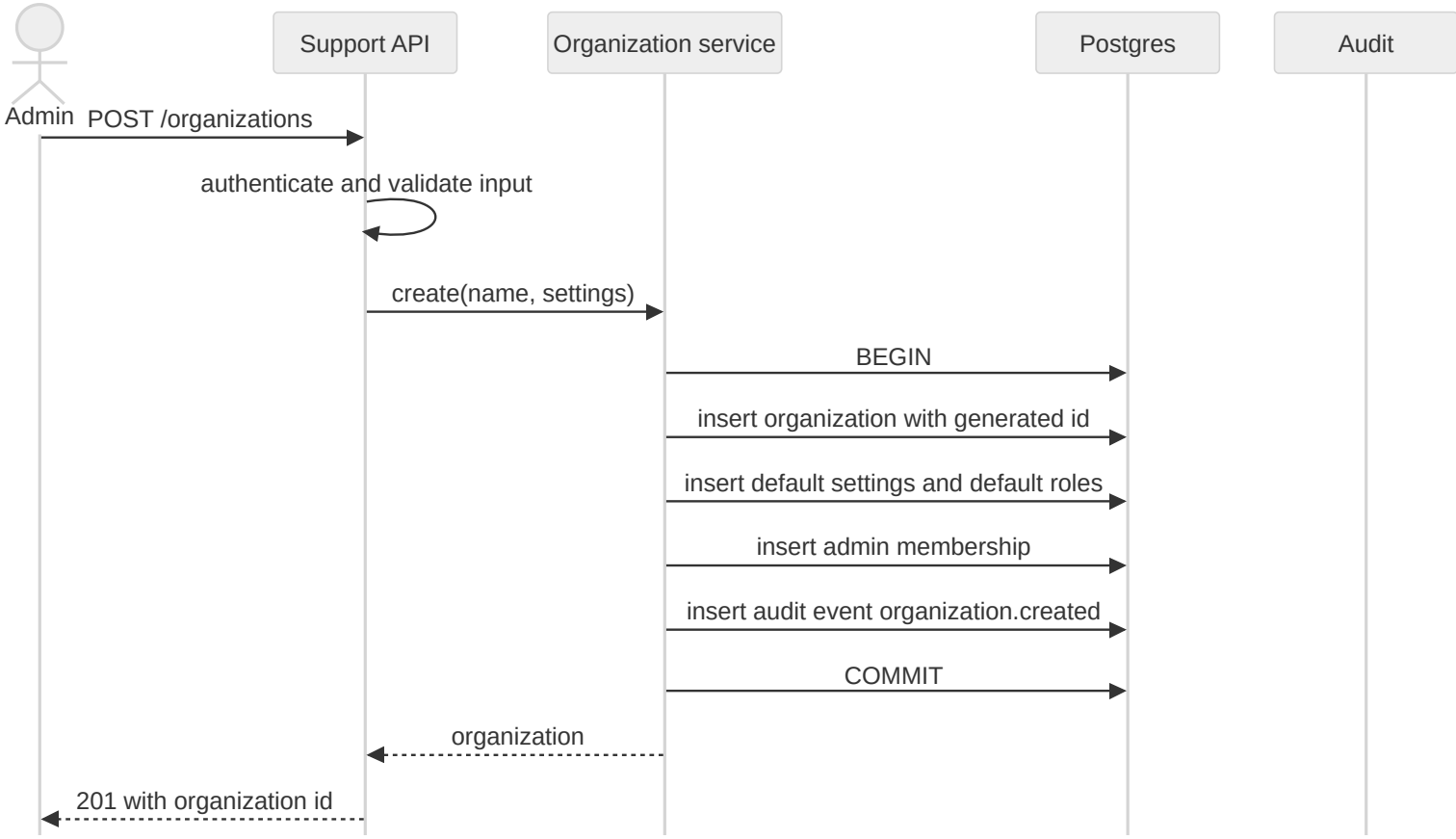


Figure 4.2: E01: US-001 — Create and configure an organization

4.3 US-002 — Invite users and assign roles

User story US-002

Story. As an administrator, I can invite users and assign roles.
Acceptance. invitation expires; role is validated server-side; revoked users lose access.

4.3.1 Invitation lifecycle

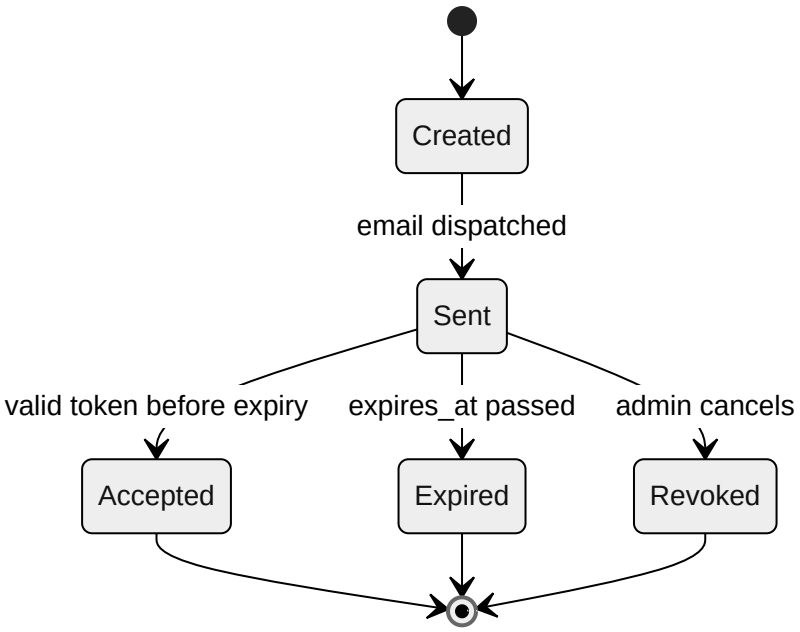


Figure 4.3: E01: US-002 — Invite users and assign roles: Invitation lifecycle

4.3.2 Role assignment and revocation

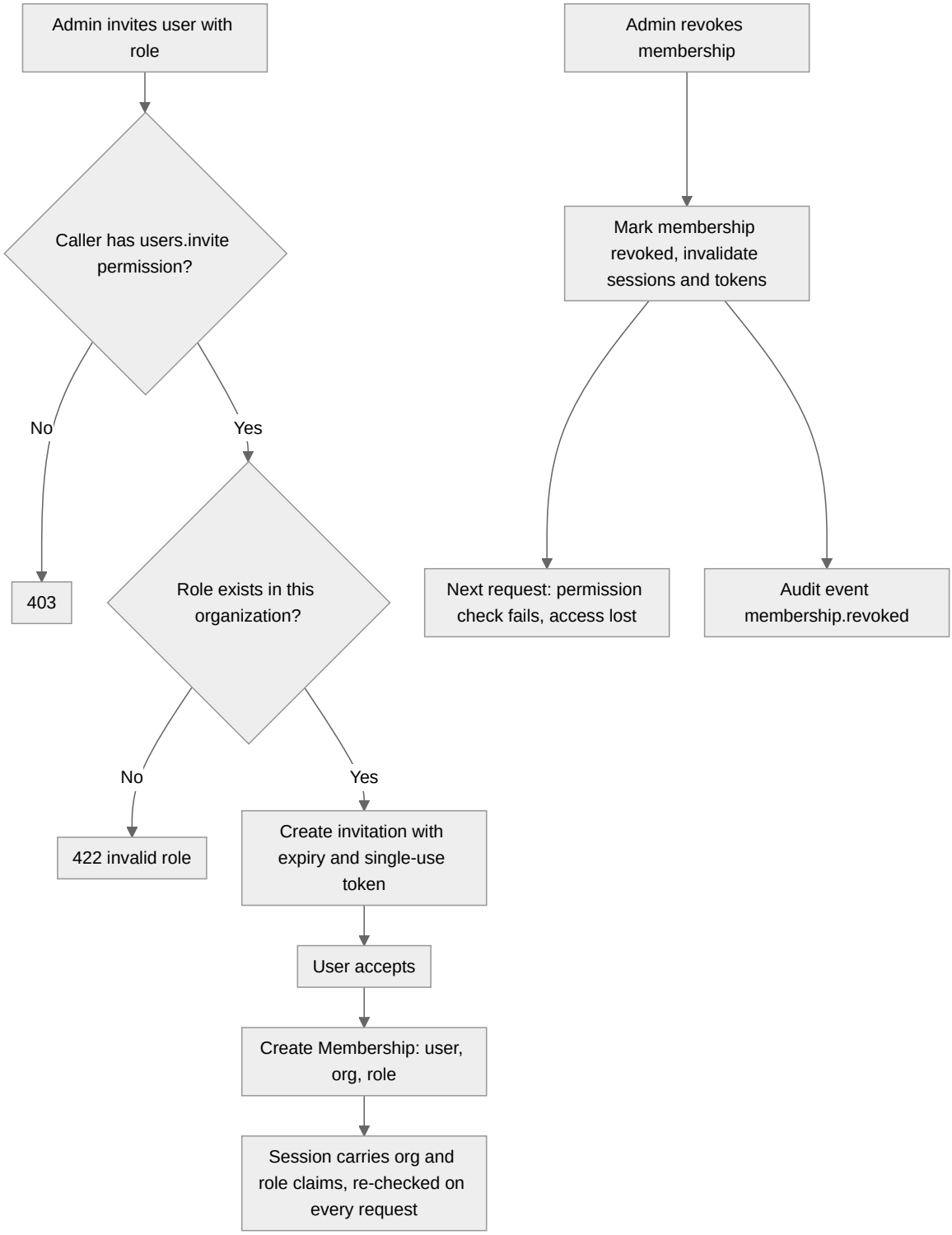


Figure 4.4: E01: US-002 — Invite users and assign roles: Role assignment and revocation

4.4 US-003 — Test tenant isolation

User story US-003

Story. As a developer, I can test tenant isolation.
Acceptance. cross-tenant read/write attempts are denied in automated tests.

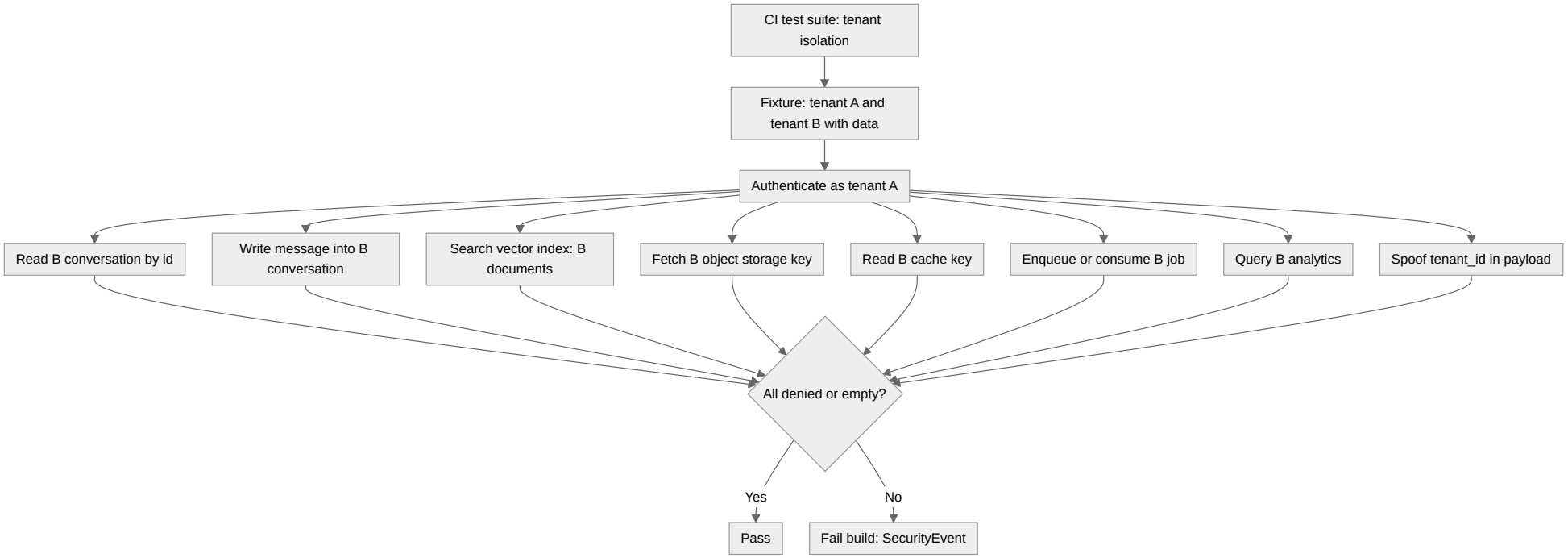


Figure 4.5: E01: US-003 — Test tenant isolation

4.4.1 Where tenant filtering is enforced

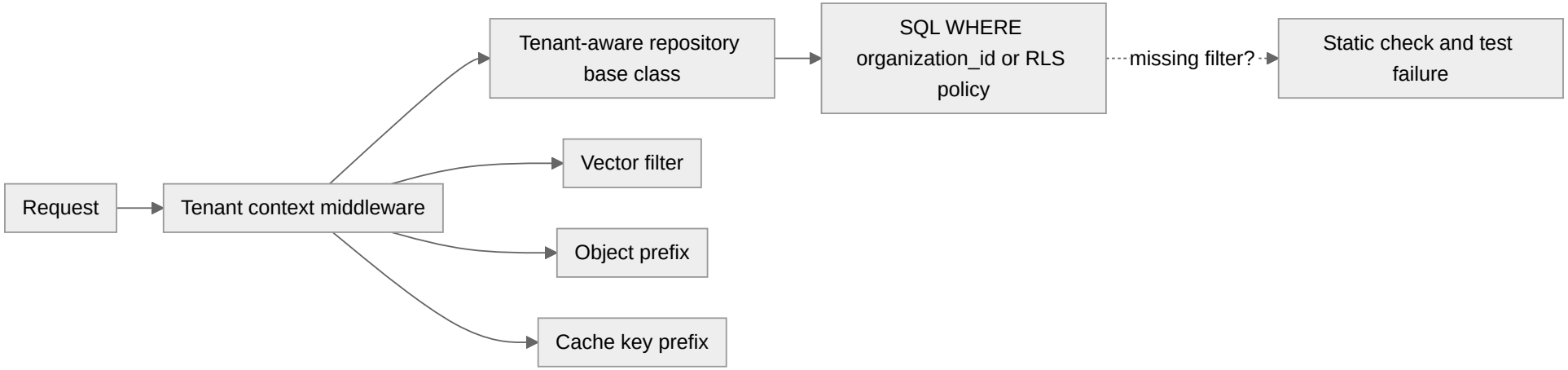


Figure 4.6: E01: US-003 — Test tenant isolation: Where tenant filtering is enforced

Chapter 5

E02 — Conversation and Message Core

Goal: persist and retrieve a reliable conversation timeline. Modules: M01, M02, M03.

Stories in this epic

ID	User story	Sprint
US-010	As a customer, I can start a conversation through website chat.	S1
US-011	As a customer, I can send and receive text messages.	S1
US-012	As a support agent, I can inspect the conversation timeline.	Backlog
US-013	As a system, I can deduplicate inbound events.	S2

5.1 Epic overview

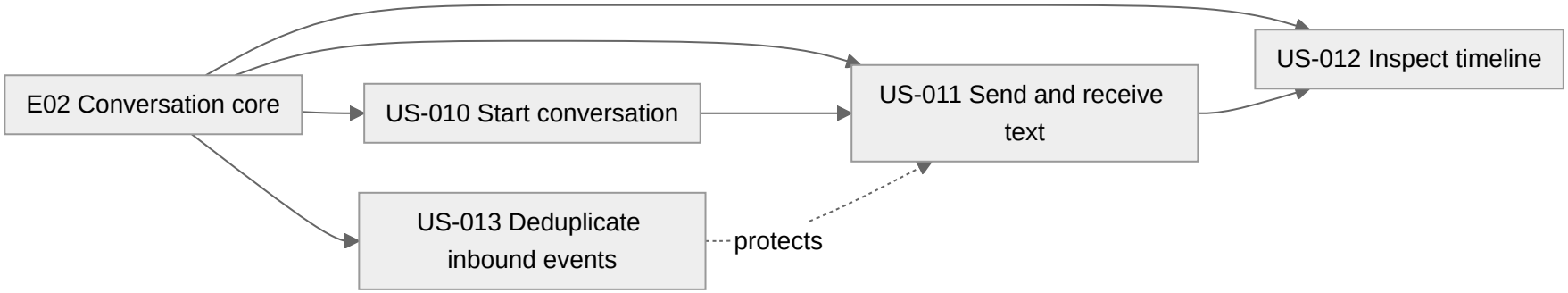


Figure 5.1: E02: Epic overview

5.2 Conversation lifecycle (used by all stories)

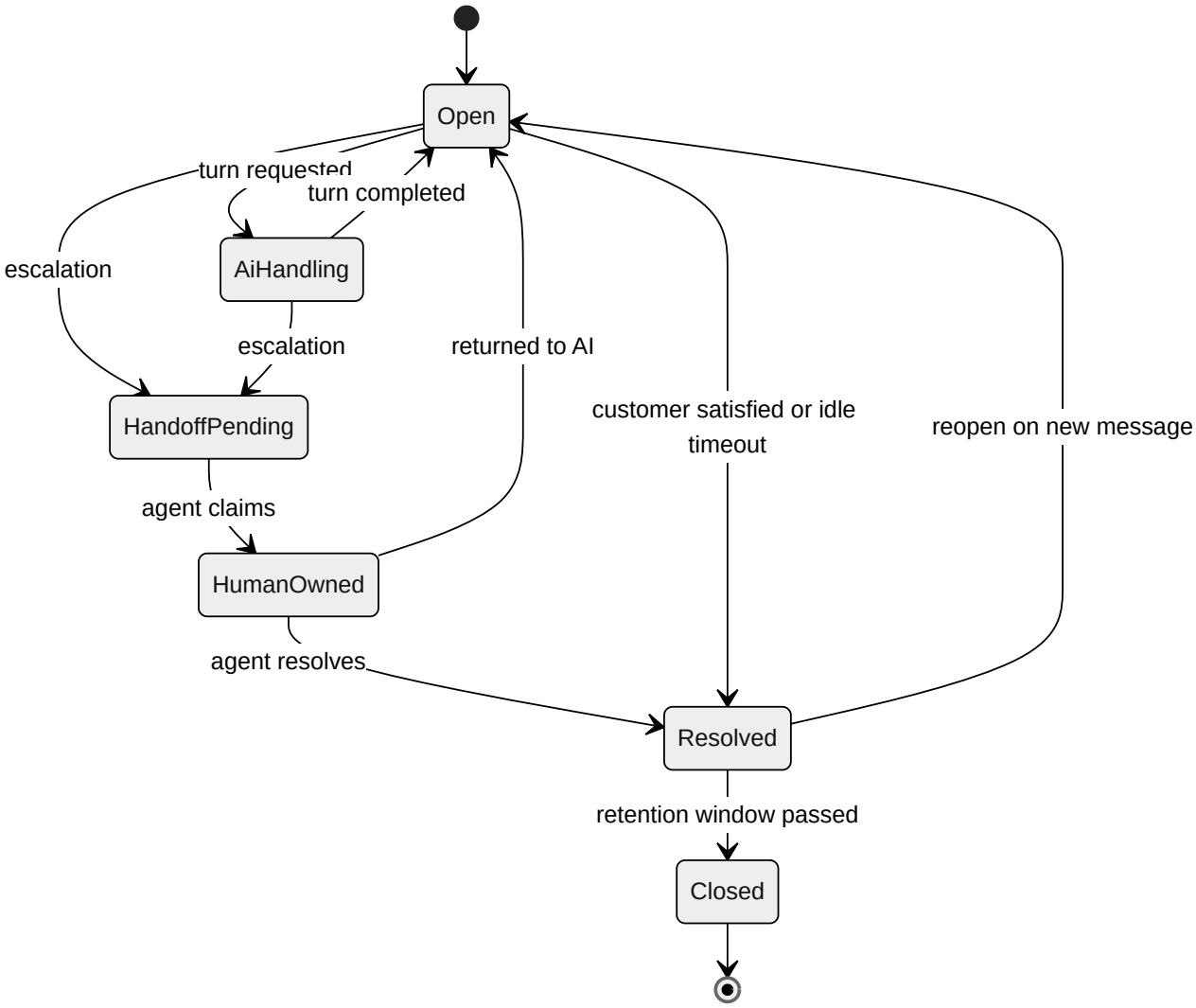


Figure 5.2: E02: Conversation lifecycle (used by all stories)

5.3 US-010 — Start a conversation through website chat

User story US-010

Story. As a customer, I can start a conversation through website chat.
Acceptance. conversation is tenant/channel scoped; initial event is persisted.

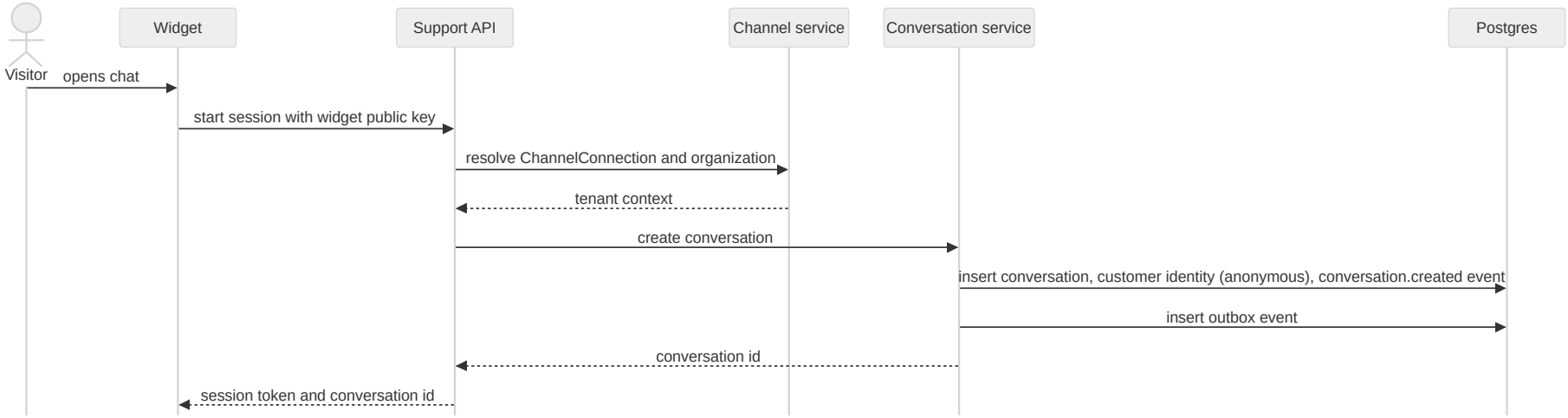


Figure 5.3: E02: US-010 — Start a conversation through website chat

5.4 US-011 — Send and receive text messages

User story US-011

Story. As a customer, I can send and receive text messages.
Acceptance. message order is stable; delivery status is represented; errors are recoverable.

5.4.1 Ordering and persistence

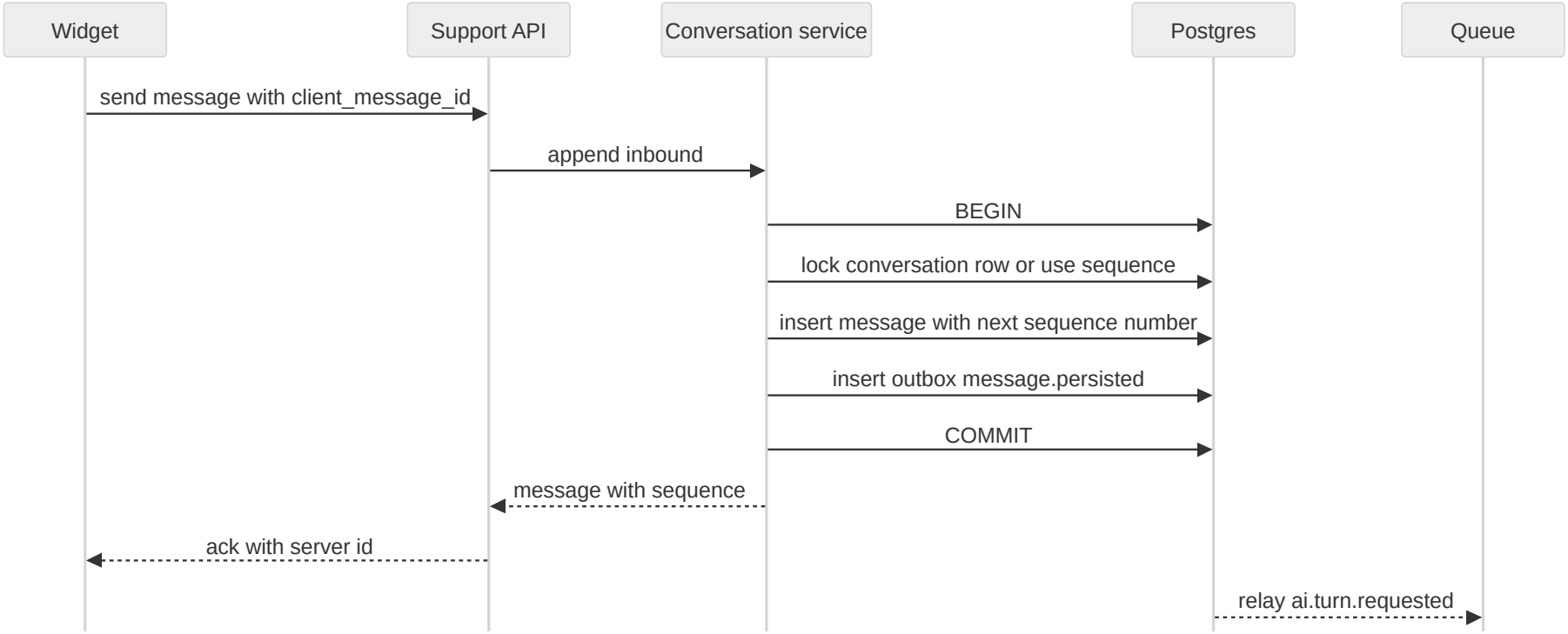


Figure 5.4: E02: US-011 — Send and receive text messages: Ordering and persistence

5.4.2 Message delivery status as seen by the customer

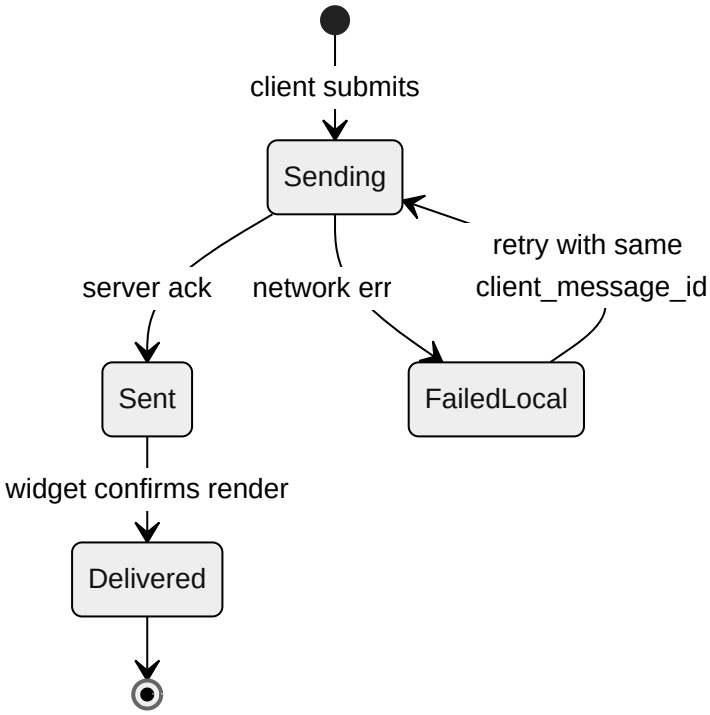


Figure 5.5: E02: US-011 — Send and receive text messages: Message delivery status as seen by the customer

5.5 US-012 — Support agent inspects the timeline

User story US-012

Story. As a support agent, I can inspect the conversation timeline.
Acceptance. messages, system events, and visibility levels are distinguished.

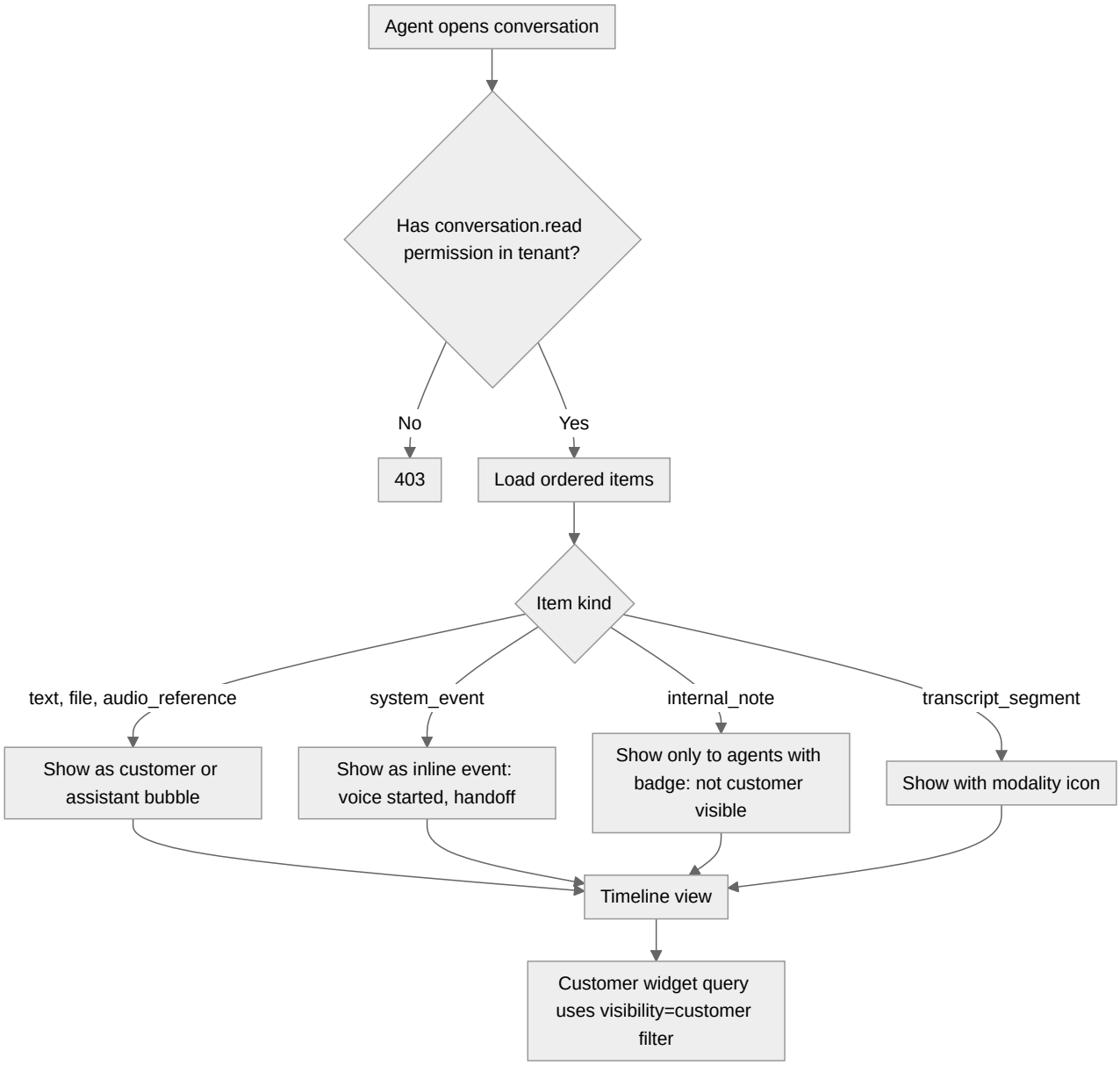


Figure 5.6: E02: US-012 — Support agent inspects the timeline

5.6 US-013 — Deduplicate inbound events

User story US-013

Story. As a system, I can deduplicate inbound events.
Acceptance. replayed provider event does not create duplicate messages.

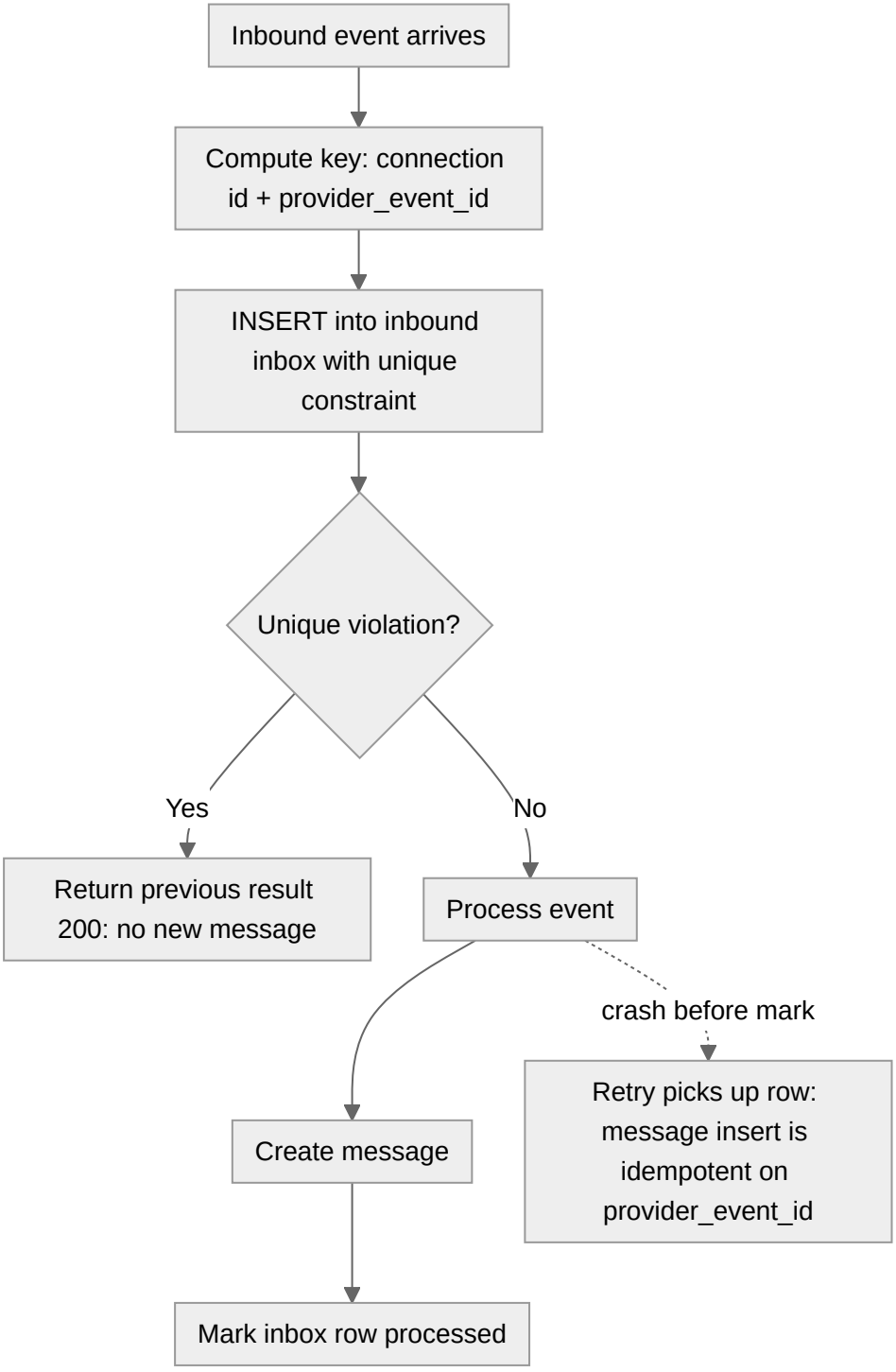


Figure 5.7: E02: US-013 — Deduplicate inbound events

Chapter 6

E03 — Website Chat Channel

Goal: first customer-facing channel. Modules: M02, M03.

Stories in this epic

ID	User story	Sprint
US-020	As an administrator, I can configure a website widget.	S2
US-021	As a visitor, I can open chat and receive a session.	S2
US-022	As a visitor, I can reconnect after a temporary network failure.	S2

6.1 Epic overview

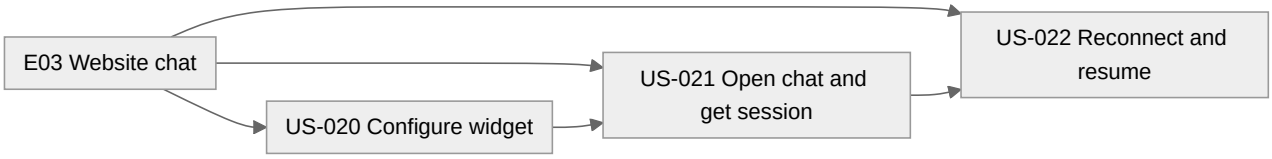


Figure 6.1: E03: Epic overview

6.2 US-020 — Administrator configures the website widget

User story US-020

Story. As an administrator, I can configure a website widget.
Acceptance. widget is associated with the correct organization and agent configuration.

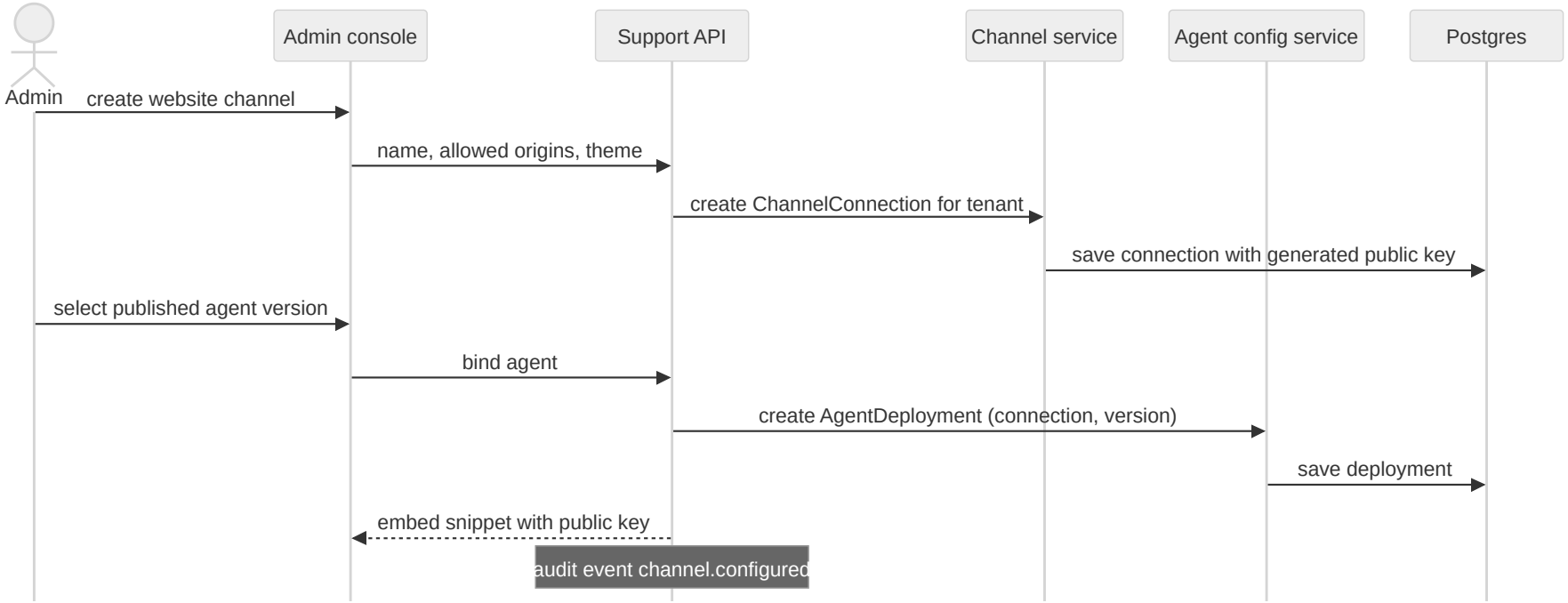


Figure 6.2: E03: US-020 — Administrator configures the website widget

6.3 US-021 — Visitor opens chat and receives a session

User story US-021

Story. As a visitor, I can open chat and receive a session.
Acceptance. session token is scoped, expires, and cannot access other tenants.

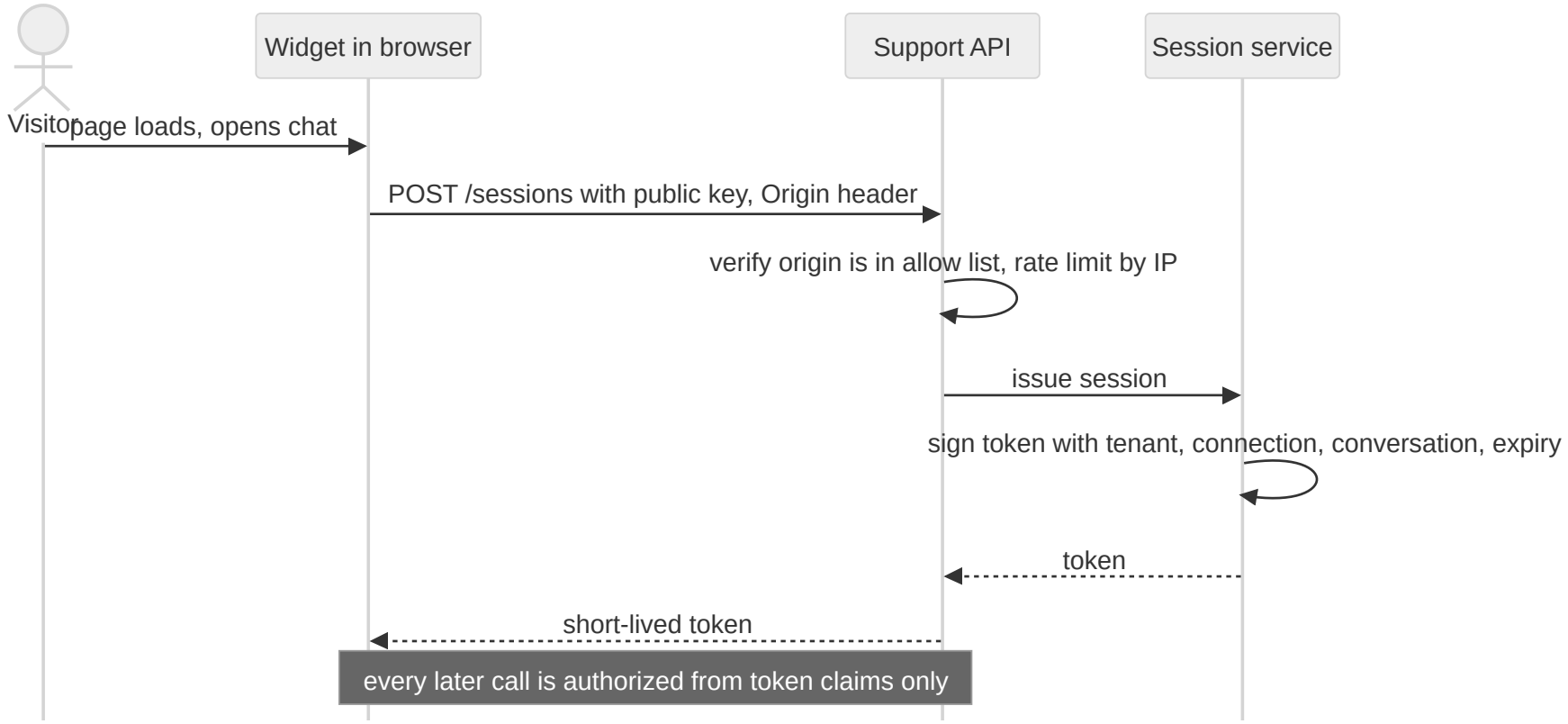


Figure 6.3: E03: US-021 — Visitor opens chat and receives a session

6.3.1 Token claim checks

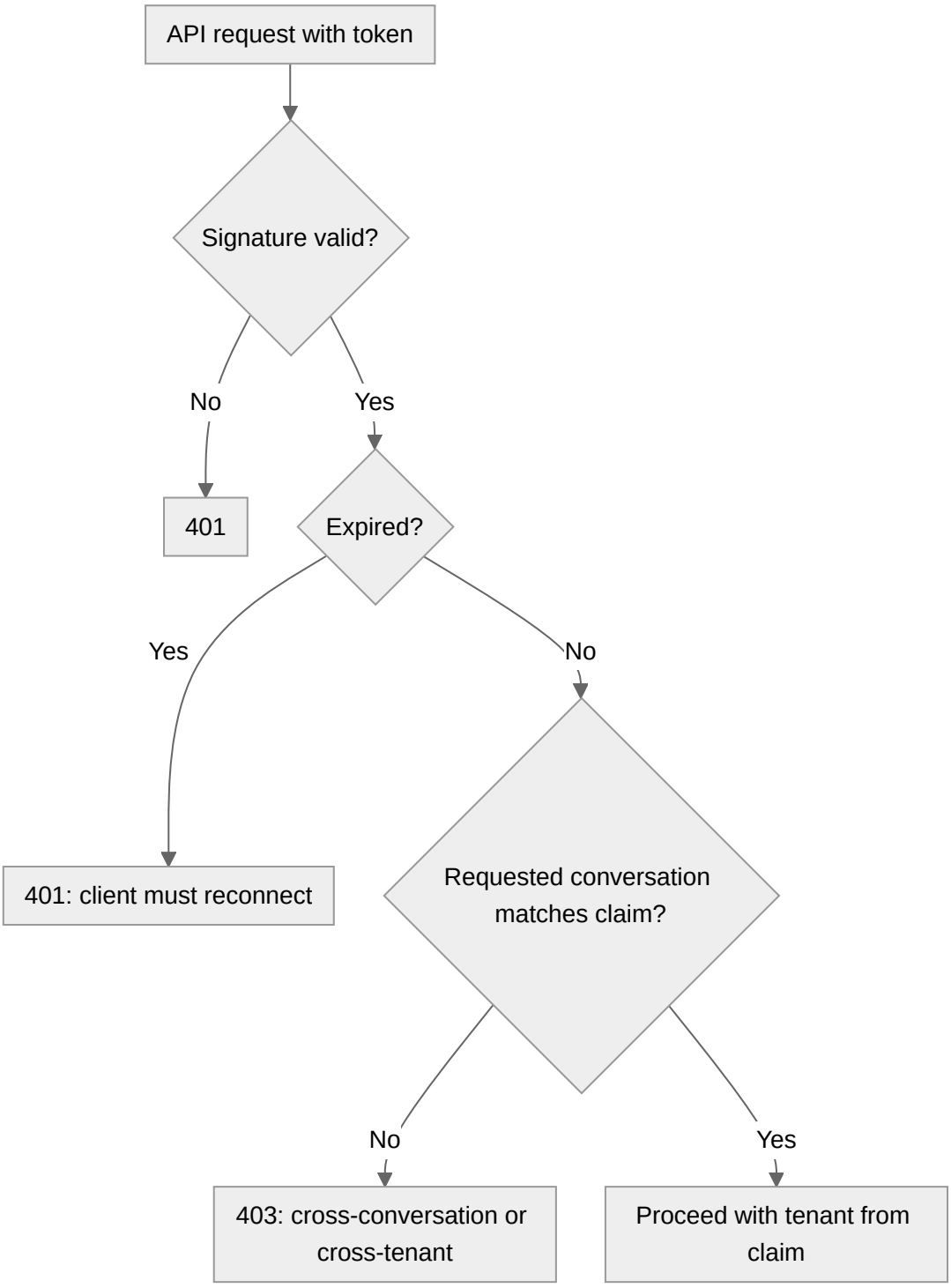


Figure 6.4: E03: US-021 — Visitor opens chat and receives a session: Token claim checks

6.4 US-022 — Reconnect after temporary network failure

User story US-022

Story. As a visitor, I can reconnect after a temporary network failure.
Acceptance. existing conversation can be resumed under a valid session policy.

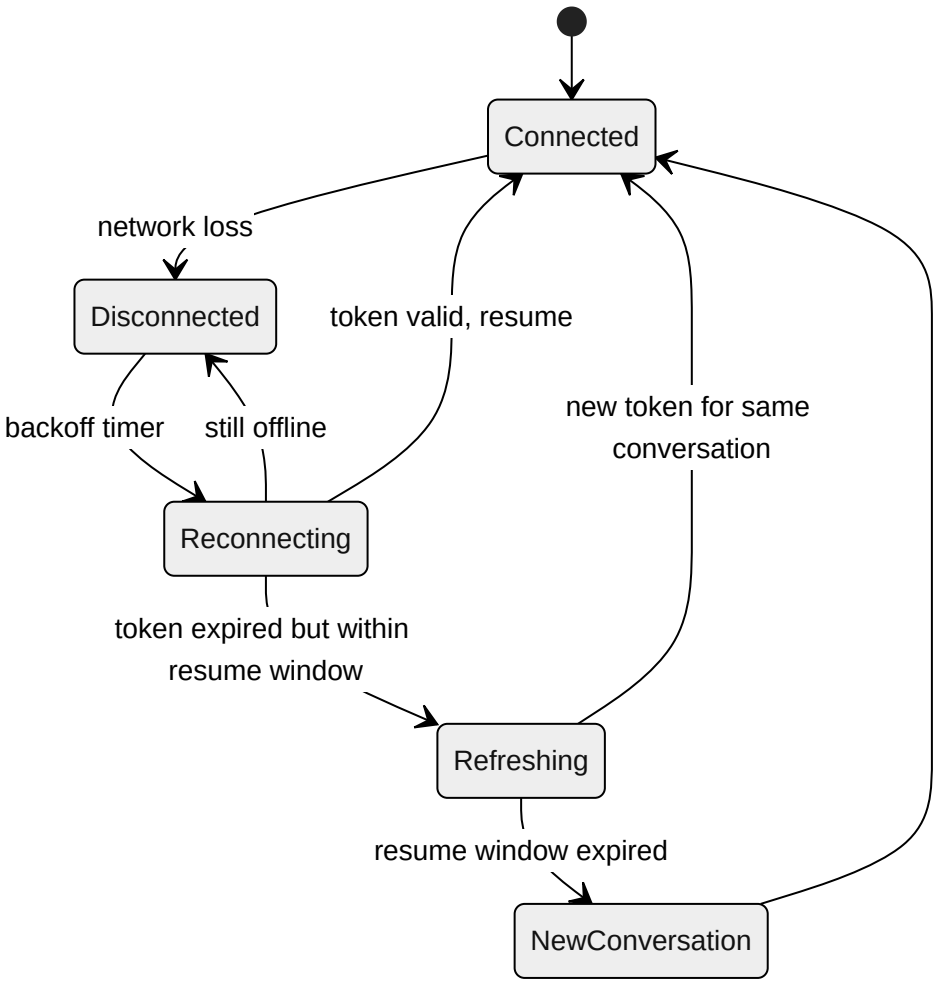


Figure 6.5: E03: US-022 — Reconnect after temporary network failure

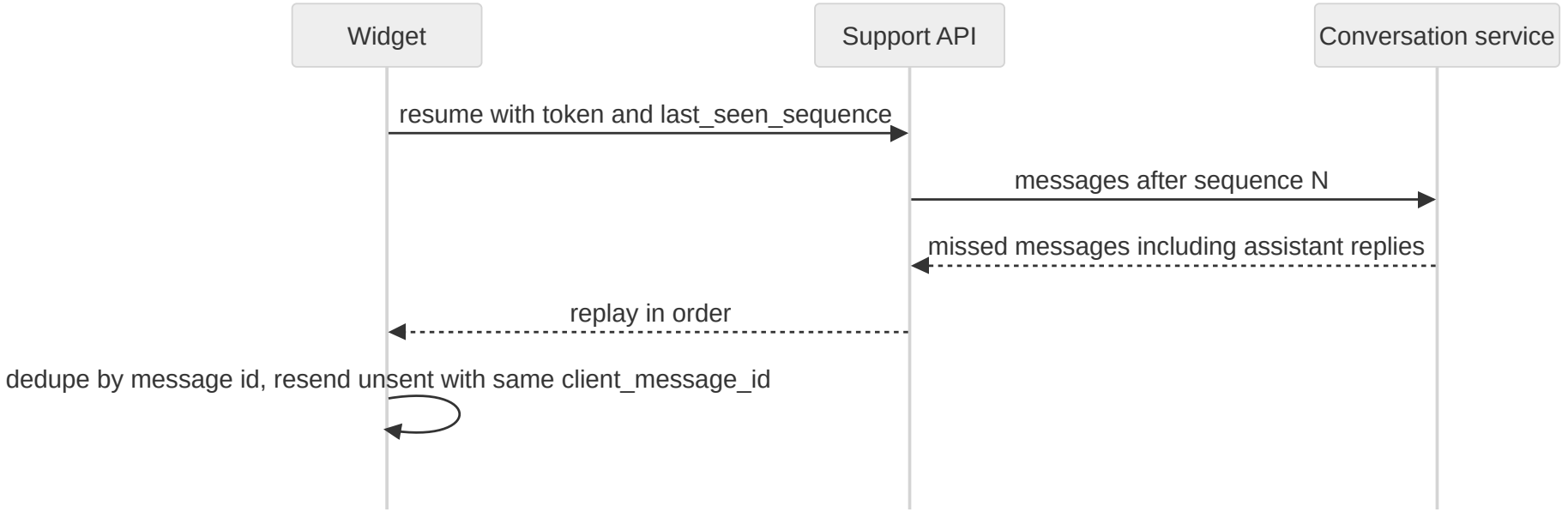


Figure 6.6: E03: US-022 — Reconnect after temporary network failure

Chapter 7

E04 — Knowledge Ingestion and RAG

Goal: ground answers in organization-approved sources. Module: M05.

Stories in this epic

ID	User story	Sprint
US-030	As an administrator, I can register and upload a knowledge source.	S3
US-031	As a system, I can parse, chunk, embed, and index a document asynchronously.	S3
US-032	As an AI agent, I can retrieve relevant chunks for a question.	S3
US-033	As a customer, I receive a grounded answer or a clear fallback.	S4

7.1 Epic overview

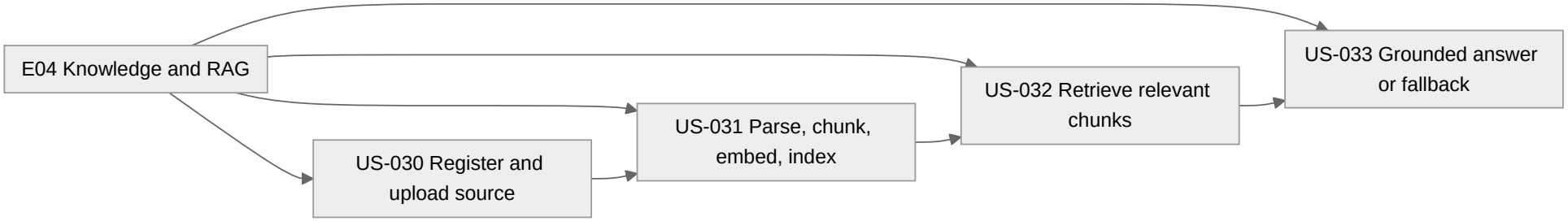


Figure 7.1: E04: Epic overview

7.2 US-030 — Register and upload a knowledge source

User story US-030

Story. As an administrator, I can register and upload a knowledge source.
Acceptance. file limits and type checks are enforced; ingestion state is visible.

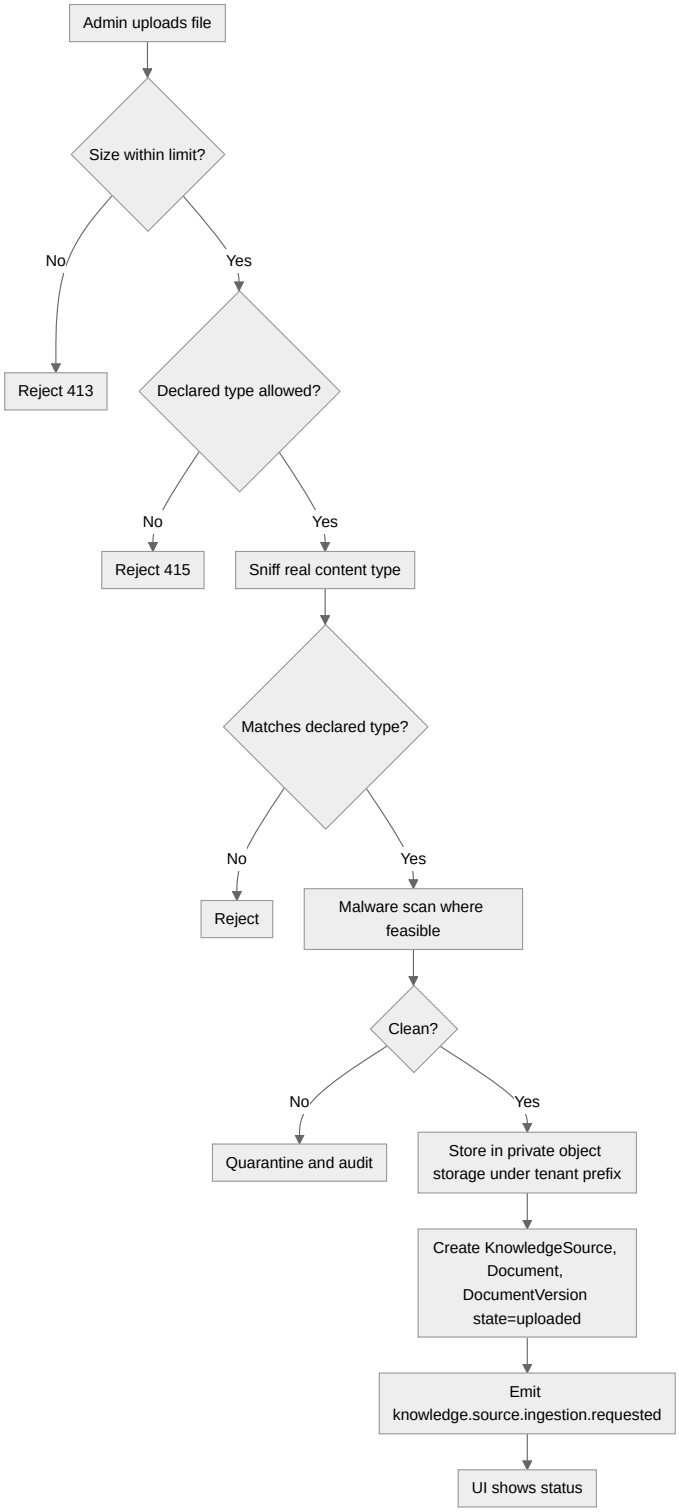


Figure 7.2: E04: US-030 — Register and upload a knowledge source

7.3 US-031 — Parse, chunk, embed and index asynchronously

User story US-031

Story. As a system, I can parse, chunk, embed, and index a document asynchronously.
Acceptance. failed ingestion can be retried; document version is traceable.

7.3.1 Pipeline

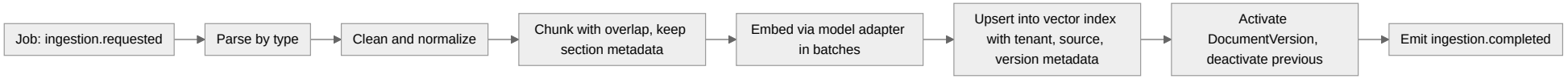


Figure 7.3: E04: US-031 — Parse, chunk, embed and index asynchronously: Pipeline

7.3.2 Ingestion state machine

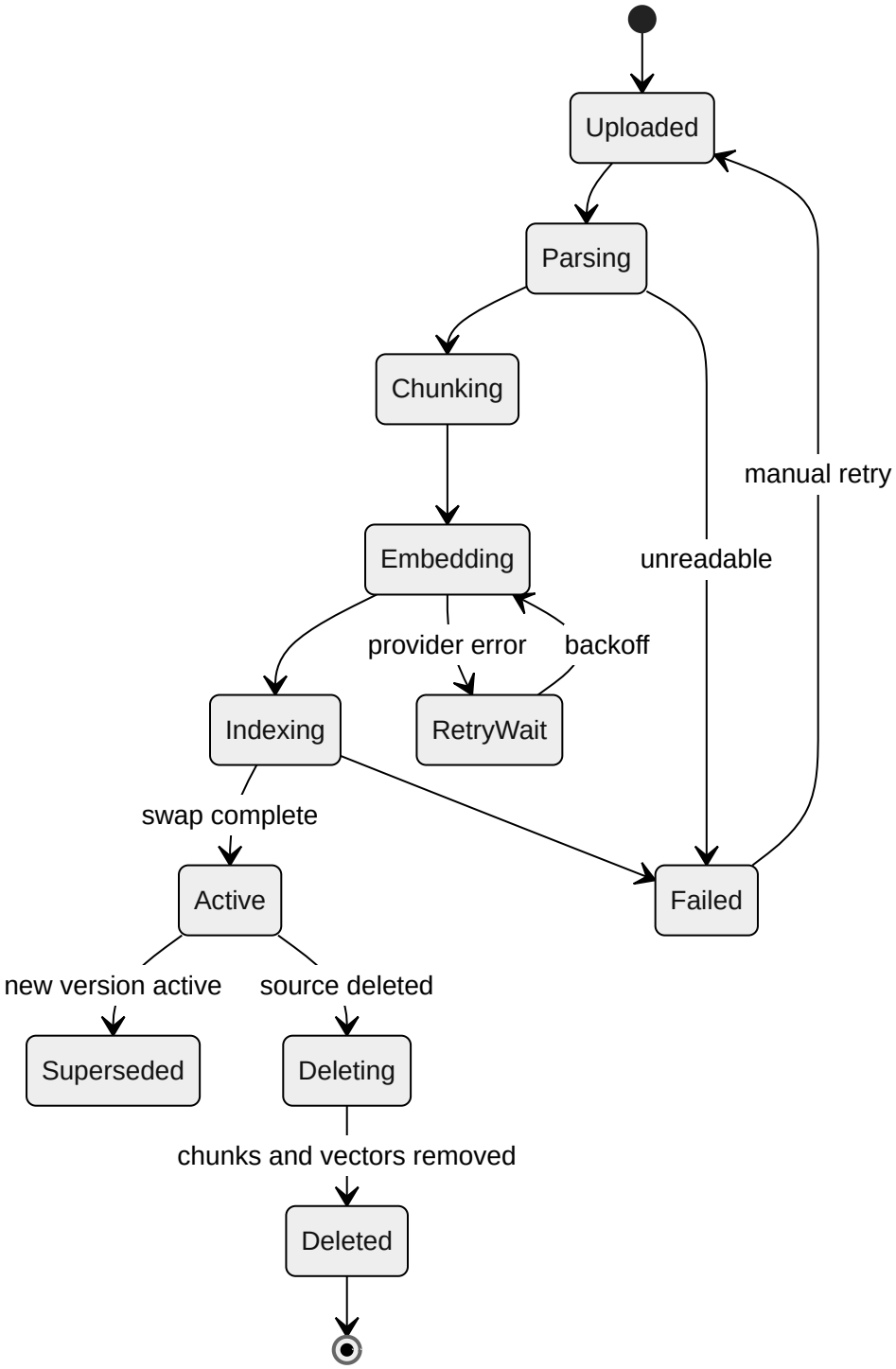


Figure 7.4: E04: US-031 — Parse, chunk, embed and index asynchronously: Ingestion state machine

7.3.3 Deletion and freshness propagation

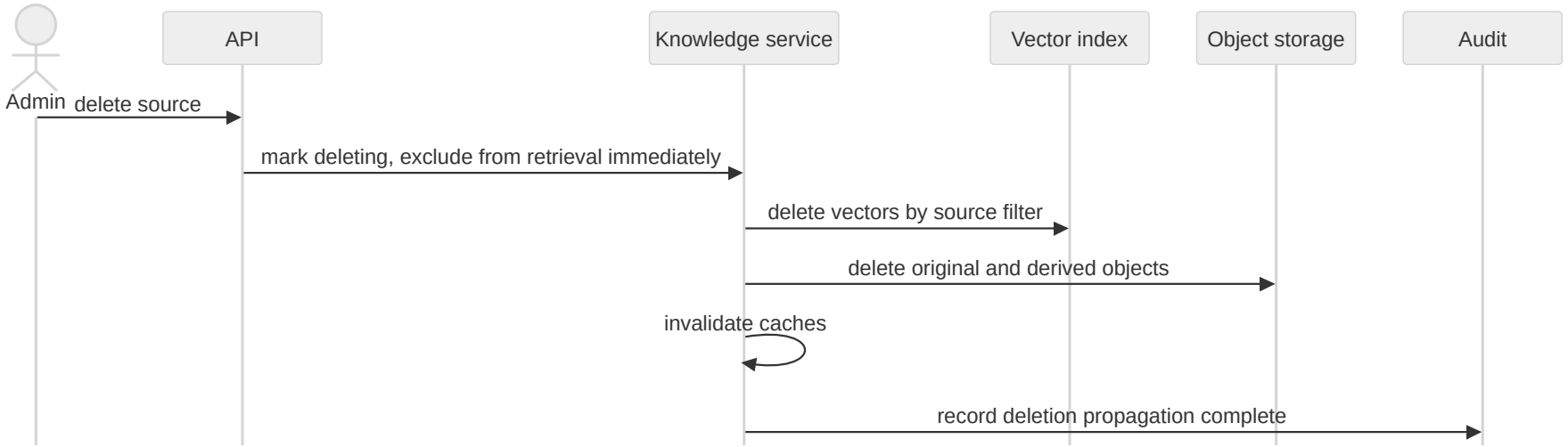


Figure 7.5: E04: US-031 — Parse, chunk, embed and index asynchronously: Deletion and freshness propagation

7.4 US-032 — Retrieve relevant chunks

User story US-032

Story. As an AI agent, I can retrieve relevant chunks for a question.
Acceptance. results are tenant-filtered and include source metadata.

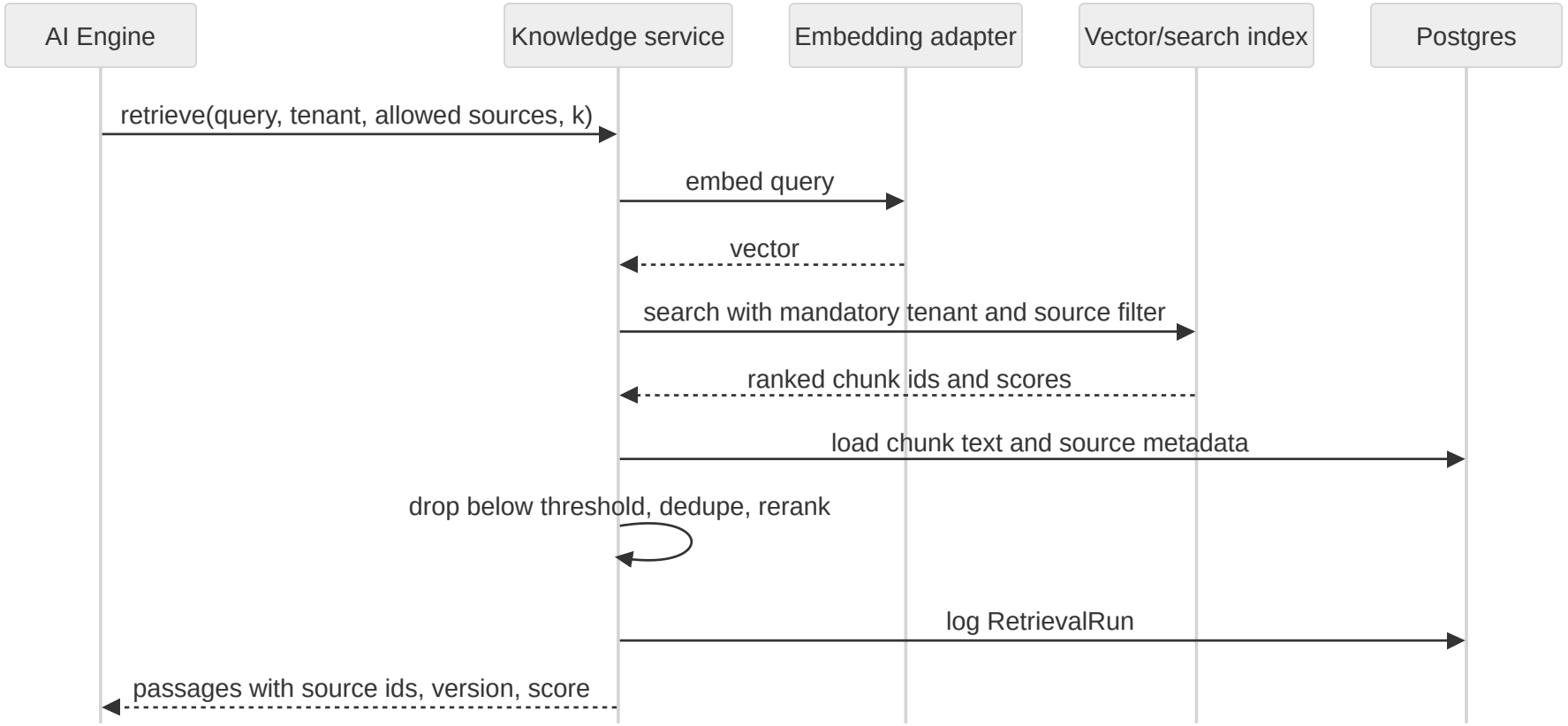


Figure 7.6: E04: US-032 — Retrieve relevant chunks

7.5 US-033 — Grounded answer or clear fallback

User story US-033

Story. As a customer, I receive a grounded answer or a clear fallback.
Acceptance. answer is not represented as sourced when retrieval returns no useful evidence.

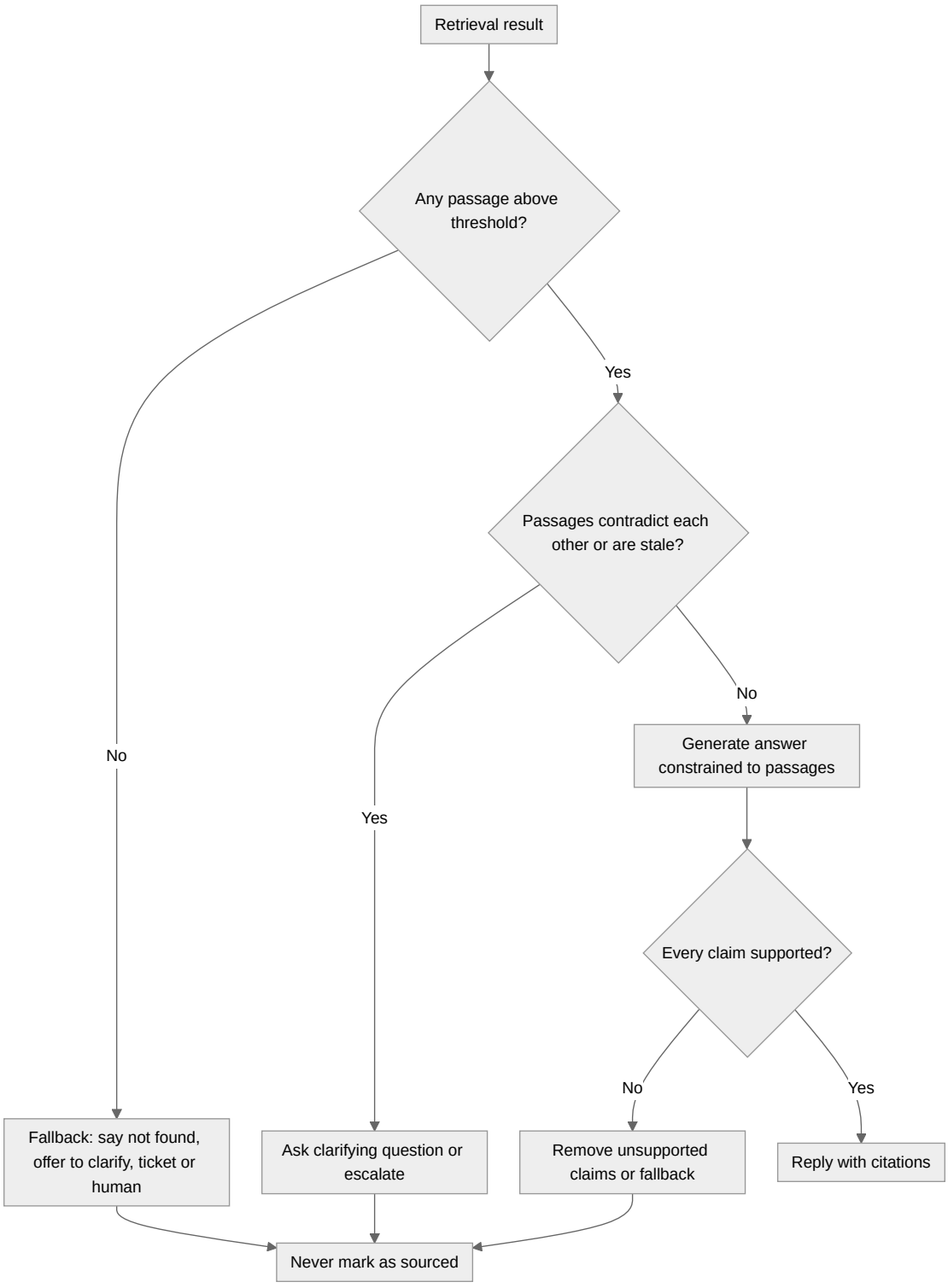


Figure 7.7: E04: US-033 — Grounded answer or clear fallback

Chapter 8

E05 — AI Orchestration

Goal: coordinate each conversation turn safely. Modules: M04, M19. Detailed turn pipeline: see the AI engine diagrams.

Stories in this epic

ID	User story	Sprint
US-040	As a system, I can process a turn through a versioned agent configuration.	S4
US-041	As a customer, I can ask a follow-up question using conversation context.	S4
US-042	As a system, I can handle model timeout or provider failure.	S4
US-043	As an administrator, I can publish and roll back an agent version.	S6

8.1 Epic overview

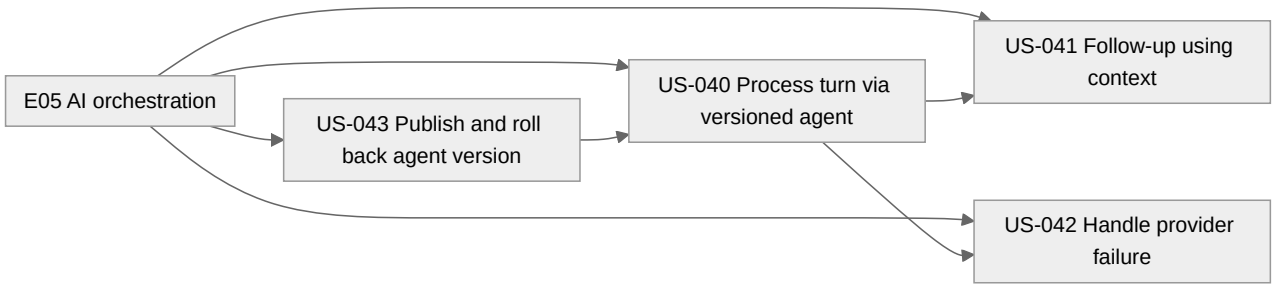


Figure 8.1: E05: Epic overview

8.2 US-040 — Process a turn through a versioned agent configuration

User story US-040

Story. As a system, I can process a turn through a versioned agent configuration.
Acceptance. model/config version, latency, and outcome are recorded.

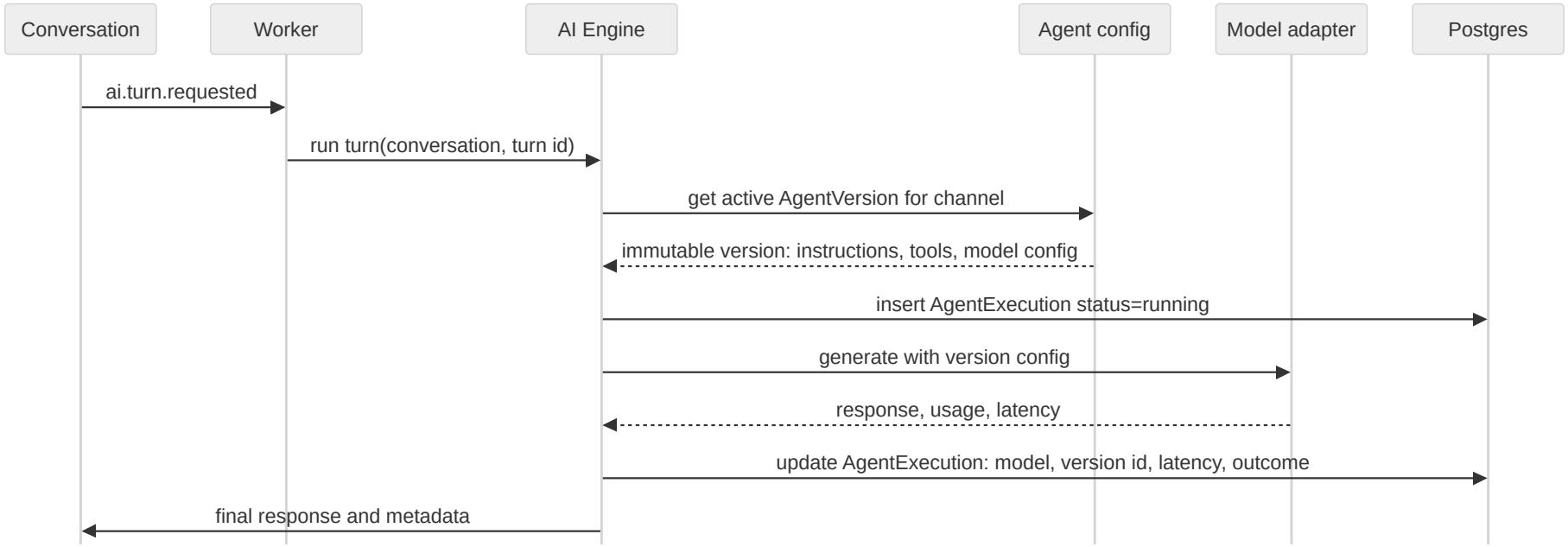


Figure 8.2: E05: US-040 — Process a turn through a versioned agent configuration

8.3 US-041 — Follow-up question using conversation context

User story US-041

Story. As a customer, I can ask a follow-up question using conversation context.
Acceptance. relevant recent context is used without exposing another customer's data.

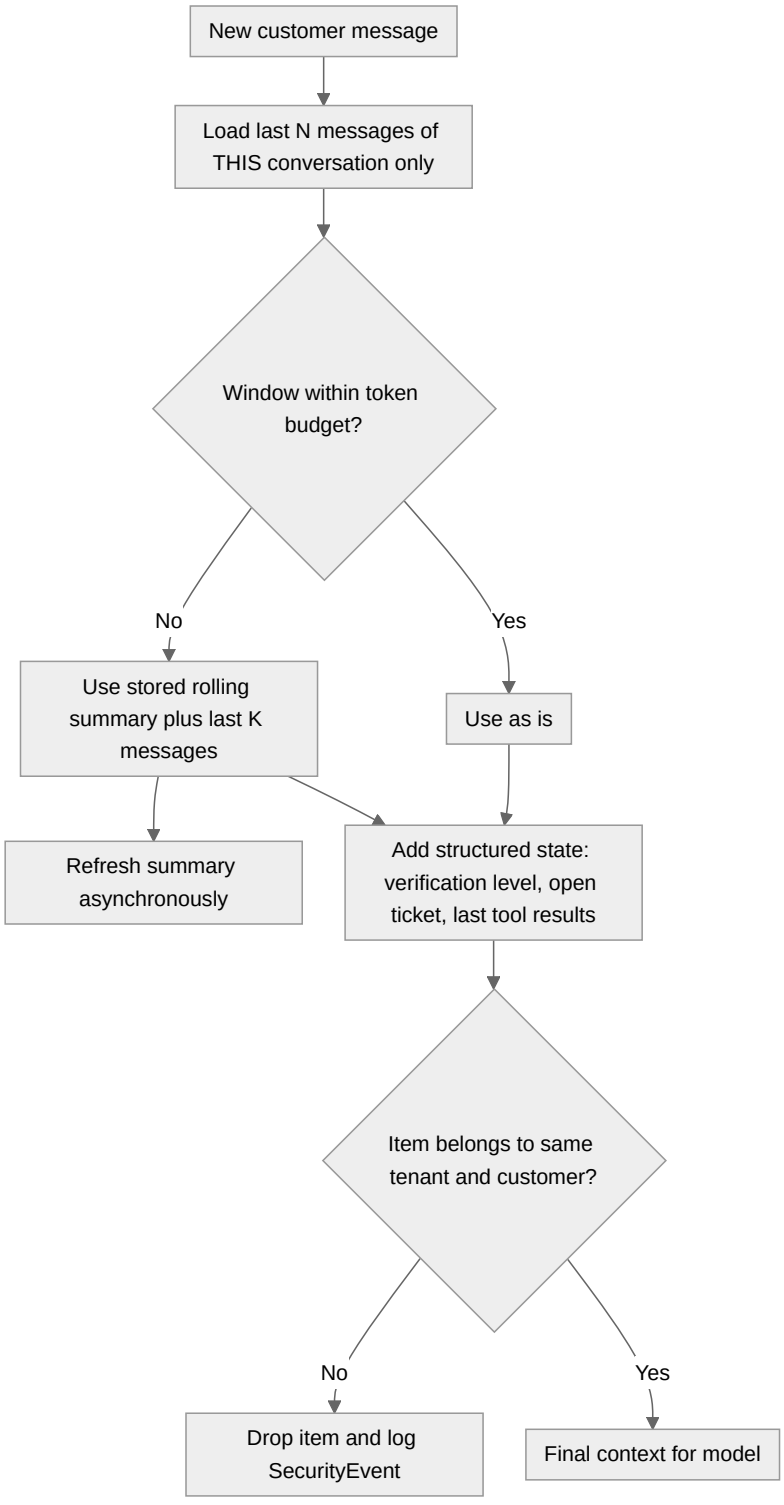


Figure 8.3: E05: US-041 — Follow-up question using conversation context

8.4 US-042 — Handle model timeout or provider failure

User story US-042

Story. As a system, I can handle model timeout or provider failure.
Acceptance. bounded retry/fallback occurs; no false success message is delivered.

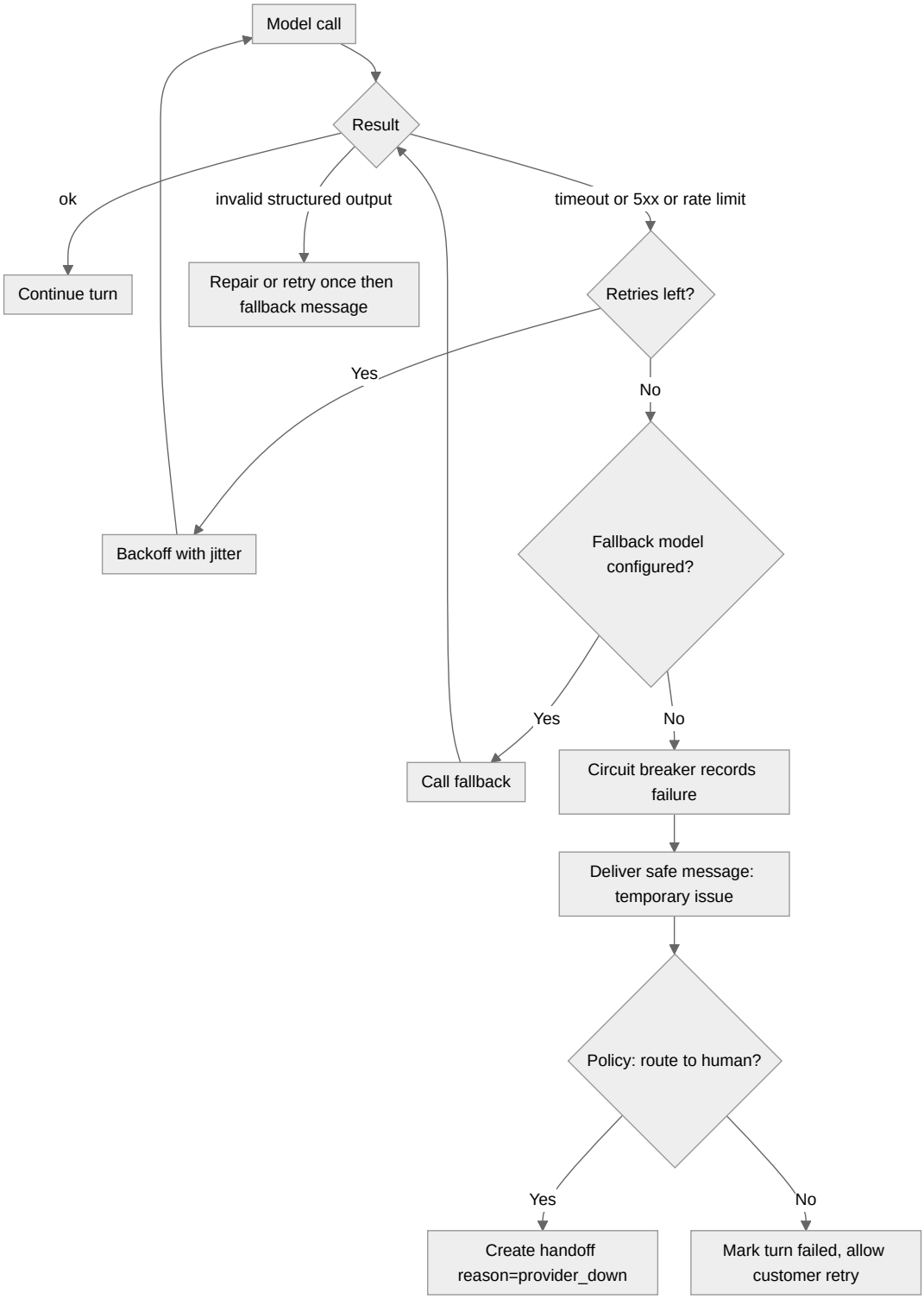


Figure 8.4: E05: US-042 — Handle model timeout or provider failure

8.5 US-043 — Publish and roll back an agent version

User story US-043

Story. As an administrator, I can publish and roll back an agent version.
Acceptance. published versions are immutable and audit logged.

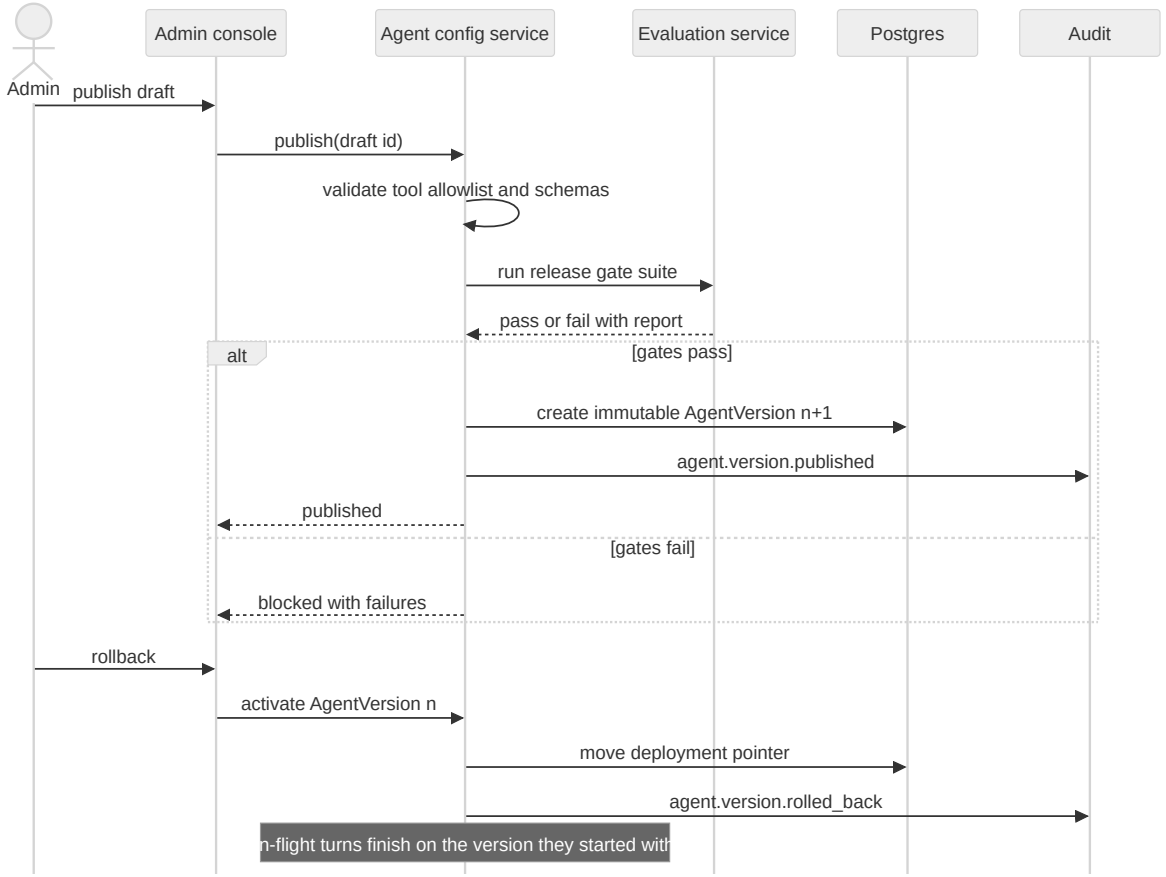


Figure 8.5: E05: US-043 — Publish and roll back an agent version

Chapter 9

E06 — Safe Business Tools

Goal: let the AI invoke a small allowlisted set of operations. Modules: M06, M15. Full pipeline: see the cross-cutting flow diagrams sections 2.5 and 2.6.

Stories in this epic

ID	User story	Sprint
US-050	As an administrator, I can enable a versioned tool for an agent.	S5
US-051	As a customer, I can ask for an order status lookup.	S5
US-052	As a system, I can safely handle tool timeout and duplicate requests.	Backlog
US-053	As a security reviewer, I can audit tool calls.	S5

9.1 Epic overview

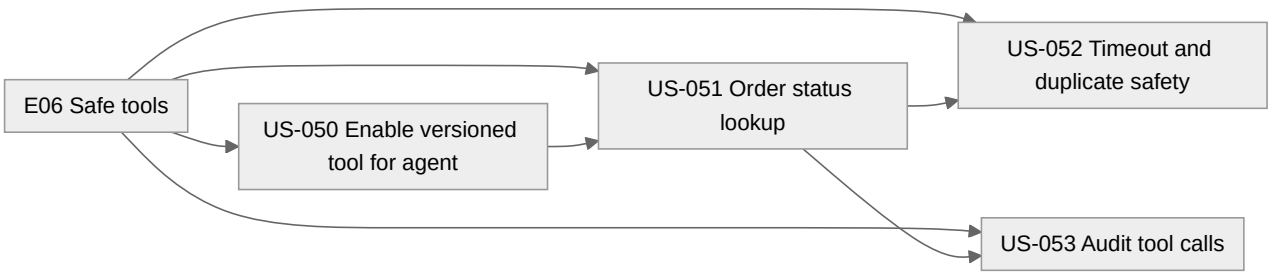


Figure 9.1: E06: Epic overview

9.2 Three-layer connector architecture

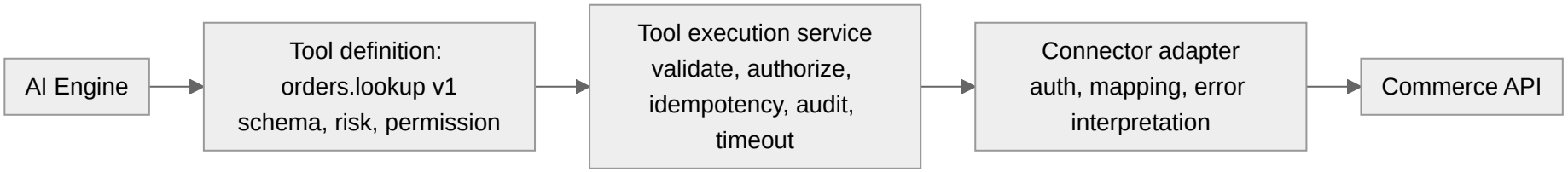


Figure 9.2: E06: Three-layer connector architecture

9.3 US-050 — Enable a versioned tool for an agent

User story US-050

Story. As an administrator, I can enable a versioned tool for an agent.
Acceptance. tool must be allowlisted and schema-valid.

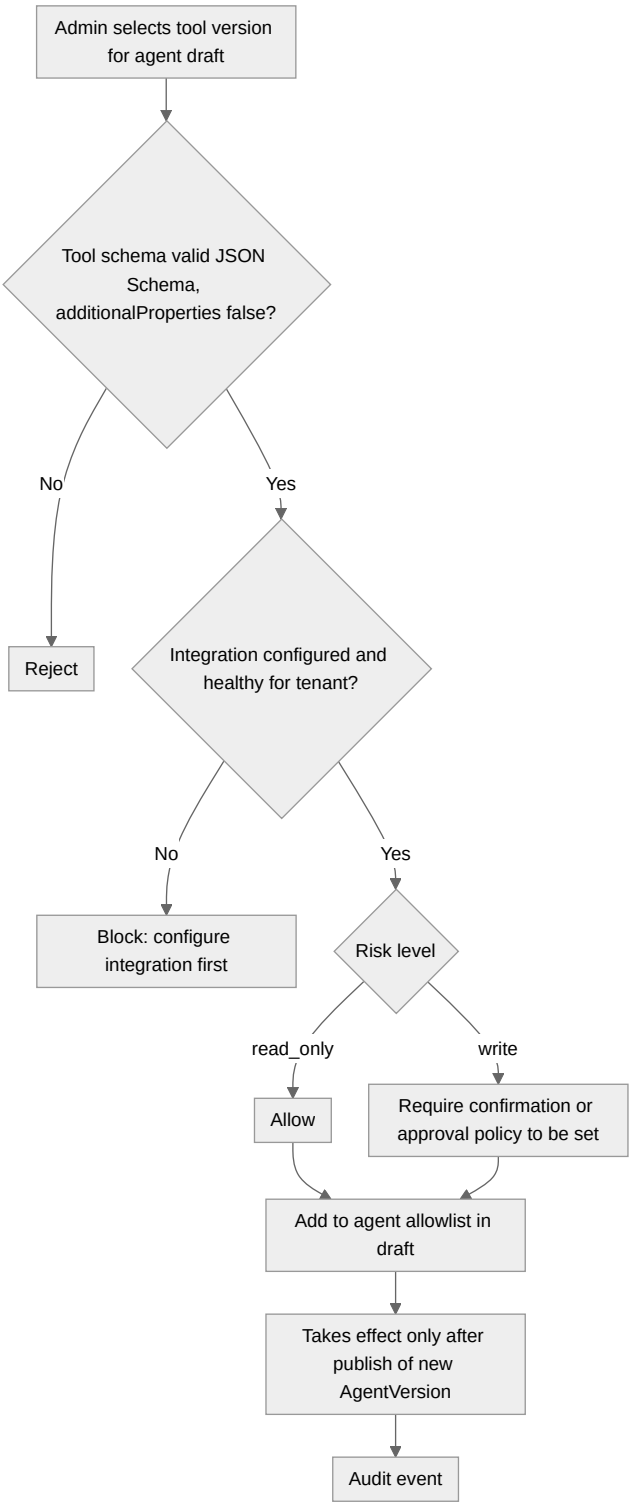


Figure 9.3: E06: US-050 — Enable a versioned tool for an agent

9.4 US-051 — Customer asks for order status

User story US-051

Story. As a customer, I can ask for an order status lookup.
Acceptance. customer verification and tenant scope are checked; result is typed and minimal.

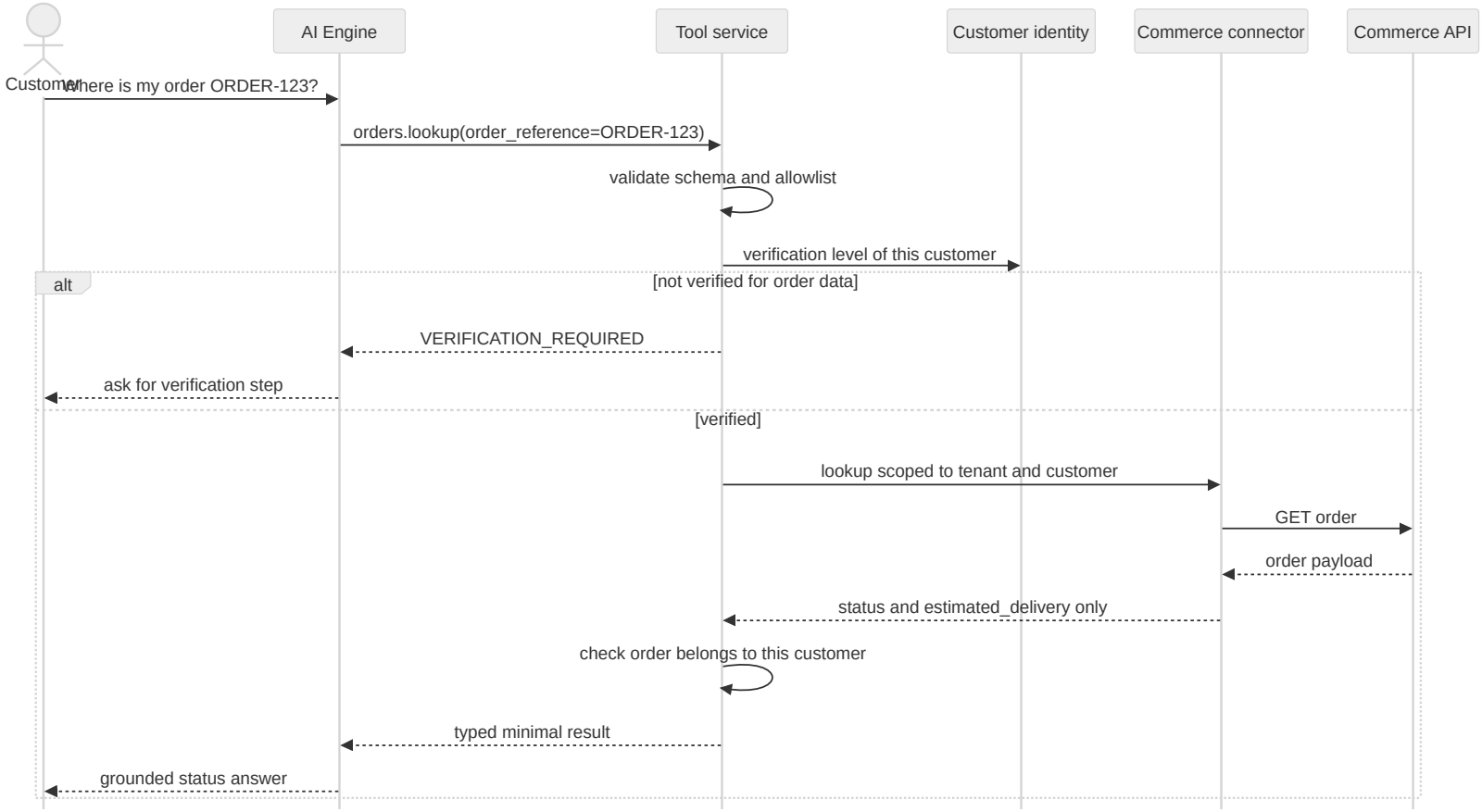


Figure 9.4: E06: US-051 — Customer asks for order status

9.5 US-052 — Handle tool timeout and duplicate requests

User story US-052

Story. As a system, I can safely handle tool timeout and duplicate requests.
Acceptance. idempotency behavior is defined; unknown write outcome is not blindly retried.

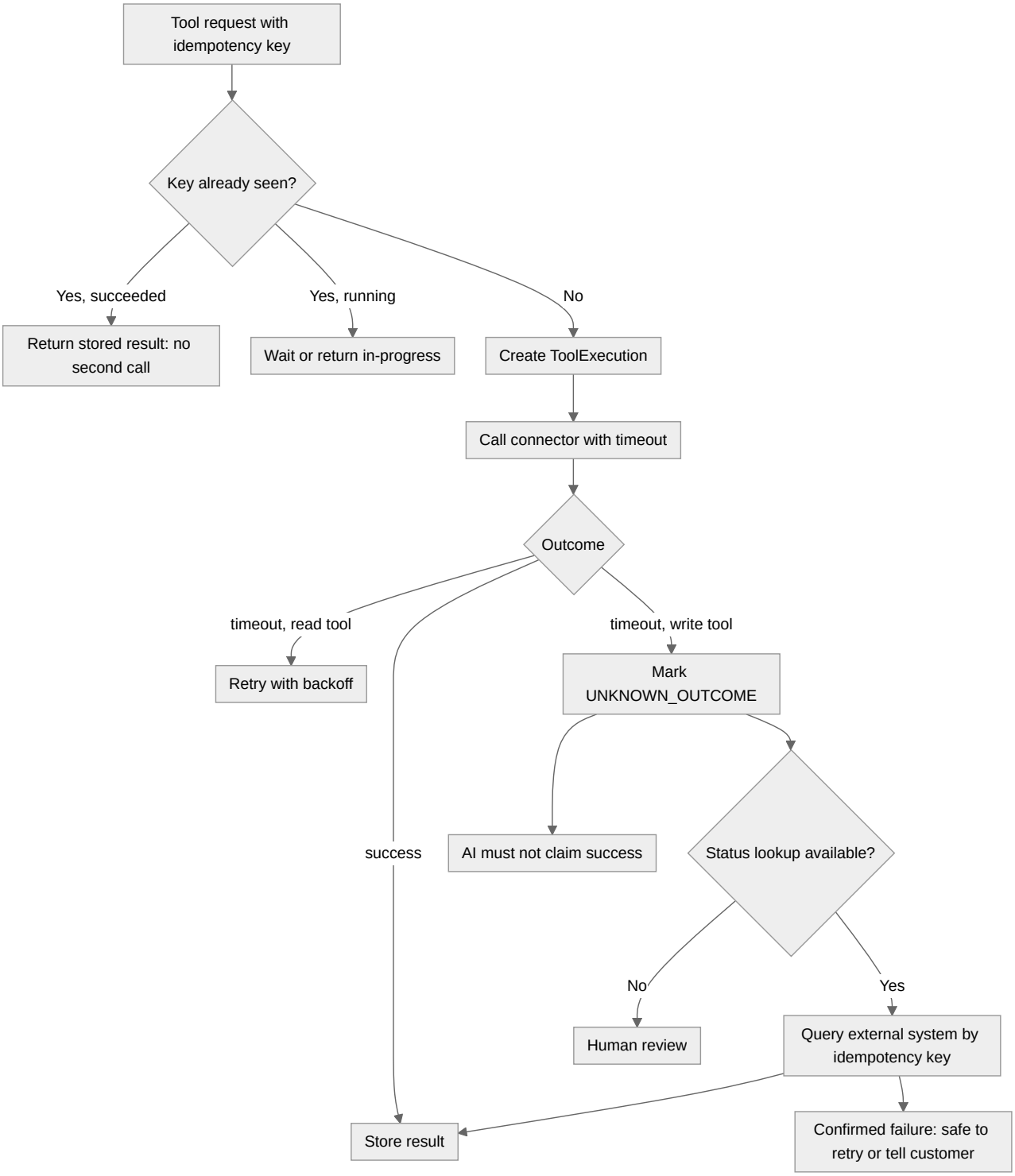


Figure 9.5: E06: US-052 — Handle tool timeout and duplicate requests

9.6 US-053 — Audit tool calls

User story US-053

Story. As a security reviewer, I can audit tool calls.
Acceptance. actor, tool/version, outcome, timestamp and correlation ID are available with sensitive values redacted.

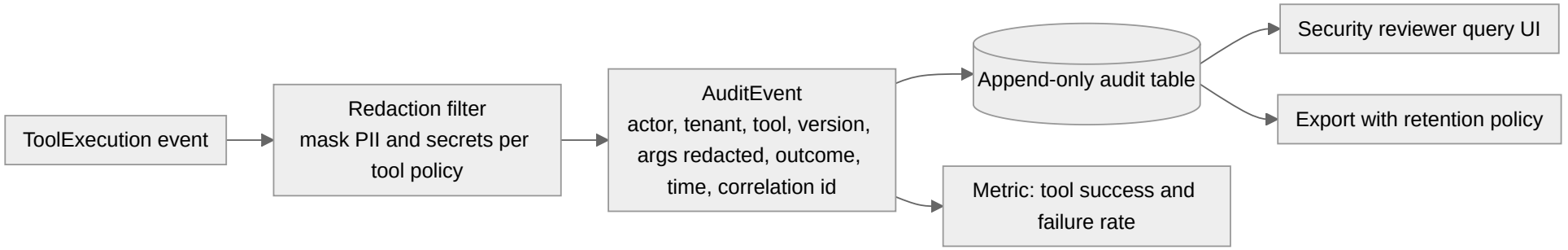


Figure 9.6: E06: US-053 — Audit tool calls

Chapter 10

E07 — Human Inbox and Handoff

Goal: a human can take over with context. Modules: M10, M13.

Stories in this epic

ID	User story	Sprint
US-060	As a support agent, I can view an unassigned queue.	S5
US-061	As a customer, I can request a human.	S5
US-062	As a support agent, I receive a handoff summary.	S5
US-063	As a support agent, I can resolve the conversation.	Backlog

10.1 Epic overview

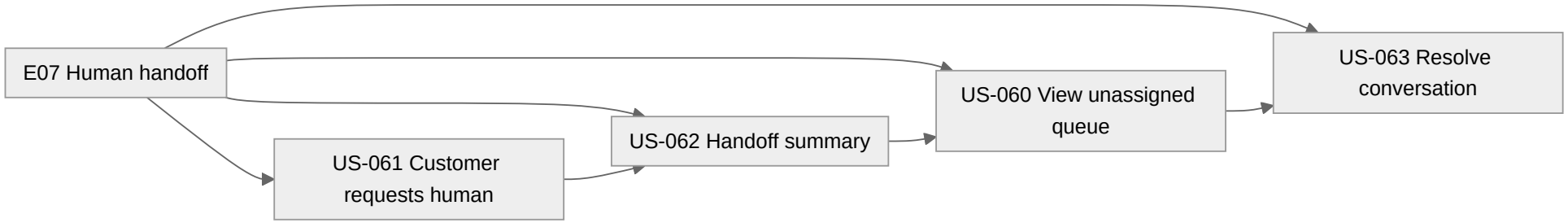


Figure 10.1: E07: Epic overview

10.2 Ownership model

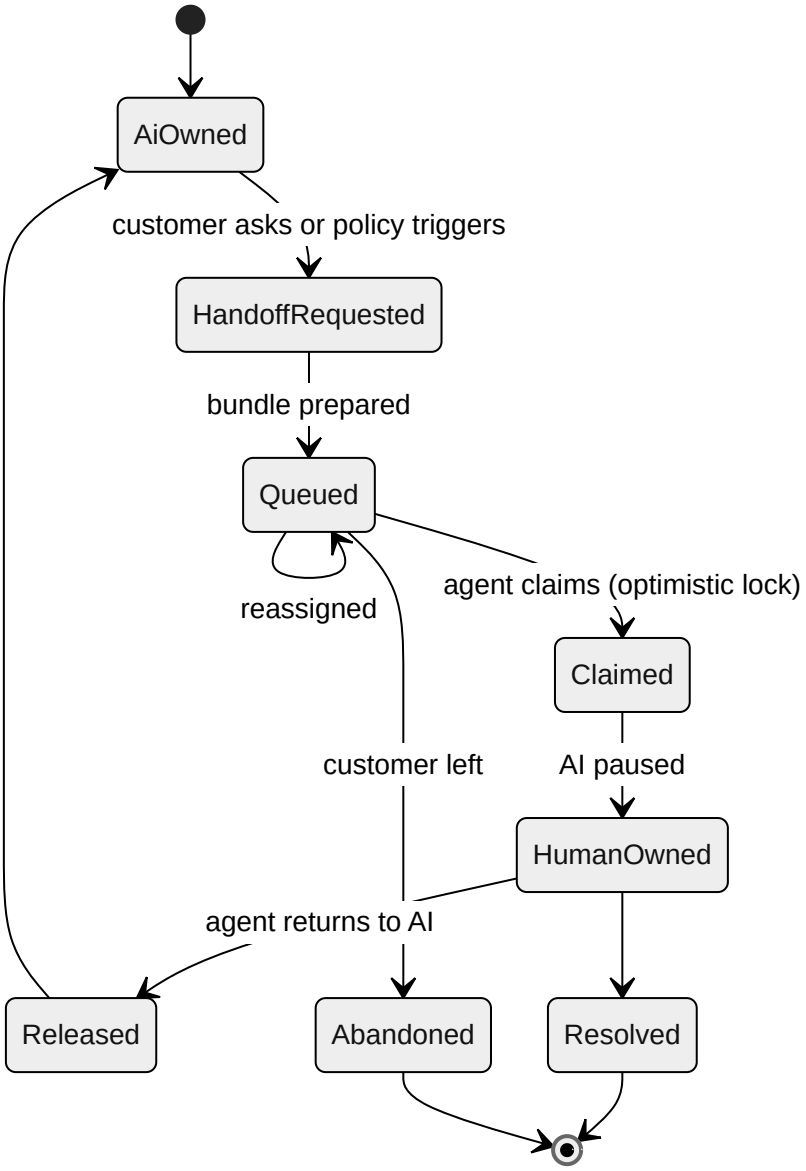


Figure 10.2: E07: Ownership model

10.3 US-060 — Support agent views the unassigned queue

User story US-060

Story. As a support agent, I can view an unassigned queue.
Acceptance. queue is tenant scoped and supports assignment.

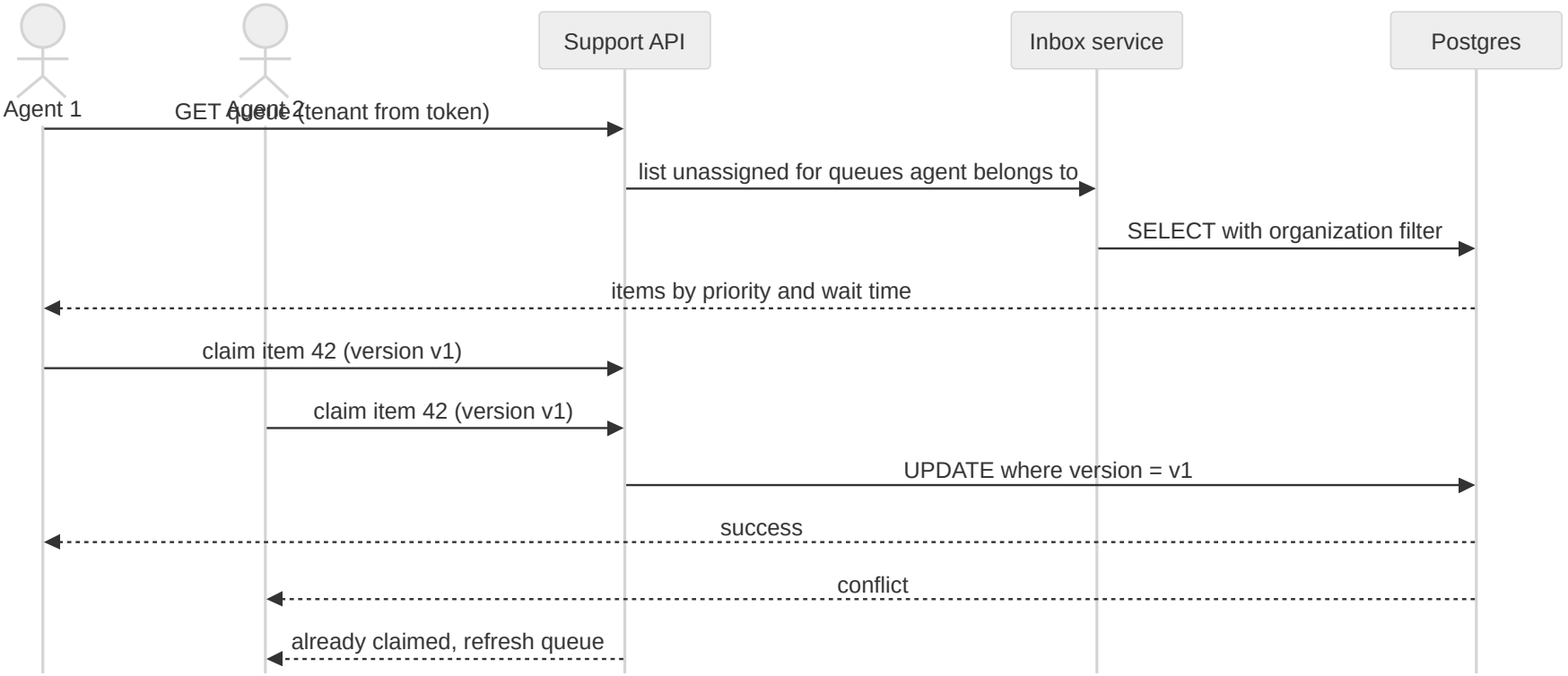


Figure 10.3: E07: US-060 — Support agent views the unassigned queue

10.4 US-061 — Customer requests a human

User story US-061

Story. As a customer, I can request a human.
Acceptance. AI stops responding when handoff policy requires; the customer receives a clear status.

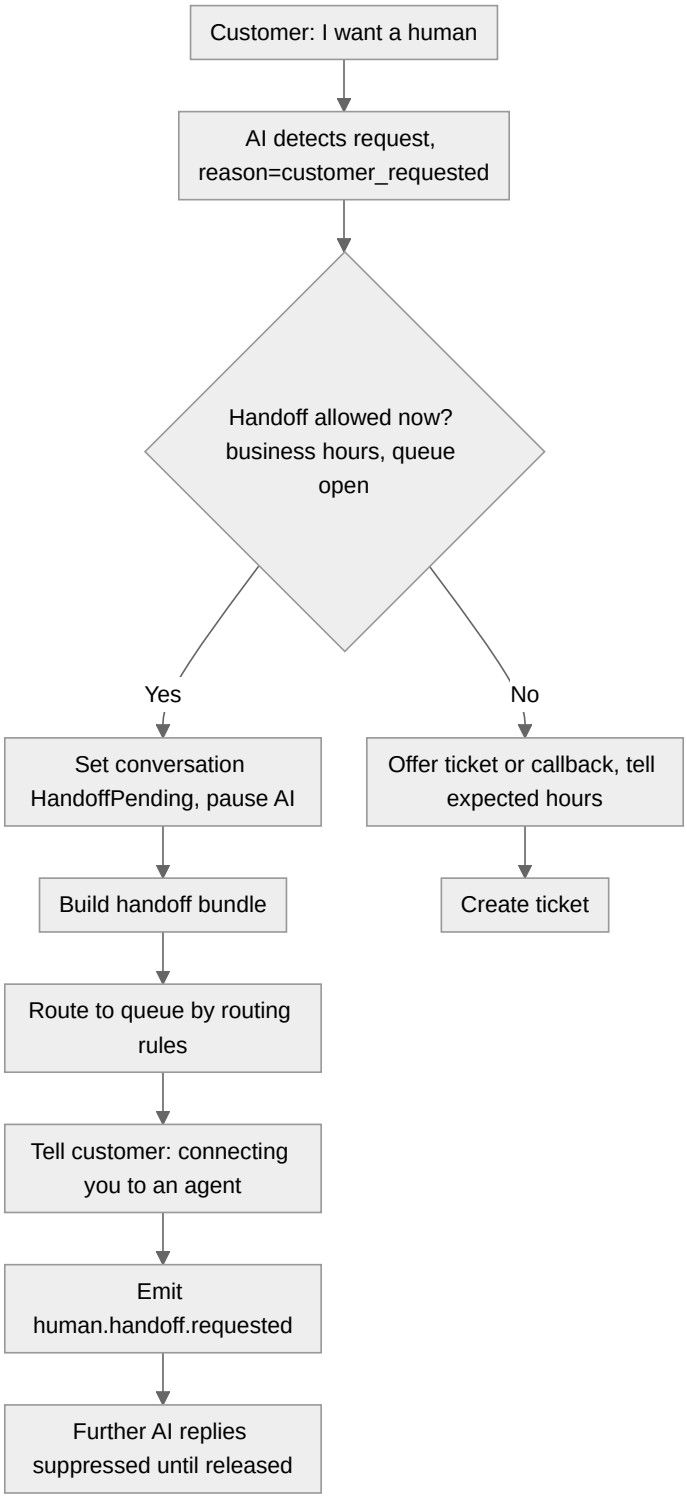


Figure 10.4: E07: US-061 — Customer requests a human

10.5 US-062 — Handoff summary for the agent

User story US-062

Story. As a support agent, I receive a handoff summary.
Acceptance. summary includes transcript, unresolved request and prior tool outcomes permitted by policy.

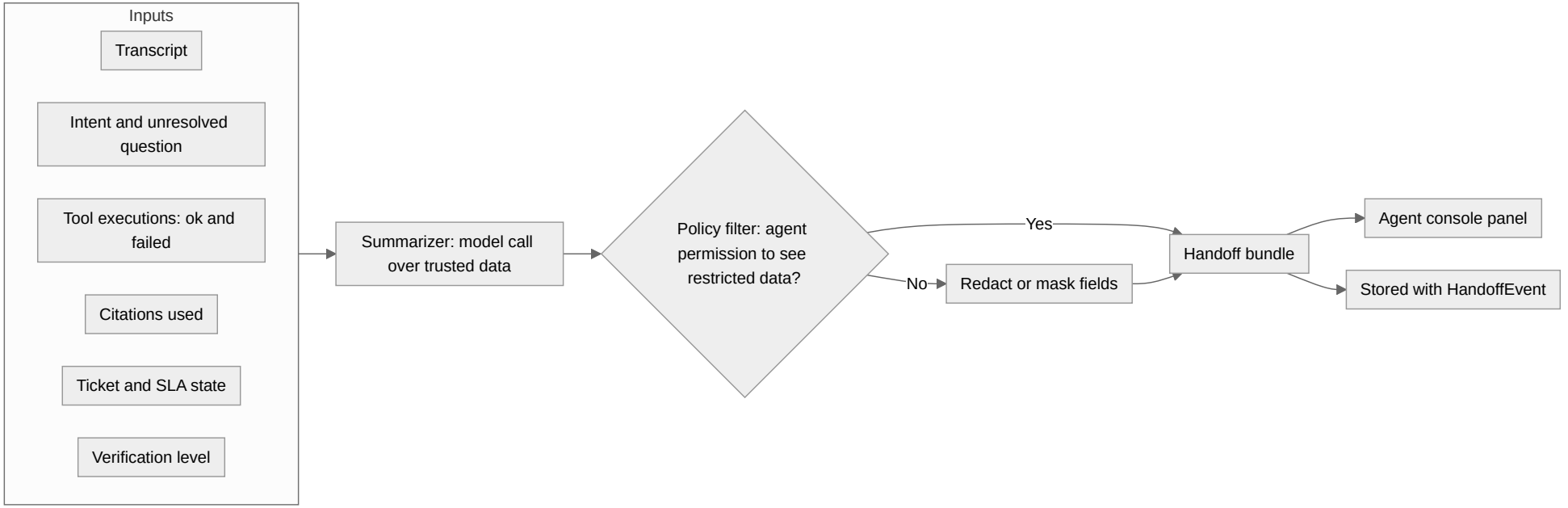


Figure 10.5: E07: US-062 — Handoff summary for the agent

10.6 US-063 — Support agent resolves the conversation

User story US-063

Story. As a support agent, I can resolve the conversation.
Acceptance. resolution and ownership changes are recorded.

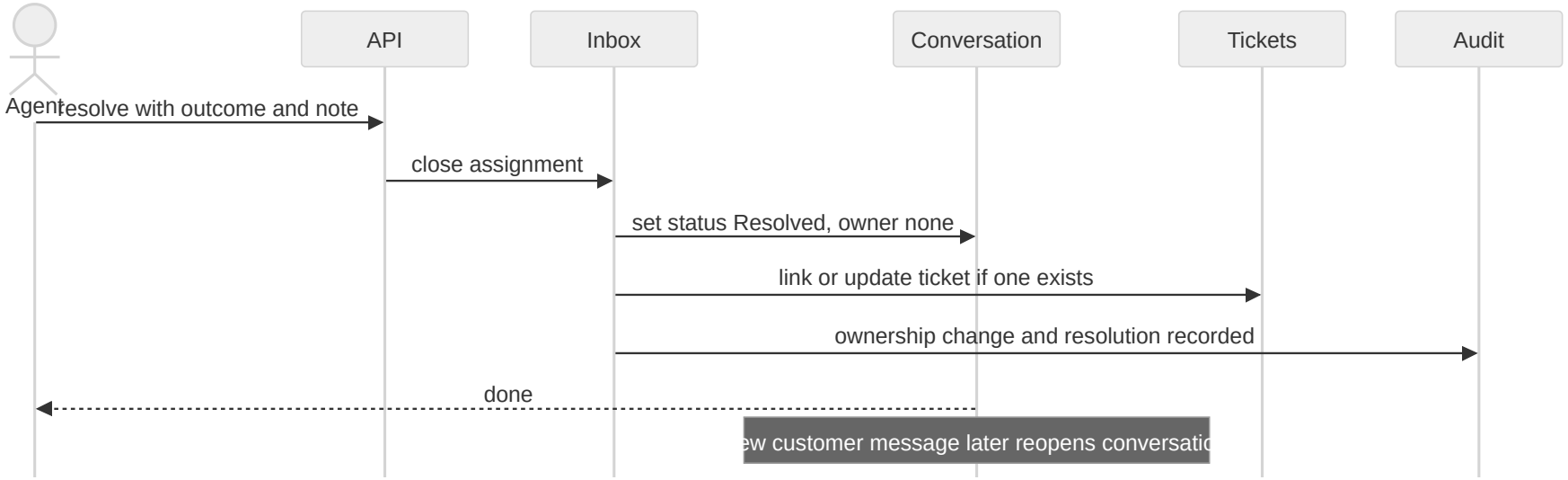


Figure 10.6: E07: US-063 — Support agent resolves the conversation

Chapter 11

E08 — Tickets and Case Tracking

Goal: track support work beyond one interaction. Module: M11.

Stories in this epic

ID	User story	Sprint
US-070	As a support agent, I can create a ticket from a conversation.	S6
US-071	As a support lead, I can filter tickets by status, priority and assignment.	S6

11.1 Epic overview

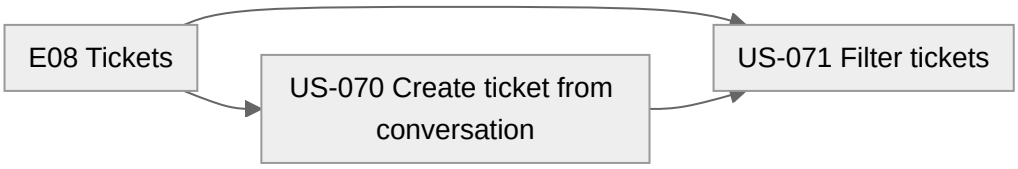


Figure 11.1: E08: Epic overview

11.2 Conversation vs ticket

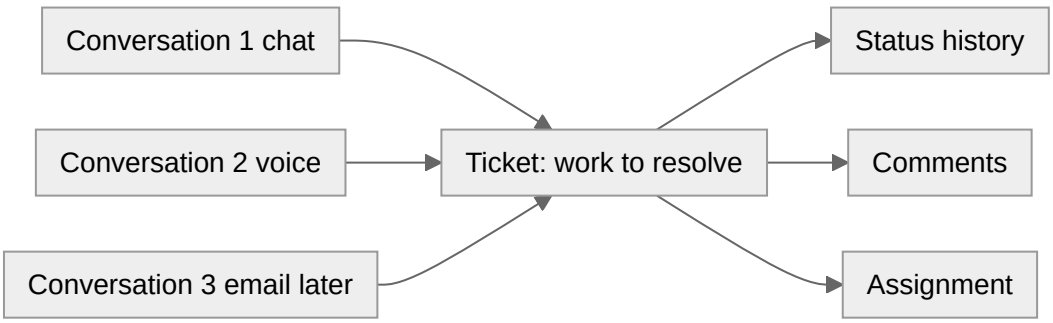


Figure 11.2: E08: Conversation vs ticket

11.3 Ticket status lifecycle

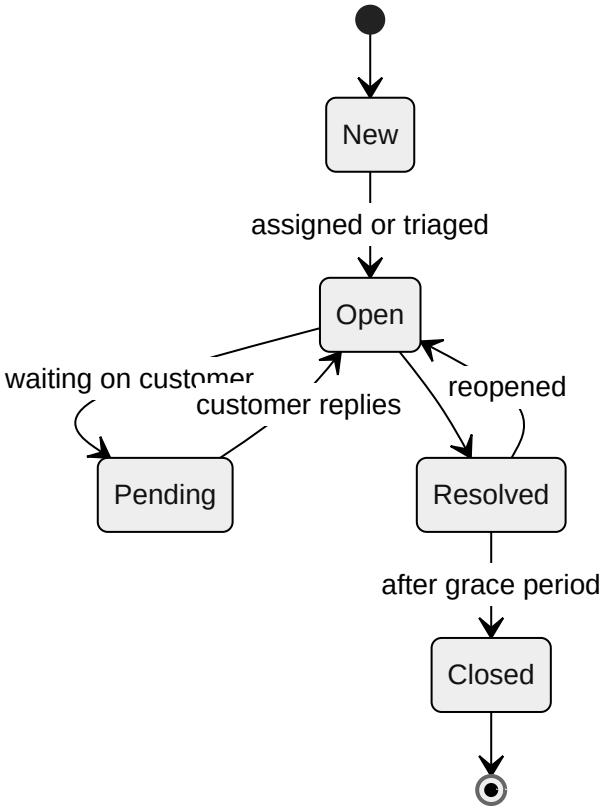


Figure 11.3: E08: Ticket status lifecycle

11.4 US-070 — Create a ticket from a conversation

User story US-070

Story. As a support agent, I can create a ticket from a conversation.
Acceptance. ticket links to source conversation and has a status history.

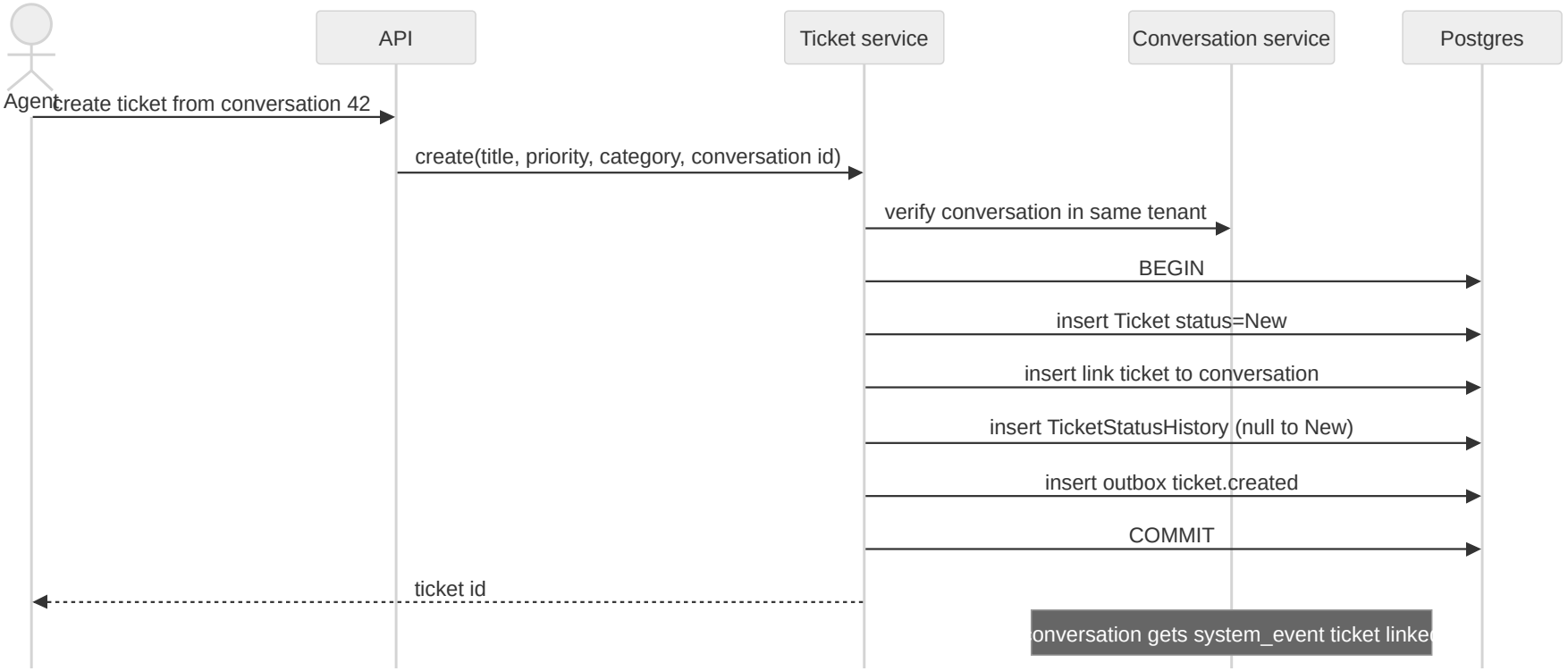


Figure 11.4: E08: US-070 — Create a ticket from a conversation

AI-initiated variant:

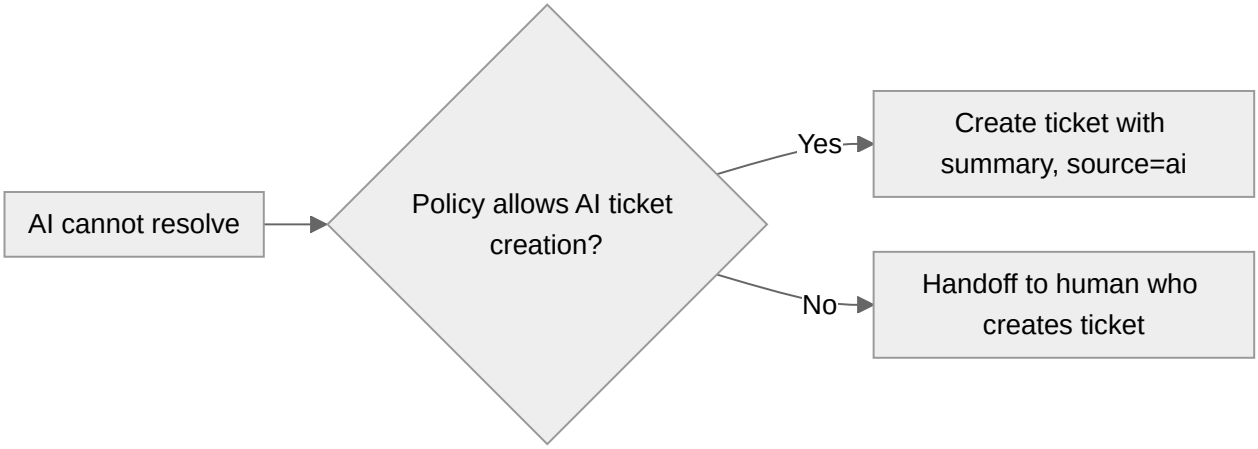


Figure 11.5: E08: US-070 — Create a ticket from a conversation

11.5 US-071 — Support lead filters tickets

User story US-071

Story. As a support lead, I can filter tickets by status, priority and assignment.
Acceptance. filters respect permissions and tenant boundaries.

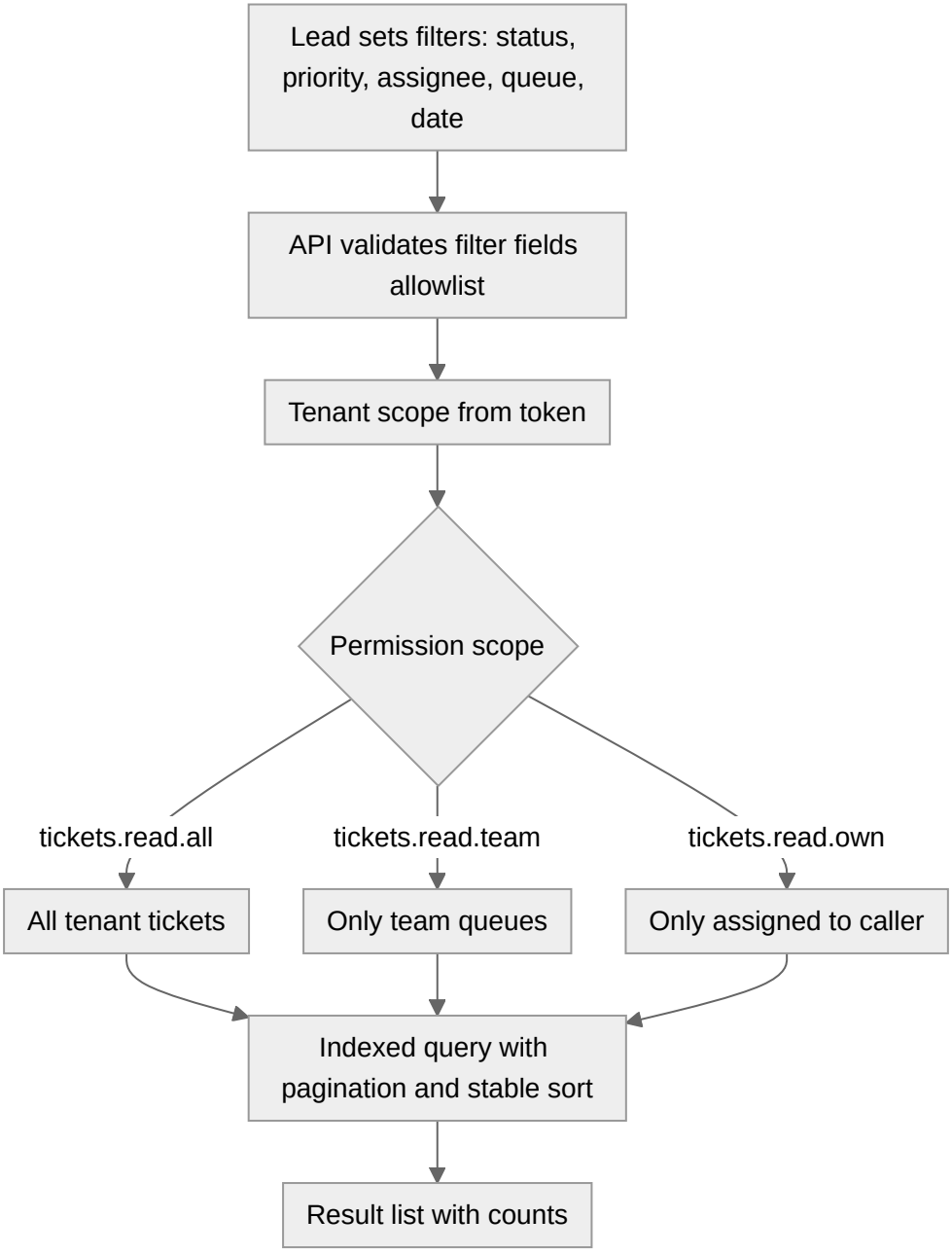


Figure 11.6: E08: US-071 — Support lead filters tickets

Chapter 12

E09 — Voice MVP

Goal: real-time browser voice using the same conversation engine. Modules: M07, M08, M09. Deeper voice diagrams: see the voice diagrams.

Stories in this epic

ID	User story	Sprint
US-080	As a customer, I can start and end a voice session.	S7
US-081	As a customer, I can speak and receive a spoken response.	S7
US-082	As a customer, I can interrupt the AI while it speaks.	S8
US-083	As a customer, I can switch between text and voice in one conversation.	S8
US-084	As a support agent, I can take over a voice conversation or receive its context.	S8

12.1 Epic overview

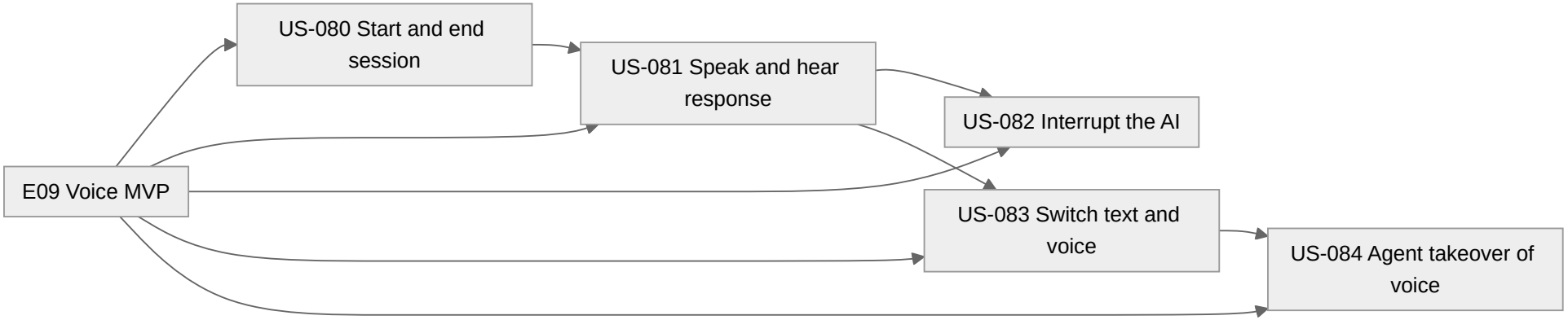


Figure 12.1: E09: Epic overview

12.2 Reuse of the text engine

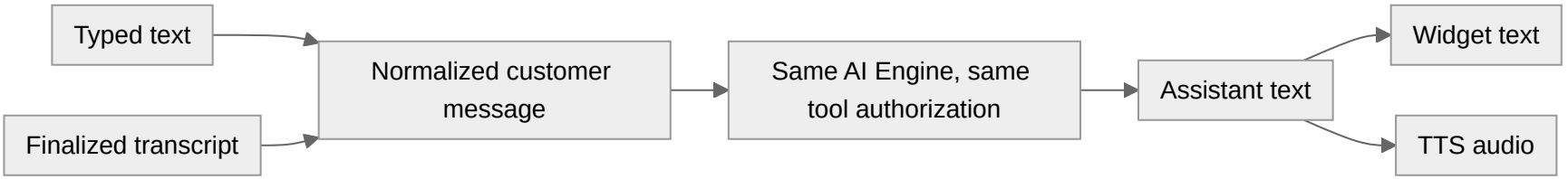


Figure 12.2: E09: Reuse of the text engine

12.3 US-080 — Start and end a voice session

User story US-080

Story. As a customer, I can start and end a voice session.
Acceptance. microphone permission is explicit; session state and failures are visible.

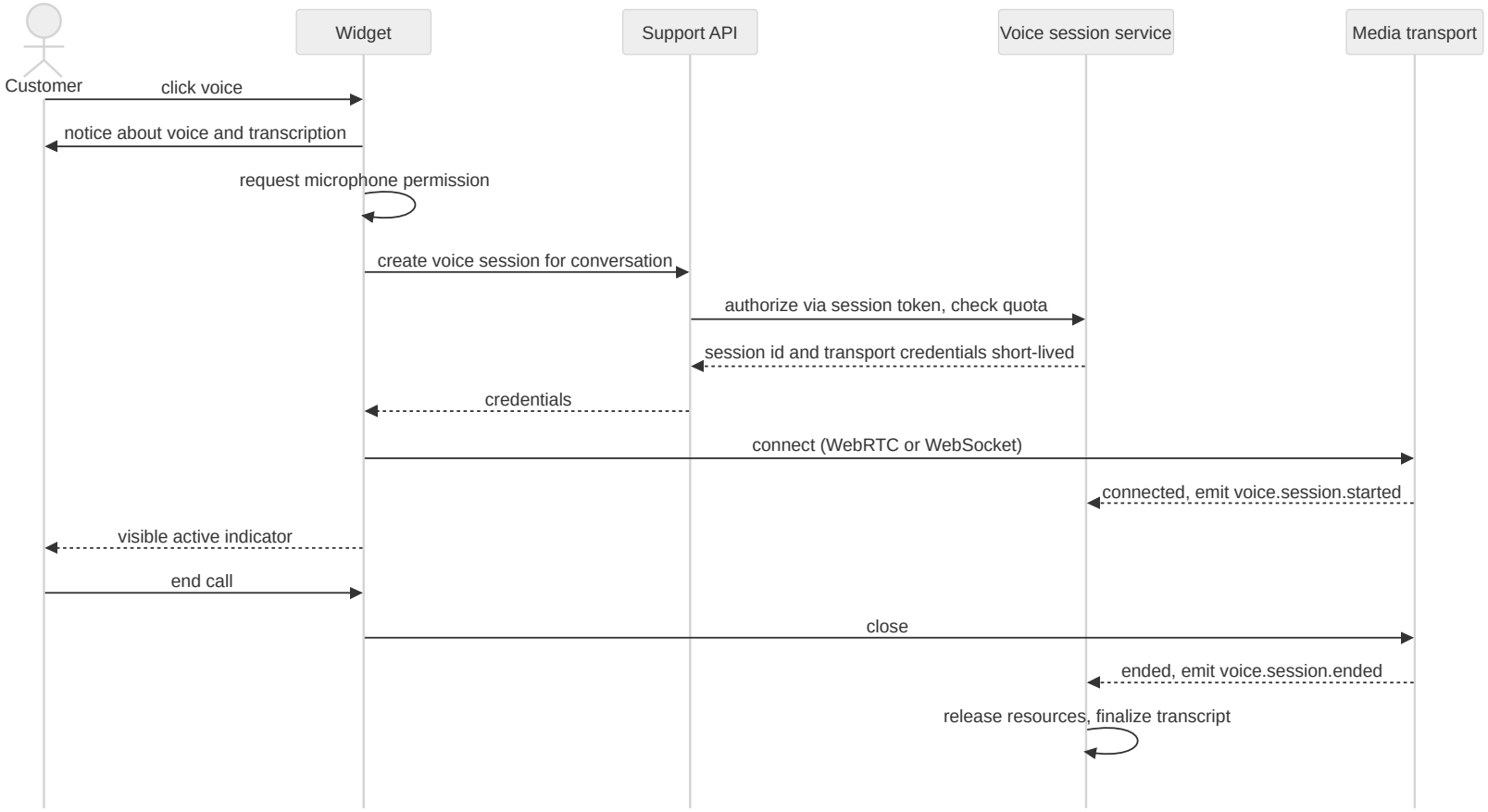


Figure 12.3: E09: US-080 — Start and end a voice session

12.4 US-081 — Speak and receive a spoken response

User story US-081

Story. As a customer, I can speak and receive a spoken response.
Acceptance. speech provider is configurable; failures have a fallback path.

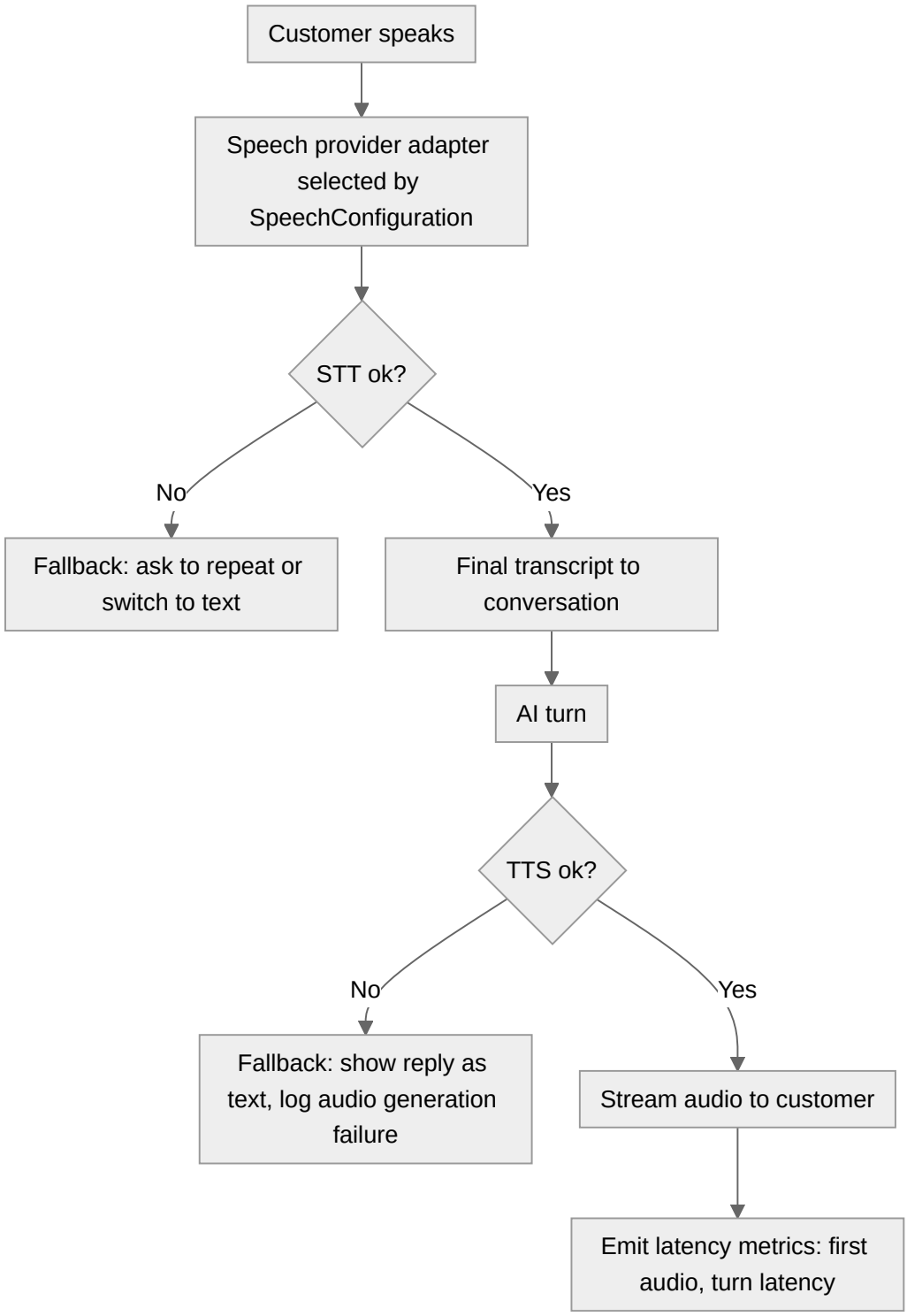


Figure 12.4: E09: US-081 — Speak and receive a spoken response

12.5 US-082 — Interrupt the AI while it speaks

User story US-082

Story. As a customer, I can interrupt the AI while it speaks.
Acceptance. obsolete audio is canceled/truncated where supported and not recorded as fully delivered.

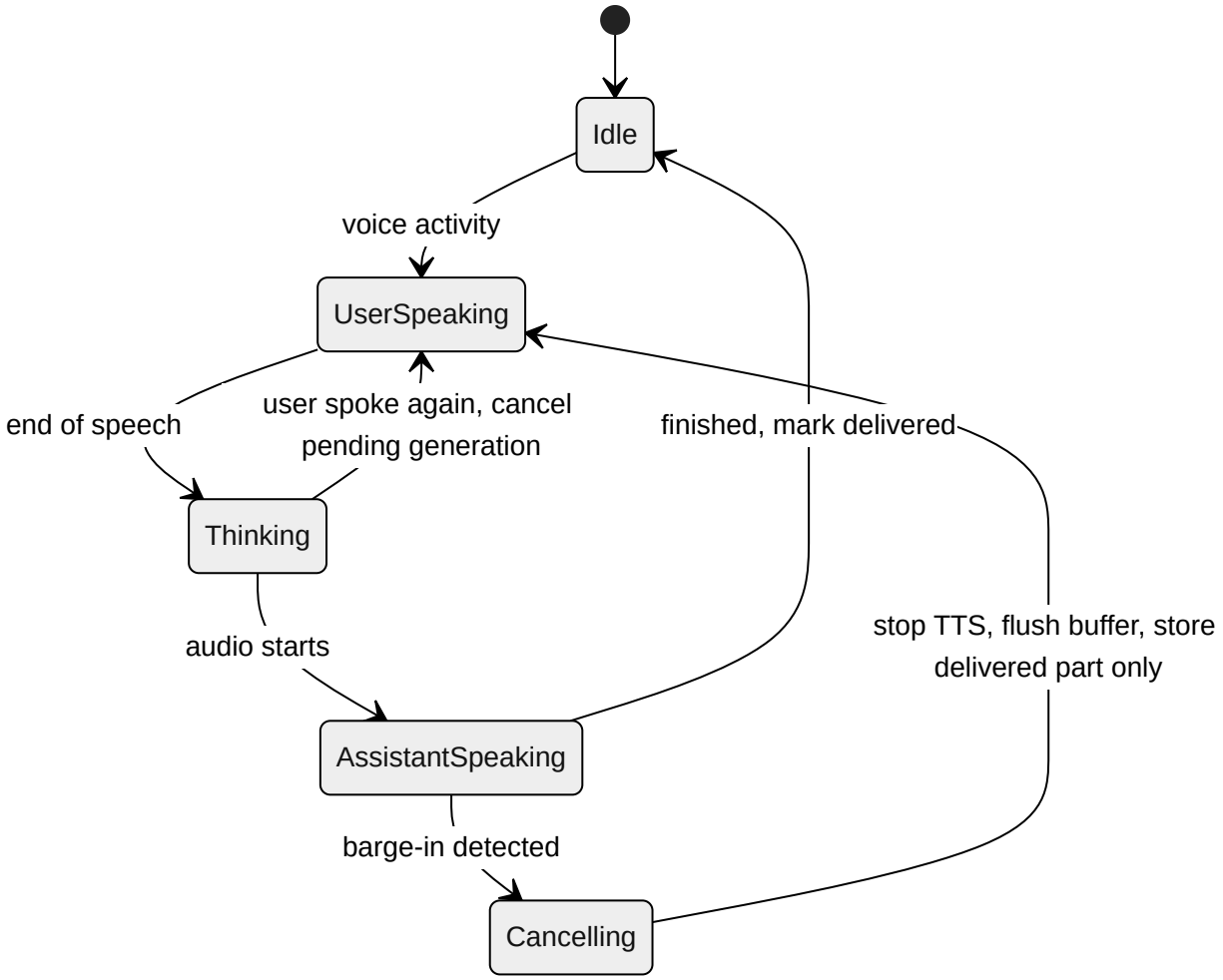


Figure 12.5: E09: US-082 — Interrupt the AI while it speaks

12.6 US-083 — Switch between text and voice

User story US-083

Story. As a customer, I can switch between text and voice in one conversation.
Acceptance. finalized transcript and message order remain coherent.

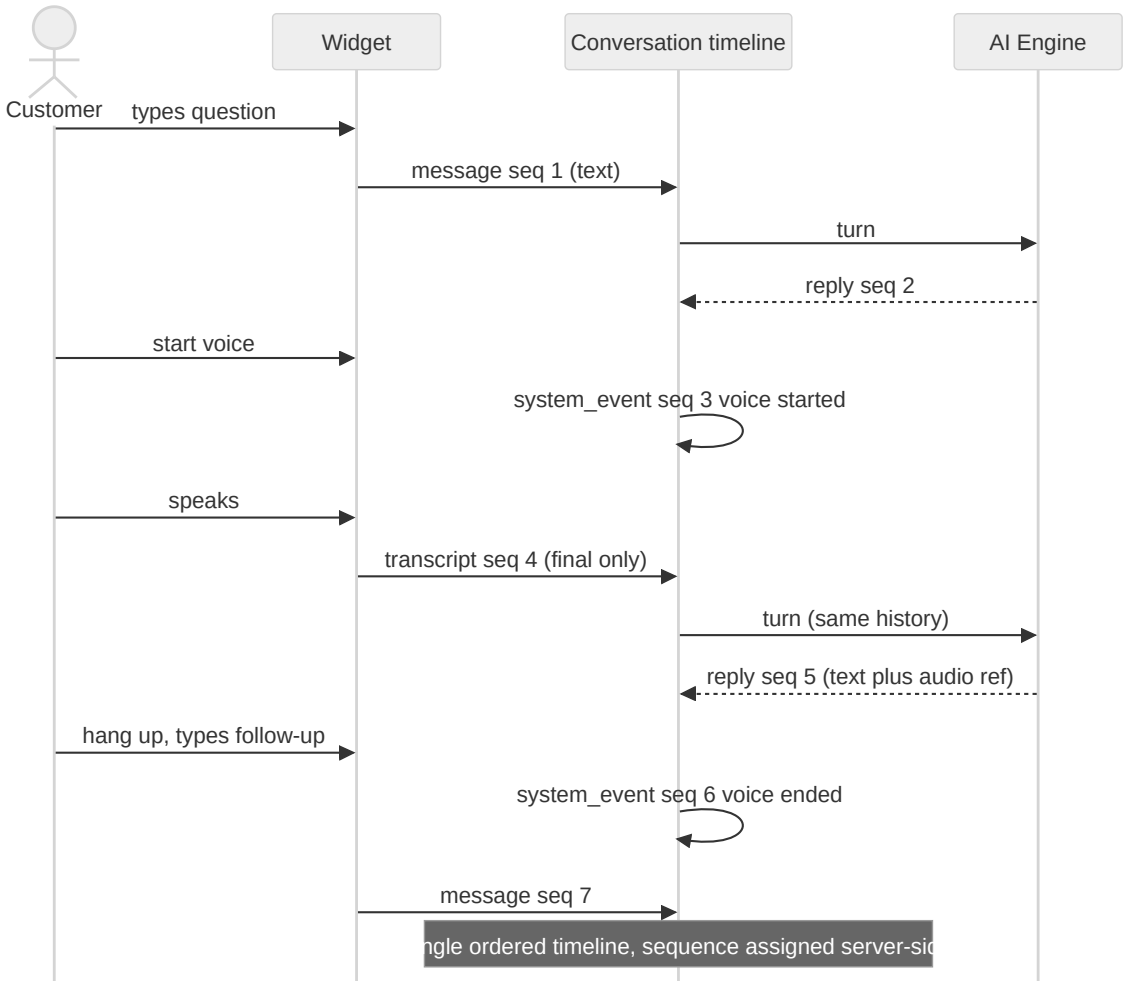


Figure 12.6: E09: US-083 — Switch between text and voice

12.7 US-084 — Agent takes over a voice conversation or receives its context

User story US-084

Story. As a support agent, I can take over a voice conversation or receive its context.
Acceptance. handoff behavior is explicit for the chosen telephony/media provider.

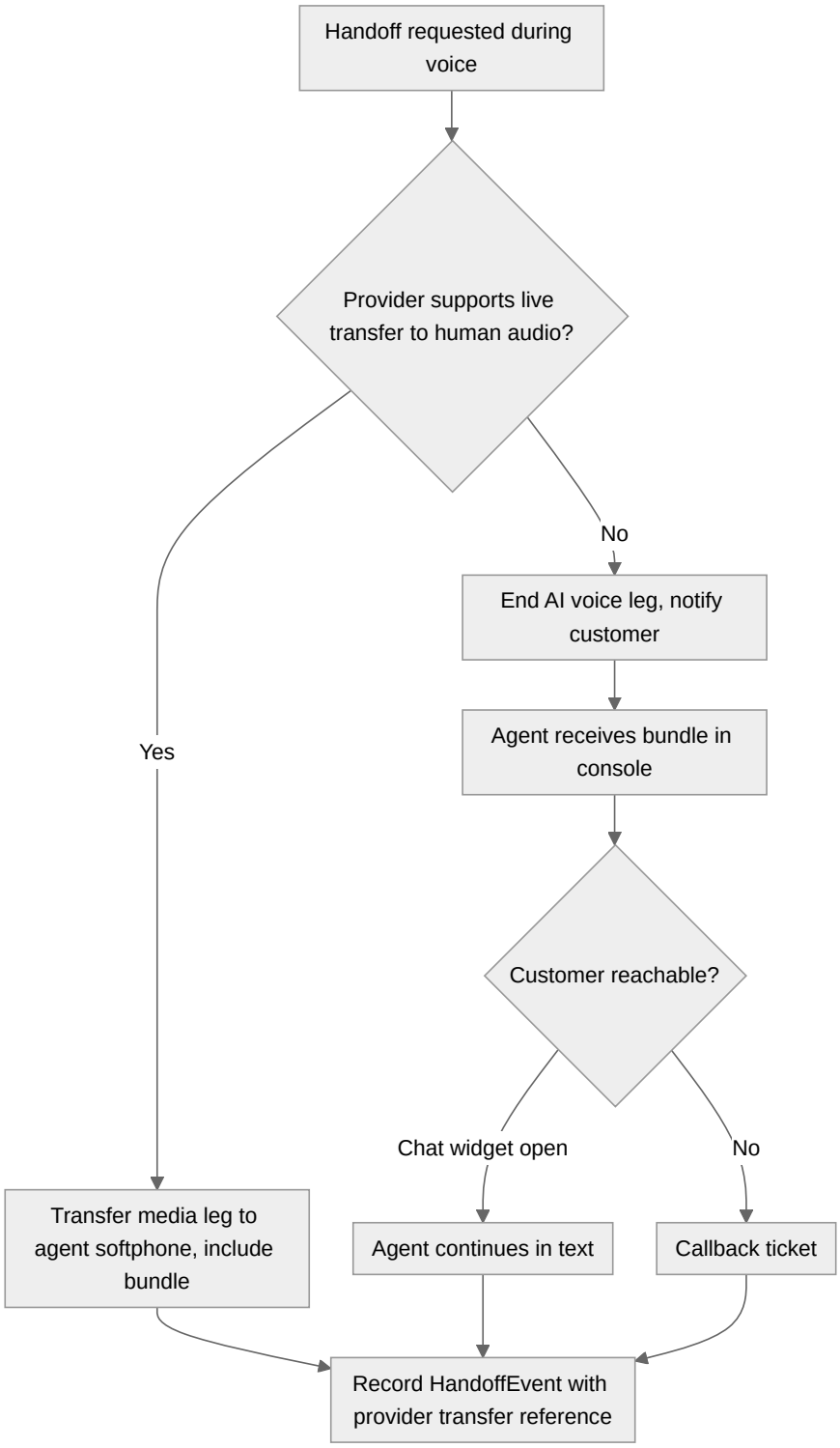


Figure 12.7: E09: US-084 — Agent takes over a voice conversation or receives its context

Chapter 13

E10 — Workflow, Routing and SLA

Goal: automate repeatable support operations. Modules: M12, M13, M14.

Stories in this epic

ID	User story	Sprint
US-090	As an administrator, I can define a workflow trigger and steps.	Backlog
US-091	As a support lead, I can define routing rules and SLA targets.	Backlog
US-092	As a system, I can retry a failed workflow step safely.	Backlog

13.1 Epic overview

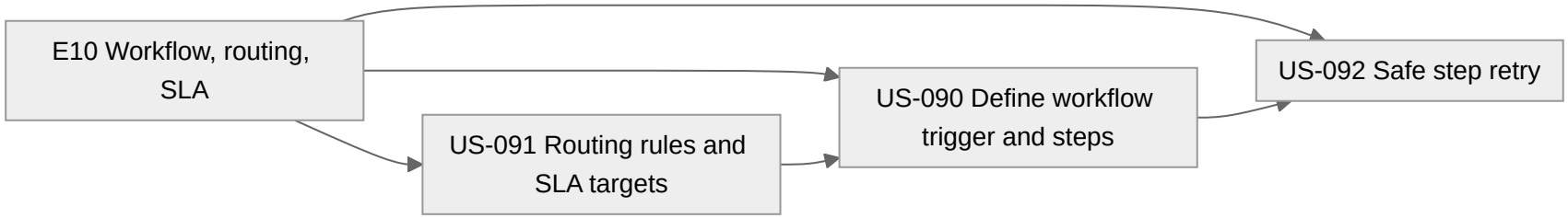


Figure 13.1: E10: Epic overview

13.2 Principle: deterministic procedure, AI proposes

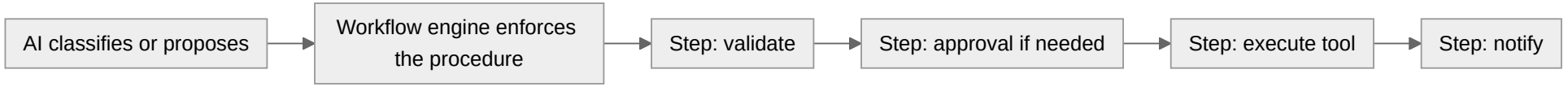


Figure 13.2: E10: Principle: deterministic procedure, AI proposes

13.3 US-090 — Define workflow trigger and steps

User story US-090

Story. As an administrator, I can define a workflow trigger and steps.
Acceptance. workflow version, run state, and failures are visible.

13.3.1 Authoring and versioning

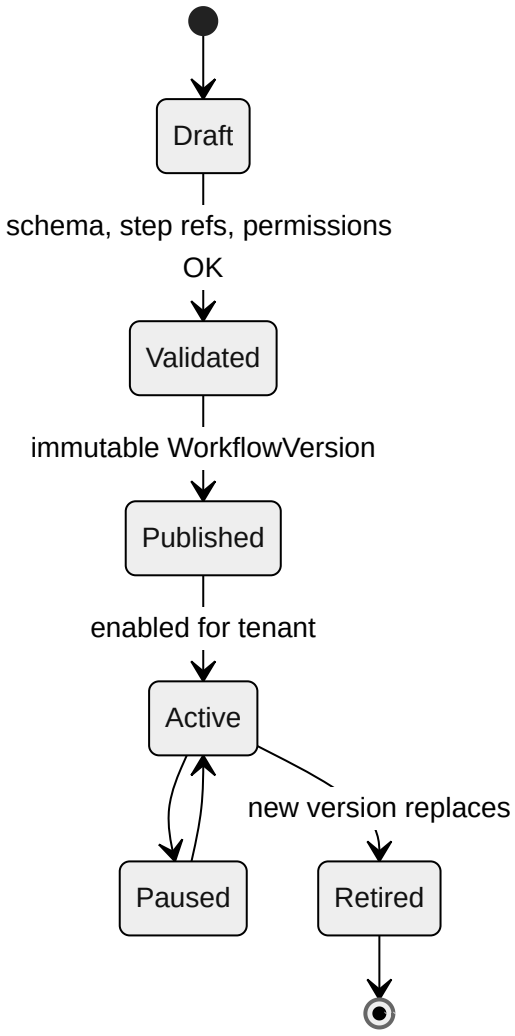


Figure 13.3: E10: US-090 — Define workflow trigger and steps: Authoring and versioning

13.3.2 Run execution

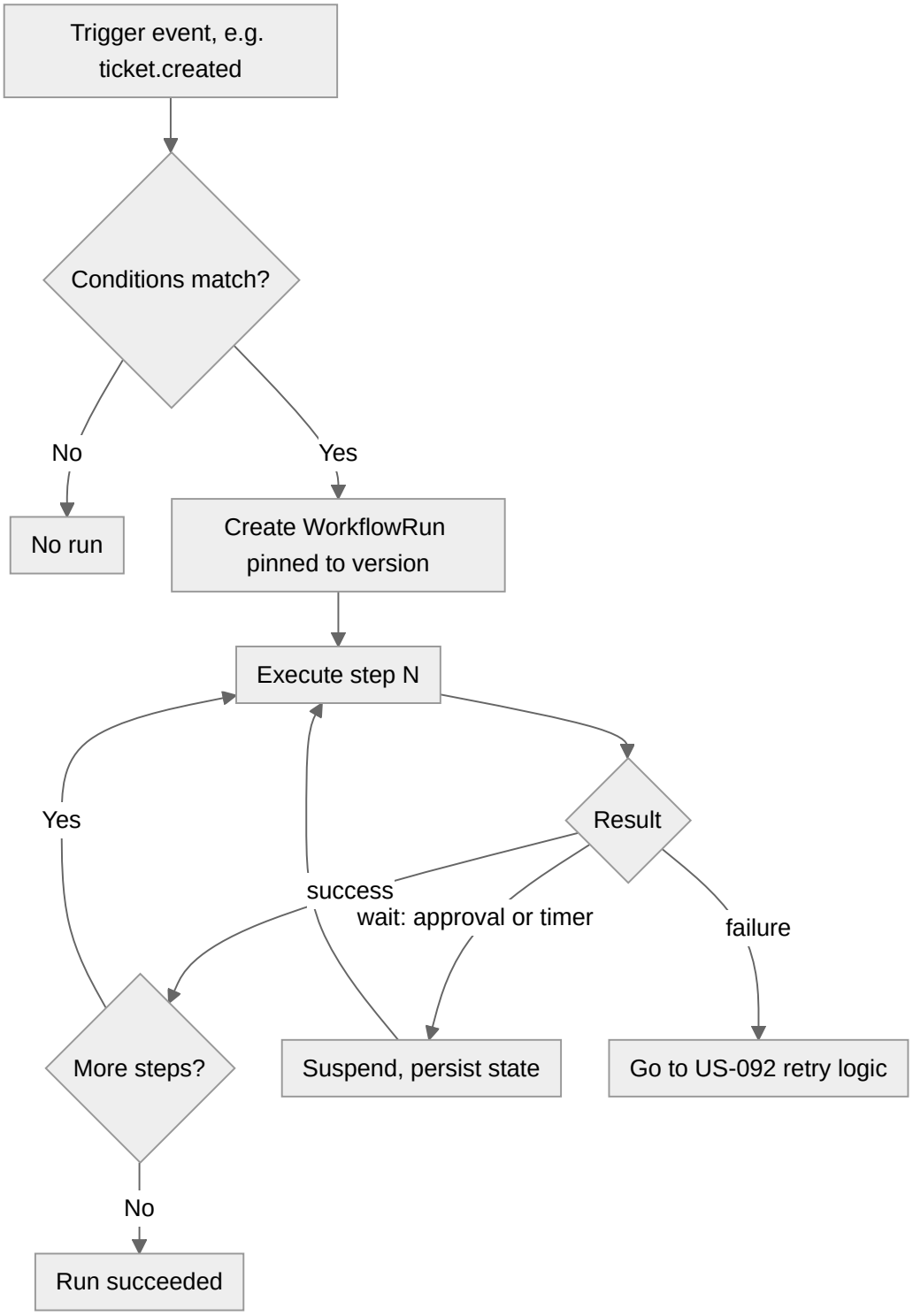


Figure 13.4: E10: US-090 — Define workflow trigger and steps: Run execution

13.4 US-091 — Routing rules and SLA targets

User story US-091

Story. As a support lead, I can define routing rules and SLA targets.
Acceptance. rule priority and fallback behavior are deterministic.

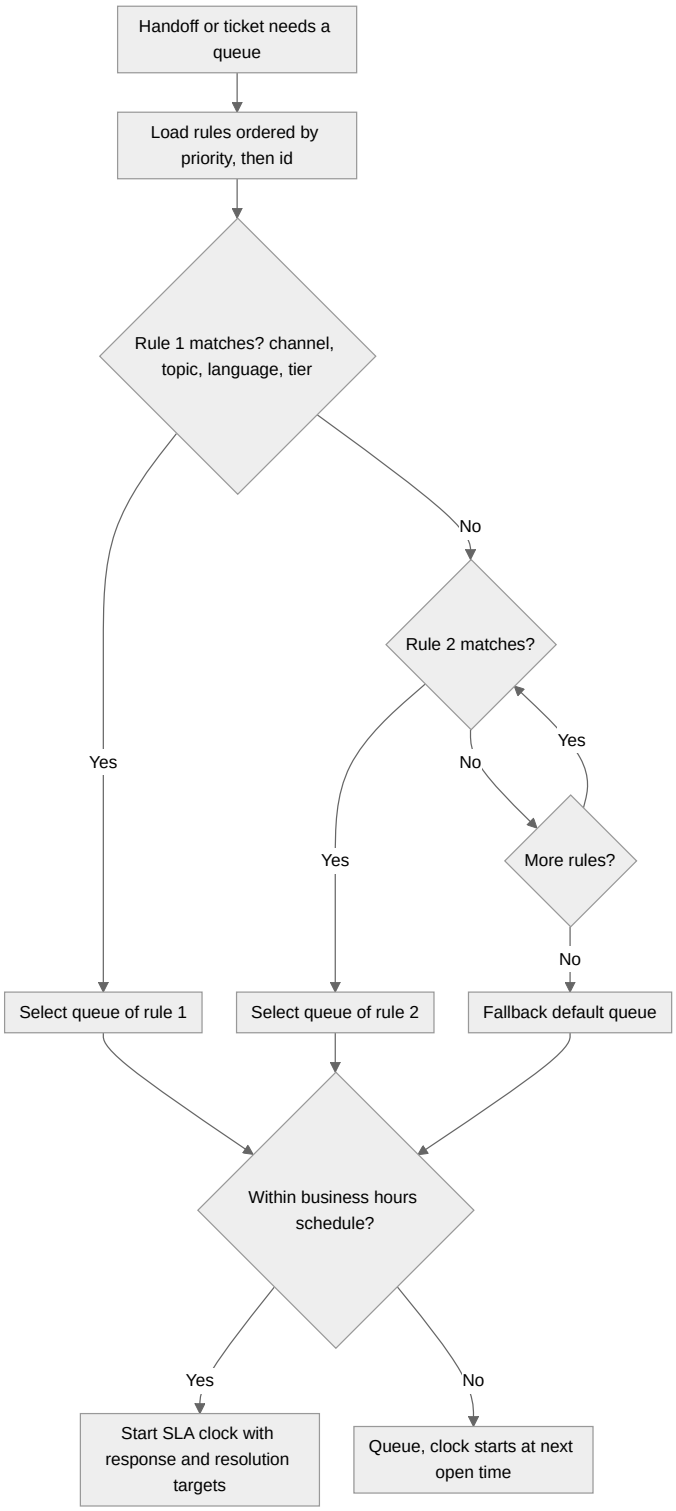


Figure 13.5: E10: US-091 — Routing rules and SLA targets

13.4.1 SLA timers

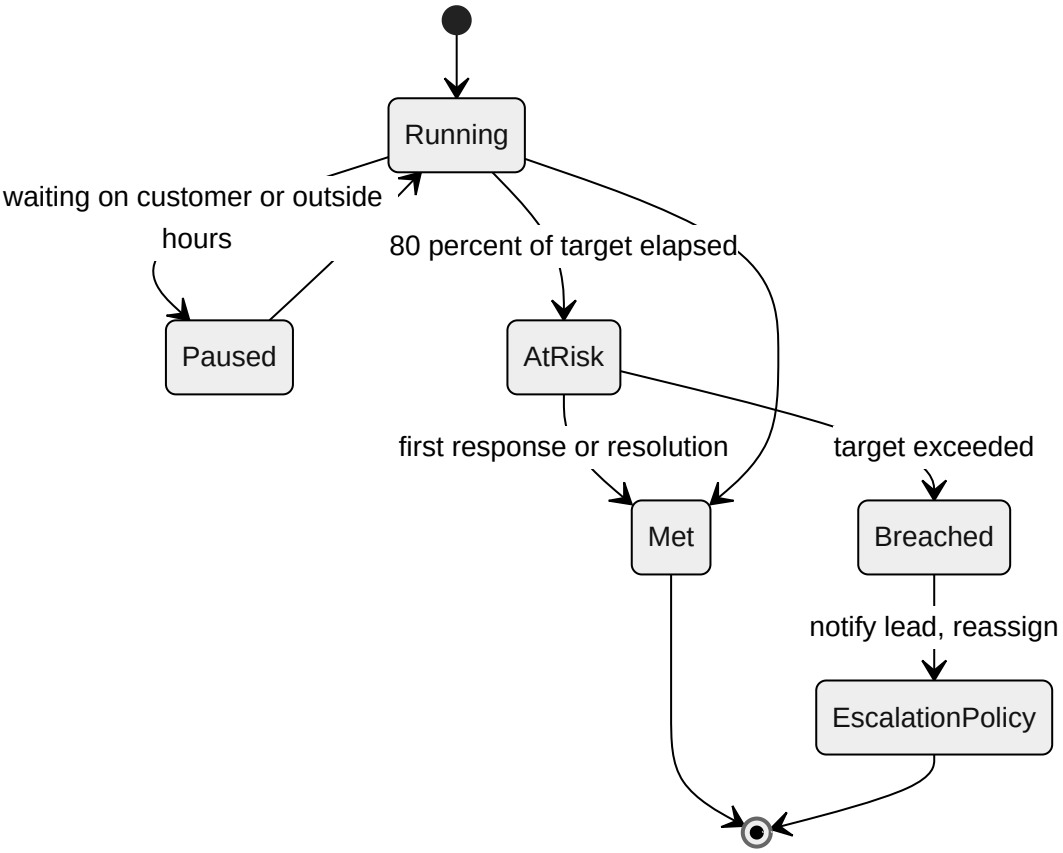


Figure 13.6: E10: US-091 — Routing rules and SLA targets: SLA timers

13.5 US-092 — Retry a failed workflow step safely

User story US-092

Story. As a system, I can retry a failed workflow step safely.
Acceptance. retry policy is bounded; idempotency is defined per step.

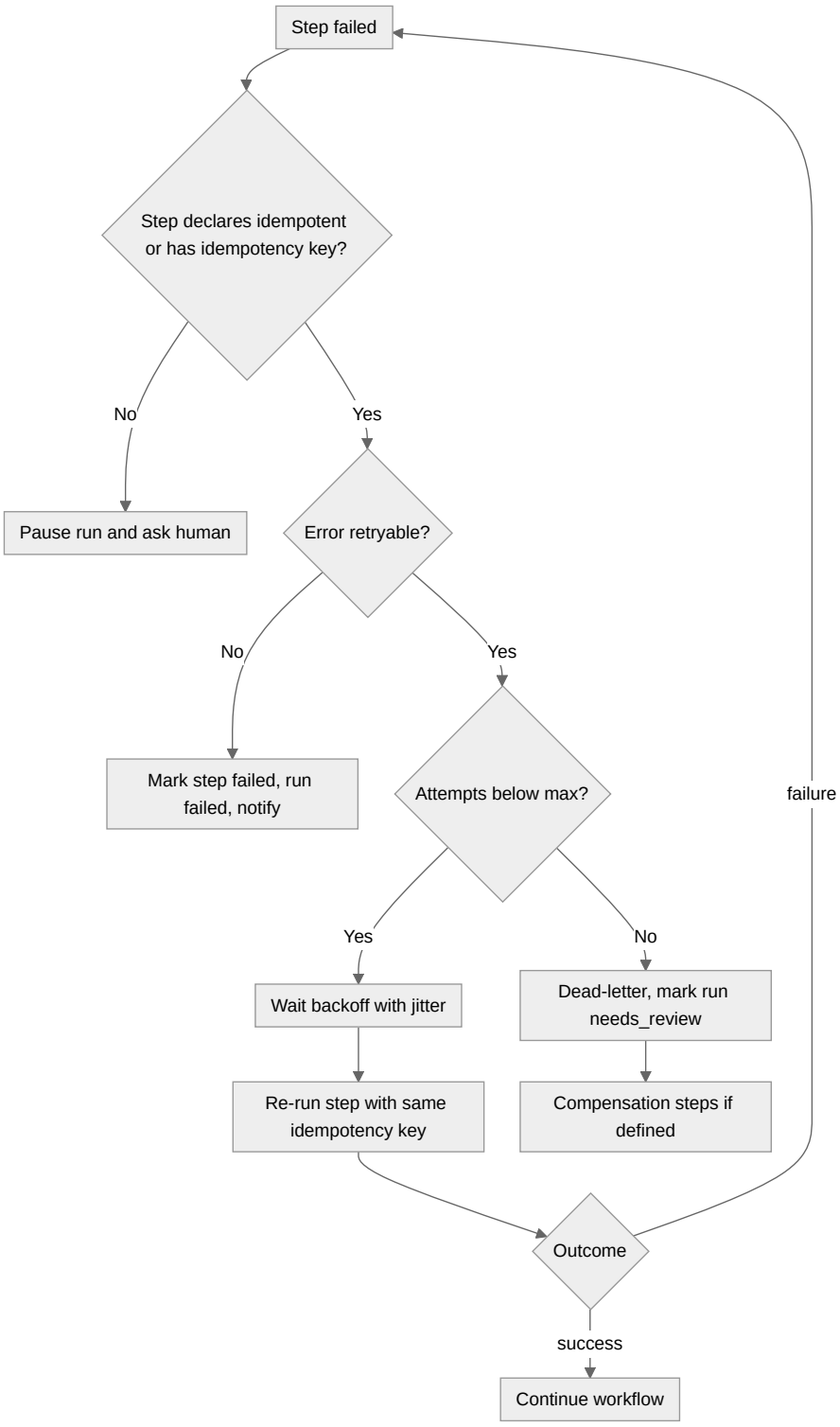


Figure 13.7: E10: US-092 — Retry a failed workflow step safely

Chapter 14

E11 — Integrations and Additional Channels

Goal: add provider connectors without contaminating core domain logic. Modules: M02, M15.

Stories in this epic

ID	User story	Sprint
US-100	As an administrator, I can configure an integration securely.	Backlog
US-101	As a system, I can normalize a provider webhook into a canonical event.	Backlog
US-102	As a customer, I can continue support through a second channel.	Backlog

14.1 Epic overview

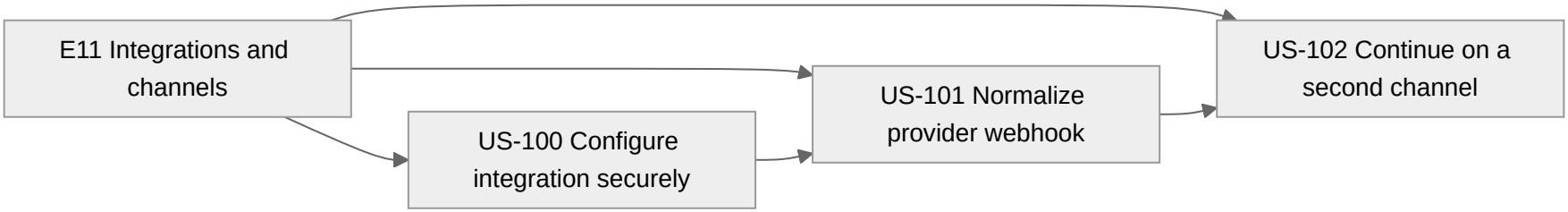


Figure 14.1: E11: Epic overview

14.2 Adapter boundary

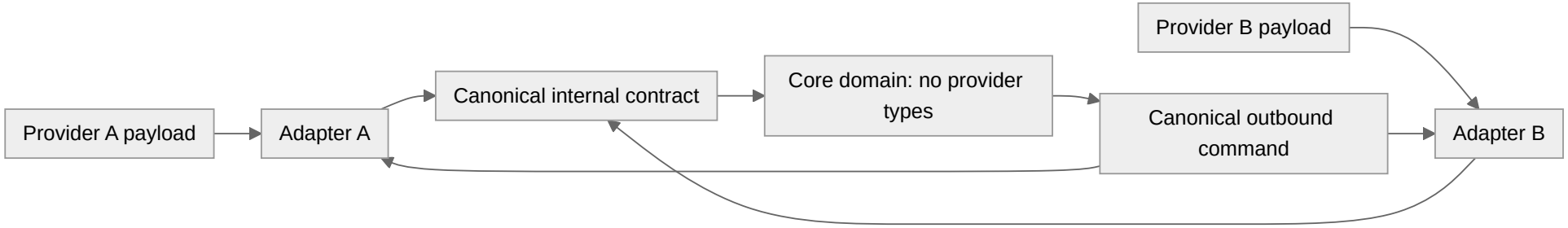


Figure 14.2: E11: Adapter boundary

14.3 US-100 — Configure an integration securely

User story US-100

Story. As an administrator, I can configure an integration securely.
Acceptance. credentials are secret references; test status and health are visible.

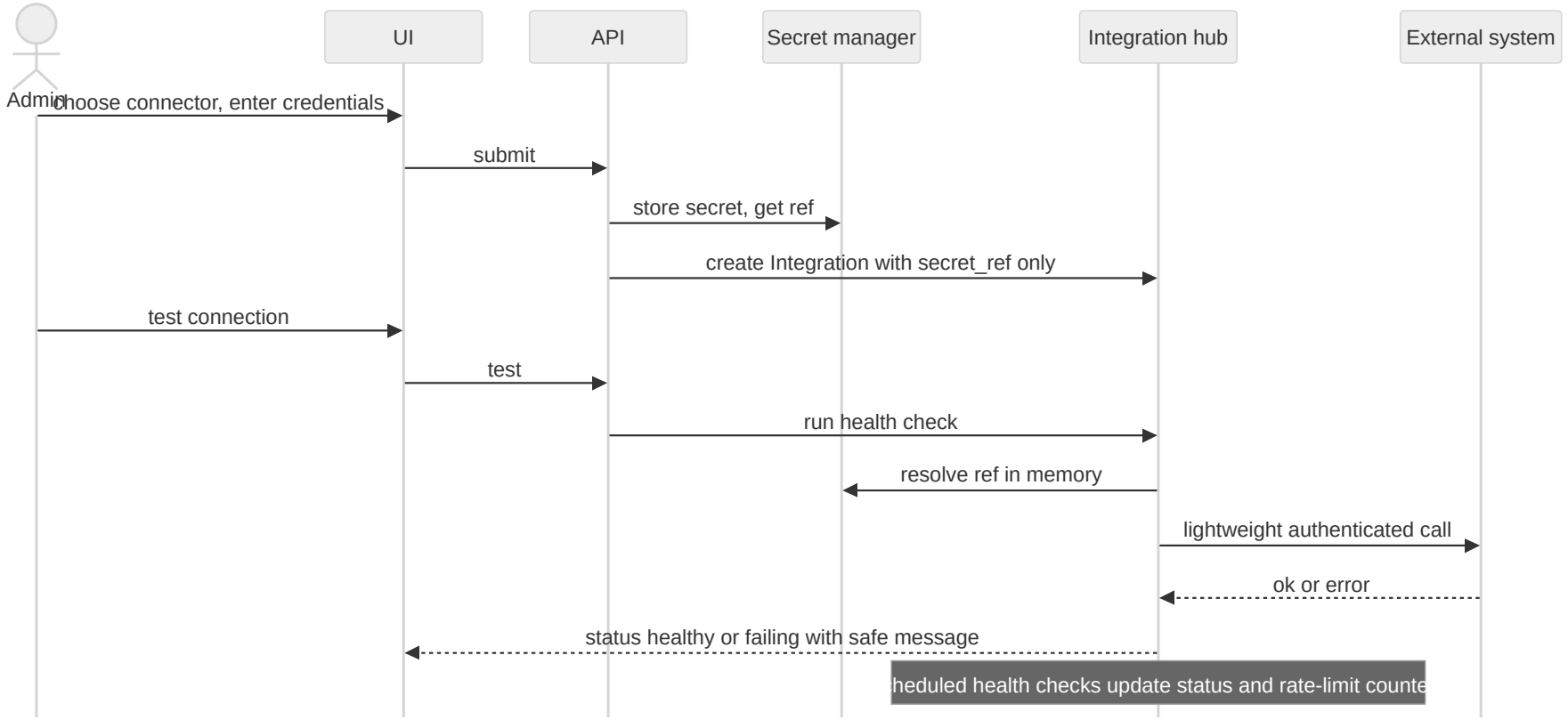


Figure 14.3: E11: US-100 — Configure an integration securely

14.3.1 Integration health states

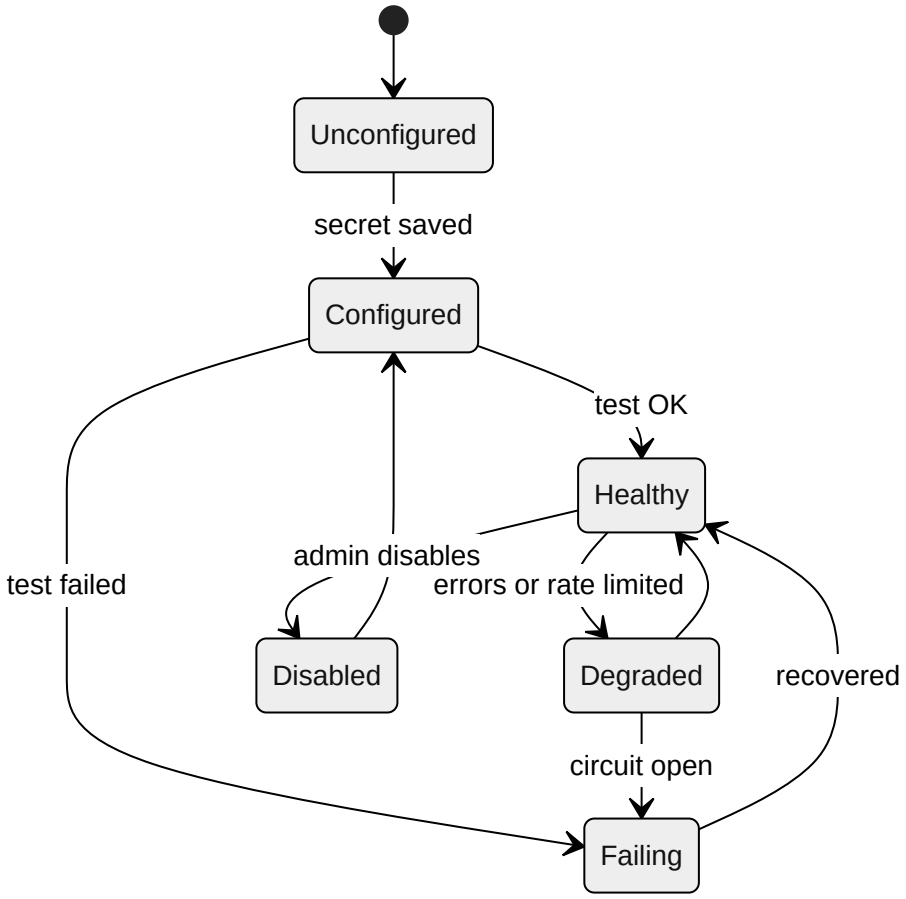


Figure 14.4: E11: US-100 — Configure an integration securely: Integration health states

14.4 US-101 — Normalize provider webhook into canonical event

User story US-101

Story. As a system, I can normalize a provider webhook into a canonical event.
Acceptance. signature verified; replay deduplicated; event processing observable.

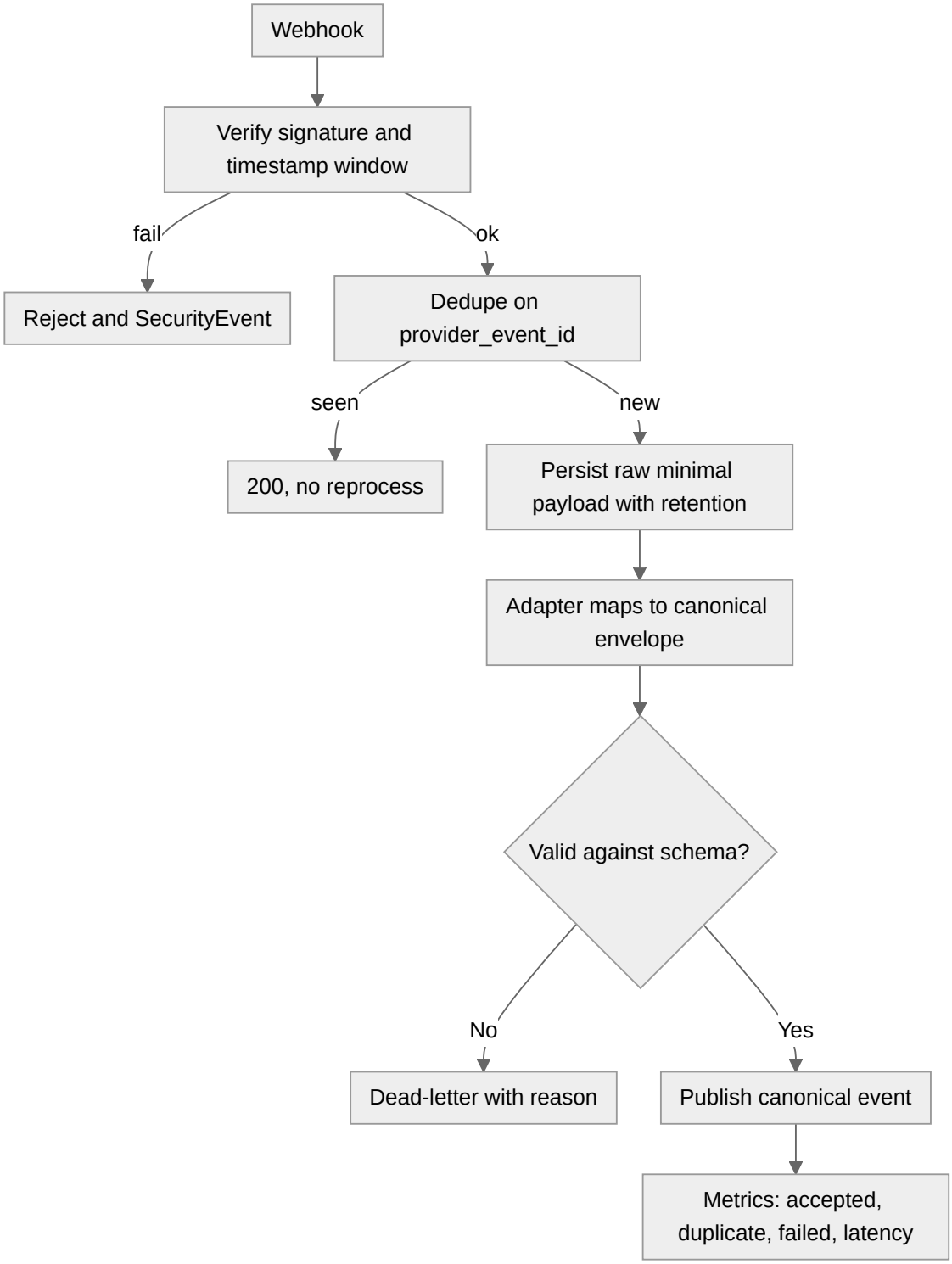


Figure 14.5: E11: US-101 — Normalize provider webhook into canonical event

14.5 US-102 — Continue support through a second channel

User story US-102

Story. As a customer, I can continue support through a second channel.
Acceptance. identity linking follows verification policy; duplicate conversations are avoided where possible.

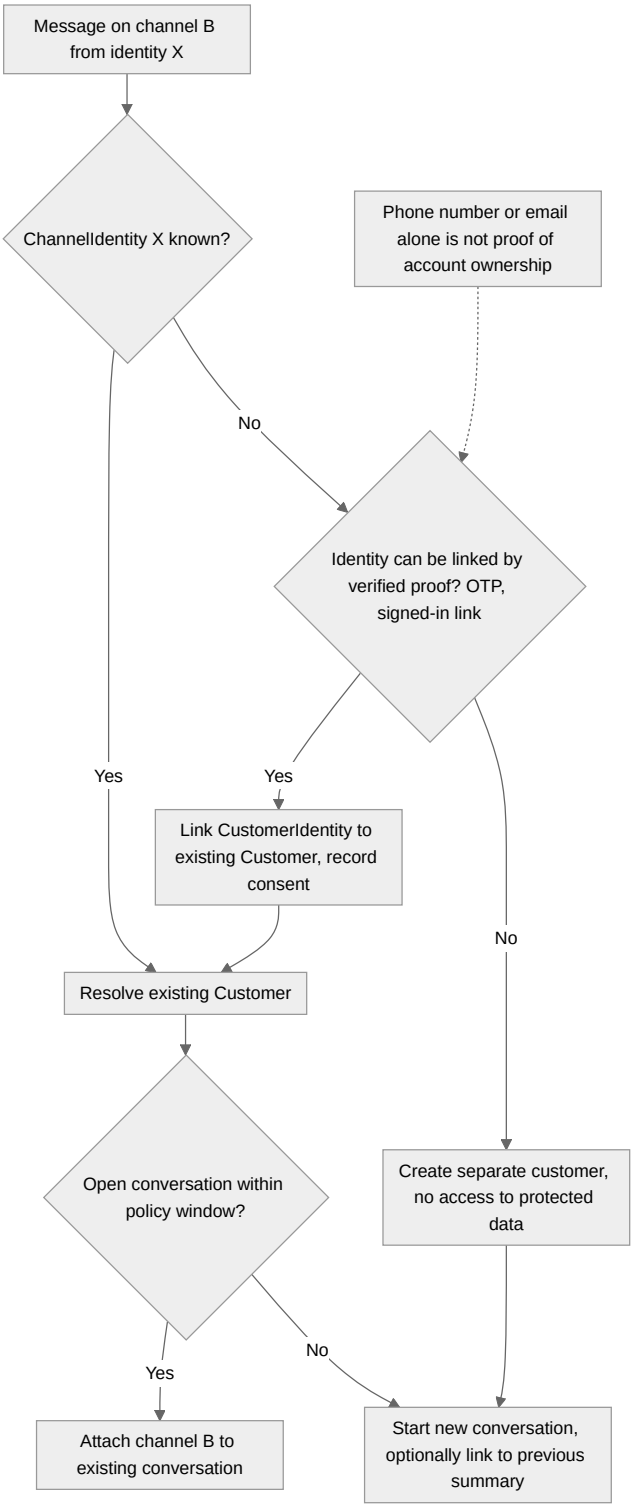


Figure 14.6: E11: US-102 — Continue support through a second channel

Chapter 15

E12 — Observability, Evaluation and Usage

Goal: operate and improve the platform with evidence. Modules: M16, M17, M20. Source: the observability chapter.

Stories in this epic

ID	User story	Sprint
US-110	As an operator, I can trace one request across modules.	S6
US-111	As a product owner, I can see resolution and handoff metrics.	Backlog
US-112	As a team, we can run AI regression evaluations.	S6
US-113	As an administrator, I can inspect usage by agent and channel.	Backlog

15.1 Epic overview

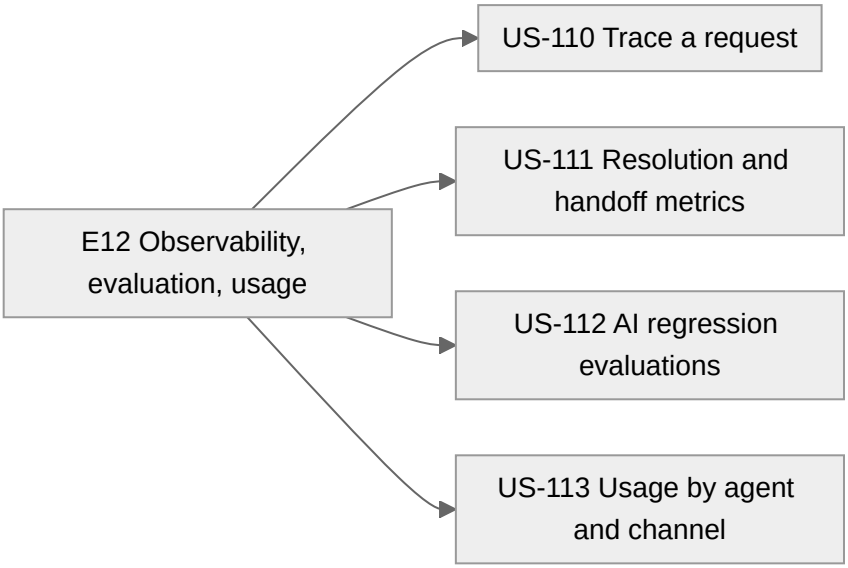


Figure 15.1: E12: Epic overview

15.2 US-110 — Trace one request across modules

User story US-110

Story. As an operator, I can trace one request across modules.
Acceptance. correlation IDs connect API, worker, model, retrieval and tool activity.

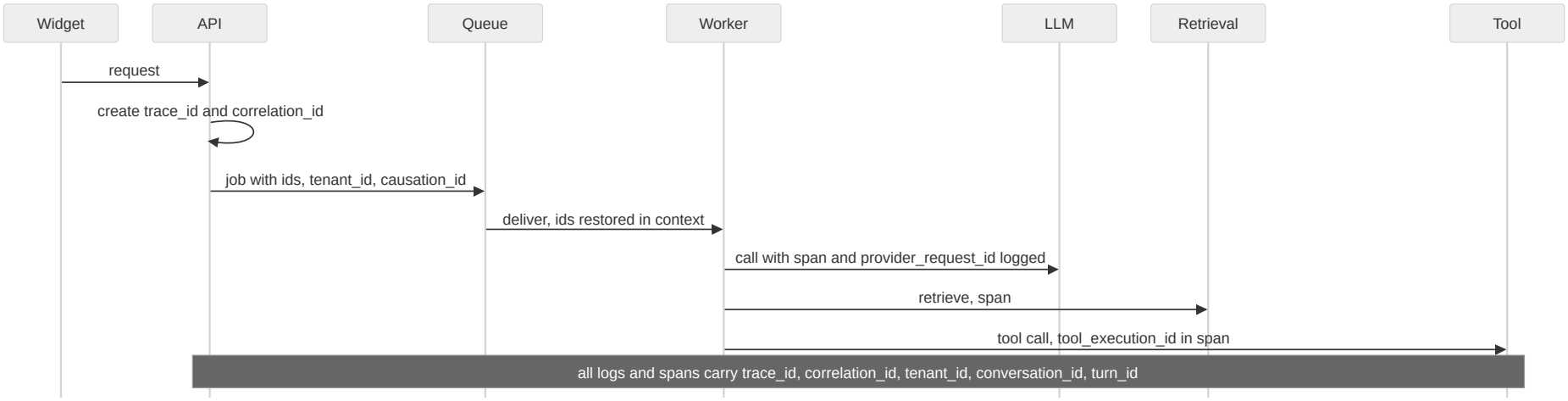


Figure 15.2: E12: US-110 — Trace one request across modules

15.2.1 Correlation identifiers

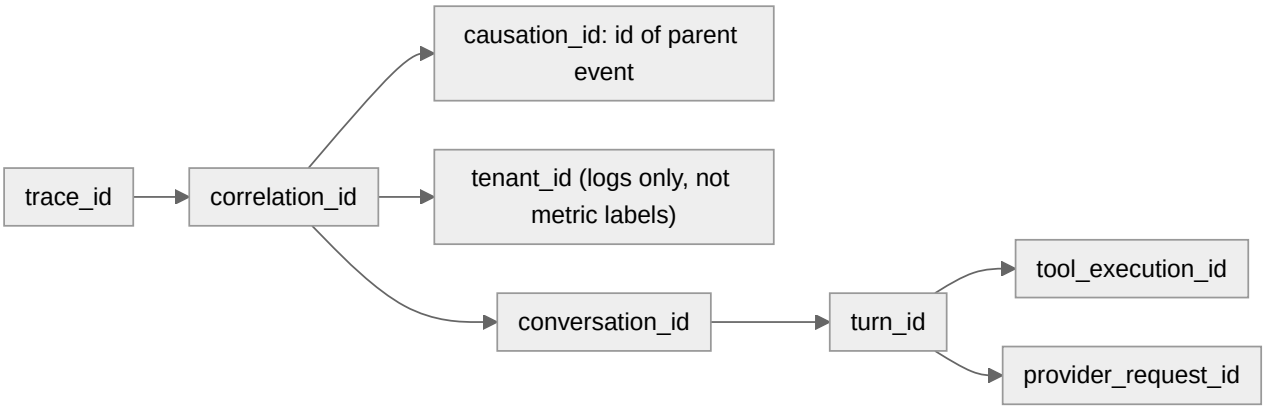


Figure 15.3: E12: US-110 — Trace one request across modules: Correlation identifiers

15.3 US-111 — Resolution and handoff metrics

User story US-111

Story. As a product owner, I can see resolution and handoff metrics.
Acceptance. definitions are documented and metrics are tenant-safe.

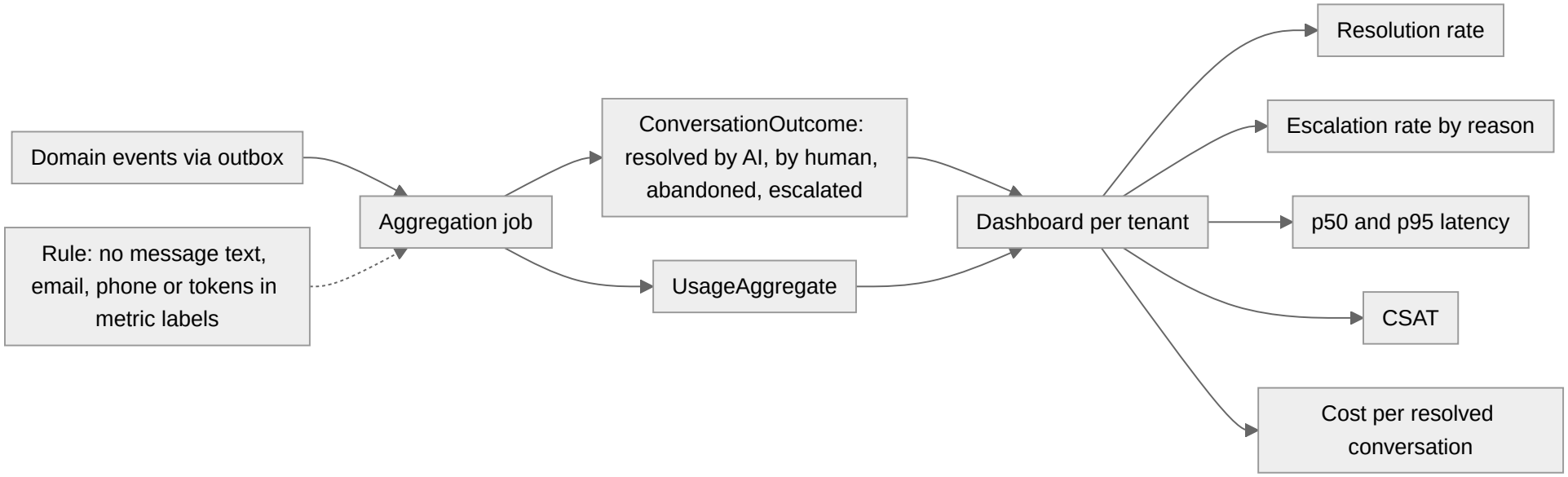


Figure 15.4: E12: US-111 — Resolution and handoff metrics

15.3.1 Platform signal groups

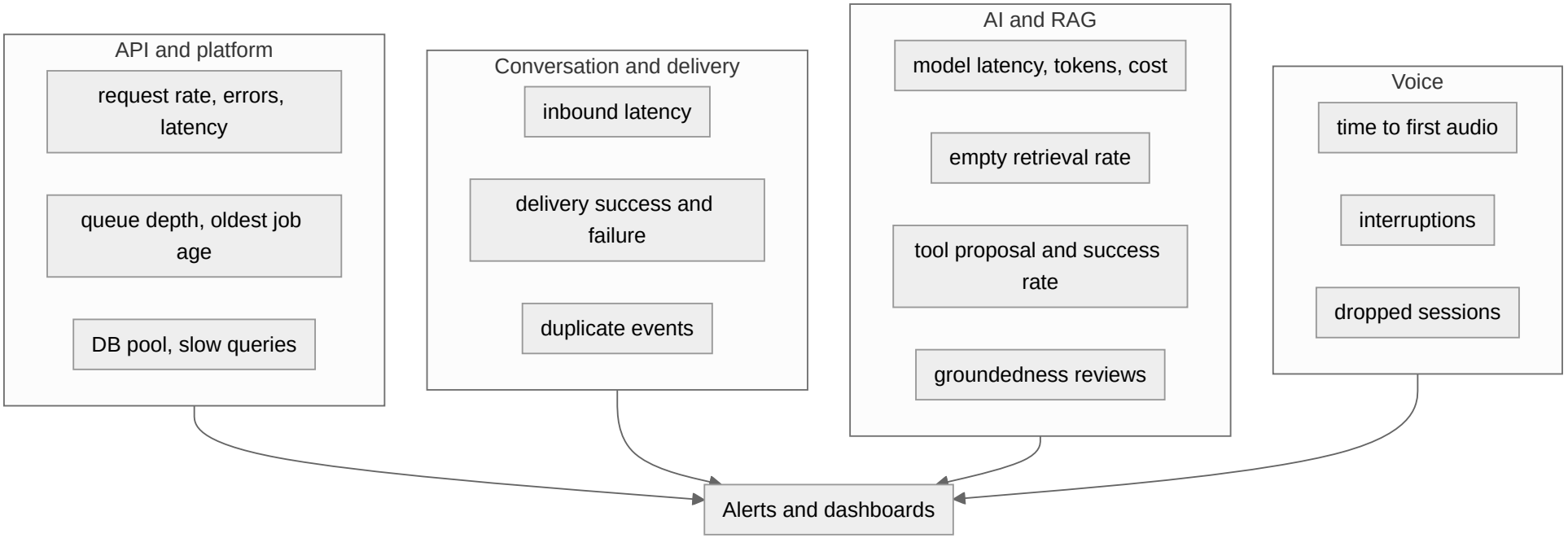


Figure 15.5: E12: US-111 — Resolution and handoff metrics: Platform signal groups

15.4 US-112 — Run AI regression evaluations

User story US-112

Story. As a team, we can run AI regression evaluations.
Acceptance. dataset and agent/model version are recorded; critical policy tests gate release.

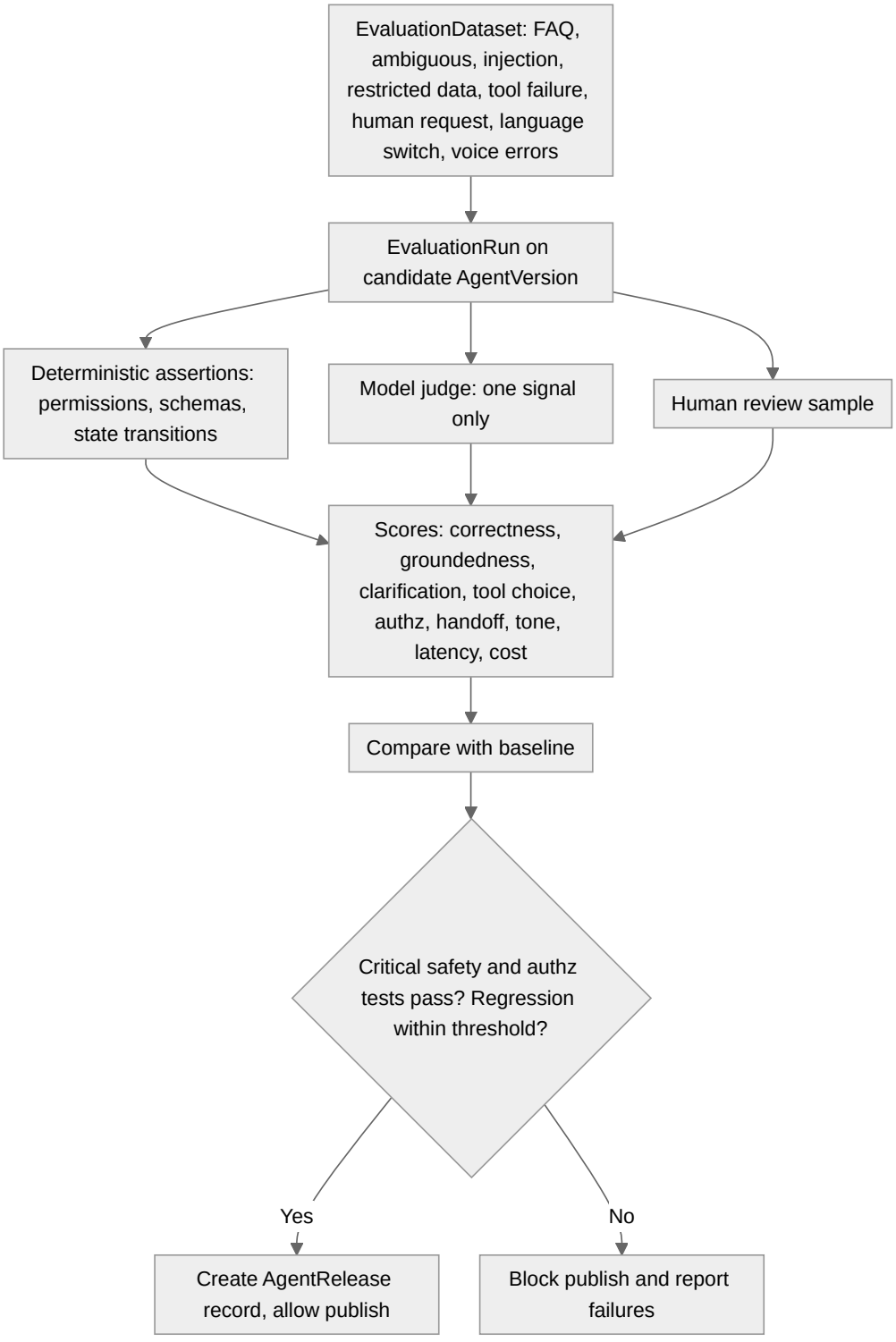


Figure 15.6: E12: US-112 — Run AI regression evaluations

15.4.1 Improvement loop

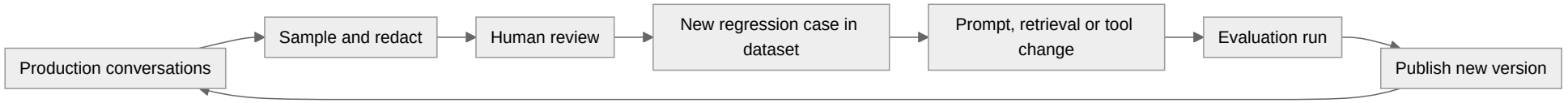


Figure 15.7: E12: US-112 — Run AI regression evaluations: Improvement loop

15.5 US-113 — Inspect usage by agent and channel

User story US-113

Story. As an administrator, I can inspect usage by agent and channel.
Acceptance. provider usage is reconciled and sensitive data is not exposed in metrics.

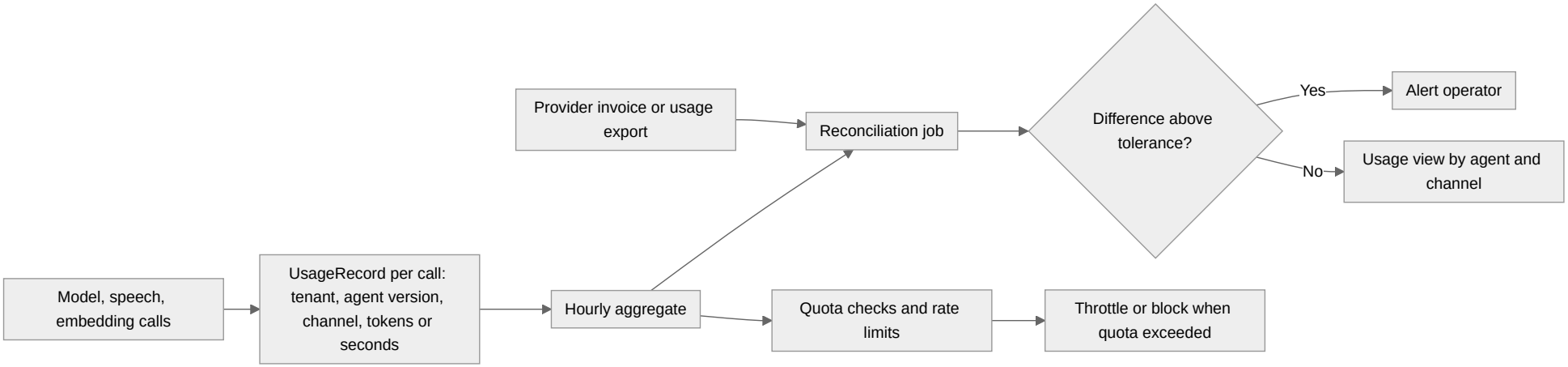


Figure 15.8: E12: US-113 — Inspect usage by agent and channel

Part III

Release and Sprint Planning

Chapter 16

Release Goals, Risks and Decision Log

16.1 Release goals

16.1.1 R1 — Text support MVP

Outcome: website visitor receives knowledge-grounded answers, one read-only business lookup is available, and human handoff preserves context.

Exit criteria:

- tenant isolation tests pass;
- inbound messages are deduplicated;
- unknown/unsupported questions have safe fallback;
- tool calls are allowlisted, authorized, and audited;
- human can take over;
- core API/worker telemetry is available;
- evaluation suite has no unresolved critical policy failure.

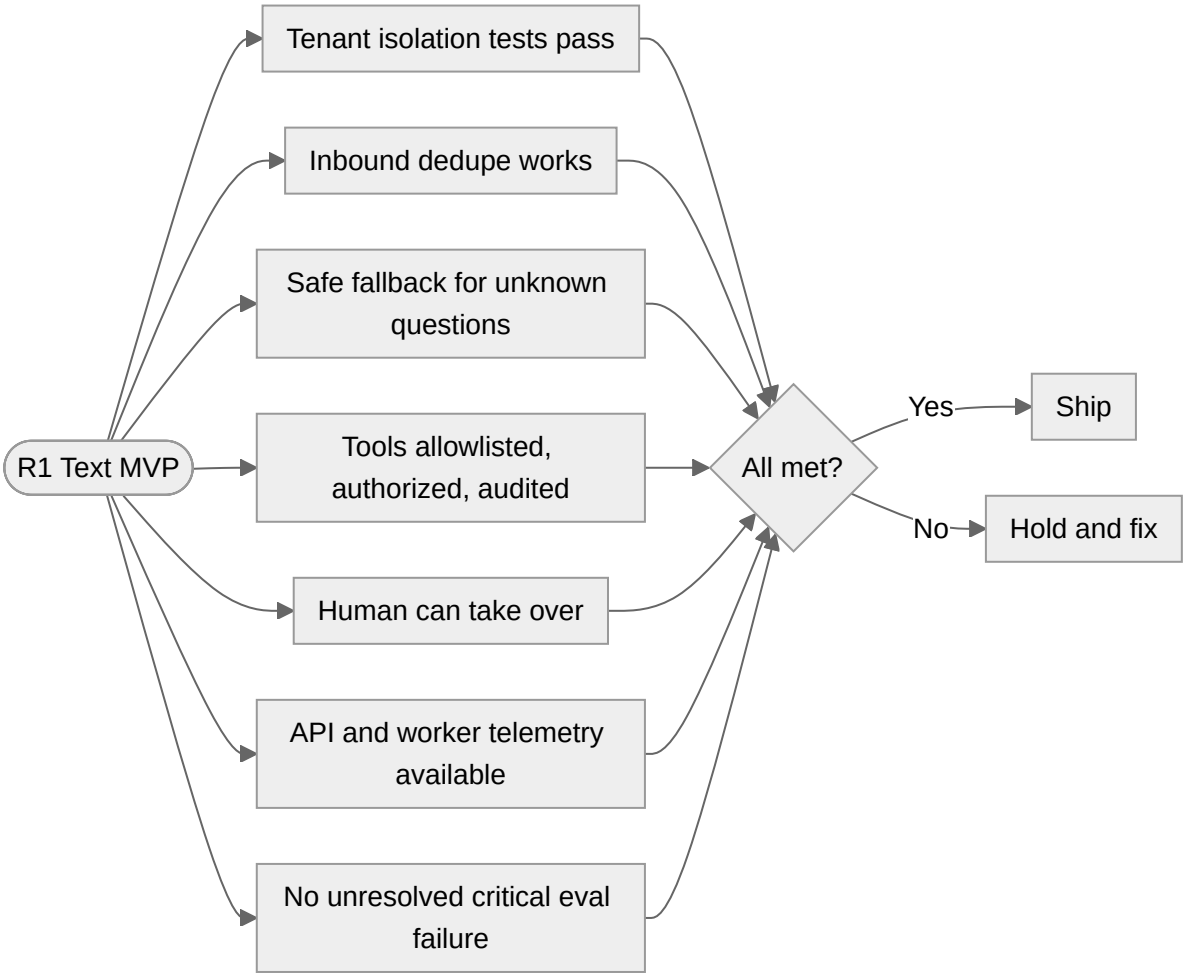


Figure 16.1: Release 1 exit criteria

16.1.2 R2 — Voice MVP

Outcome: customer can speak with the same AI engine and see a coherent transcript.

Exit criteria:

- session start/stop and cleanup are reliable;
- transcript finalization and ordering work;
- interruption behavior is tested;
- audio/transcript privacy policy is implemented;
- latency/cost metrics are visible;
- human handoff behavior is validated for the selected provider.

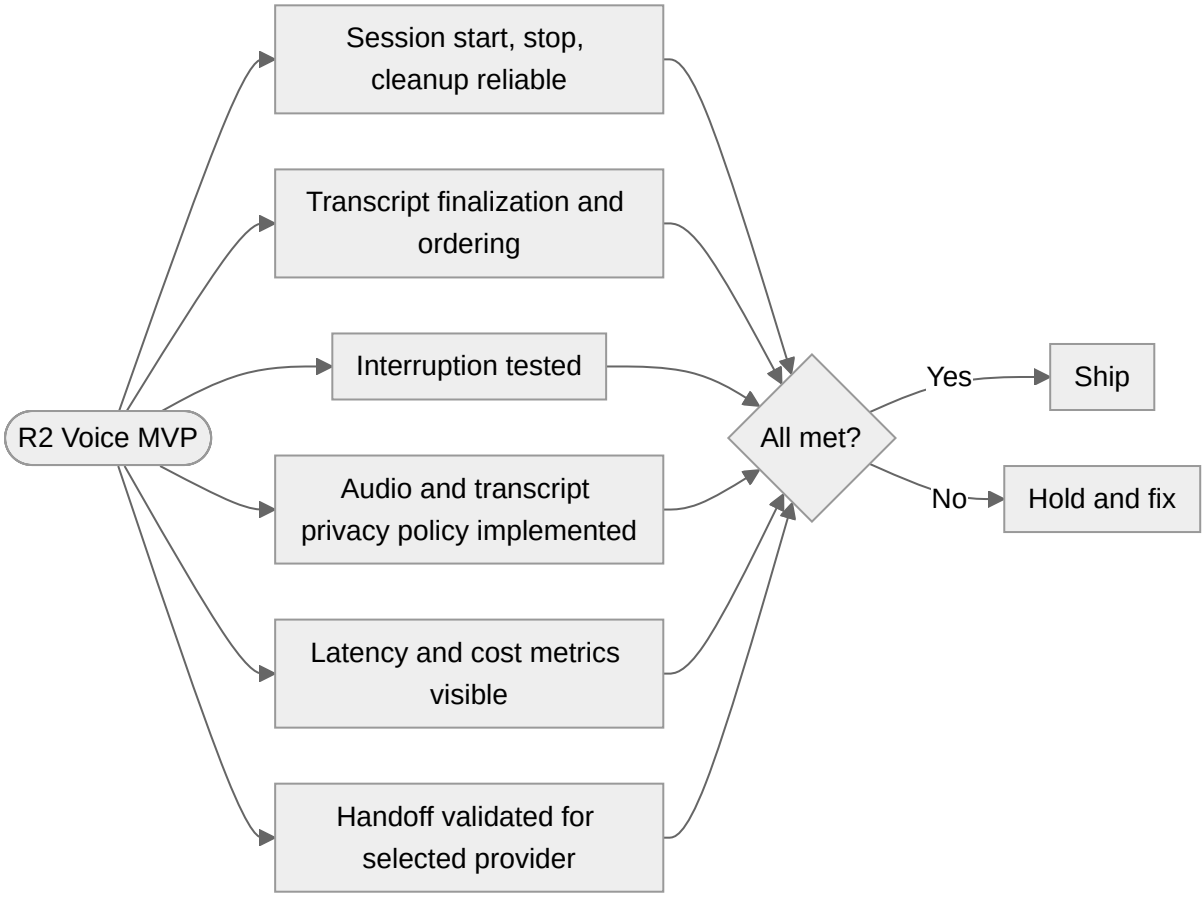


Figure 16.2: Release 2 exit criteria

16.1.3 R3 — Operational expansion

Outcome: support teams can scale operations with workflows, routing, additional channels and stronger evaluation.

16.2 Initial risks

Risk	Impact	Mitigation
Prompt injection via customer or retrieved content	Unauthorized or incorrect actions	Tool allowlist, server authorization, untrusted-content boundaries, adversarial tests
Tenant data leakage in retrieval	Severe privacy/security issue	Tenant-scoped indexes/filters, access tests, per-tenant metadata and object controls
LLM latency/cost	Poor customer experience and high cost	Budgets, timeouts, model routing, caching only safe content, measurement
Duplicate webhook or tool execution	Duplicate messages/actions	Durable inbox, idempotency keys, deduplication
Voice interruption/transcript race	Confusing conversation history	Sequence IDs, finalized vs partial states, cancellation semantics
Provider lock-in	Cost and migration risk	Provider adapters, normalized contracts, contract tests
AI gives unsupported answer	Loss of trust	Retrieval evaluation, evidence thresholds, safe fallback and handoff
Scope too broad	Delayed first usable product	Text MVP vertical slice, ordered backlog, defer nonessential channels

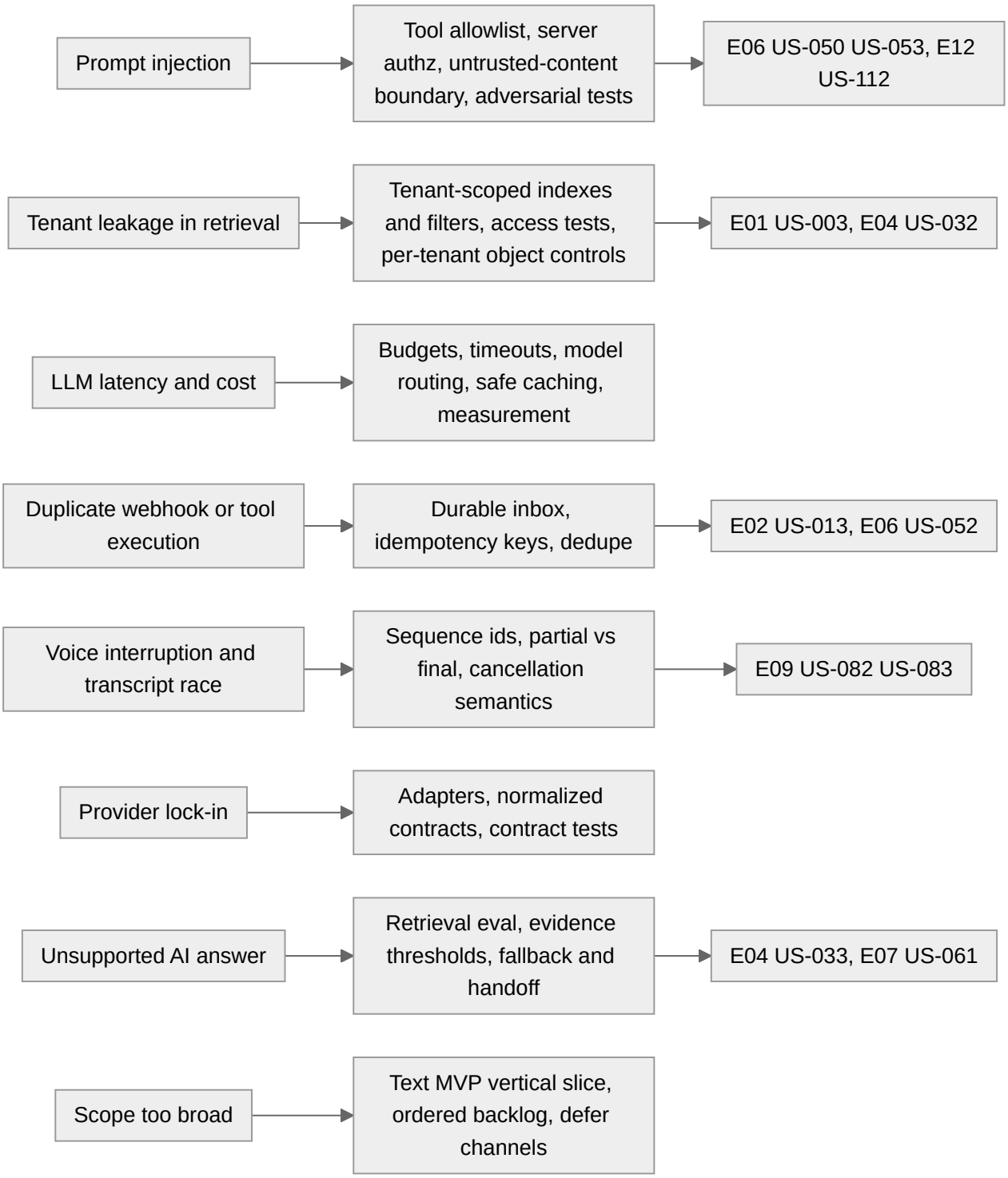


Figure 16.3: Risk map

16.3 Architecture decision records to create

- ADR-001: Modular monolith boundaries and package ownership
- ADR-002: Relational database and tenant isolation strategy
- ADR-003: Queue and outbox/inbox strategy
- ADR-004: Knowledge ingestion and vector/search technology
- ADR-005: LLM provider abstraction and data policy
- ADR-006: Voice strategy: modular STT/TTS versus speech-to-speech
- ADR-007: Browser voice transport and telephony provider
- ADR-008: Transcript/audio retention and consent
- ADR-009: Authentication and customer verification
- ADR-010: Tool authorization and approval model

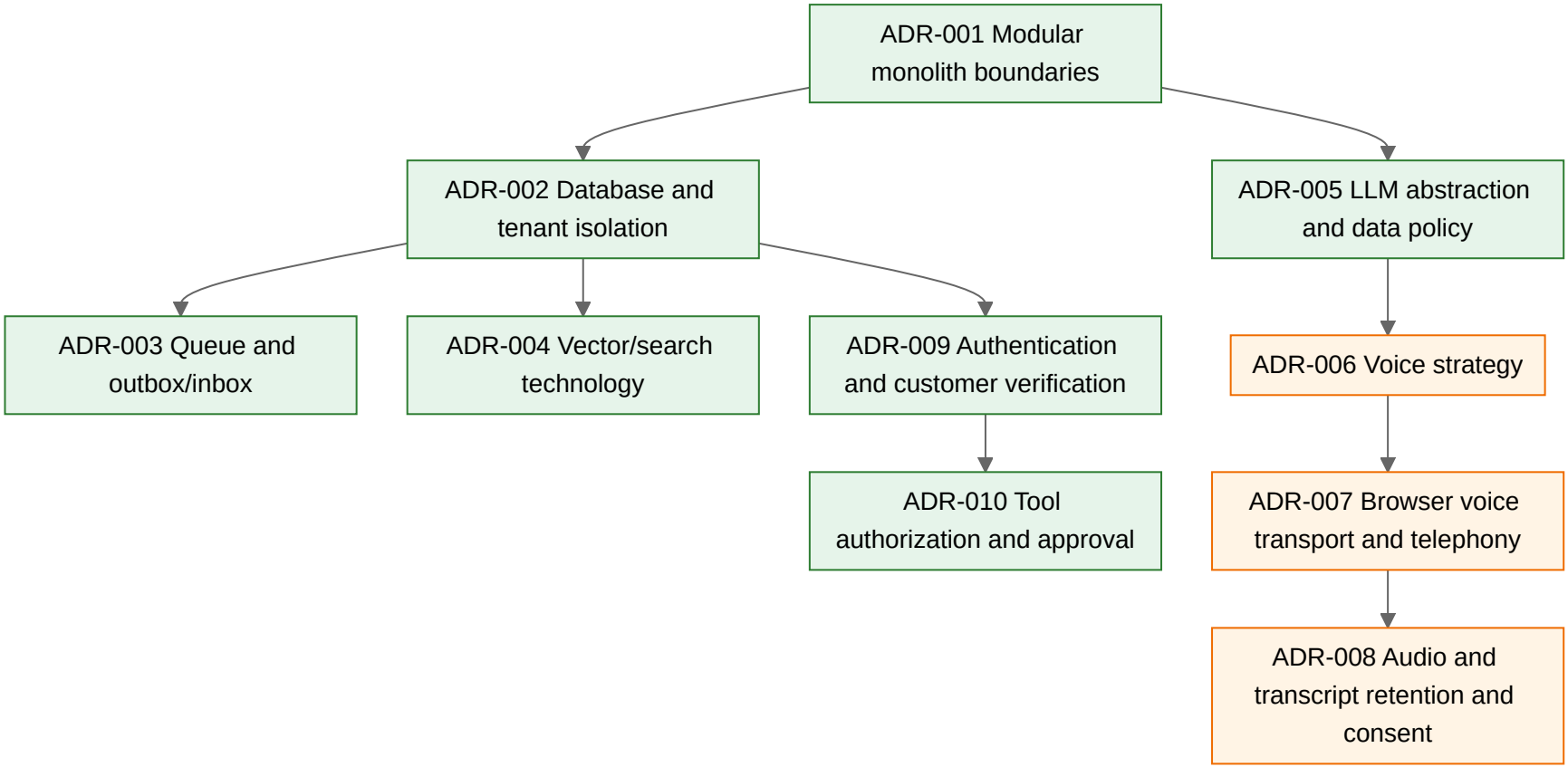


Figure 16.4: ADR dependency order

16.4 Decisions not to make prematurely

- One microservice per module
- A universal workflow builder before core support works
- Multiple speech providers before the first voice use case is validated
- Autonomous write access to every customer system
- Permanent raw audio retention
- Complex subscription/billing before pricing and usage requirements are known

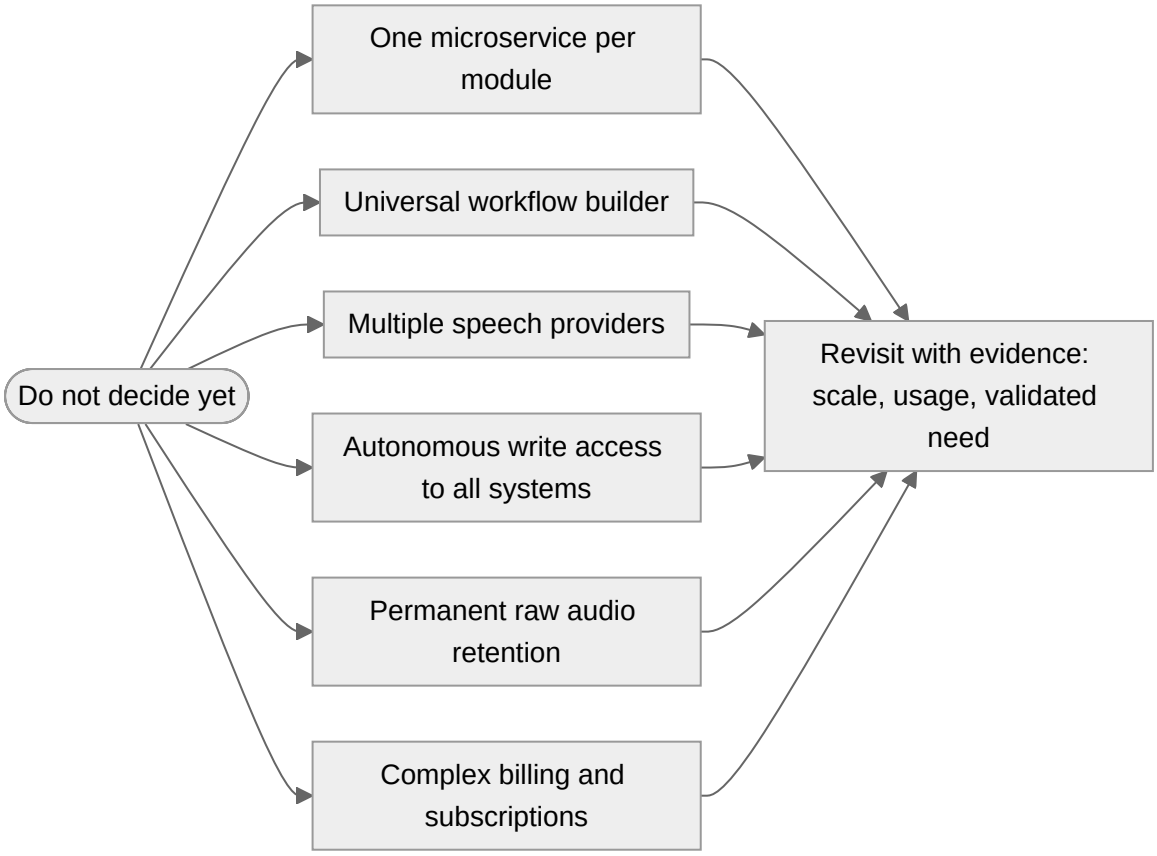


Figure 16.5: Decisions to defer

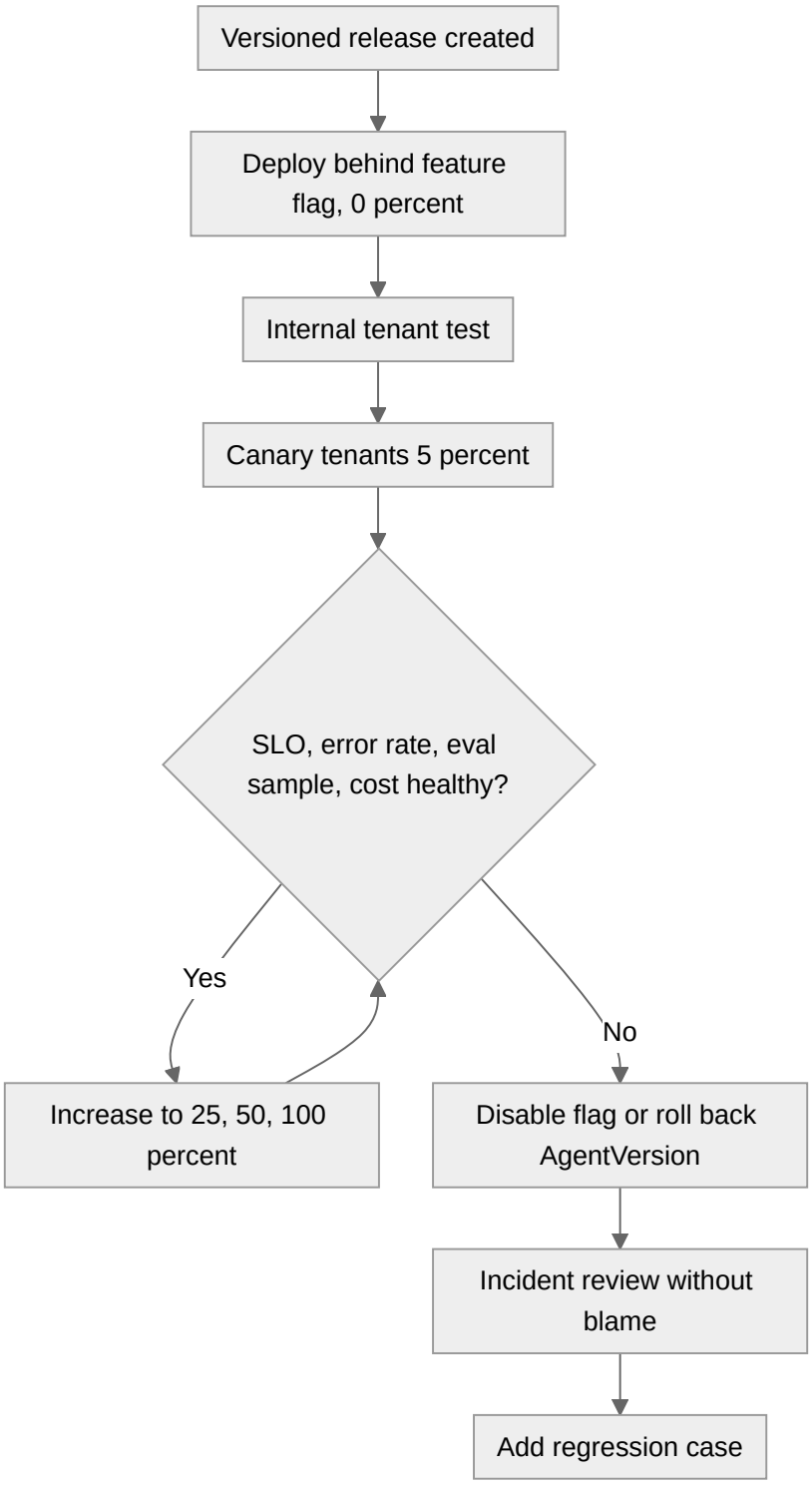


Figure 16.6: Rollout and rollback

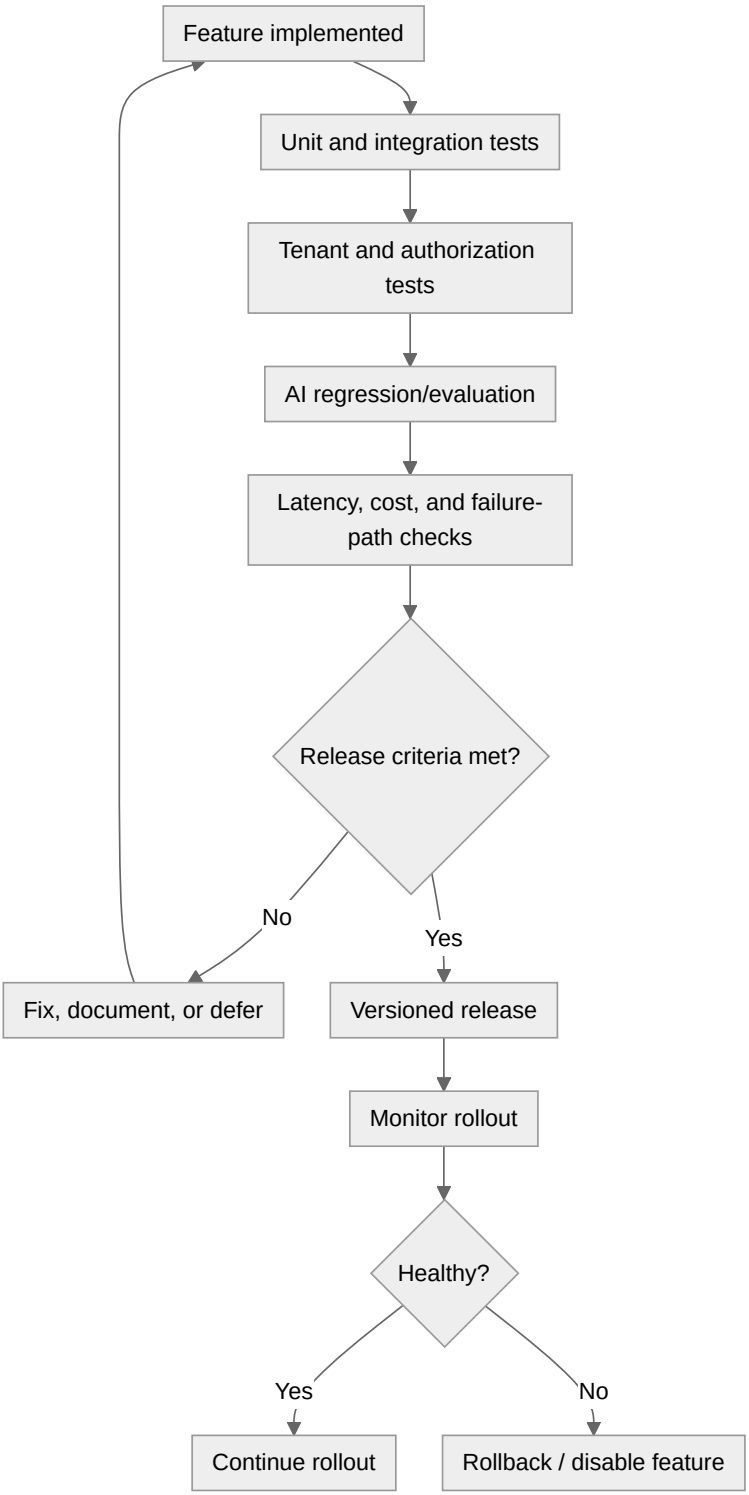


Figure 16.7: Release acceptance flow

Chapter 17

Provisional Sprint Plan

Assumption: two-week Sprints. This sequence is a planning hypothesis, not a fixed commitment. Reforecast after the first Sprint using actual capacity and discovered complexity.

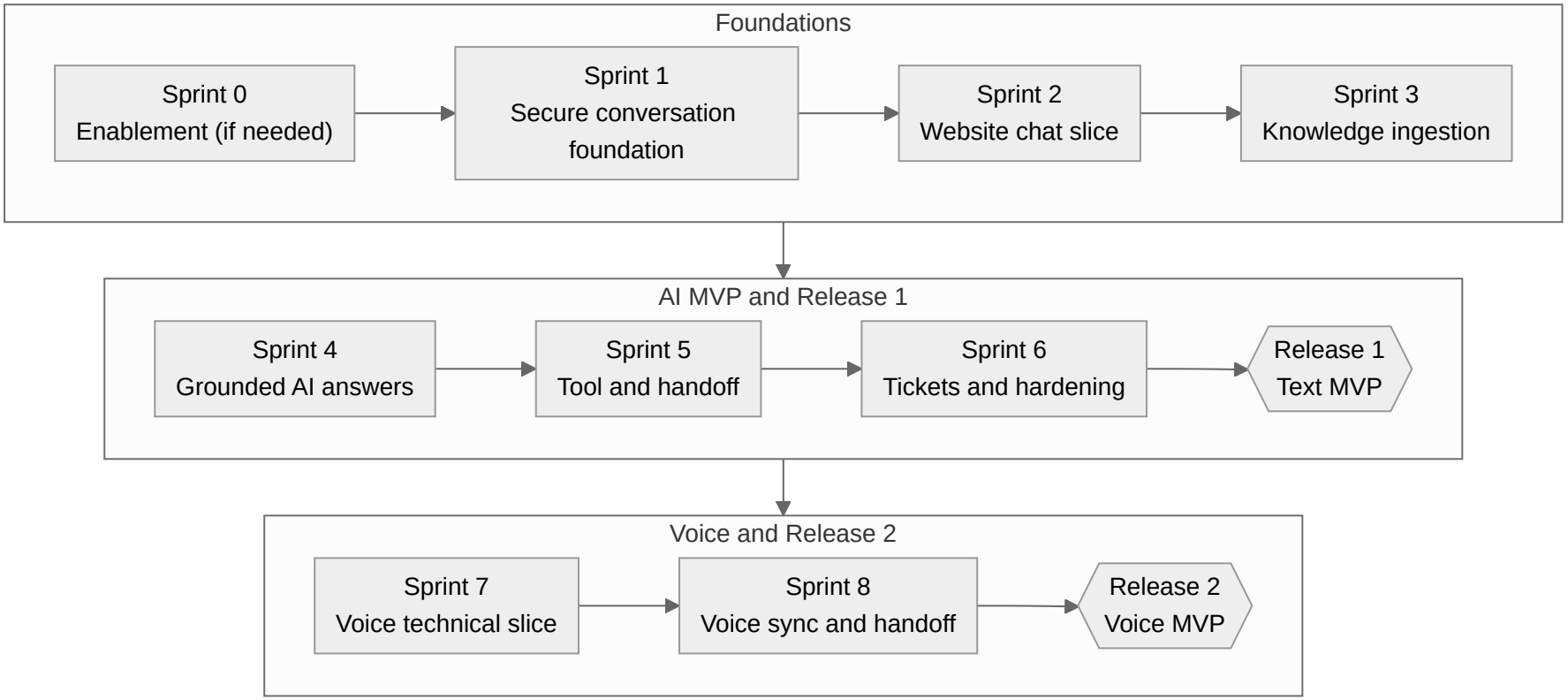


Figure 17.1: Sprint sequence and goals

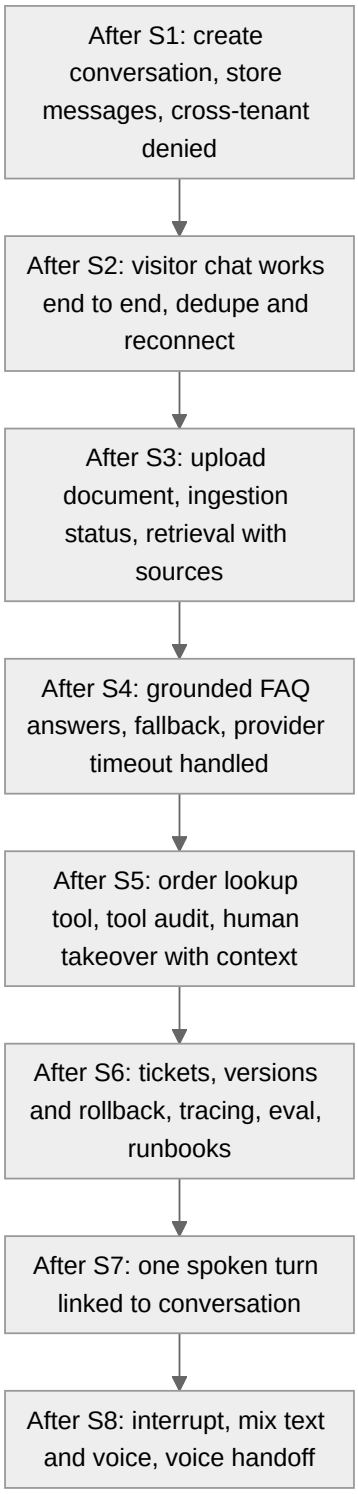


Figure 17.2: What the demo can show after each Sprint

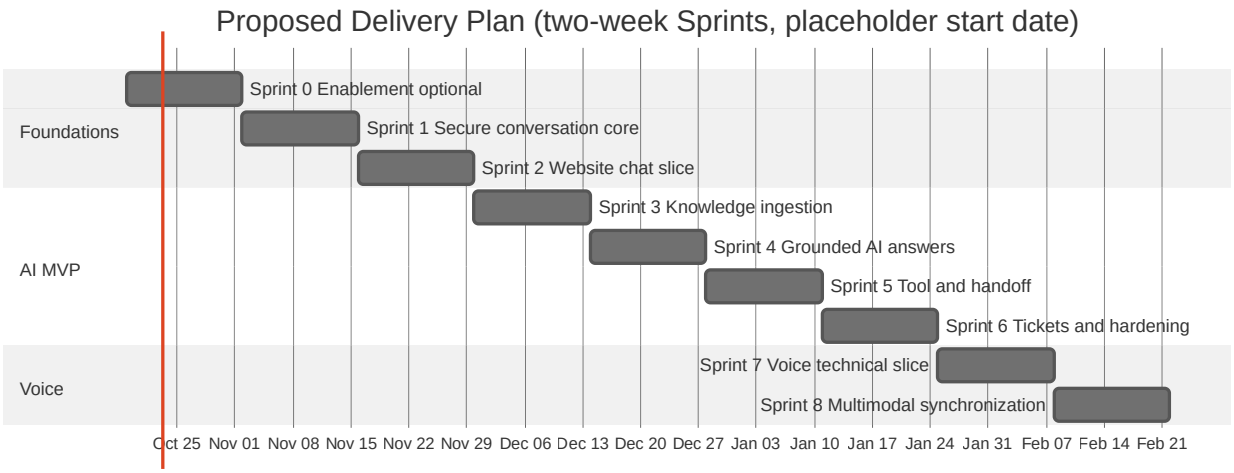


Figure 17.3: Provisional timeline (placeholder dates)

17.1 Sprint 0 / Enablement (only if needed)

Sprint Goal: make the team able to build, test, and release safely.

- Repository and module/package structure
- CI pipeline, formatting, static analysis, test setup
- Local development environment and database migrations
- Initial threat model and architecture decision records
- Minimal deployment and secret handling
- Definition of Done agreement

Do not use Sprint 0 as a permanent phase that postpones delivering product value.

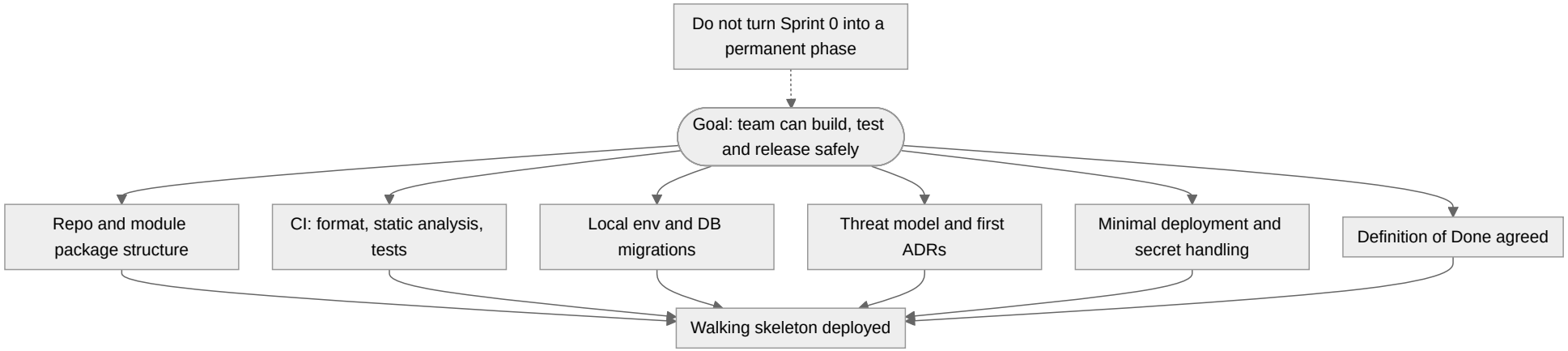


Figure 17.4: Sprint 0 detail

17.2 Sprint 1 — Secure conversation foundation

Sprint Goal: a tenant-scoped conversation and message can be created, persisted, and retrieved.

- US-001 organization foundation
- US-003 tenant-isolation test foundation
- US-010 conversation creation
- US-011 text message persistence
- Initial audit/correlation ID conventions

Review demo: create a conversation, append messages, retrieve timeline, prove cross-tenant access is denied.

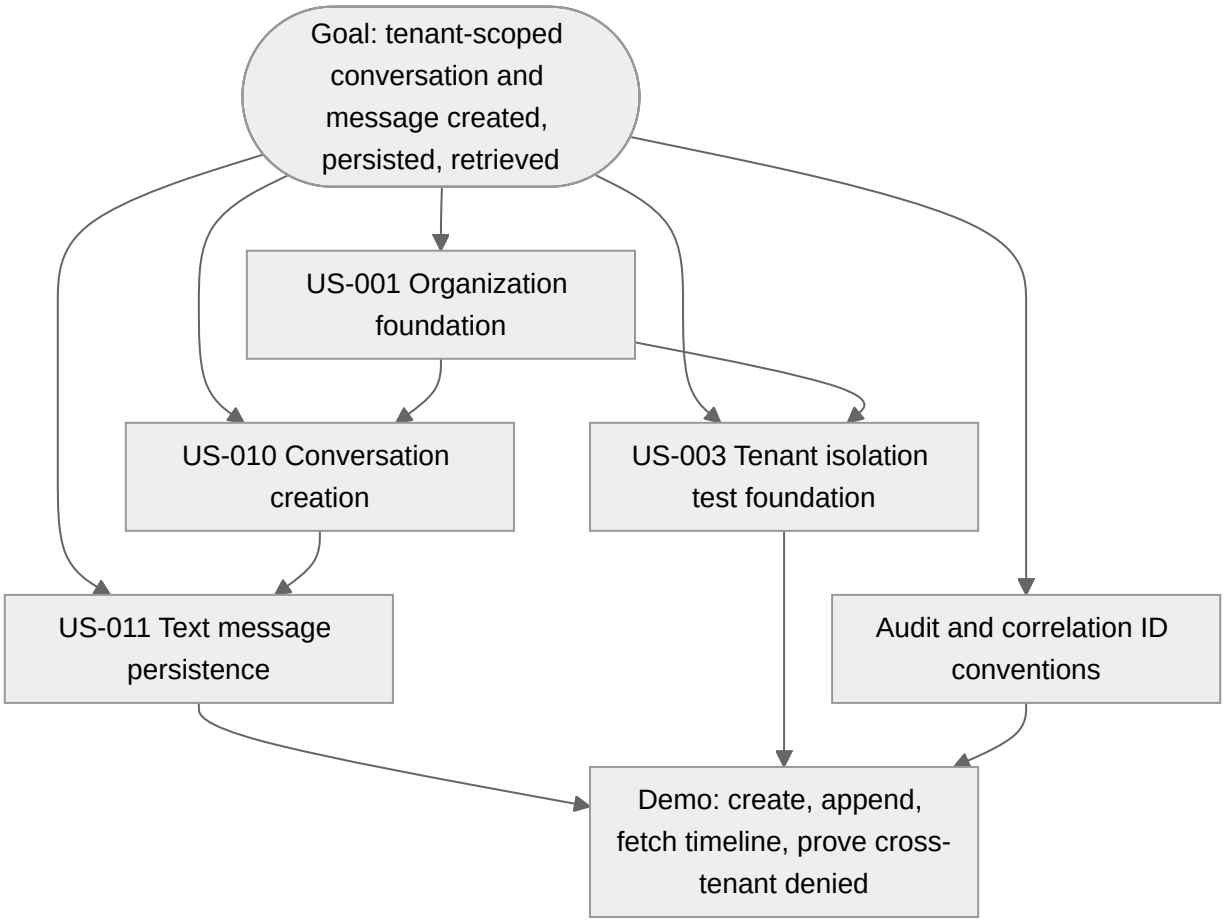


Figure 17.5: Sprint 1 detail

17.3 Sprint 2 — Website chat vertical slice

Sprint Goal: a visitor can exchange text messages with the backend reliably.

- US-020 widget configuration
- US-021 chat session
- US-022 reconnect/resume
- US-013 inbound deduplication
- Basic delivery/error states and tracing

Review demo: customer sends a message; it is persisted and returned; duplicate/reconnect tests are shown.

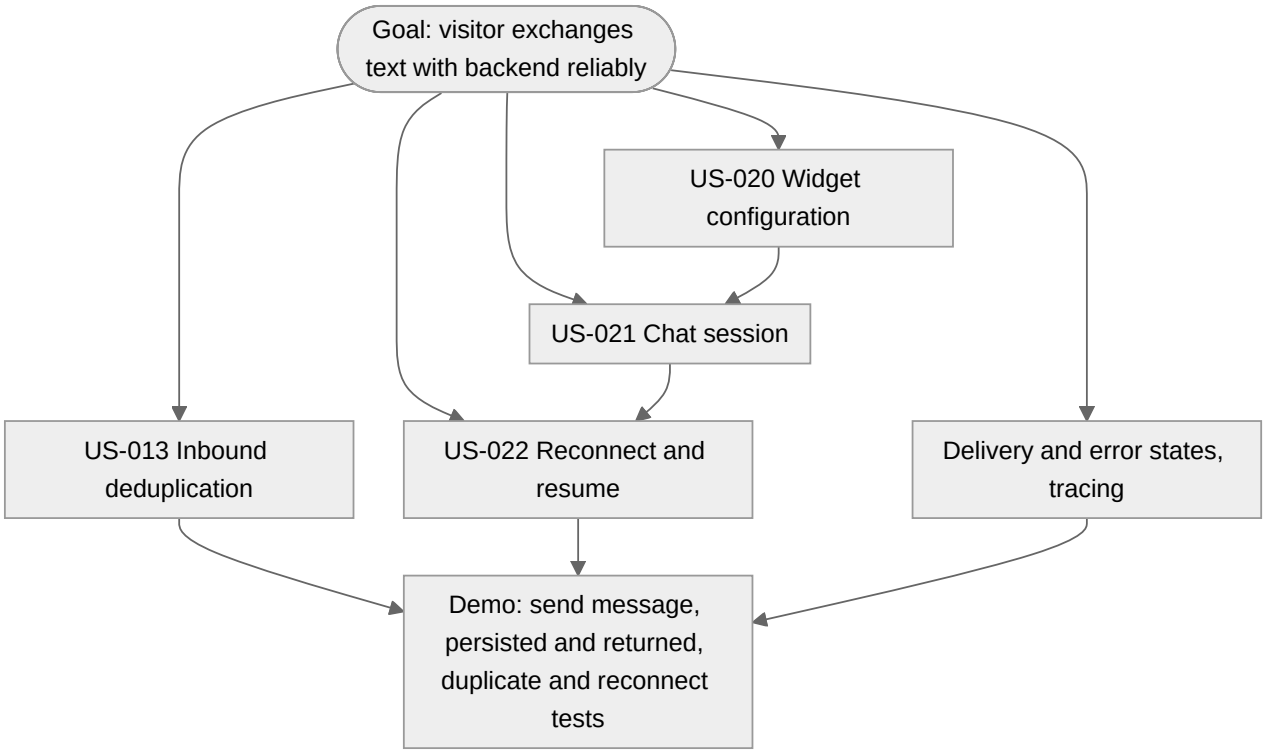


Figure 17.6: Sprint 2 detail

17.4 Sprint 3 — Knowledge ingestion and retrieval

Sprint Goal: organization knowledge can be ingested and retrieved with tenant boundaries.

- US-030 source registration/upload
- US-031 asynchronous parse/chunk/index
- US-032 retrieval with source metadata
- Basic ingestion status and failure retry

Review demo: upload a document, inspect ingestion status, query it, see source references.

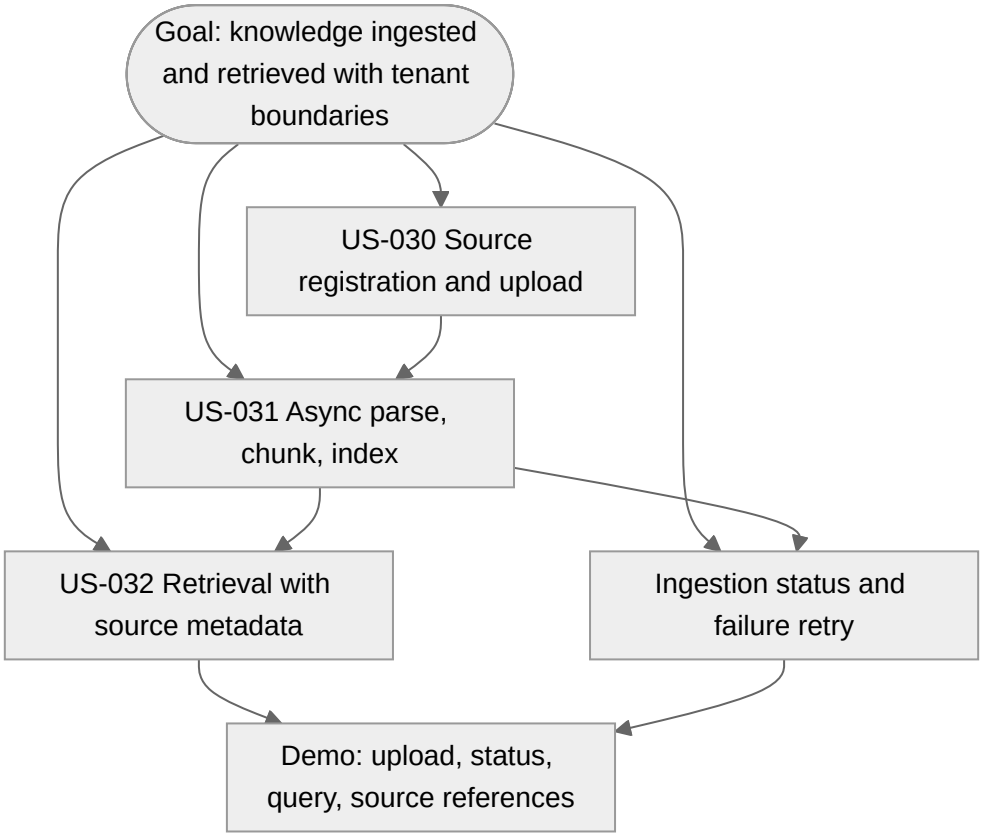


Figure 17.7: Sprint 3 detail

17.5 Sprint 4 — Grounded AI answers

Sprint Goal: website chat answers common questions from approved knowledge and handles missing evidence safely.

- US-040 turn orchestration
- US-033 grounded response/fallback
- US-041 follow-up context
- US-042 model failure handling
- Minimal evaluation set and AI usage recording

Review demo: grounded FAQ, follow-up question, unknown answer fallback, provider timeout behavior.

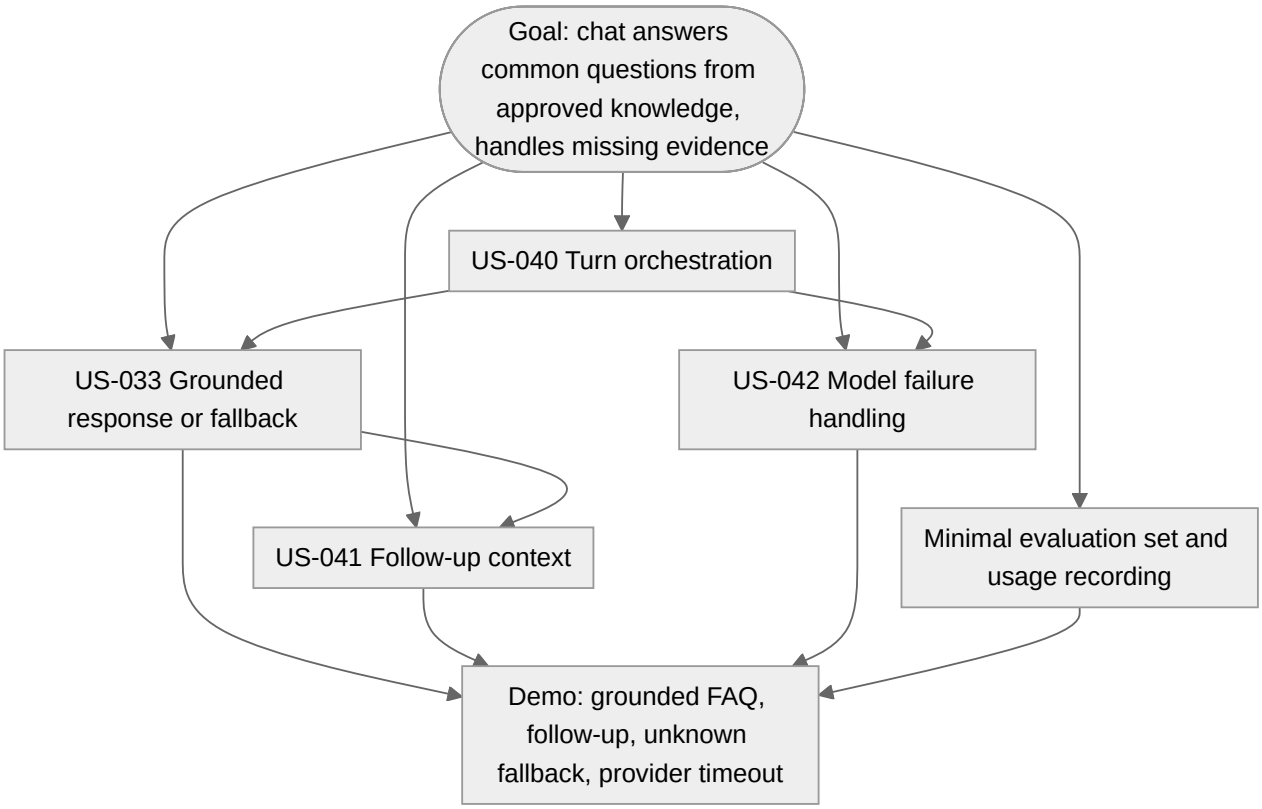


Figure 17.8: Sprint 4 detail

17.6 Sprint 5 — Safe business tool and human handoff

Sprint Goal: the AI can perform one authorized read-only lookup and transfer unresolved conversations to a human.

- US-050 tool registry and allowlist
- US-051 order lookup (or equivalent safe read-only tool)
- US-053 audit tool calls
- US-060 inbox/queue
- US-061 customer-requested handoff
- US-062 handoff summary

Review demo: tool success, tool denial/failure, and human takeover with preserved context.

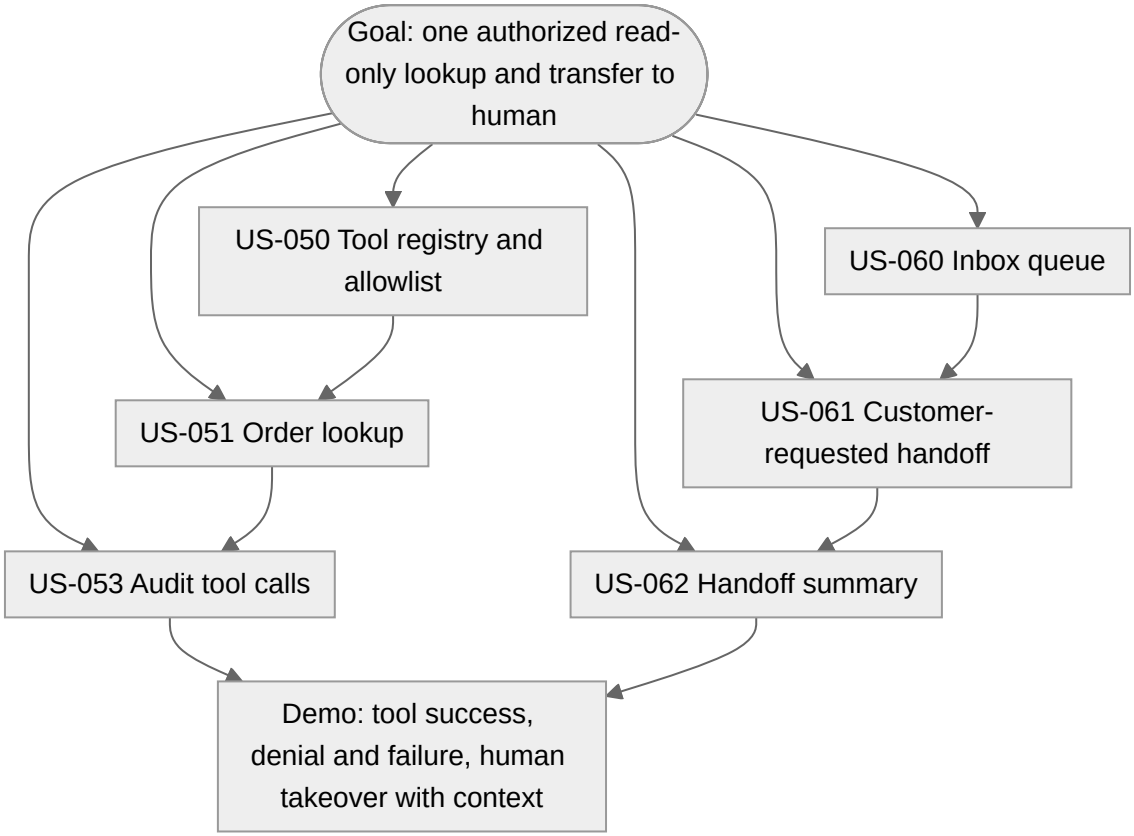


Figure 17.9: Sprint 5 detail

17.7 Sprint 6 — Tickets and release hardening

Sprint Goal: support staff can track unresolved work and the MVP meets agreed release gates.

- US-070 ticket creation
- US-071 ticket filtering
- US-043 agent version publication/rollback
- US-110 end-to-end tracing
- US-112 initial regression evaluation
- Security, load, backup/restore, and runbook work

Review demo: conversation becomes a ticket; team shows metrics, audit, test report, and rollback path.

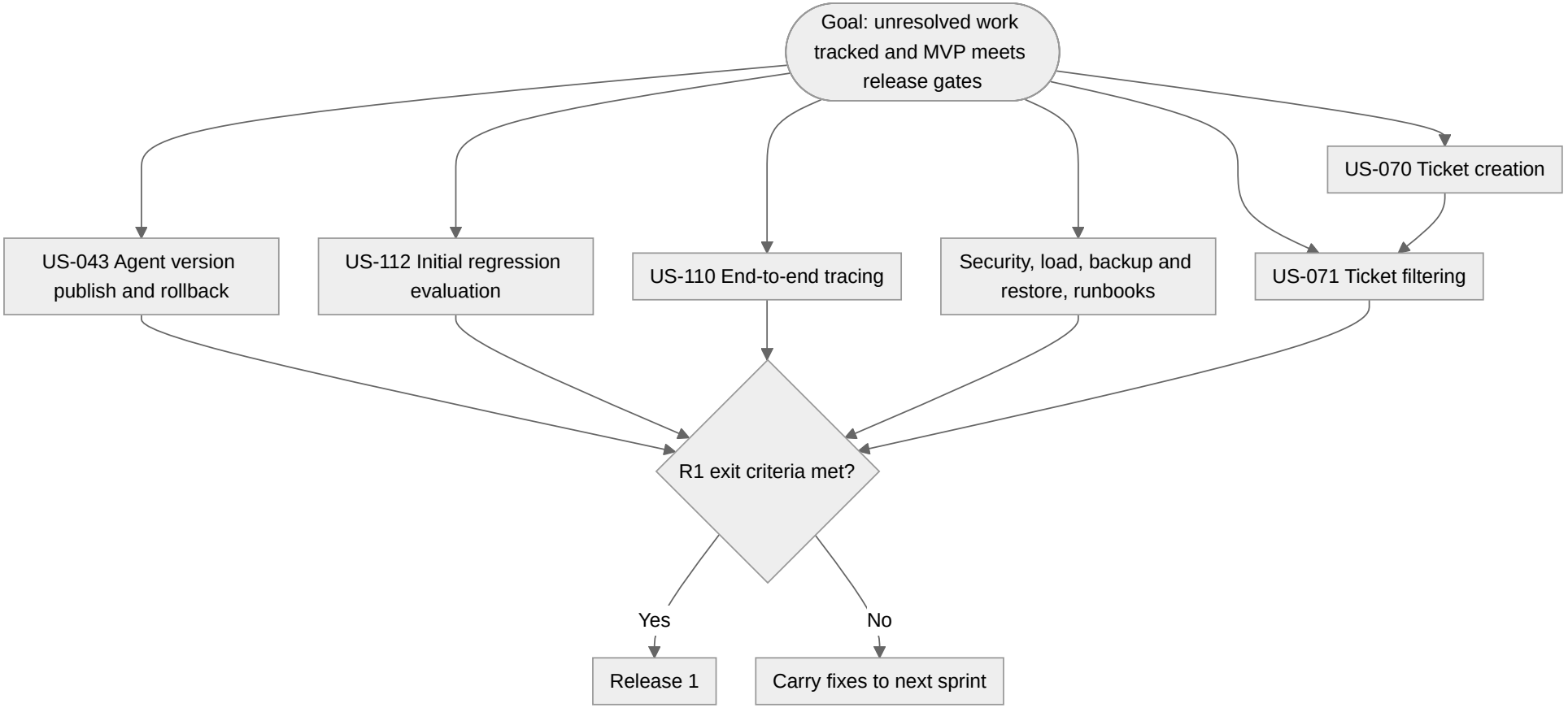


Figure 17.10: Sprint 6 detail

17.8 Sprint 7 — Voice technical spike / MVP slice

Sprint Goal: validate the selected voice architecture and complete a minimal browser voice turn.

- Select speech strategy and provider based on latency, language, cost, and privacy requirements
- US-080 start/end session
- US-081 basic speech exchange
- Voice telemetry and fallback
- Security/consent review

Review demo: customer speaks, receives response, and transcript is linked to conversation.

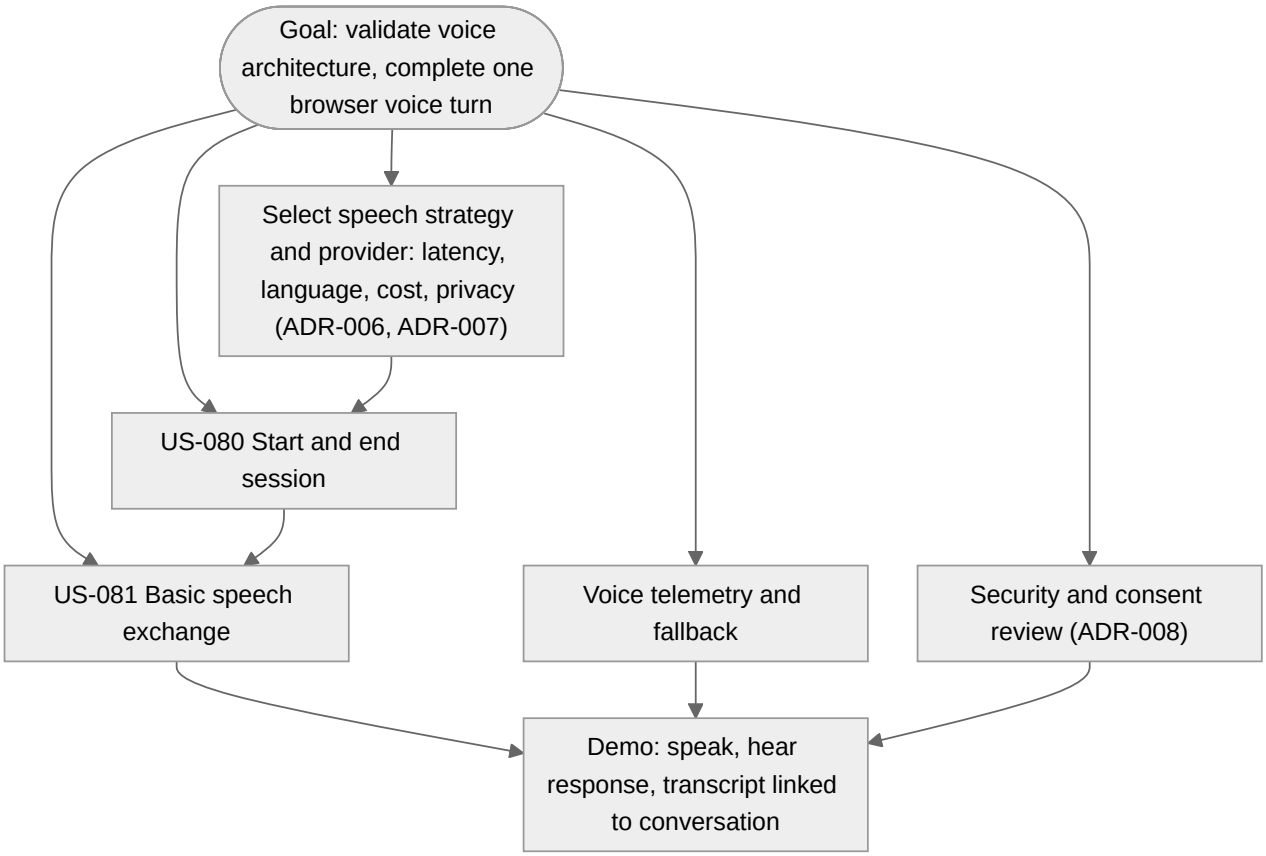


Figure 17.11: Sprint 7 detail

17.9 Sprint 8 — Voice synchronization and handoff

Sprint Goal: voice and text share a coherent conversation timeline.

- US-082 interruption
- US-083 modality switching
- US-084 voice handoff behavior for chosen provider
- Voice quality regression scenarios
- Cost/latency dashboard

Review demo: speak, interrupt, type a follow-up, and transfer context to human support.

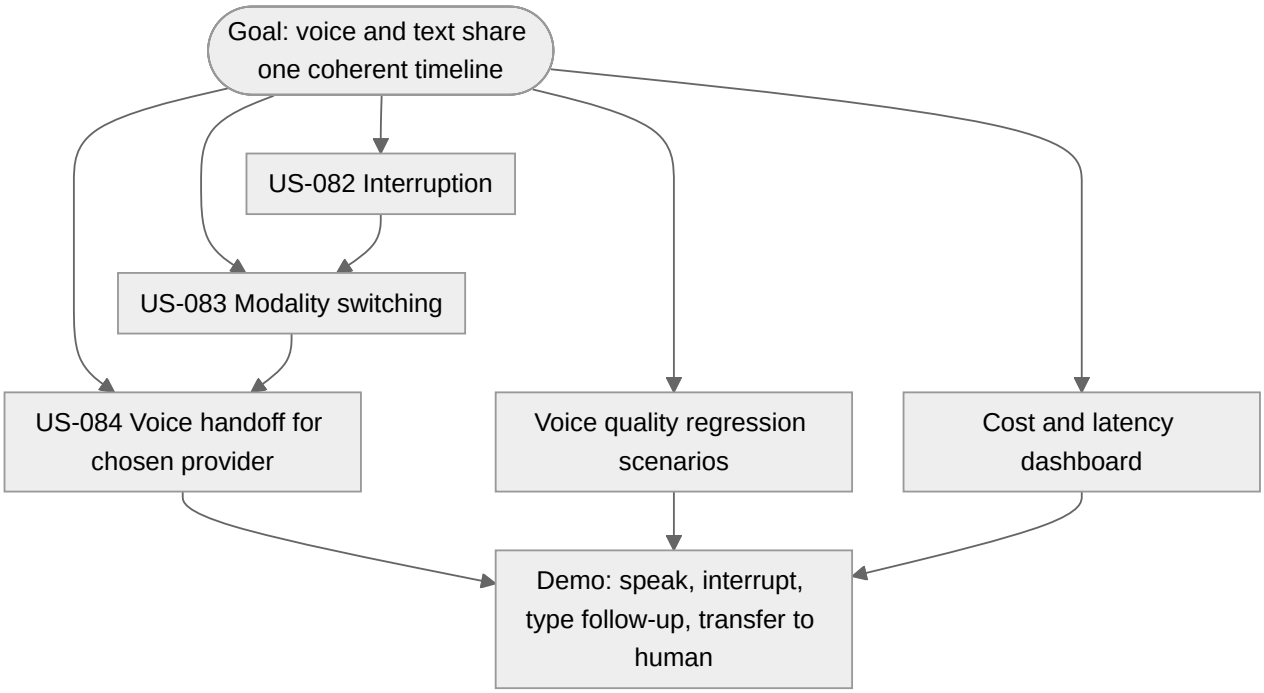


Figure 17.12: Sprint 8 detail

17.10 Later Sprints

- Additional channels/connectors
- Workflow builder, routing, SLA, notifications
- Advanced evaluation and release gates
- Usage plans, quotas and billing
- Additional organization administration and reporting

17.11 Planning caveats

- Team size and capacity are unknown; sprint count may change substantially.
- Voice and integration work often depend on vendor capabilities and credentials.
- Pull security, observability, and tests into each Sprint rather than deferring them.
- Each Sprint should end with a usable Increment, not only design documents.

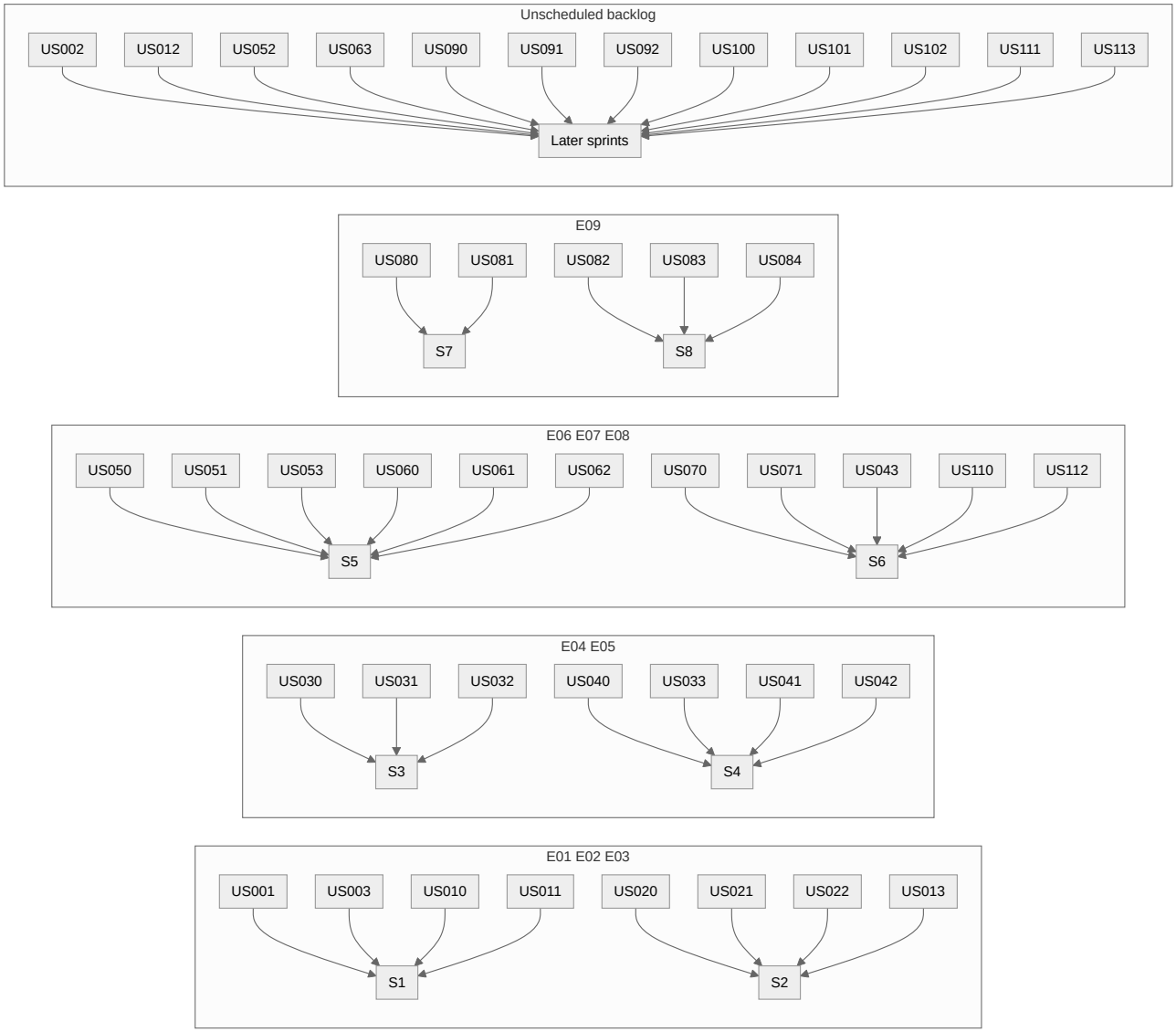


Figure 17.13: Story-to-Sprint traceability

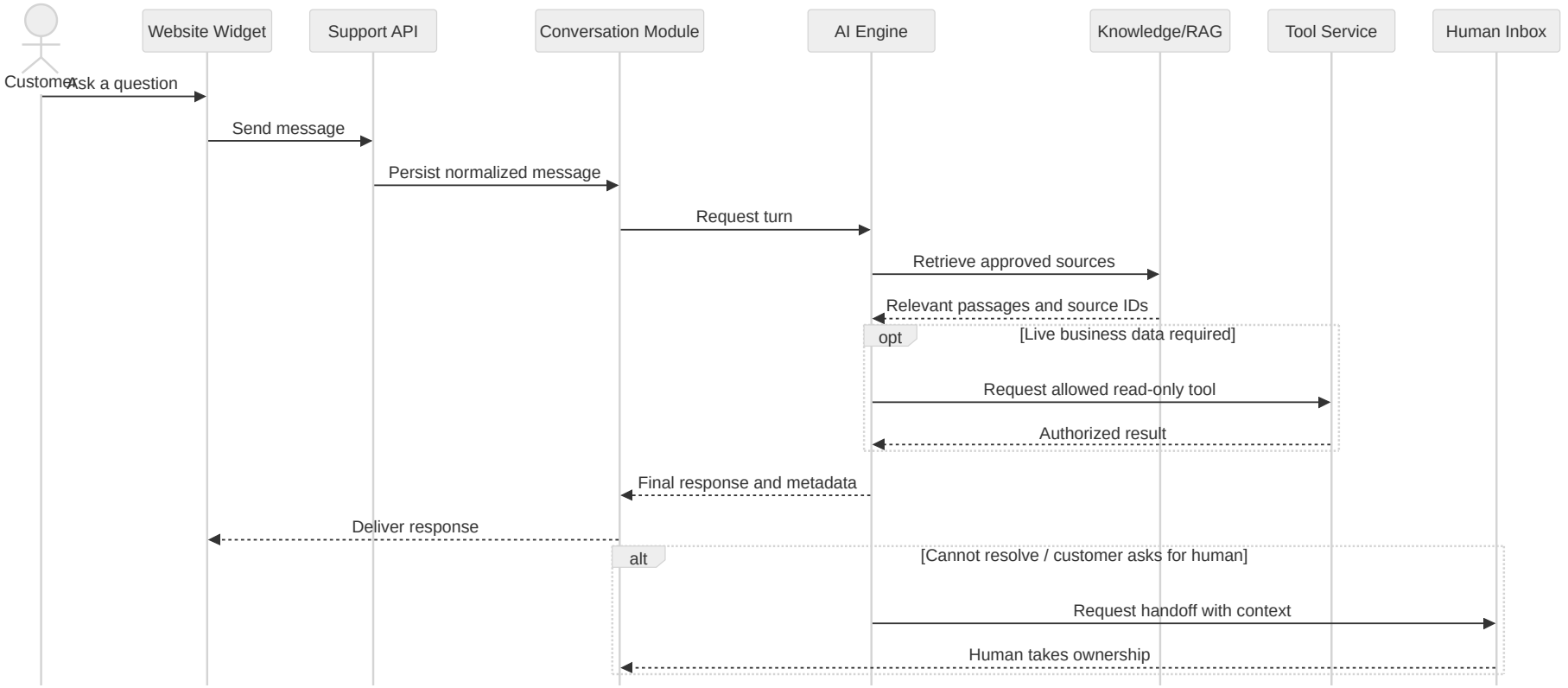


Figure 17.14: Sprint Review demo path for the Text MVP

Chapter 18

Definition of Ready and Definition of Done

18.1 Definition of Ready (team working agreement)

A backlog item is ready for Sprint Planning when:

- user/business outcome is clear;
- acceptance criteria are testable;
- dependencies and external-provider assumptions are identified;
- UX/API/data behavior is sufficiently understood;
- security/privacy implications have been considered;
- the item is small enough to complete in one Sprint (split it if not);
- the team has an estimate or agreed sizing approach;
- required credentials, test data, and environments are available or have a plan.

This is a team aid, not a gate that should block all discovery work.

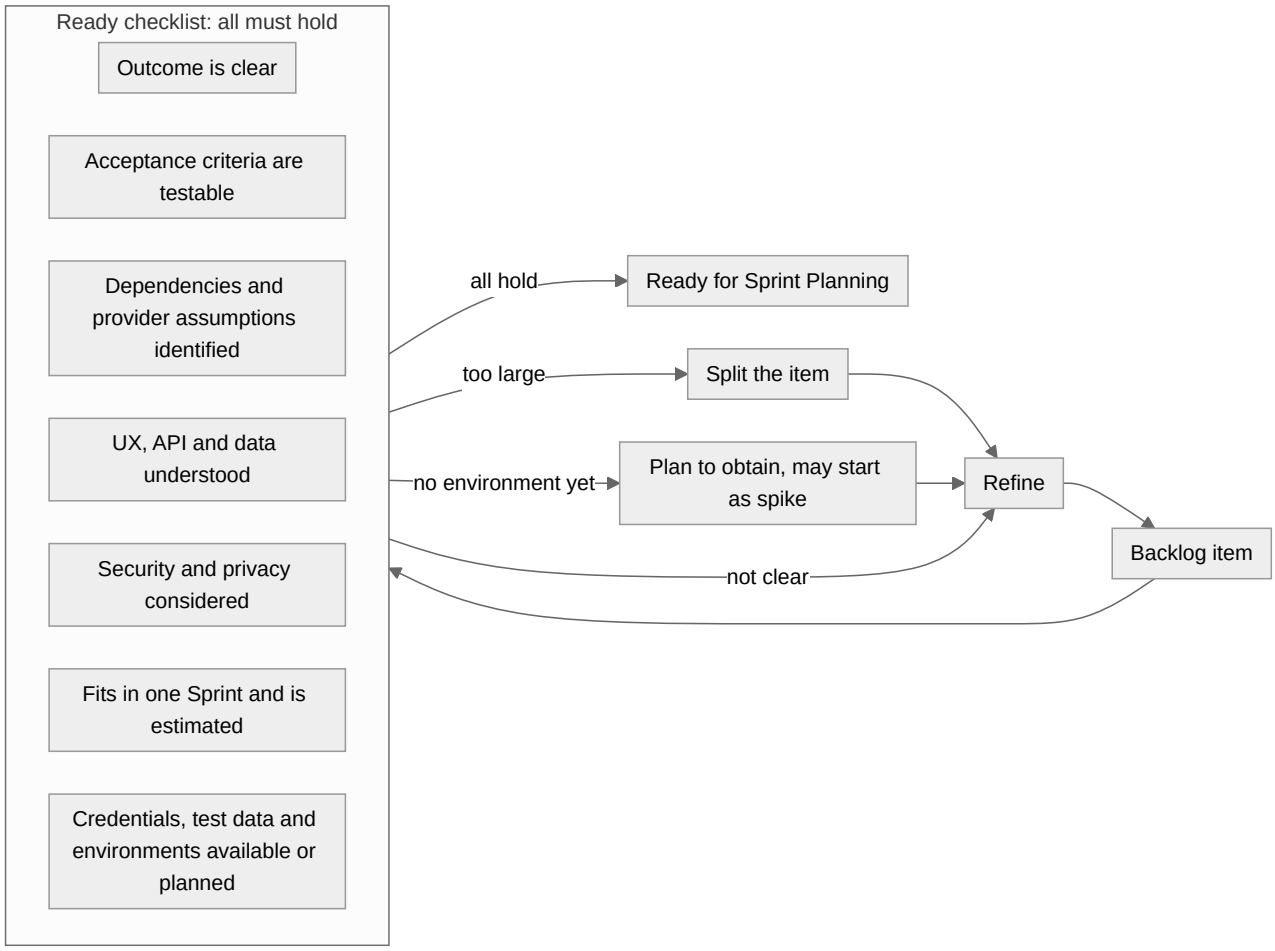


Figure 18.1: Definition of Ready check

18.2 Definition of Done

A Product Backlog Item is Done only when:

- implementation meets acceptance criteria;
- code is reviewed and follows module boundaries;
- automated tests cover relevant behavior and failure cases;
- tenant isolation and authorization are tested where relevant;
- input validation and error handling are implemented;
- database migrations are reviewed and repeatable;
- logs/metrics/traces include safe correlation fields;
- sensitive values are redacted;
- documentation/API contracts are updated;
- no known critical/high-severity defect remains without explicit risk acceptance;
- deployment/rollback or feature-flag behavior is understood;
- product owner can inspect the completed behavior.

18.3 AI-specific Done checks

- model and prompt/config version are identifiable;
- tool schemas are validated;
- tool permissions are enforced in code;
- retrieval is tenant-scoped;
- failure, empty retrieval, and uncertainty paths are tested;
- a regression example is added for important behavior;
- no secrets or hidden reasoning are persisted in logs.

18.4 Voice-specific Done checks

- microphone permission and active state are clear;
- session cleanup works on disconnect;
- interruption/cancellation behavior is tested;
- transcript finalization and ordering are tested;
- audio/transcript retention follows policy;
- latency and failure metrics are emitted.

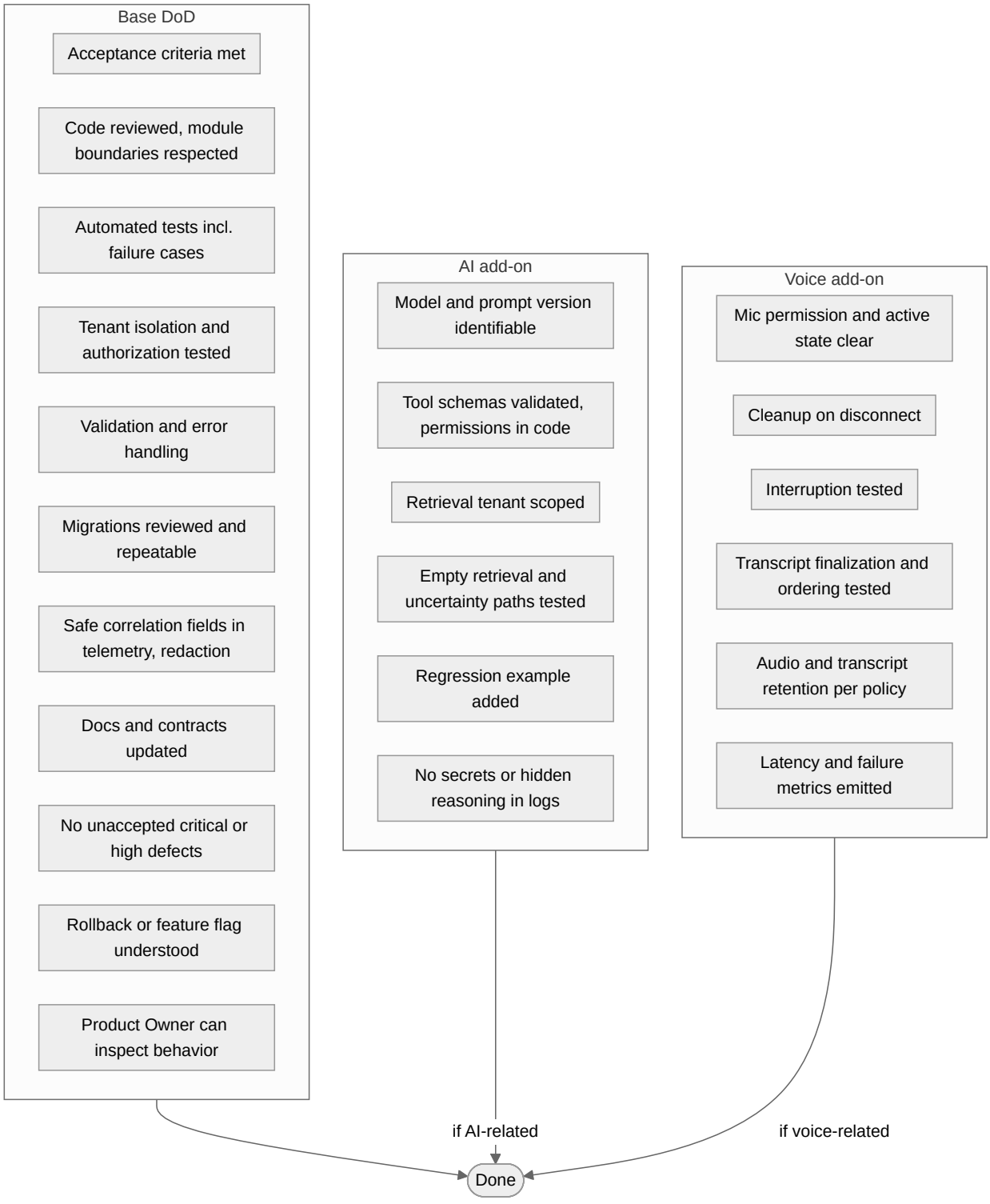


Figure 18.2: Definition of Done layers

Part IV

Architecture and Design Reference

Chapter 19

System Architecture

19.1 Recommended starting point

Use a modular monolith with:

- a single primary application deployment;
- strict module packages and interfaces;
- a relational database for transactional state;
- object storage for documents and permitted media;
- a queue/job runner for ingestion, outbound delivery, and other retryable work;
- a vector/search index for knowledge retrieval;
- provider adapters for LLMs, speech, channels, and business integrations.

Avoid creating a microservice for every module in the first release. Voice streaming may later need an independently scaled real-time component.

19.2 Logical layers

1. **Edge and channel adapters:** website widget, webhook adapters, voice transport, email/messaging adapters.
2. **Application API:** authentication, tenant resolution, validation, rate limiting.
3. **Conversation domain:** sessions, messages, turns, lifecycle, identity references.
4. **AI orchestration:** model routing, policy evaluation, retrieval, tool planning, response generation.
5. **Capability modules:** knowledge, business tools, workflow, tickets, human inbox.
6. **Infrastructure adapters:** SQL, object storage, vector index, queue, LLM provider, speech provider.
7. **Cross-cutting controls:** security, audit, observability, evaluation, usage metering.

19.3 Architecture constraints

- The core domain must not depend on a specific LLM vendor or messaging provider.
- Every request/job must have a trace/correlation ID and tenant context.
- Every tool execution must record actor, tenant, tool version, validated arguments (redacted where needed), outcome, and idempotency key.
- Durable state must be committed before acknowledging a webhook or inbound message when loss would be unacceptable.
- Use a transactional outbox when database changes must reliably publish events.
- External calls require timeouts, bounded retries, backoff, and idempotency where possible.
- Do not hold a database transaction open while waiting for an LLM or external provider.
- Streaming tokens/audio are not equivalent to a committed final message.

19.4 Suggested deployable components at MVP

- `support-api`: HTTP API and modular domain/application modules

- `support-worker`: background jobs and event consumers; may share code with API
- `support-web-widget`: lightweight customer-facing widget
- `support-agent-console`: human support and organization admin UI
- `postgres`: transactional database
- `object-storage`: uploaded source documents and approved attachments
- `queue`: durable asynchronous work
- `vector-search`: managed or self-hosted vector/search capability
- `telemetry`: logs, metrics, traces, alerts

These can initially be combined where appropriate. Keep logical ownership even if API and worker share a codebase.

19.5 System context

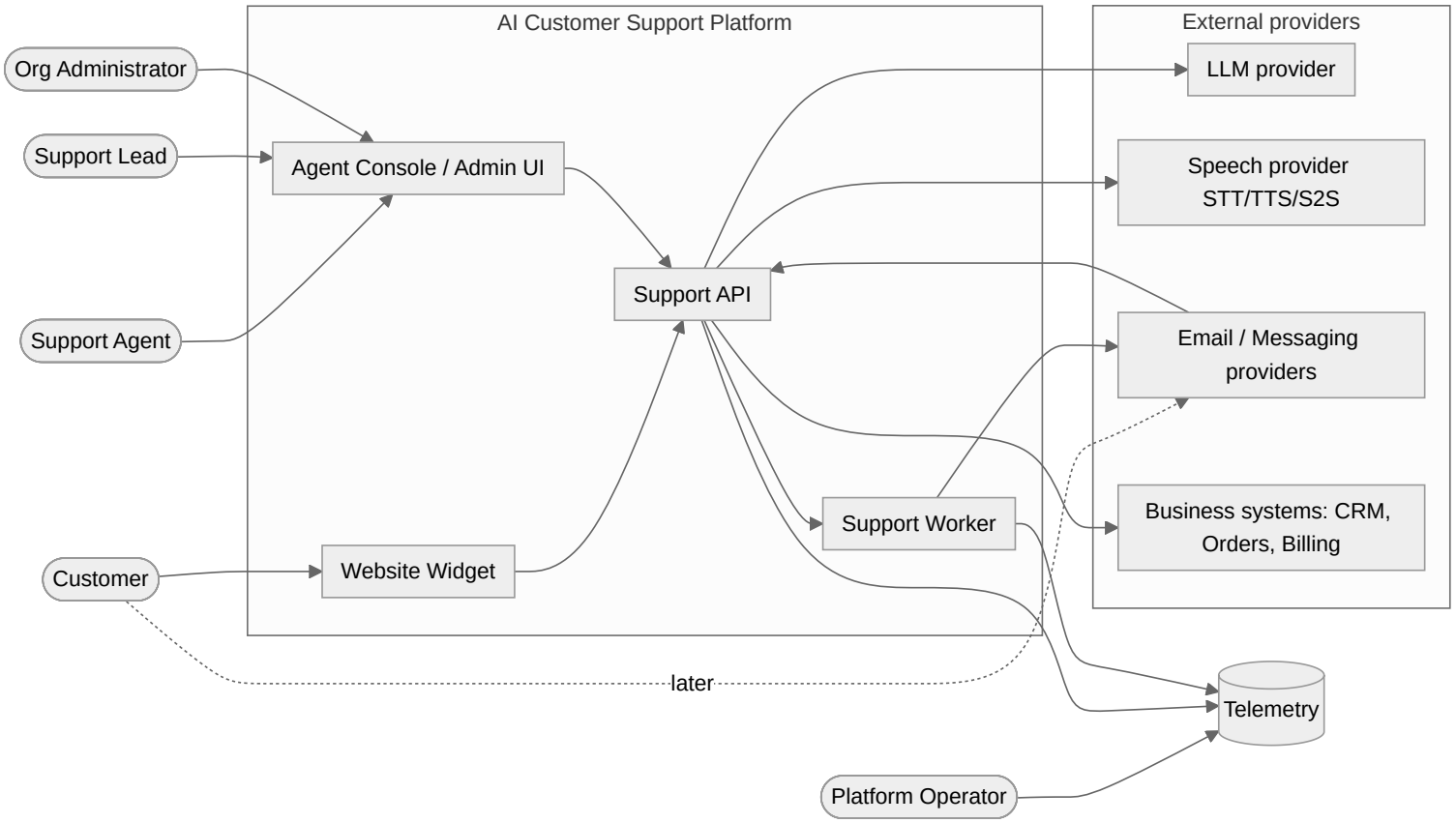


Figure 19.1: System context

19.6 Logical layers

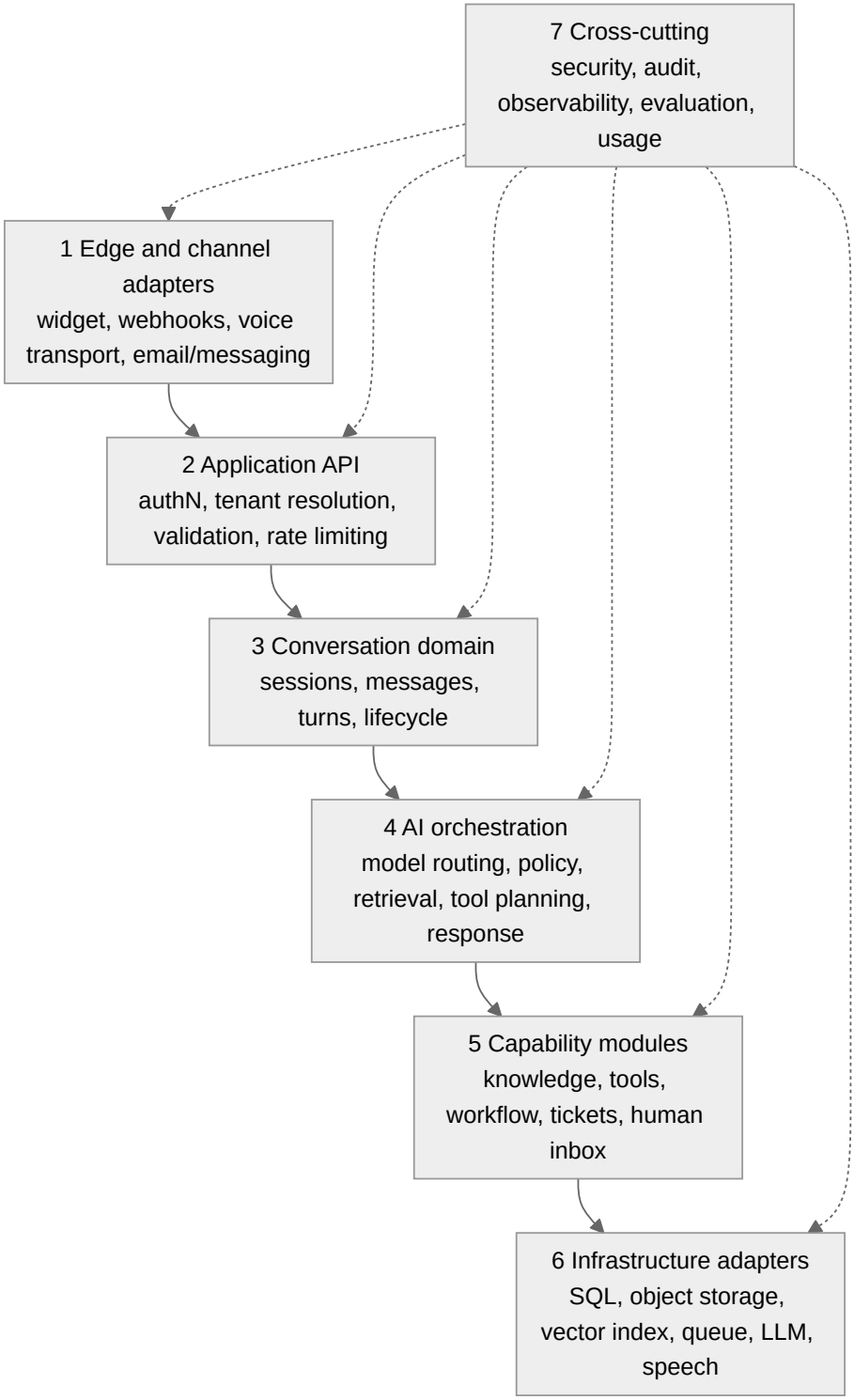


Figure 19.2: Logical layers

19.7 MVP deployable components

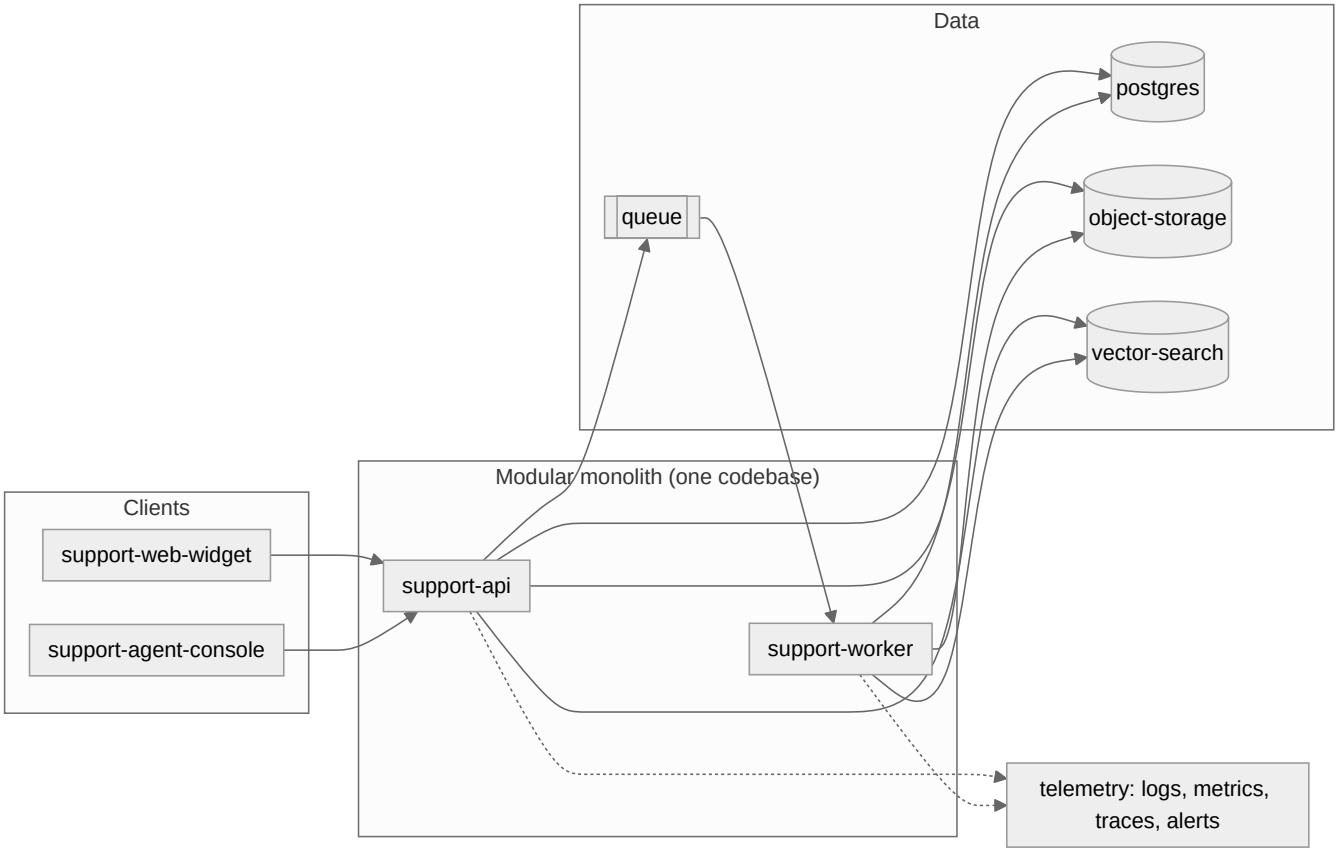


Figure 19.3: MVP deployable components

19.8 Module map — split by domain

19.8.1 A. Customer interaction and conversation (M01–M03)

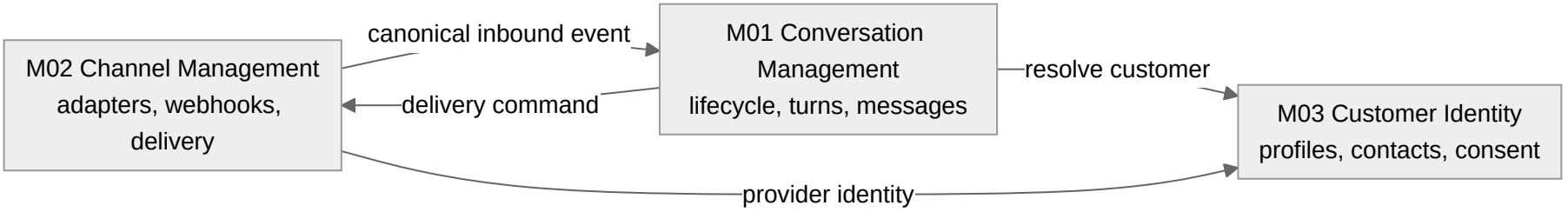


Figure 19.4: Module map — split by domain: A. Customer interaction and conversation (M01–M03)

19.8.2 B. AI conversation and voice (M04–M09)

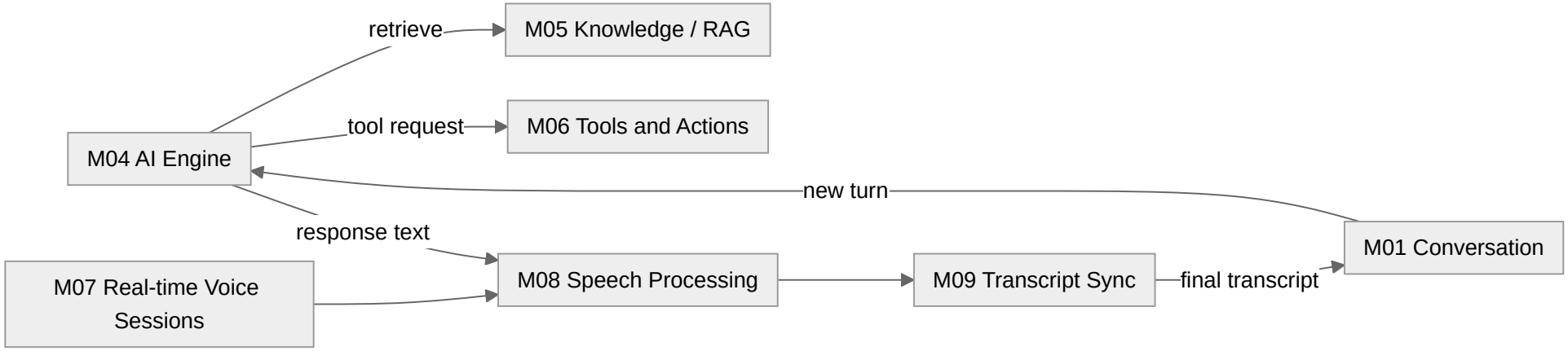


Figure 19.5: Module map — split by domain: B. AI conversation and voice (M04–M09)

19.8.3 C. Support operations (M10–M14)

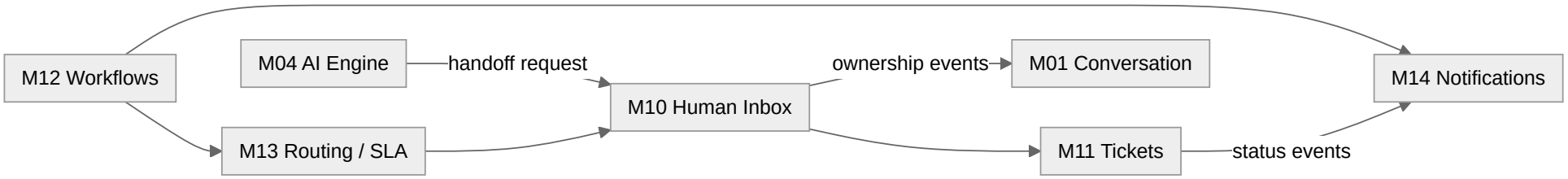


Figure 19.6: Module map — split by domain: C. Support operations (M10–M14)

19.8.4 D. Integrations and insights (M15–M17)

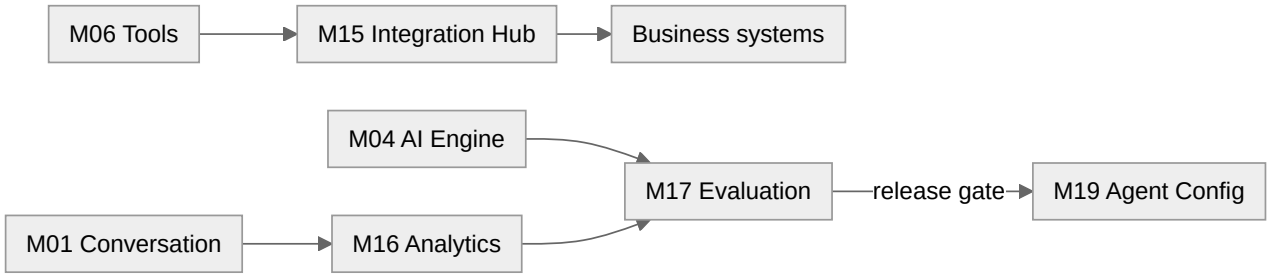


Figure 19.7: Module map — split by domain: D. Integrations and insights (M15–M17)

19.8.5 E. Platform and governance (M18–M20)

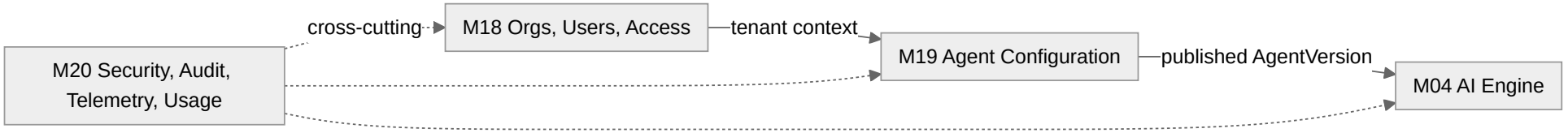


Figure 19.8: Module map — split by domain: E. Platform and governance (M18–M20)

19.9 Combined module dependency view

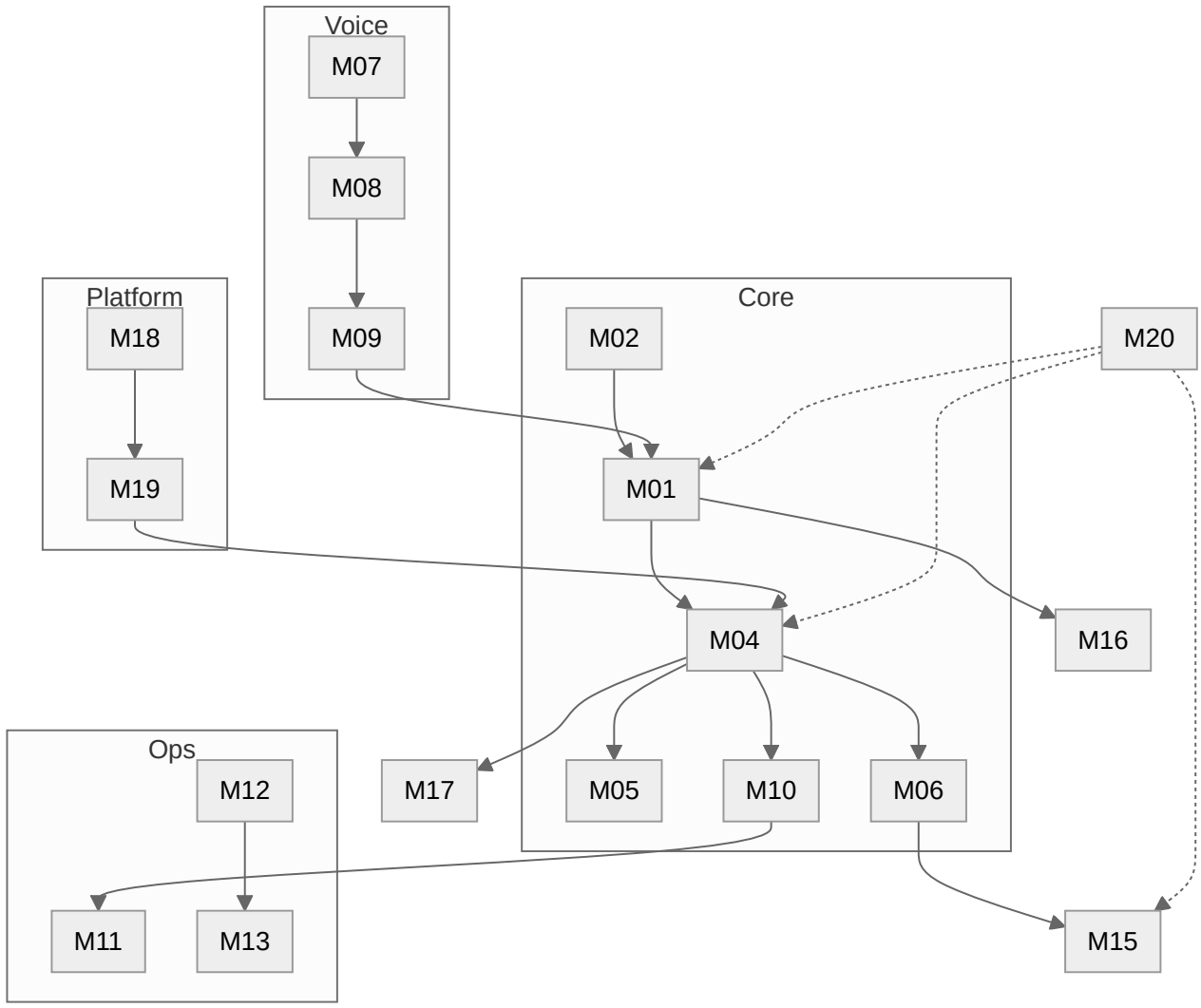


Figure 19.9: Combined module dependency view

19.10 Event catalog by lifecycle

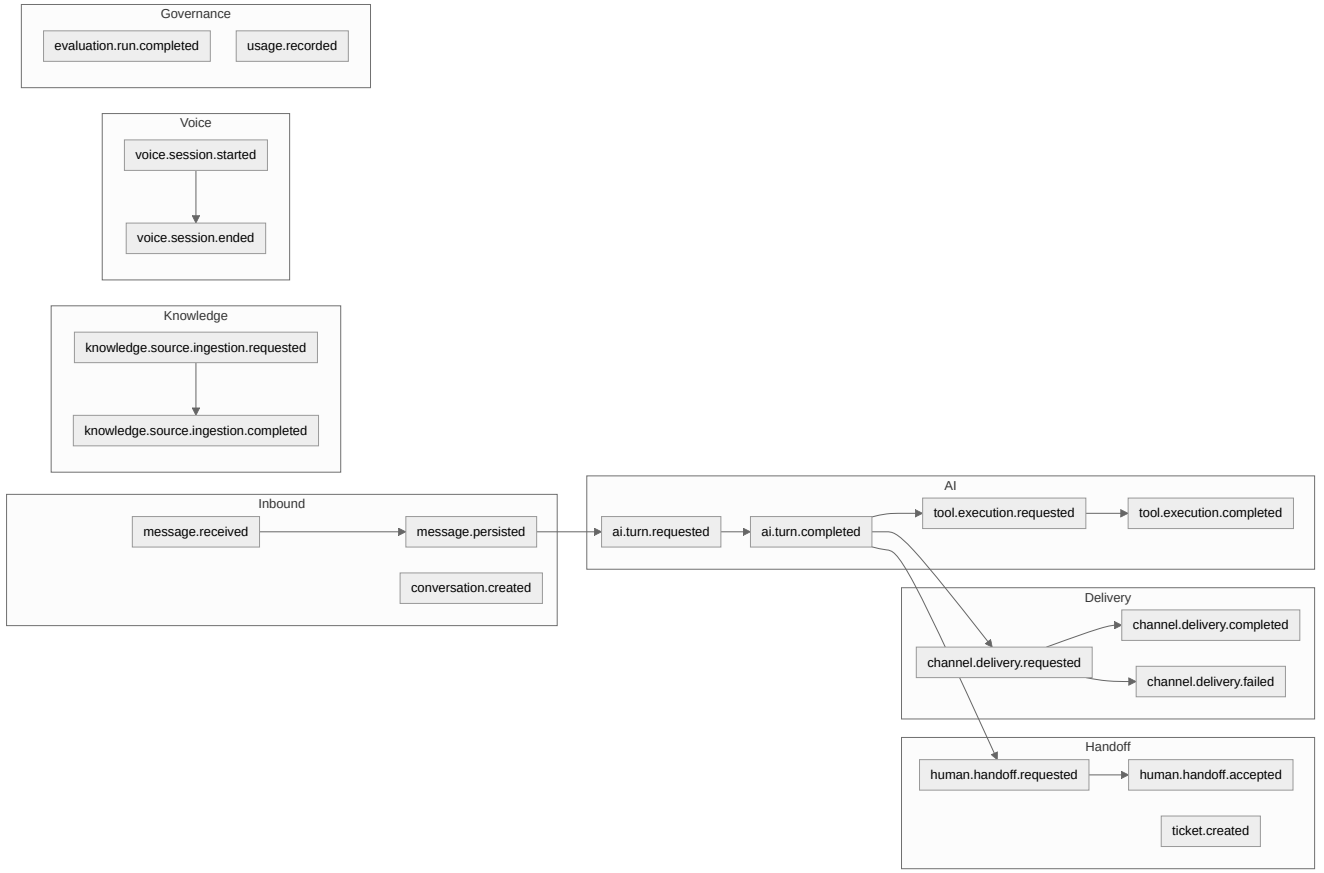


Figure 19.10: Event catalog by lifecycle

Chapter 20

Full Module Catalog

The module names describe logical ownership. They do not imply one deployable service per module.

20.1 A. Customer interaction and conversation

20.1.1 M01 — Conversation Management

Owns: conversation lifecycle, turns, participants, message ordering, status, links to cases, and event history.
Does not own: provider transport, AI reasoning, or business-system data.
Core records: Conversation, ConversationParticipant, ConversationTurn, Message, ConversationEvent.
Key interfaces: create/resume conversation; append inbound message; claim/release ownership; close/reopen; retrieve timeline.

20.1.2 M02 — Channel Management

Owns: channel connections, provider adapters, webhook intake, outbound delivery, provider identities, retries, and delivery state.
Core records: ChannelConnection, ChannelIdentity, InboundEvent, OutboundDelivery.
Key interfaces: receive provider event; normalize event; enqueue delivery; report delivery status.
Initial channel: website chat. Add email and messaging providers after the core flow is stable.

20.1.3 M03 — Customer Identity and Profiles

Owns: customer profile, contact points, identity linking, preferences, and consent records.
Core records: Customer, CustomerIdentity, CustomerContact, CustomerConsent.
Rule: a phone number or email address is not, by itself, proof that a person is authorized to access a protected account.

20.2 B. AI conversation and voice

20.2.1 M04 — AI Conversation Engine

Owns: turn orchestration, intent/context interpretation, policy-aware response planning, model selection, and decision trace.
Core records: AgentDefinition, AgentVersion, AgentExecution, ModelConfiguration.
May request: retrieval, tool execution, clarification, response generation, or human handoff.
Must not: bypass authorization or execute arbitrary model-generated code.

20.2.2 M05 — Knowledge Base and Retrieval

Owns: source registration, ingestion jobs, parsing, chunking, embeddings, indexing, retrieval, source versions, and source metadata.
Core records: KnowledgeSource, Document, DocumentVersion, DocumentChunk, RetrievalRun.
Requirements: tenant-scoped retrieval, source freshness, deletion propagation, and traceable citations where supported.

20.2.3 M06 — AI Tools and Business Actions

Owns: tool registry, schema validation, permission checks, execution, results, idempotency, approvals, and action audit.
Core records: ToolDefinition, ToolVersion, ToolExecution, ActionApproval.
Rule: model-selected arguments are untrusted. Validate inputs, customer identity, tenant, and authorization before executing.

20.2.4 M07 — Real-Time Voice Sessions

Owns: live audio session lifecycle, streaming transport, turn-taking signals, interruption handling, timeout, and reconnection.
Core records: VoiceSession, VoiceSessionEvent, AudioSegmentReference.
Transport: WebRTC is a strong browser-media candidate; WebSockets may suit text/control or some audio-streaming APIs. Telephony requires a separate provider adapter.

20.2.5 M08 — Speech Processing

Owns: speech provider abstraction, STT, TTS, speech-to-speech sessions, language/voice settings, and speech errors.
Core records: SpeechProvider, SpeechConfiguration, TranscriptionSegment, SpeechGeneration.
Design: support a modular STT → AI → TTS pipeline and optionally a direct speech-to-speech provider behind an interface.

20.2.6 M09 — Transcript and Multimodal Synchronization

Owns: durable transcript alignment, partial/final transcription state, text/voice message correlation, event ordering, and modality switching.
Core records: TranscriptSegment, ModalityEvent, MessageMediaReference.
Rule: ephemeral partials/audio chunks should not automatically become durable customer-visible messages.

20.3 C. Support operations and automation

20.3.1 M10 — Human Support Inbox and Handoff

Owns: agent queues, assignment, human takeover, AI pause/resume, handoff context, internal notes, and handoff outcome.
Core records: SupportAgent, SupportQueue, Assignment, HandoffEvent.
Handoff bundle: transcript, summary, customer context permitted for the agent, previous tool actions, unresolved questions, and reason for escalation.

20.3.2 M11 — Ticket and Case Management

Owns: durable support work item, status, priority, category, assignment, comments, due dates, and resolution.
Core records: Ticket, TicketComment, TicketStatusHistory.
Rule: a conversation is an interaction; a ticket is work to resolve. One ticket may link to multiple conversations.

20.3.3 M12 — Workflow Automation

Owns: event-triggered business workflows, steps, conditions, schedules, retries, approvals, and execution history.
Core records: WorkflowDefinition, WorkflowVersion, WorkflowRun, WorkflowStepRun.
Rule: use deterministic workflows for business-critical procedures; AI can classify or propose, while the workflow enforces the procedure.

20.3.4 M13 — Routing, Policies and SLAs

Owns: routing rules, queue selection, business hours, response/resolution targets, escalation policy, and fallback behavior.
Core records: RoutingRule, SlaPolicy, BusinessSchedule, EscalationPolicy.

20.3.5 M14 — Notifications and Follow-Ups

Owns: operational/customer notification templates, preferences, dispatch, delivery state, and retry.
Core records: Notification, NotificationTemplate, NotificationDelivery.
Boundary: notification delivery is separate from an ordinary conversational response.

20.4 D. Business integrations and insights

20.4.1 M15 — Integration Hub

Owns: connector definitions, credentials references, API/webhook mapping, health checks, rate limits, retries, and external event normalization.
Core records: Integration, IntegrationCredentialReference, ConnectorDefinition, IntegrationEvent.
Rule: credentials are stored in a secret manager or protected encrypted store, never in plain logs or model prompts.

20.4.2 M16 — Analytics and Reporting

Owns: operational/product metrics, conversation outcomes, channel usage, SLA reporting, voice latency, and cost summaries.
Core records: MetricEvent, ConversationOutcome, Feedback, UsageAggregate.

20.4.3 M17 — AI Quality, Evaluation and Improvement

Owns: evaluation datasets, test runs, quality scores, regression comparison, human review, and release gates.
Core records: EvaluationDataset, EvaluationRun, EvaluationResult, AgentRelease.
Evaluates: correctness, grounding, retrieval, tool choice, policy adherence, refusal/escalation, latency, and voice quality.

20.5 E. Platform administration and governance

20.5.1 M18 — Organizations, Users and Access Control

Owns: tenant, memberships, roles, permissions, service identities, invitations, and tenant context.
Core records: Organization, User, Membership, Role, Permission.
Rule: enforce tenant boundaries in application authorization and data access, not only in the UI.

20.5.2 M19 — Agent Configuration and Administration

Owns: agent instructions, persona, model/speech configuration references, knowledge assignments, tool allowlist, channel mapping, versions, publication, and rollback.
Core records: AgentConfiguration, AgentVersion, AgentDeployment, AgentPolicy.

20.5.3 M20 — Security, Audit, Observability and Usage

Owns: audit records, retention policy, abuse controls, application telemetry conventions, provider usage, quotas, and usage metering.
Core records: AuditEvent, UsageRecord, ProviderUsage, RetentionPolicy, SecurityEvent.
Note: billing/subscription management can be a separate module if commercial plans are introduced.

20.6 Shared technical capabilities

These are not necessarily standalone business modules:

- API gateway / edge rate limiting
- Background job runner and durable queue
- Transactional outbox/inbox
- Object storage for approved attachments and documents
- Search/vector index
- Secret manager
- Configuration and feature flags
- Structured logs, metrics, and distributed tracing
- Database migrations and tenant-aware repository patterns

Chapter 21

Module Communication and Contracts

21.1 Communication principles

1. Use direct application interfaces for short, local, request/response operations.
2. Use asynchronous jobs/events for slow, retryable, or externally dependent operations.
3. Use durable event records for inbound webhooks and outbound deliveries.
4. Keep public provider payloads at the adapter boundary; convert them to canonical internal contracts.
5. Never use an LLM prompt as the only interface contract between modules.
6. Carry `tenant_id`, `correlation_id`, `causation_id`, and `occurred_at` on internal events.
7. Version public/event contracts and make consumers tolerant of additive fields.
8. Do not expose internal-only endpoints publicly.

21.2 Synchronous interactions

Caller	Callee	Purpose	Contract expectations
Channel adapter	Conversation application service	Normalize and append inbound message	Idempotent on provider event ID
Conversation orchestrator	AI engine	Process a new turn	Bounded timeout; trace ID
AI engine	Knowledge module	Retrieve relevant chunks	Tenant/source filters; citations/IDs
AI engine	Tool execution module	Request a named tool	Schema validation and policy check
Tool module	Business connector	Execute an allowed operation	Timeout, idempotency, typed result
Admin API	Agent configuration	Publish agent version	Immutable version; audit record
Human inbox	Conversation module	Claim/take over conversation	Optimistic concurrency or explicit lock

21.3 Asynchronous events/jobs

Suggested event names:

- `conversation.created`
- `message.received`
- `message.persisted`
- `ai.turn.requested`
- `ai.turn.completed`
- `tool.execution.requested`
- `tool.execution.completed`
- `human.handoff.requested`
- `human.handoff.accepted`

- `ticket.created`
- `knowledge.source.ingestion.requested`
- `knowledge.source.ingestion.completed`
- `channel.delivery.requested`
- `channel.delivery.completed`
- `channel.delivery.failed`
- `voice.session.started`
- `voice.session.ended`
- `evaluation.run.completed`
- `usage.recorded`

Events communicate facts that have happened. Commands request work. Do not use event names that imply success before the work has succeeded.

21.4 Canonical inbound message envelope

```
{
  "event_id": "provider-or-internal-unique-id",
  "tenant_id": "tenant-id",
  "channel_connection_id": "channel-id",
  "provider": "website-chat",
  "provider_event_id": "provider-event-id",
  "conversation_id": "conversation-id-or-null",
  "customer_identity": {
    "provider_identity_id": "external-identity-id",
    "display_name": "Customer-provided-name-or-null"
  },
  "message": {
    "message_id": "internal-message-id",
    "direction": "inbound",
    "kind": "text",
    "text": "Where is my order?",
    "attachments": []
  },
  "occurred_at": "2026-01-01T12:00:00Z",
  "correlation_id": "trace-or-correlation-id"
}
```

This is illustrative. Production contracts should define schemas, limits, optional fields, and validation rules. Never trust a provider-supplied tenant ID without resolving the configured channel connection.

21.5 Tool invocation envelope

```
{
  "tool_execution_id": "execution-id",
  "tenant_id": "tenant-id",
  "conversation_id": "conversation-id",
  "agent_version_id": "agent-version-id",
  "tool_name": "orders.lookup",
  "arguments": {
    "order_reference": "ORDER-123"
  },
  "requested_by": "ai-agent",
  "idempotency_key": "stable-operation-key",
  "correlation_id": "correlation-id"
}
```

The tool service must derive authorization from trusted application context. It must not accept model-generated claims of identity or permissions.

21.6 Delivery state machine

Recommended outbound states: `queued` → `sending` → `sent` → `delivered` with side paths: `queued/sending` → `retry_wait` → `sending` and terminal or review states: `failed`, `cancelled`, `unknown`.

`sent` and `delivered` are distinct when the provider exposes a delivery receipt. An uncertain provider timeout must not automatically mean the message was not sent; retrying may duplicate it.

21.7 Reliability patterns

- **Webhook inbox:** persist provider event IDs and processing state to deduplicate replays.
- **Transactional outbox:** commit domain changes and pending event records in one transaction, then publish.
- **Idempotent commands:** use stable idempotency keys for external write operations.
- **Retry policy:** exponential backoff with jitter, maximum attempts, and dead-letter/review path.
- **Concurrency:** protect conversation ownership and turn sequencing.
- **Ordering:** order by conversation/stream key when the broker supports partitioning.
- **Timeouts:** bound all provider calls.
- **Circuit breaking:** stop repeatedly calling a failing dependency.
- **Compensation:** define recovery for workflows that span systems without distributed transactions.

21.8 Error contract

Return stable machine-readable error codes, safe user-facing messages, retryability, and correlation ID. Do not return stack traces, secret values, provider tokens, or internal prompts.

21.9 Cross-cutting flow diagrams

21.9.1 Inbound webhook handling

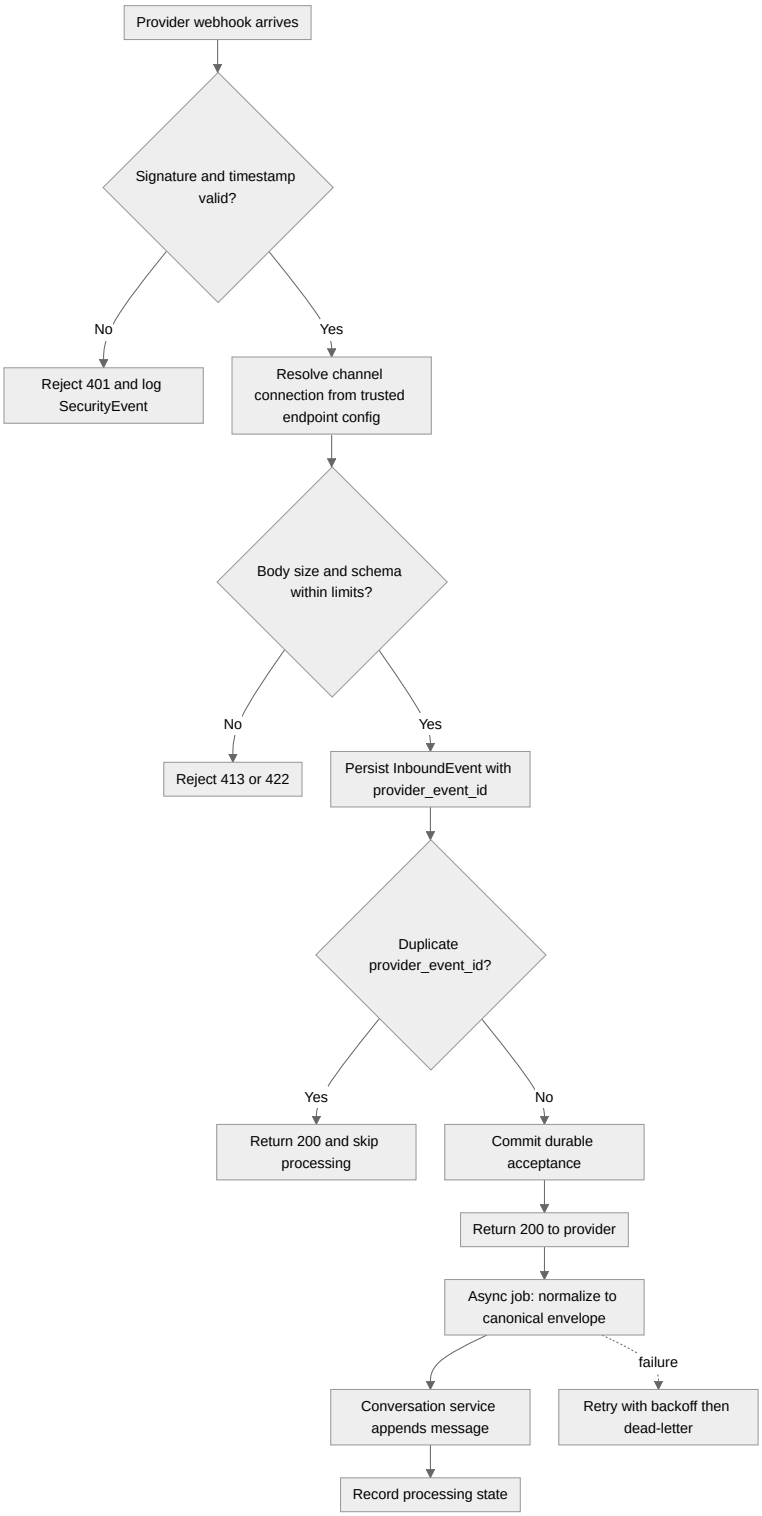


Figure 21.1: Inbound webhook handling

21.9.2 Transactional outbox and inbox

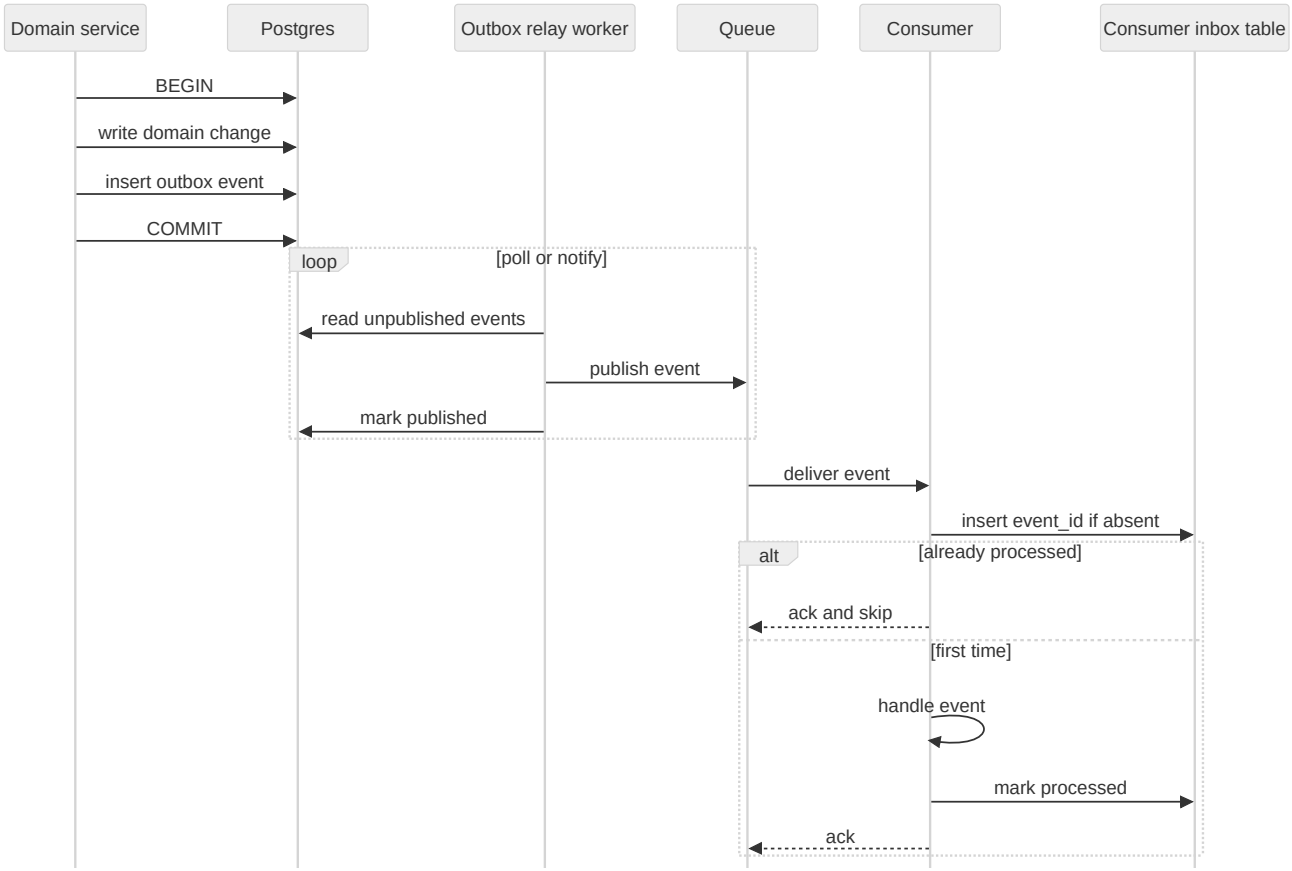


Figure 21.2: Transactional outbox and inbox

21.9.3 Outbound delivery state machine

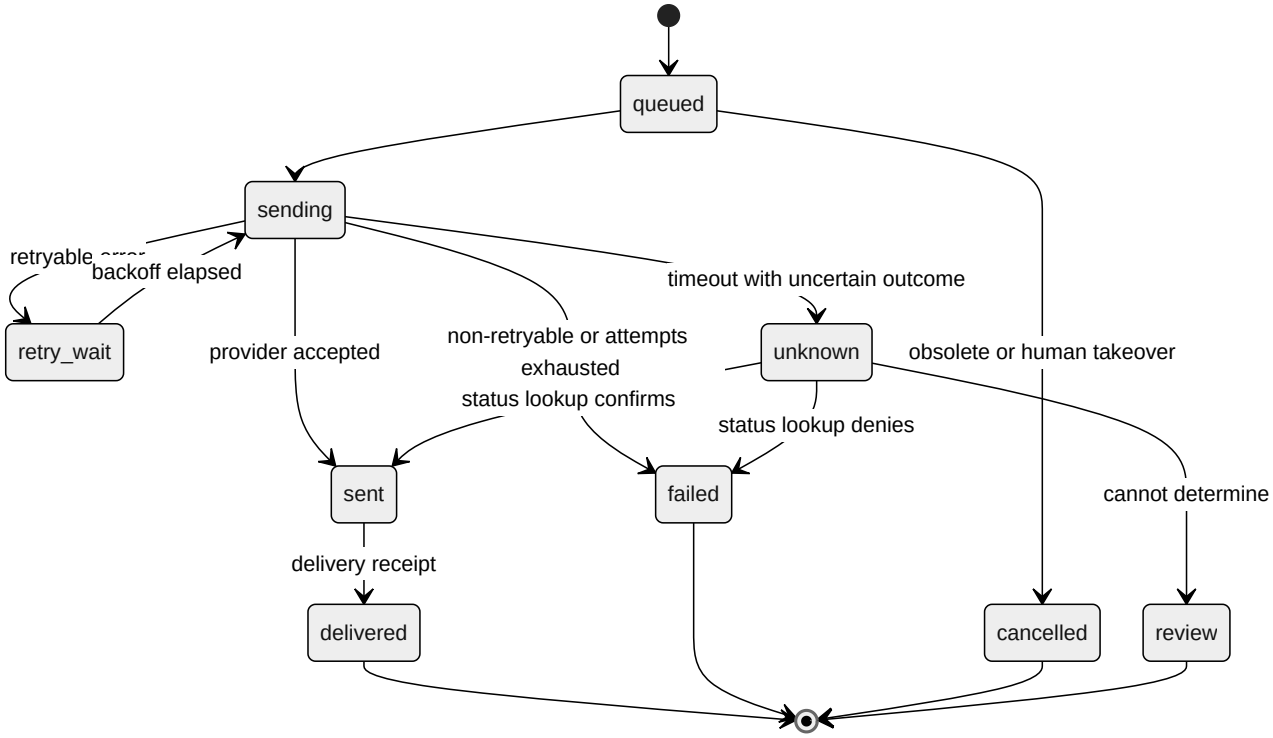


Figure 21.3: Outbound delivery state machine

21.9.4 Retry decision

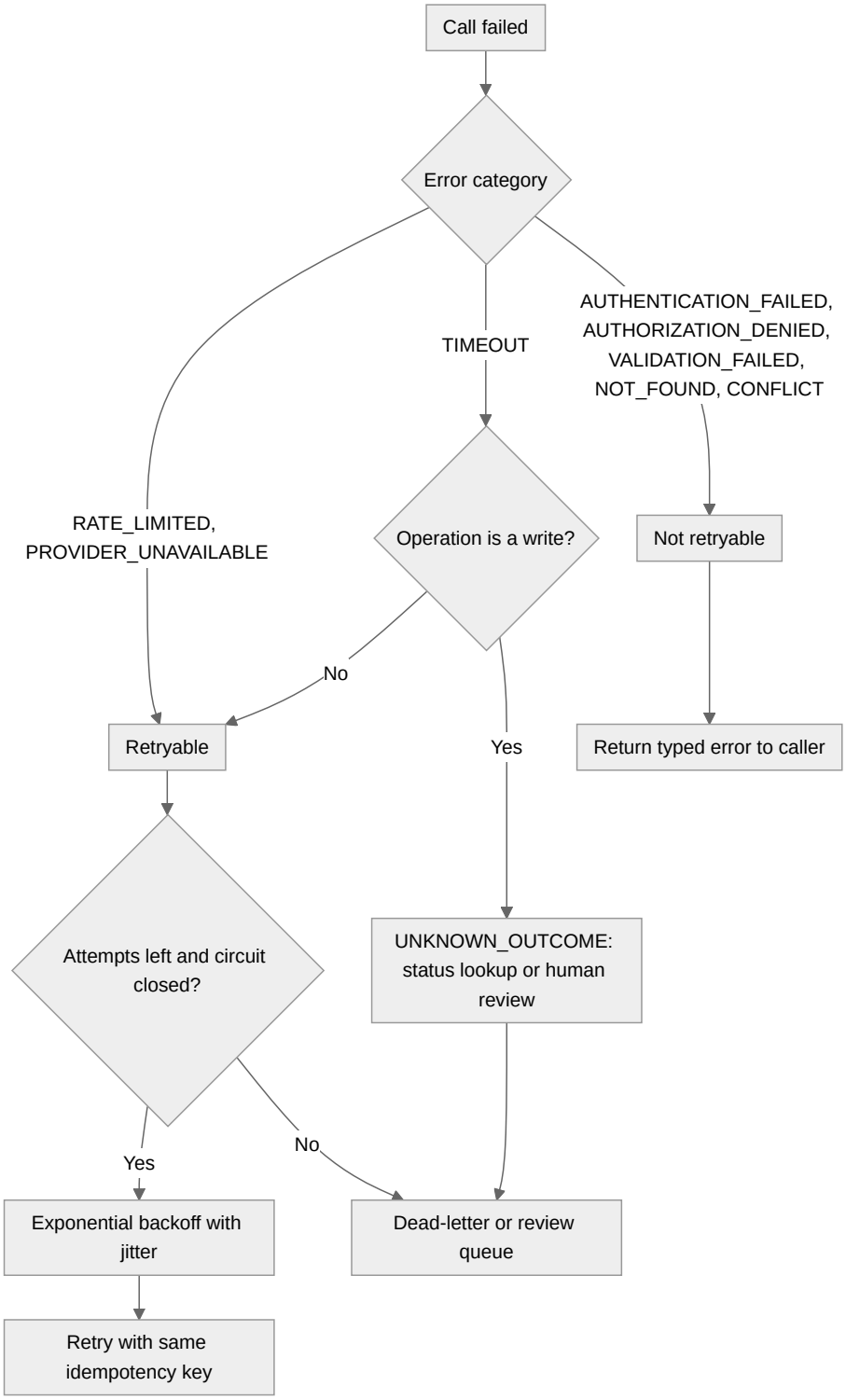


Figure 21.4: Retry decision

21.9.5 Tool execution — full pipeline (split in two parts)

21.9.5.1 Part A: validation and authorization

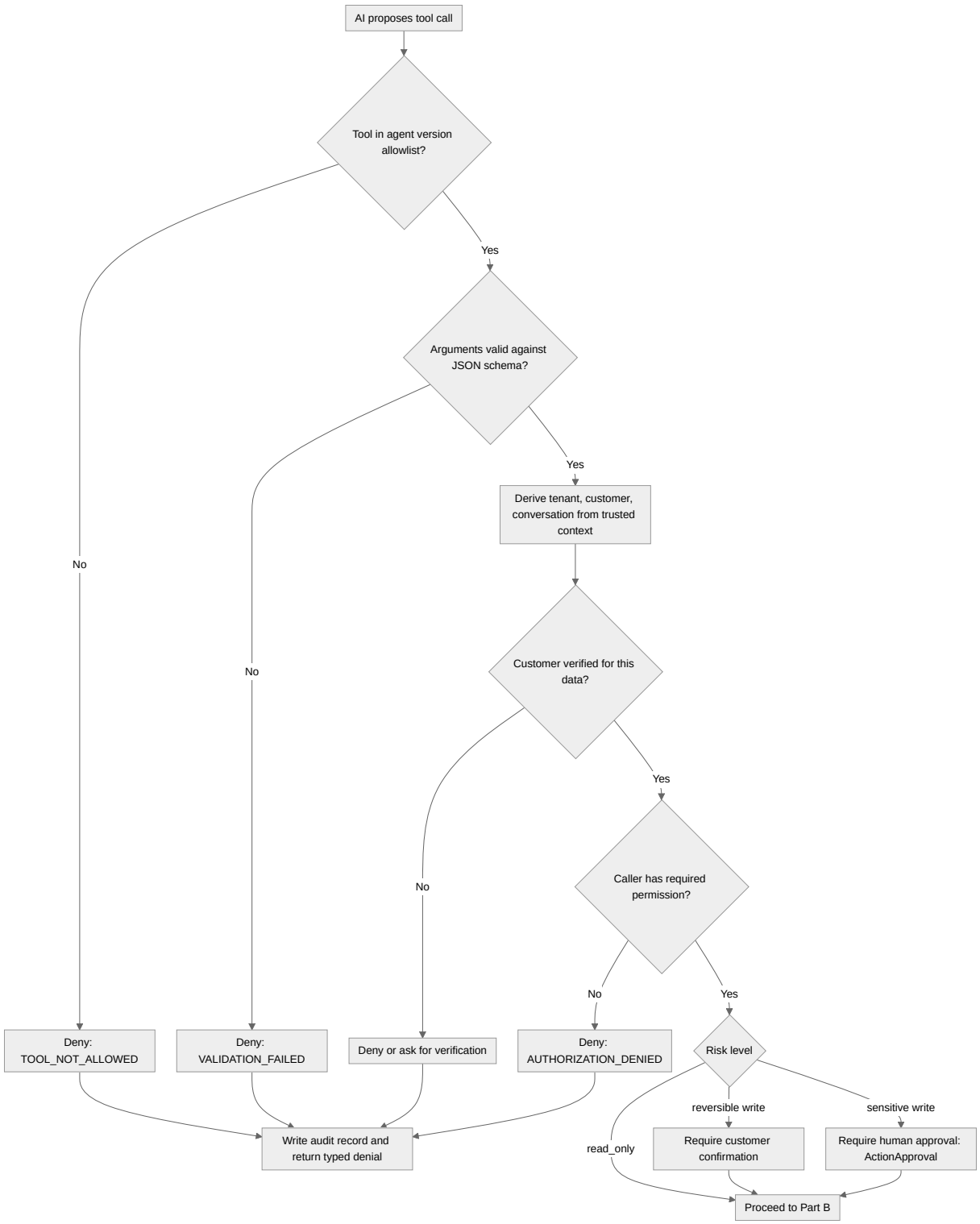


Figure 21.5: Tool execution — full pipeline (split in two parts): Part A: validation and authorization

21.9.5.2 Part B: execution and result handling

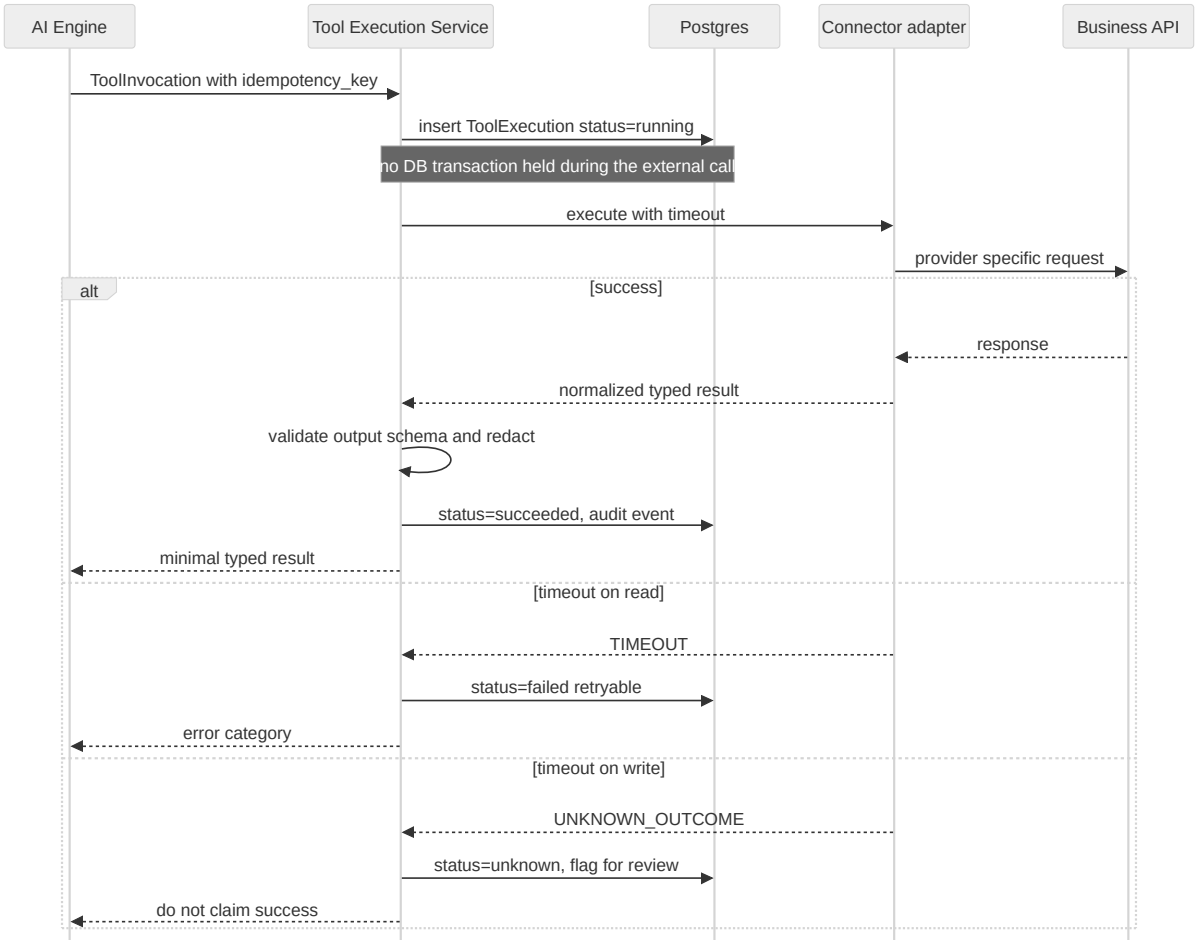


Figure 21.6: Tool execution — full pipeline (split in two parts): Part B: execution and result handling

21.9.6 Tool execution state machine

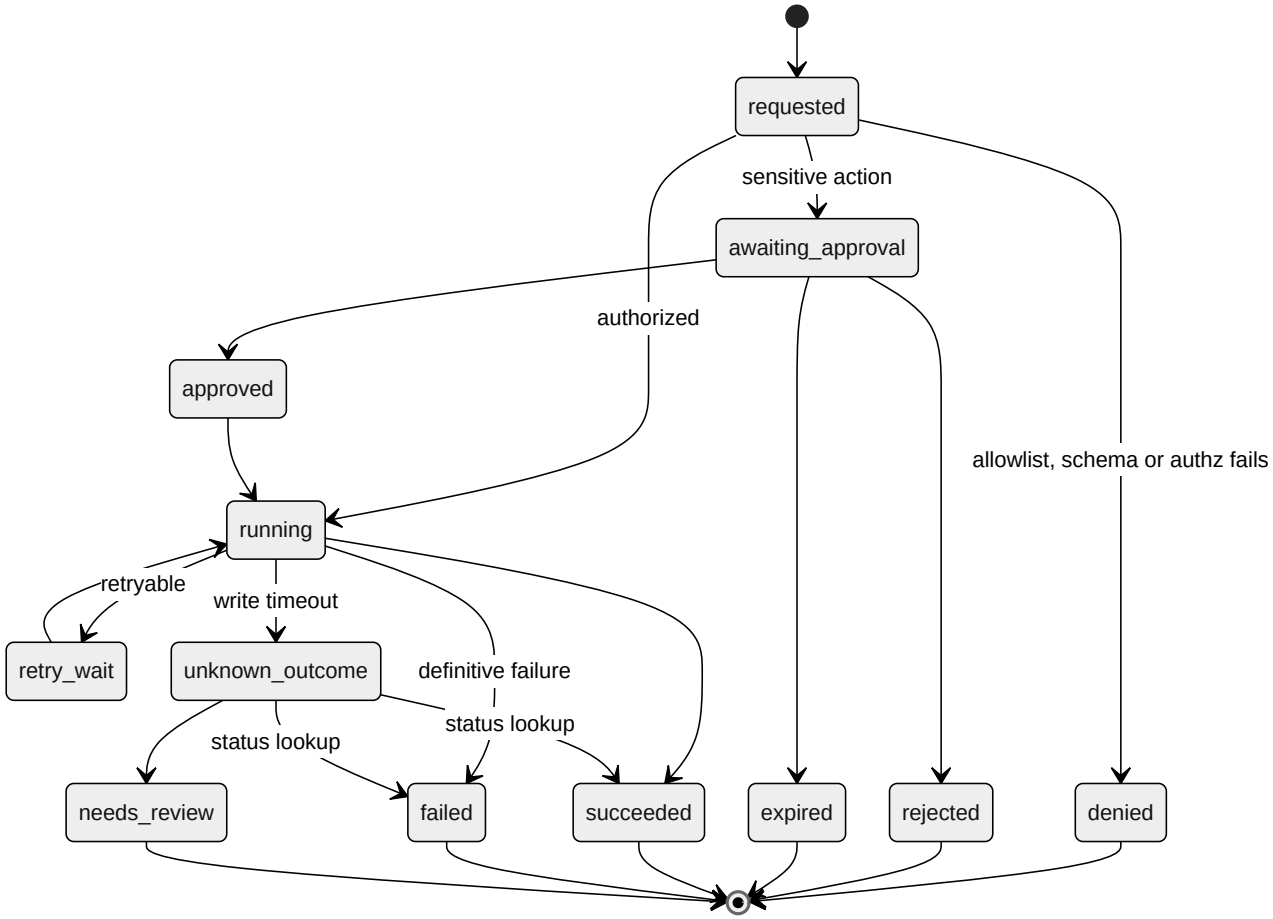


Figure 21.7: Tool execution state machine

21.9.7 Tenant resolution and isolation

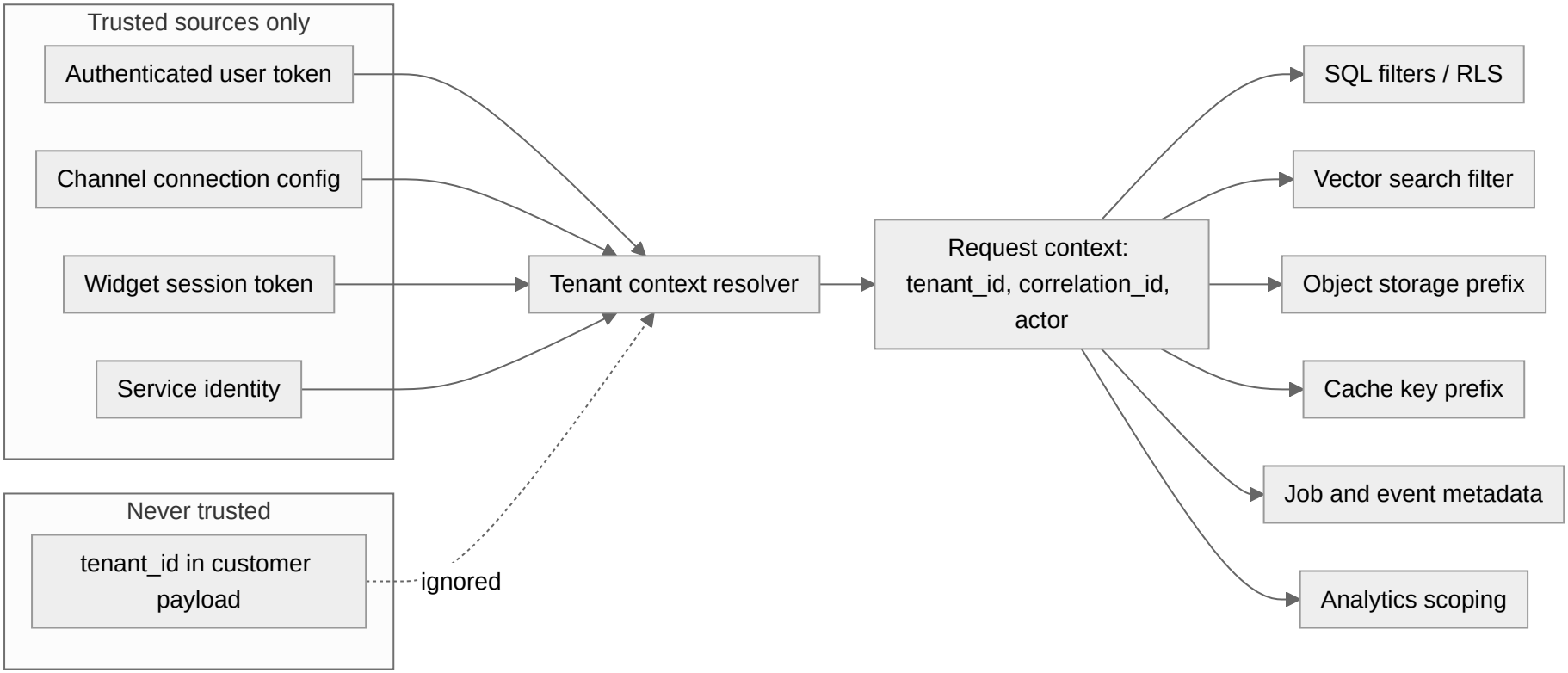


Figure 21.8: Tenant resolution and isolation

21.9.8 Secrets and credential flow

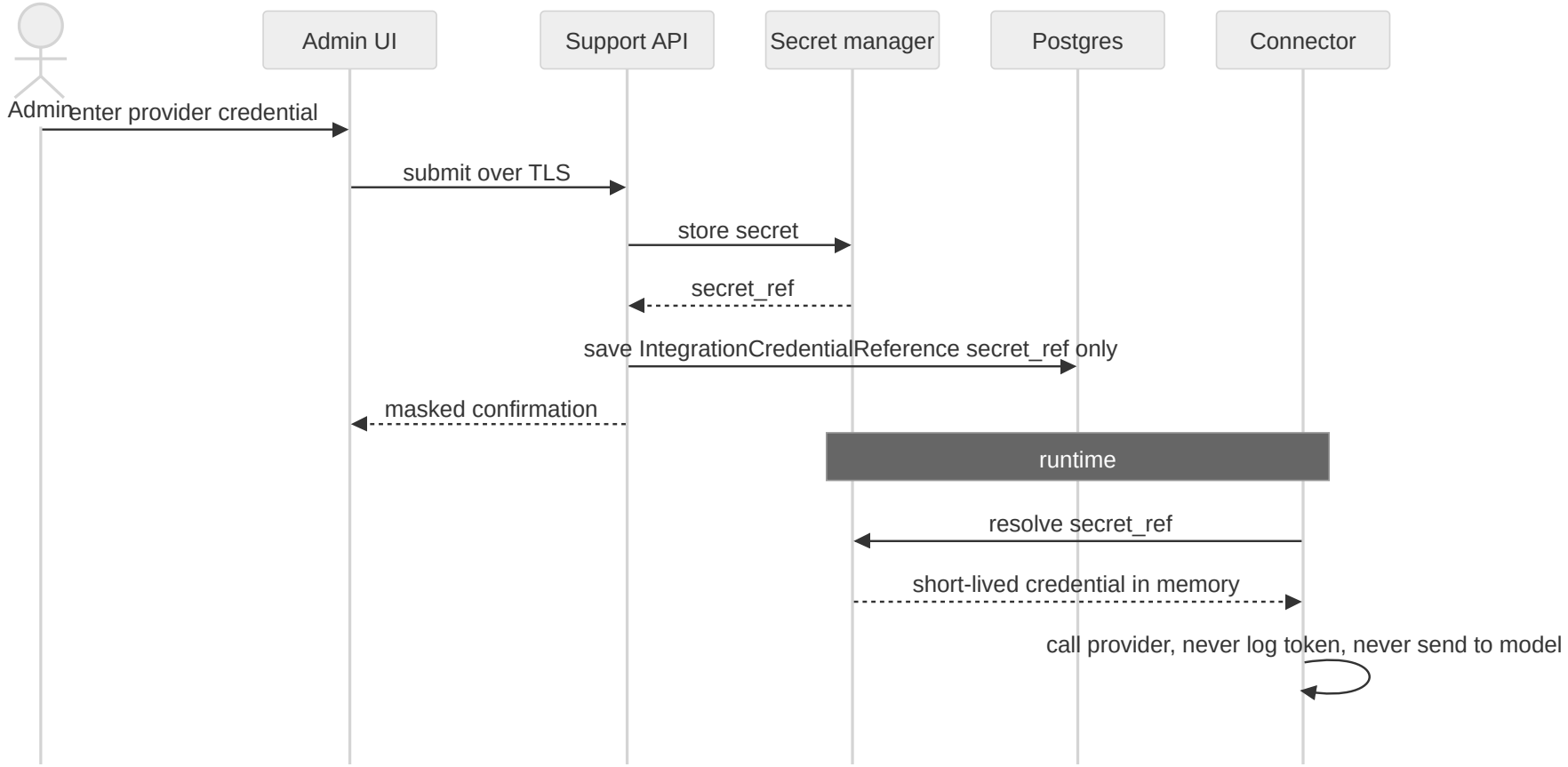


Figure 21.9: Secrets and credential flow

21.9.9 Graceful degradation

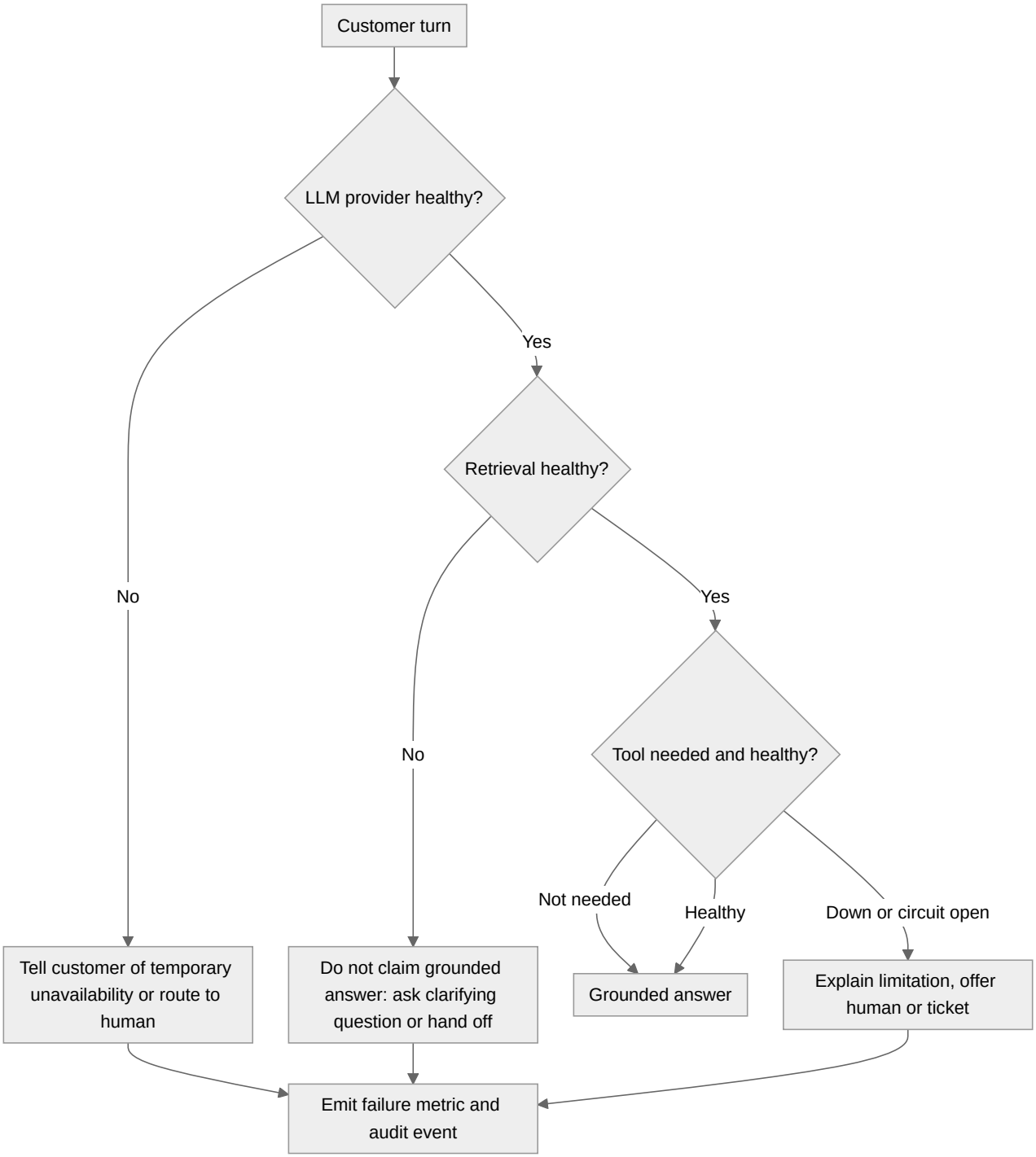


Figure 21.10: Graceful degradation

21.9.10 Data lifecycle and retention

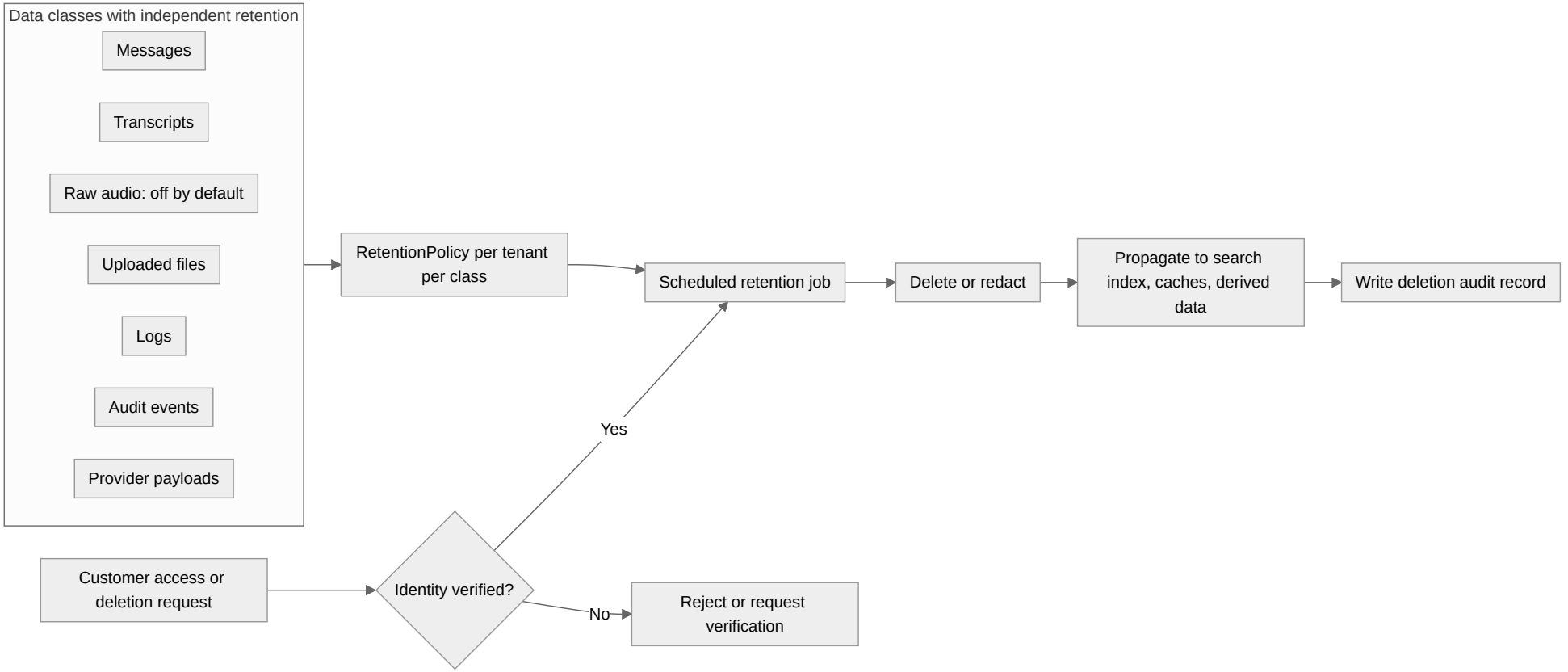


Figure 21.11: Data lifecycle and retention

21.9.11 Security test matrix

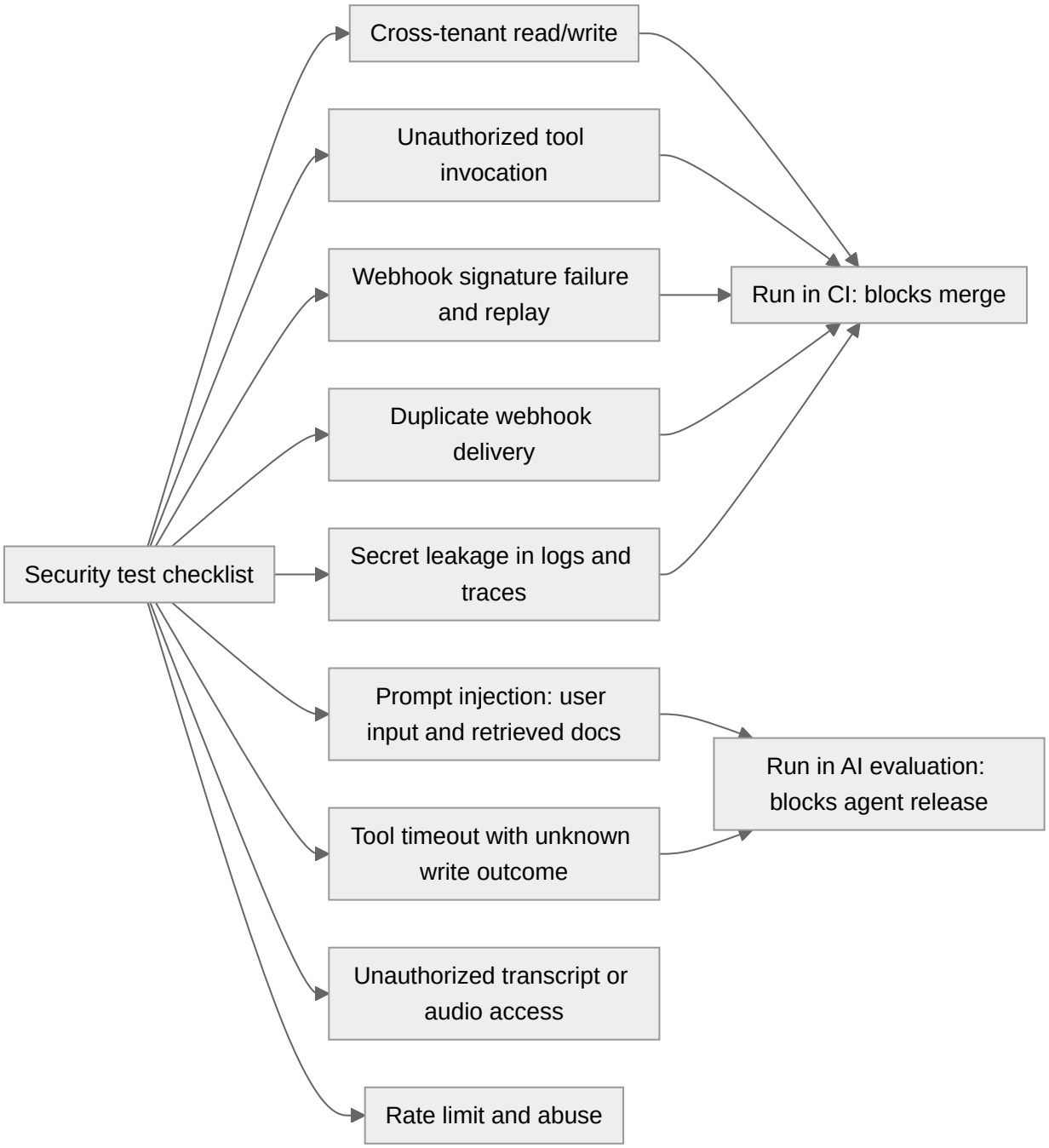


Figure 21.12: Security test matrix

Chapter 22

Domain Model

22.1 Core aggregate boundaries

- **Organization:** tenant boundary and configuration owner.
- **Conversation:** durable interaction and message timeline.
- **Customer:** profile and verified identity links.
- **Agent configuration/version:** published AI behavior and allowed capabilities.
- **Knowledge source/document version:** ingestion and retrieval lifecycle.
- **Tool execution:** authorization, idempotency, result, and audit.
- **Ticket:** support work item that can span conversations.
- **Voice session:** live media lifecycle linked to a conversation.
- **Workflow run:** deterministic multi-step execution.
- **Audit/usage event:** immutable record for governance and measurement.

22.2 Relationship overview

22.3 Data modeling rules

- All tenant-owned rows must include or be reachable through a trusted `organization_id`.
- Use internal UUIDs/ULIDs; keep provider IDs in separate fields with uniqueness scoped to provider/connection.
- Avoid using phone number/email as the primary key for a customer.
- Store timestamps in UTC; render local time at the edge.
- Keep external provider payloads only when needed for debugging/compliance and with explicit retention/redaction.
- Prefer immutable version rows for agent configuration, knowledge documents, workflows, and tool schemas.
- Keep messages, transcripts, audio references, and attachments distinct so retention/access policies can differ.
- Store object references rather than large binary payloads in the primary relational database.
- Record deletion and re-indexing propagation for knowledge sources.
- Do not store hidden chain-of-thought. Store concise decision metadata, tool calls, sources, and outcomes needed for debugging and audit.

22.4 Suggested message kinds

- `text`
- `image_reference`
- `file_reference`
- `audio_reference`
- `system_event`
- `internal_note` (not customer-visible)

- `transcript_segment` (if transcript is modeled in the message timeline)

Use explicit visibility and direction fields rather than relying on message text conventions.

22.5 Data model diagrams

22.5.1 Aggregate map

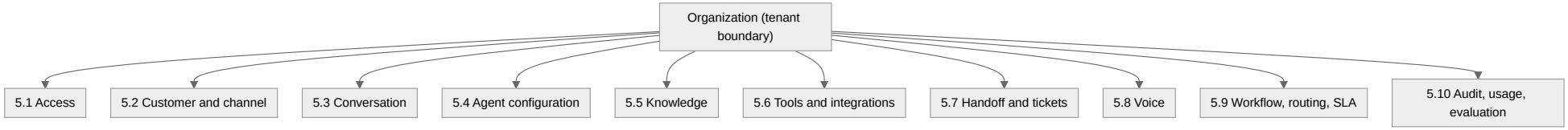


Figure 22.1: Aggregate map

22.5.2 Organizations, users and access

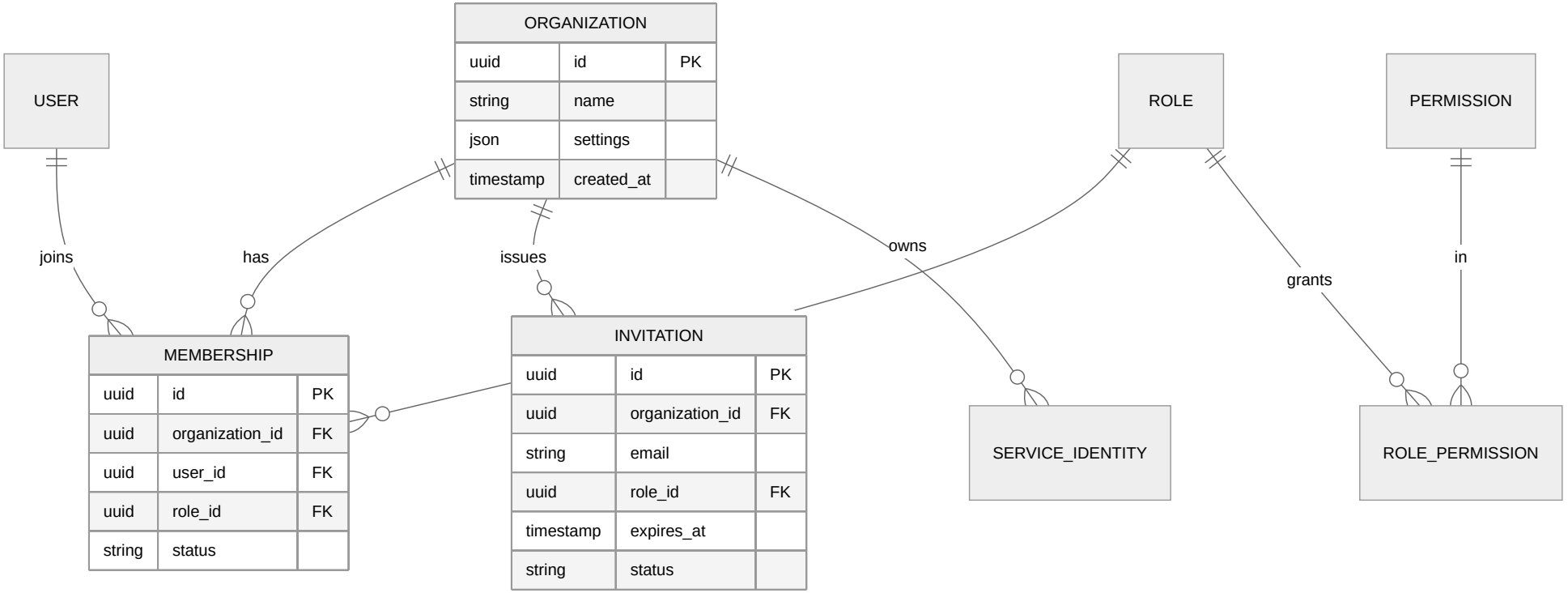


Figure 22.2: Organizations, users and access

22.5.3 Customer and channel

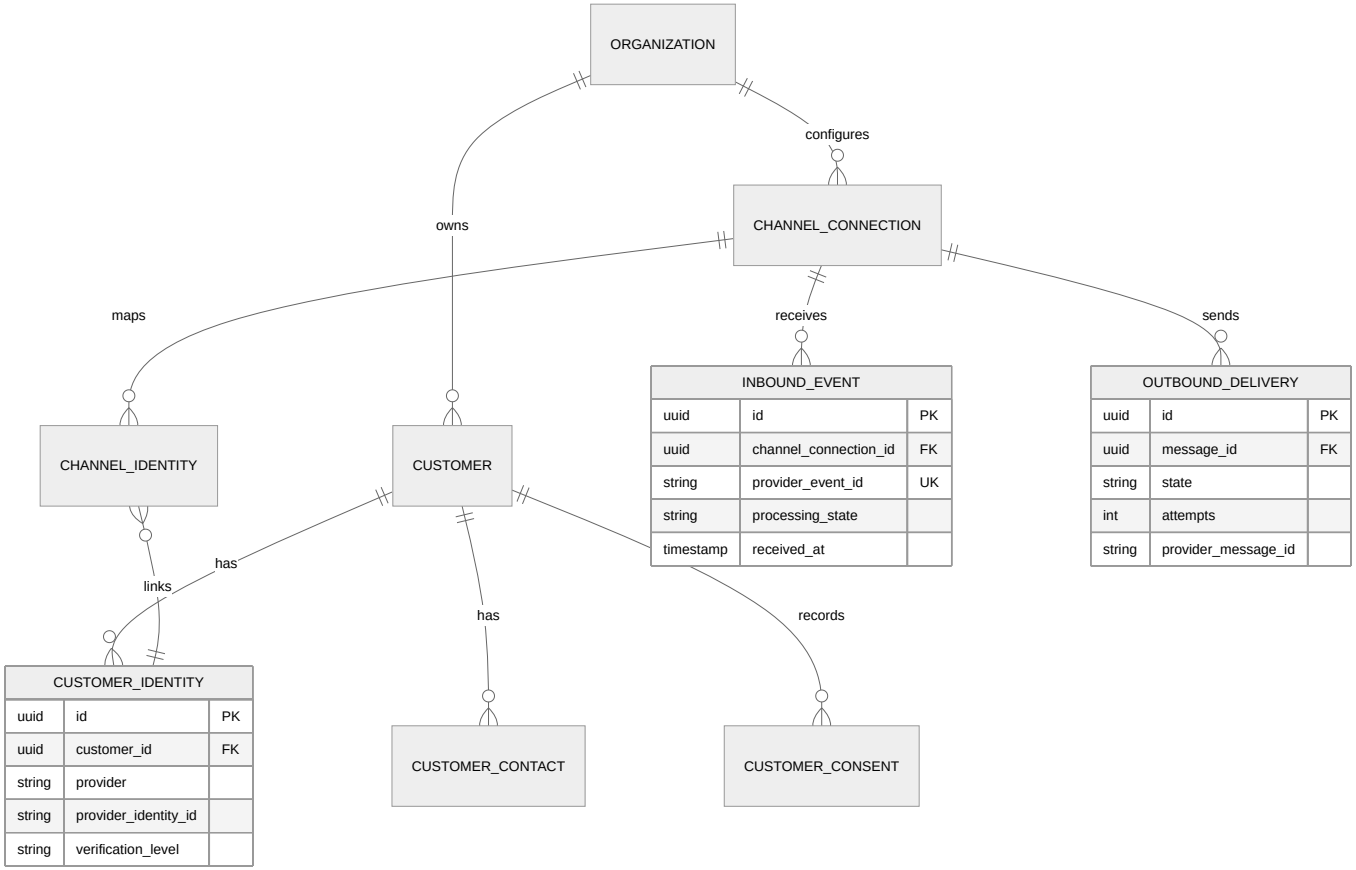


Figure 22.3: Customer and channel

22.5.4 Conversation

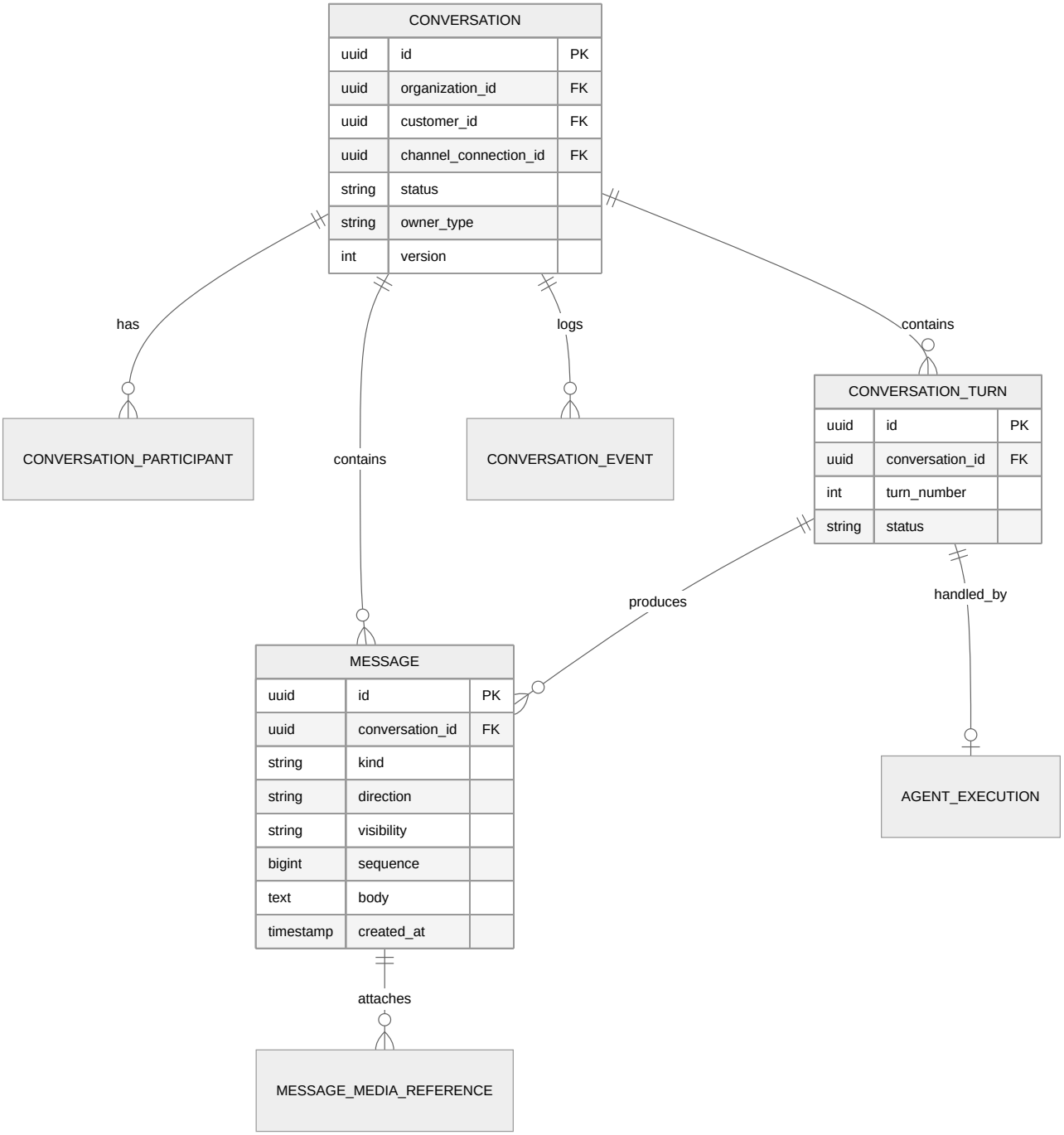


Figure 22.4: Conversation

22.5.5 Agent configuration and execution

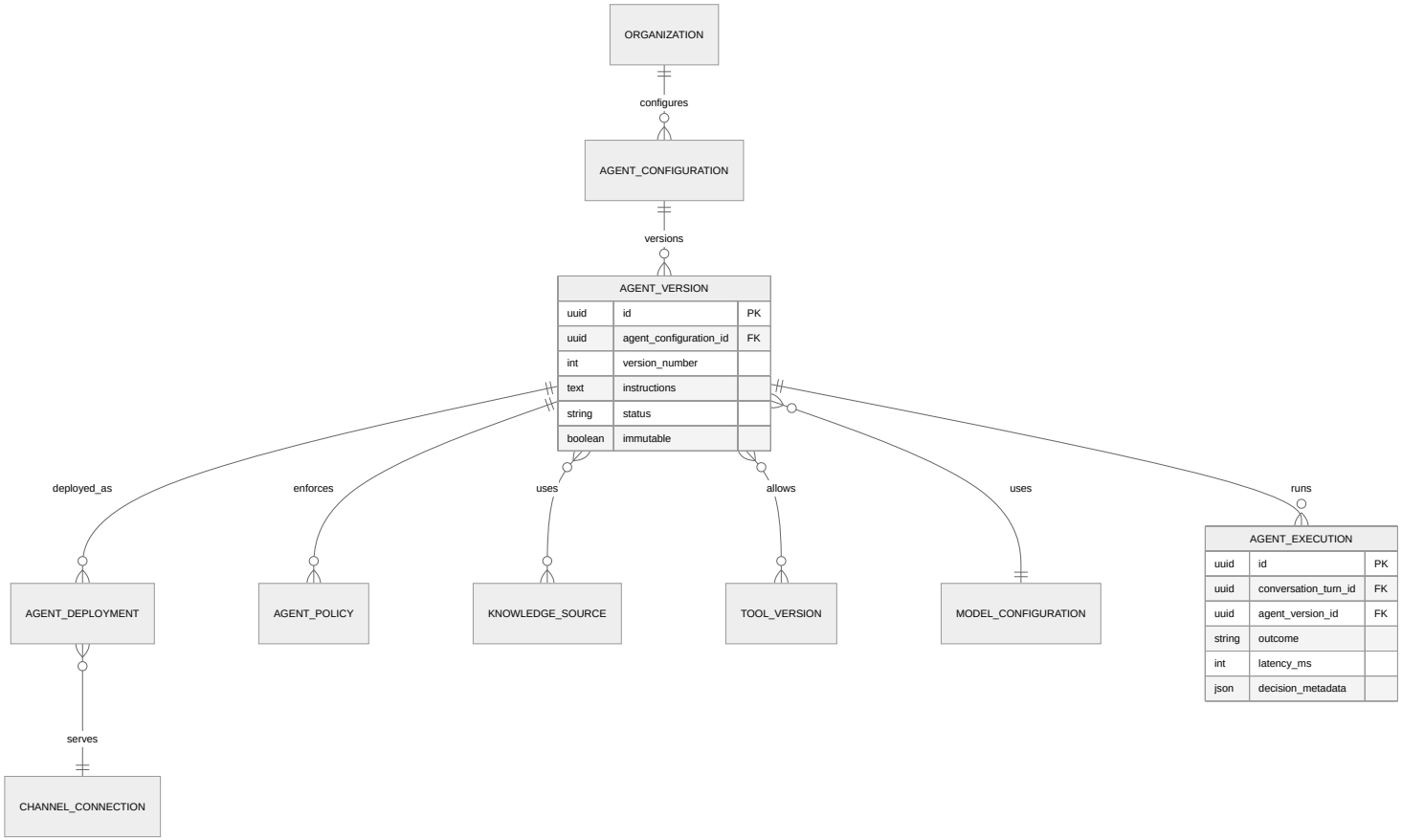


Figure 22.5: Agent configuration and execution

22.5.6 Knowledge

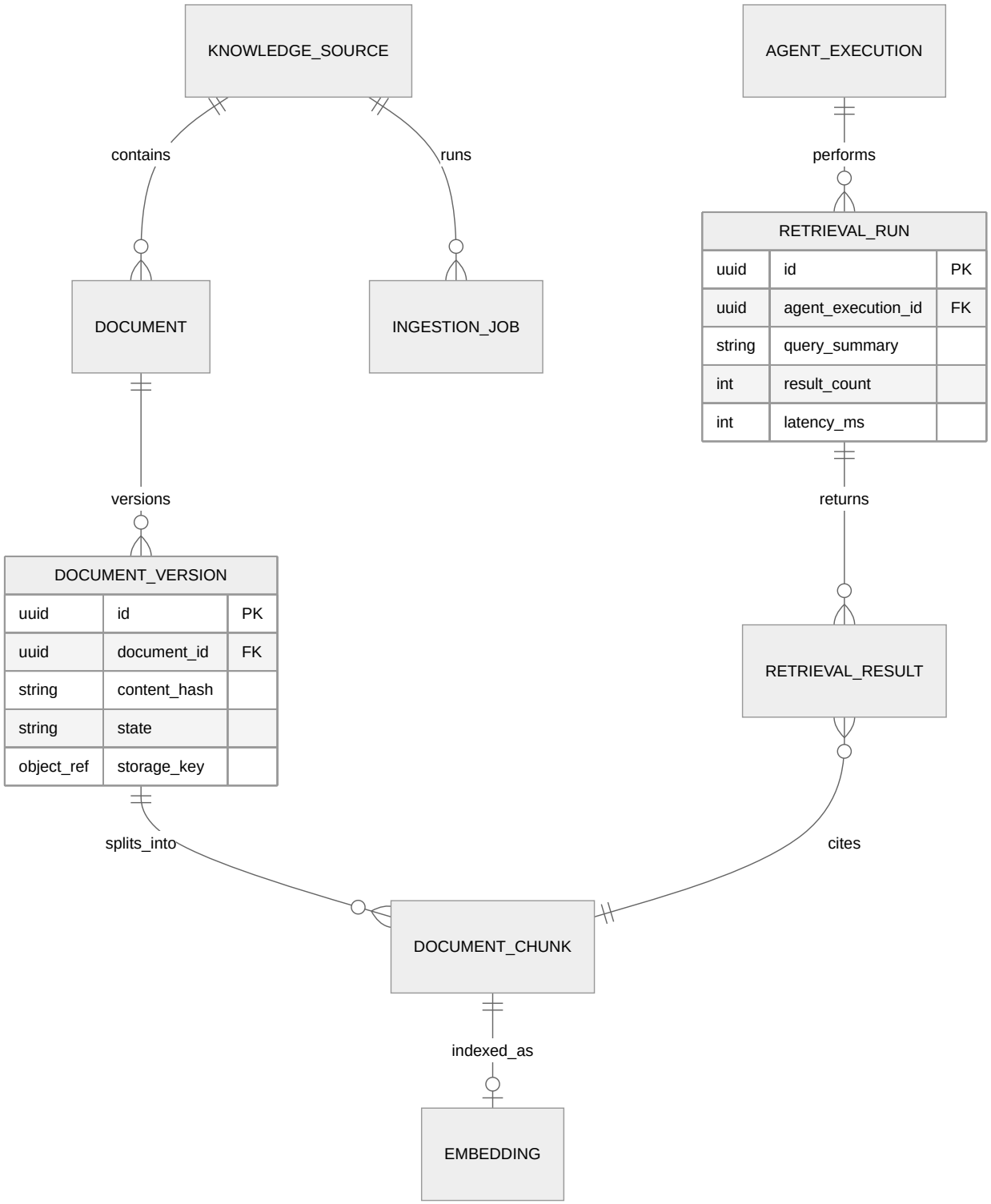


Figure 22.6: Knowledge

22.5.7 Tools and integrations

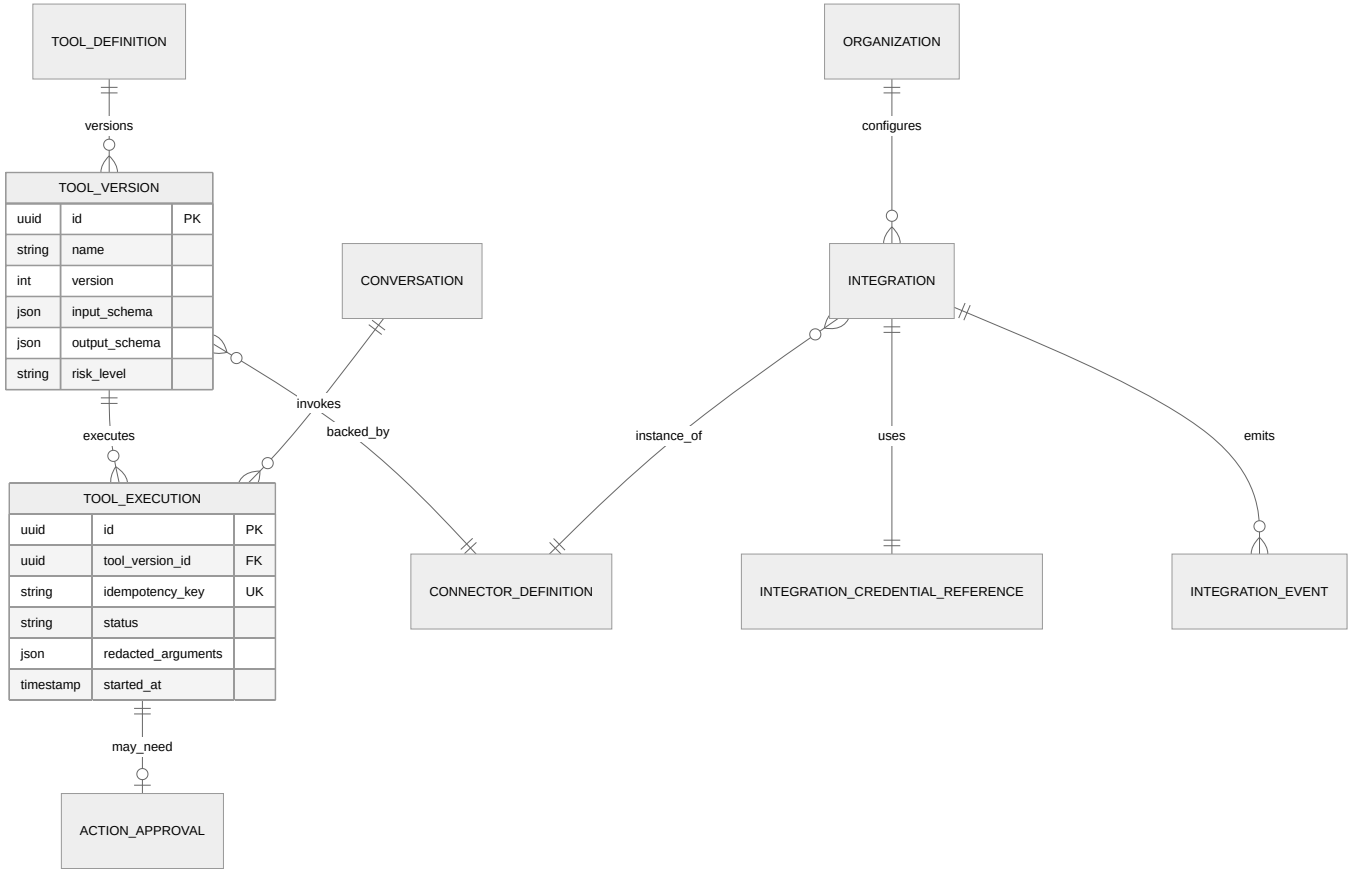


Figure 22.7: Tools and integrations

22.5.8 Handoff and tickets

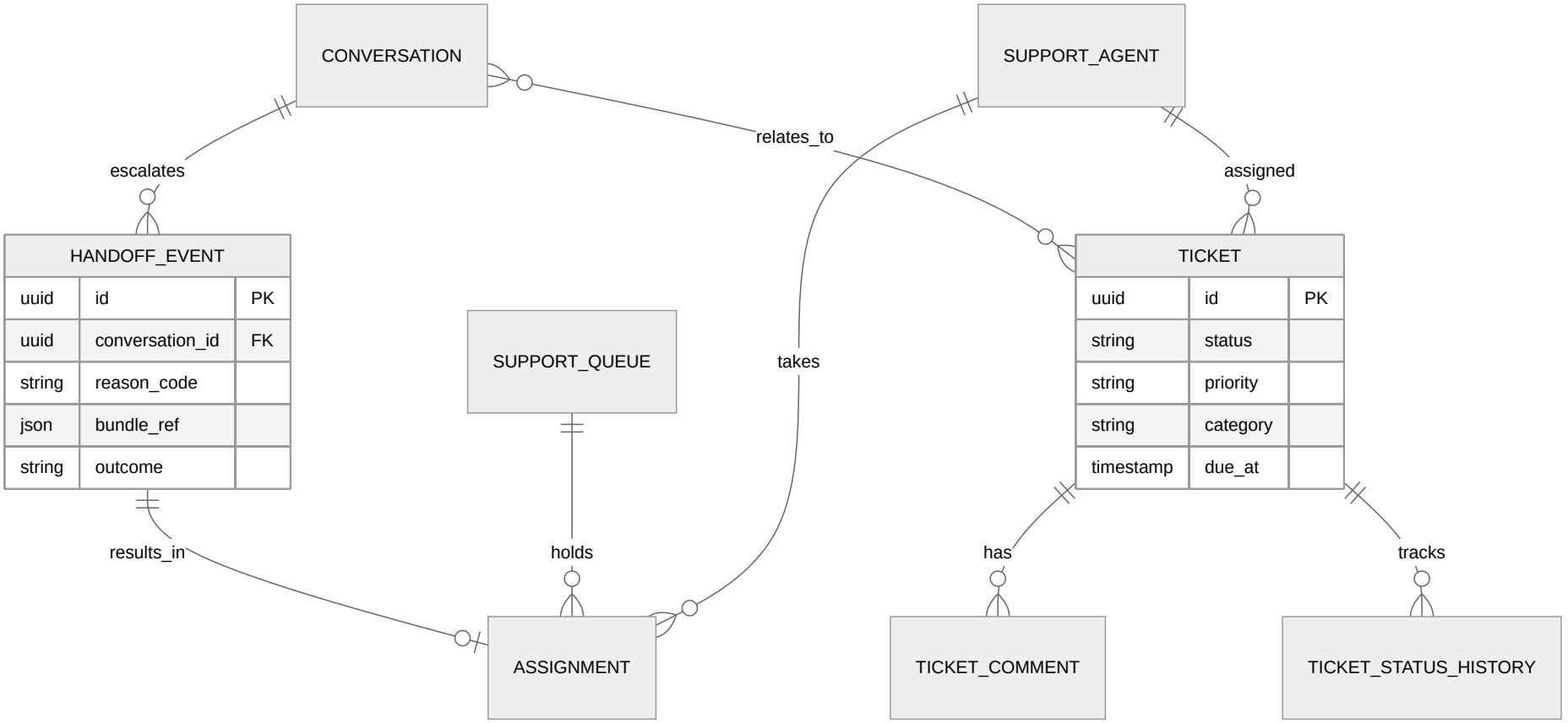


Figure 22.8: Handoff and tickets

22.5.9 Voice

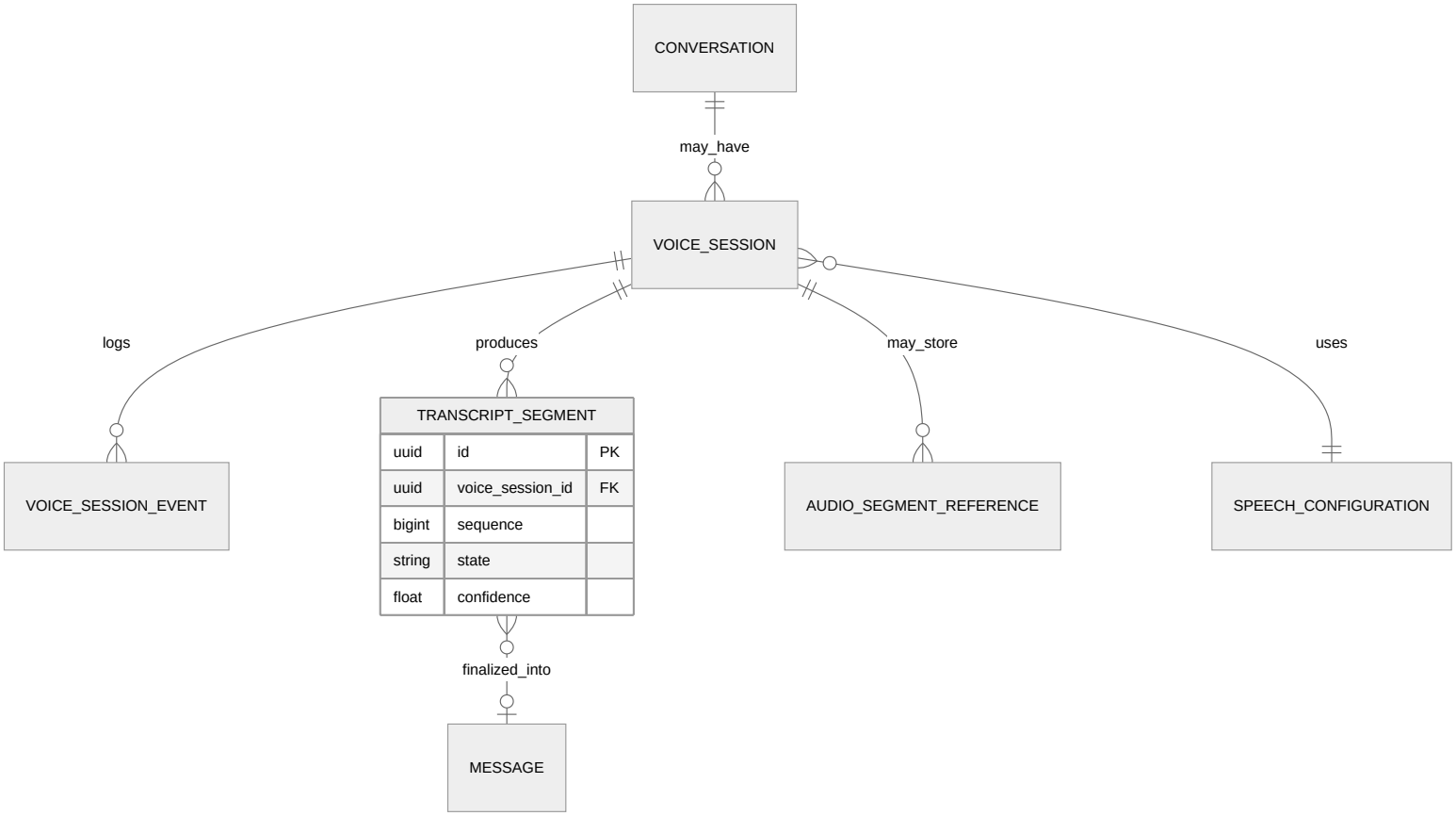


Figure 22.9: Voice

22.5.10 Workflow, routing and SLA

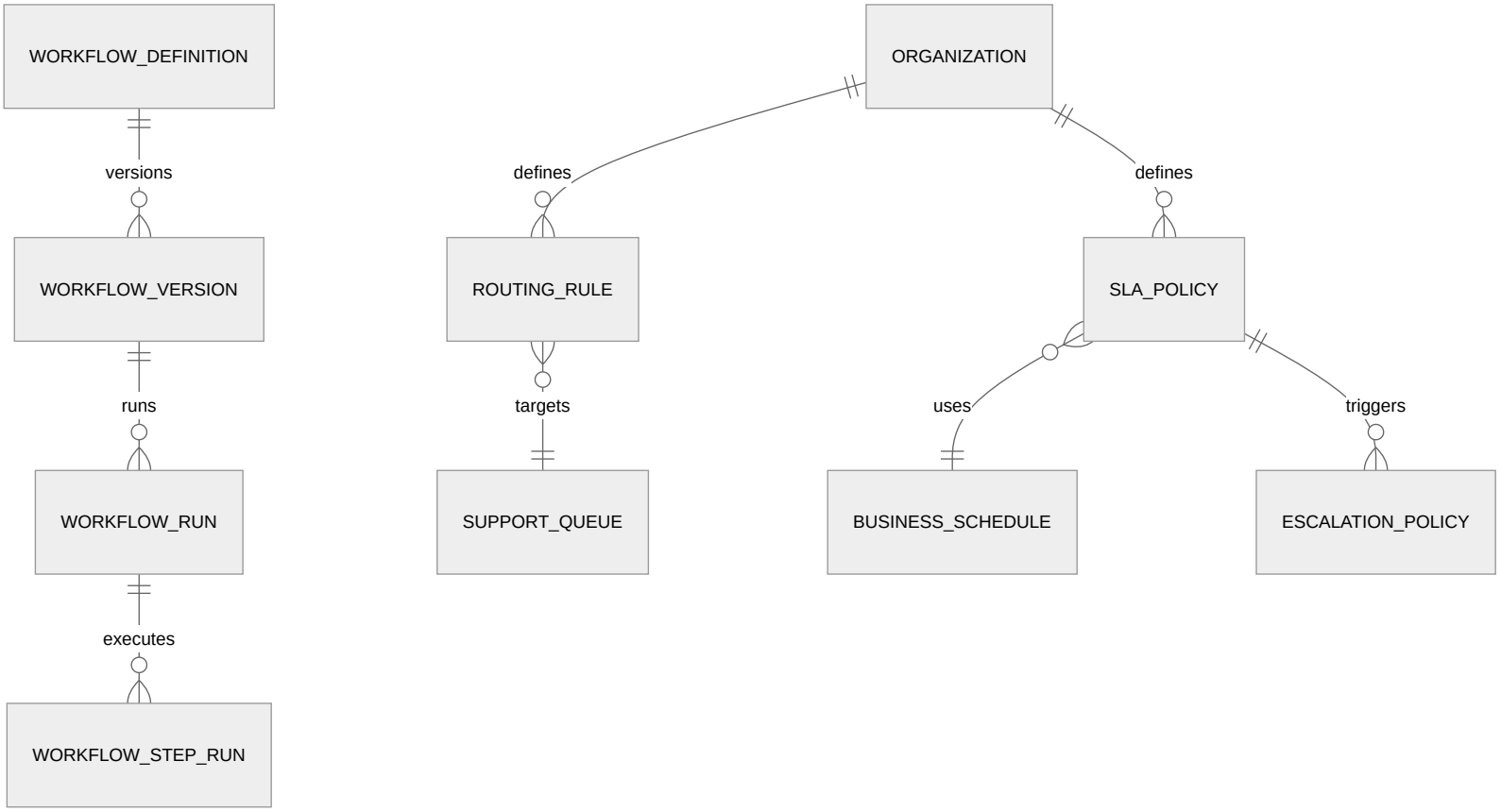


Figure 22.10: Workflow, routing and SLA

22.5.11 Audit, usage and evaluation

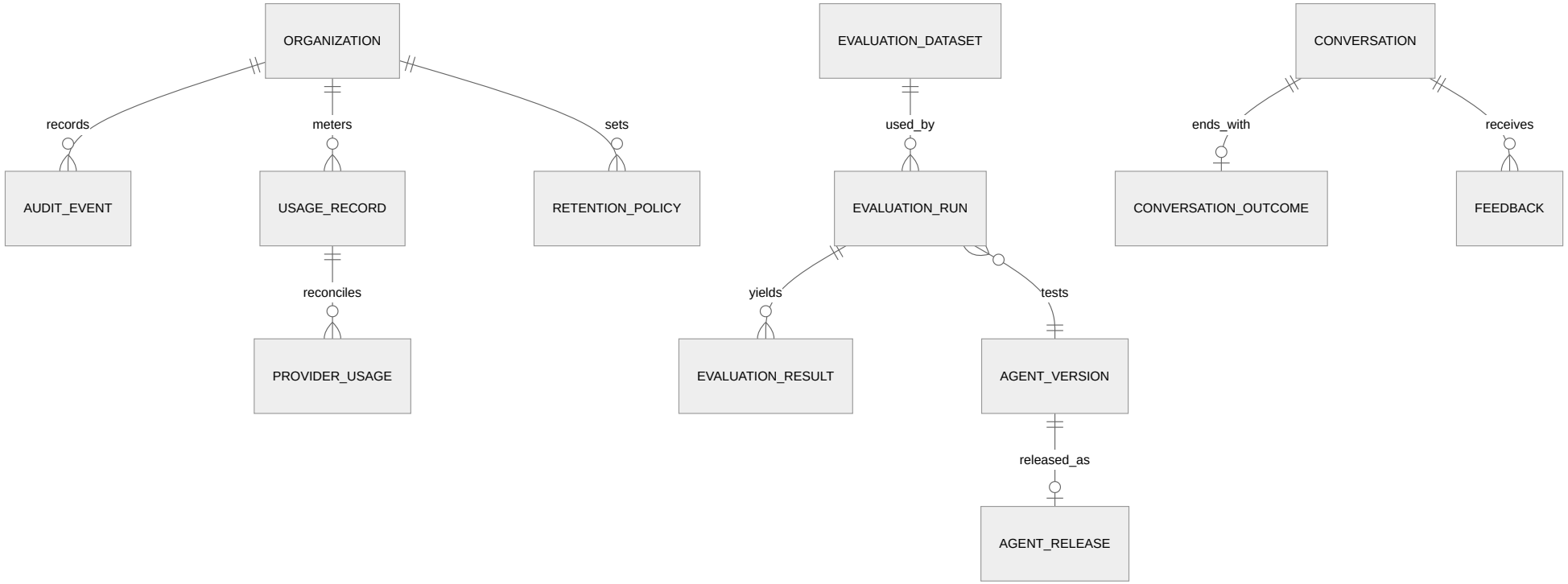


Figure 22.11: Audit, usage and evaluation

Chapter 23

AI Conversation Engine

23.1 Responsibilities

The engine processes one customer turn and chooses among:

- answer from approved knowledge;
- ask a clarifying question;
- invoke an allowed business tool;
- create/update a ticket or workflow request;
- refuse or safely redirect a disallowed request;
- hand off to a human;
- recover from provider or tool failures.

23.2 Recommended turn pipeline

1. Resolve trusted tenant, conversation, customer identity, and active agent version.
2. Check conversation state: open, closed, human-owned, rate-limited, or otherwise blocked.
3. Build a minimal context window from recent messages, a safe summary, and relevant structured state.
4. Classify intent/risk and identify missing information.
5. Retrieve approved knowledge when useful.
6. Plan the next step using only the agent's allowed tool registry.
7. Validate any proposed tool call against a schema and policy.
8. Execute authorized tool calls and validate returned data.
9. Generate a customer-facing response using verified results and source context.
10. Apply output safety and disclosure rules.
11. Persist final message, citations, tool results, timing, usage, and outcome metadata.
12. Emit events for delivery, evaluation, and analytics.

23.3 State machine

23.4 Prompt/context construction

Use explicit, separately labeled context sections:

- system-level behavior and non-negotiable policies;
- organization-approved agent instructions;
- customer-visible conversation history;
- trusted application state;
- retrieved source passages with source IDs;
- tool descriptions and JSON schemas;
- output format requirements.

Treat customer text, documents, retrieved content, and tool results as untrusted data. Instructions embedded in a retrieved document must not override system policy or tool permissions.

23.5 Tool execution policy

- Allowlist tools per agent version and tenant.
- Validate argument schema before execution.
- Check tenant scope and user/customer authorization.
- Require confirmation or human approval for sensitive actions.
- Use idempotency keys for non-idempotent external operations where supported.
- Apply timeout, retry, and rate-limit policies.
- Return minimal, typed results to the model.
- Redact secrets and unnecessary personal data.
- Record outcome and audit metadata.

23.6 Response policy

- Do not fabricate order status, account data, policies, or actions.
- Distinguish retrieved policy from live transactional data.
- If evidence is missing or contradictory, ask a question or escalate.
- Do not claim an action succeeded until the tool confirms success.
- Keep customer-facing responses concise and understandable.
- Support language selection and text/audio rendering separately from reasoning.

23.7 Model-provider interface

Conceptual interface:

- `generate_response(input, output_schema, options)`
- `stream_response(input, options)`
- `classify(input, taxonomy, options)`
- `embed(texts, model_configuration)`
- optional `open_realtime_session(configuration)`

Keep vendor-specific request/response shapes in adapters. Capture model name/version, latency, usage, and provider errors without storing secrets.

23.8 AI engine diagrams

23.8.1 Turn pipeline overview

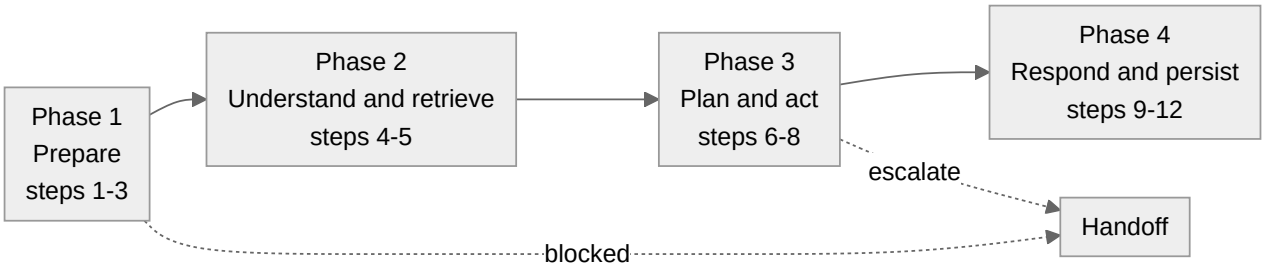


Figure 23.1: Turn pipeline overview

23.8.1.1 Phase 1 — Prepare (resolve trust, check state, build context)

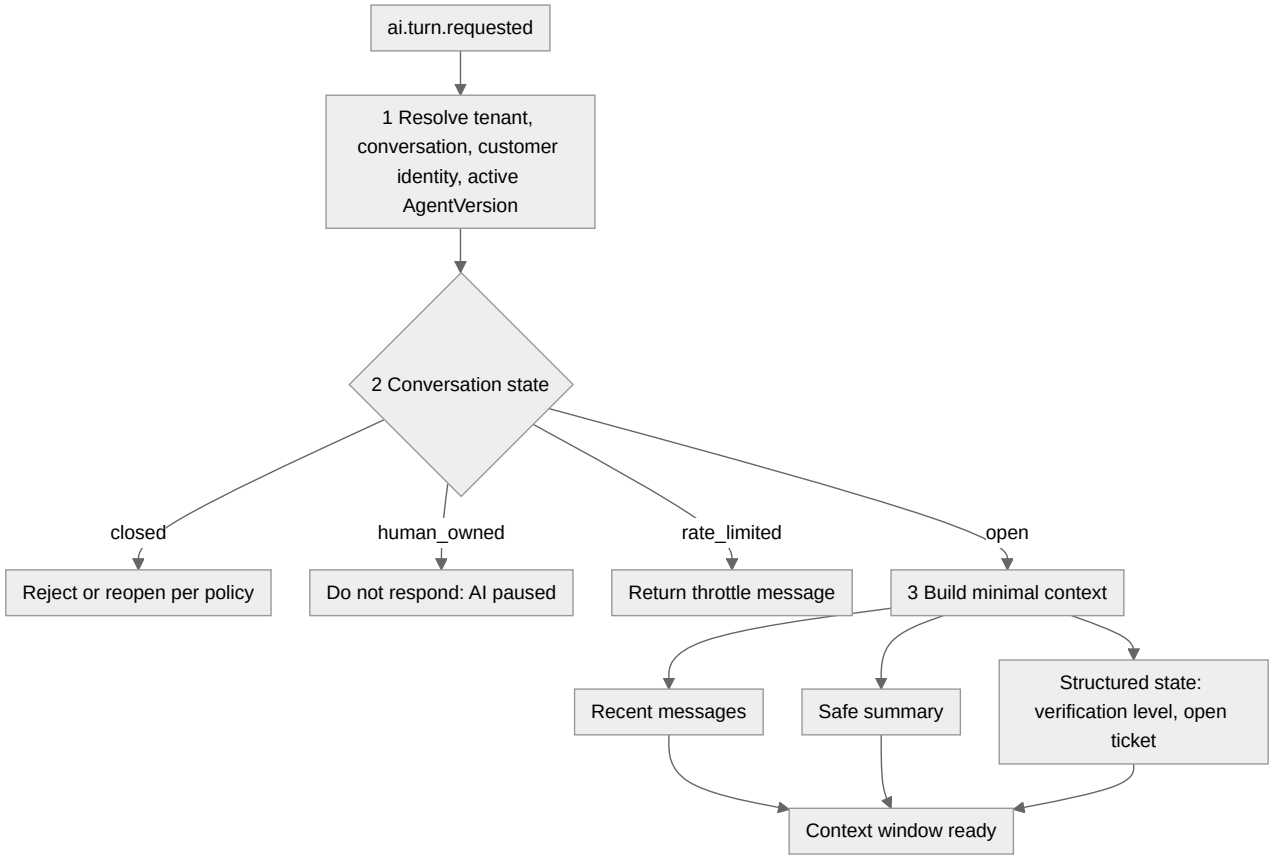


Figure 23.2: Turn pipeline overview: Phase 1 — Prepare (resolve trust, check state, build context)

23.8.1.2 Phase 2 — Understand and retrieve

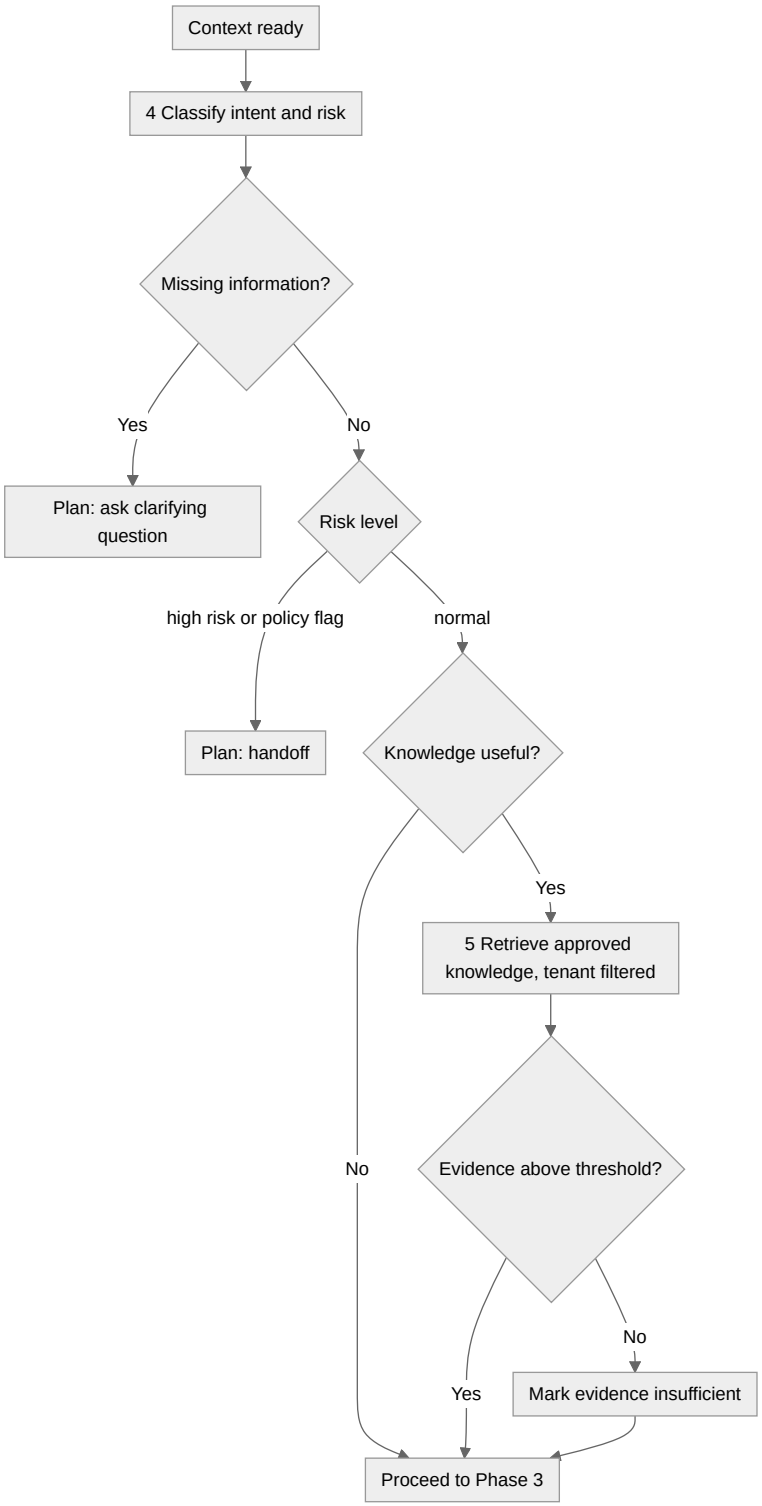


Figure 23.3: Turn pipeline overview: Phase 2 — Understand and retrieve

23.8.1.3 Phase 3 — Plan and act (tool loop)

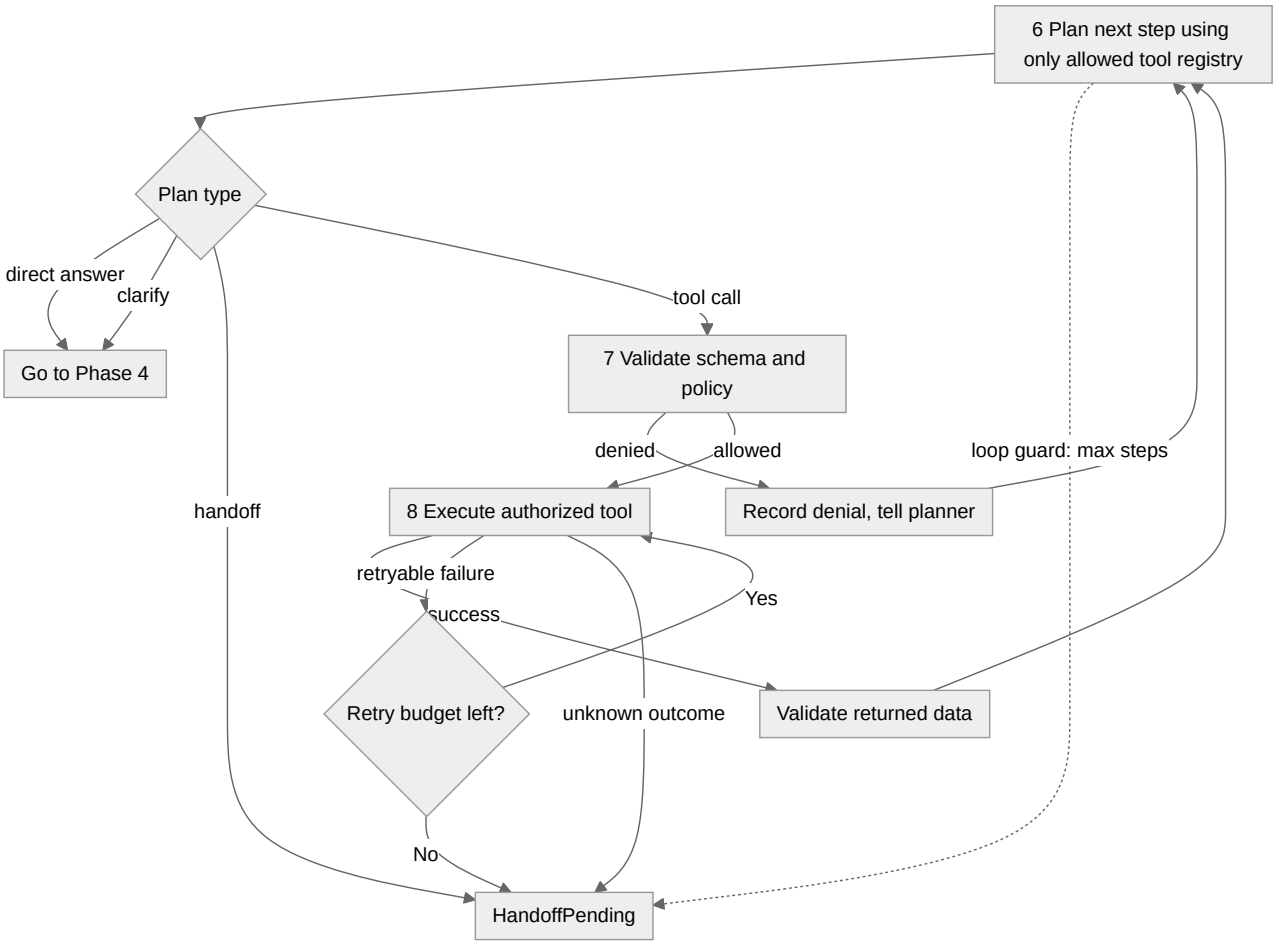


Figure 23.4: Turn pipeline overview: Phase 3 — Plan and act (tool loop)

23.8.1.4 Phase 4 — Respond and persist

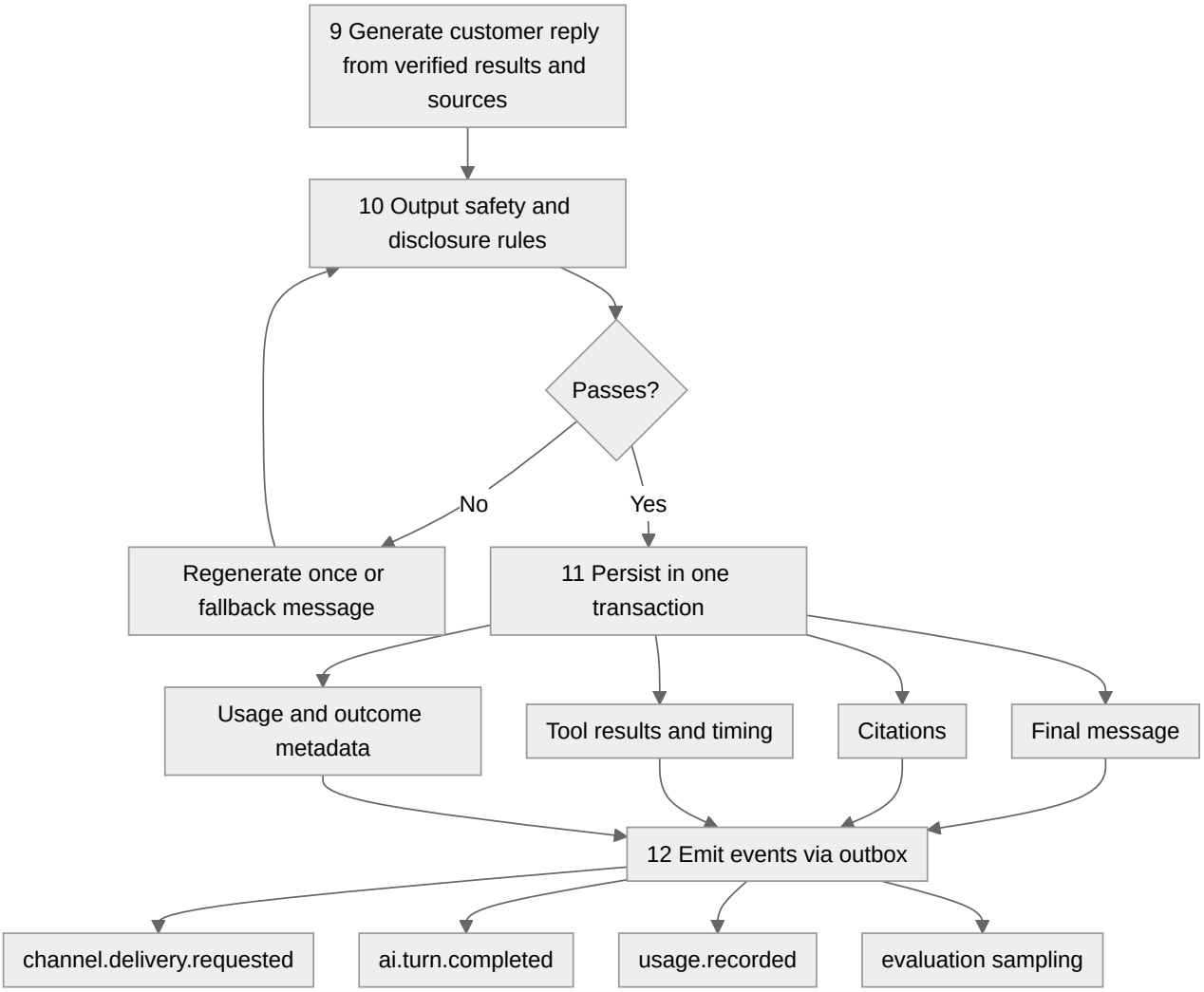


Figure 23.5: Turn pipeline overview: Phase 4 — Respond and persist

23.8.2 Turn state machine

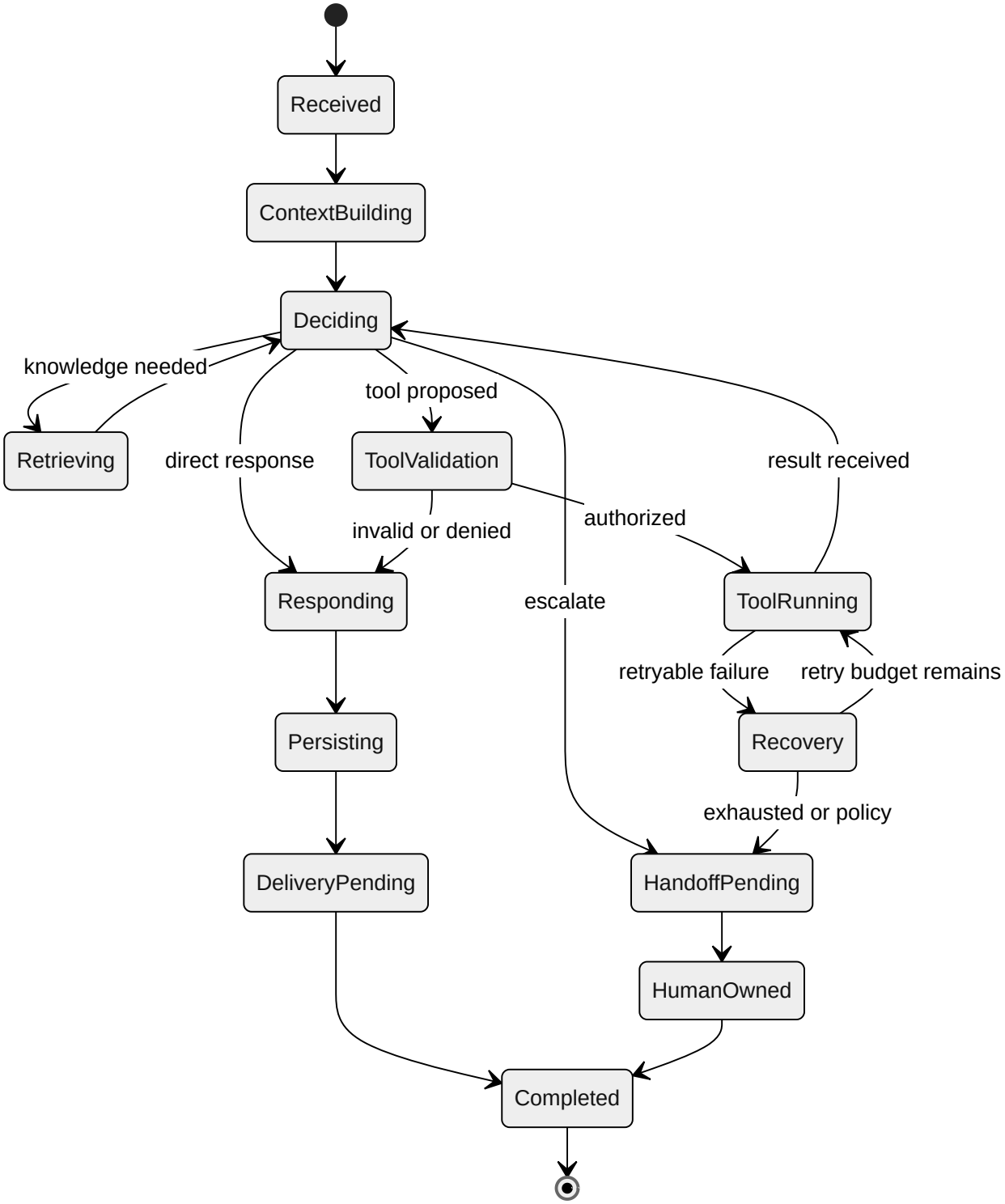


Figure 23.6: Turn state machine

23.8.3 Context sections and trust levels

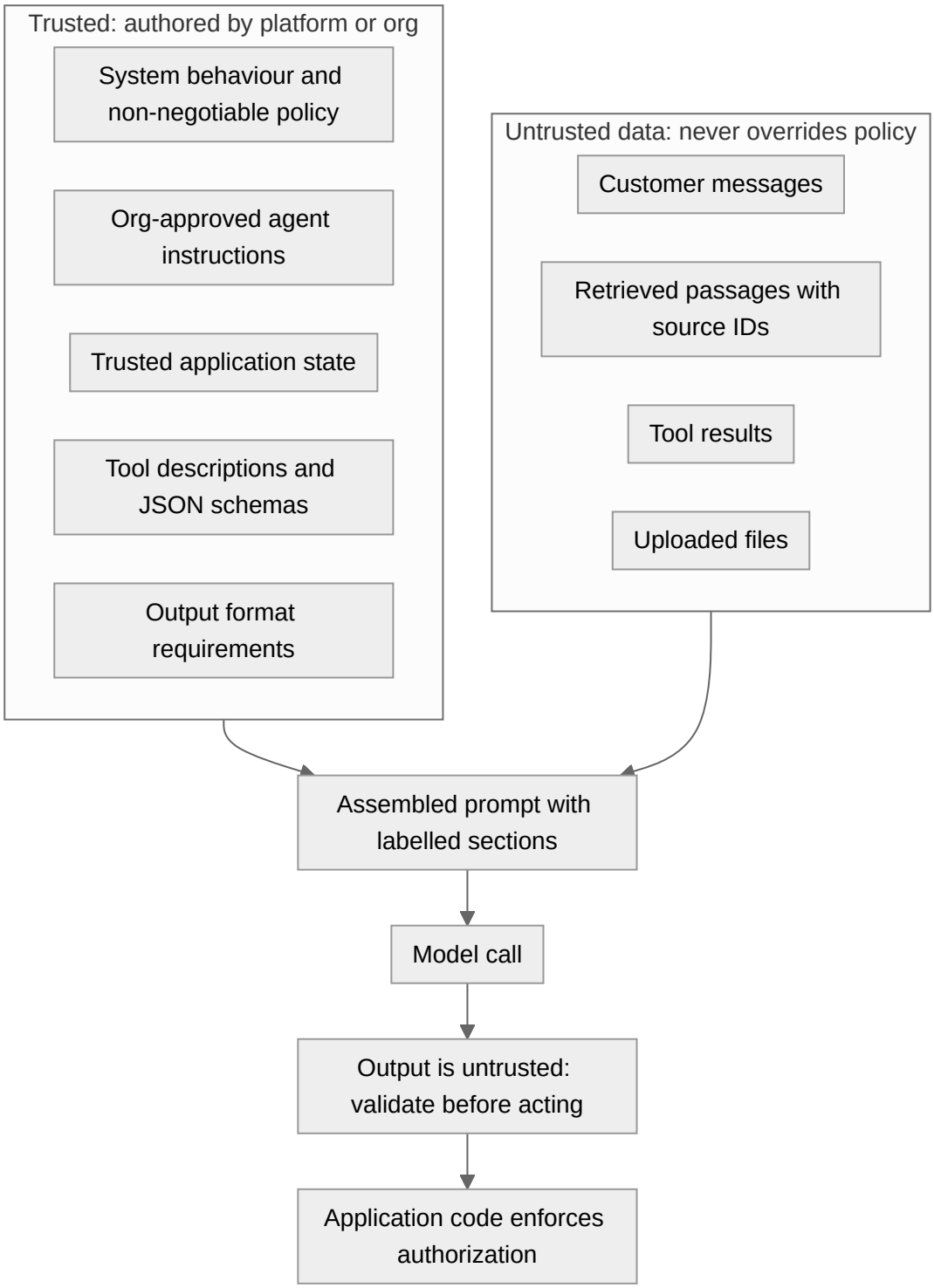


Figure 23.7: Context sections and trust levels

23.8.4 Escalation decision tree

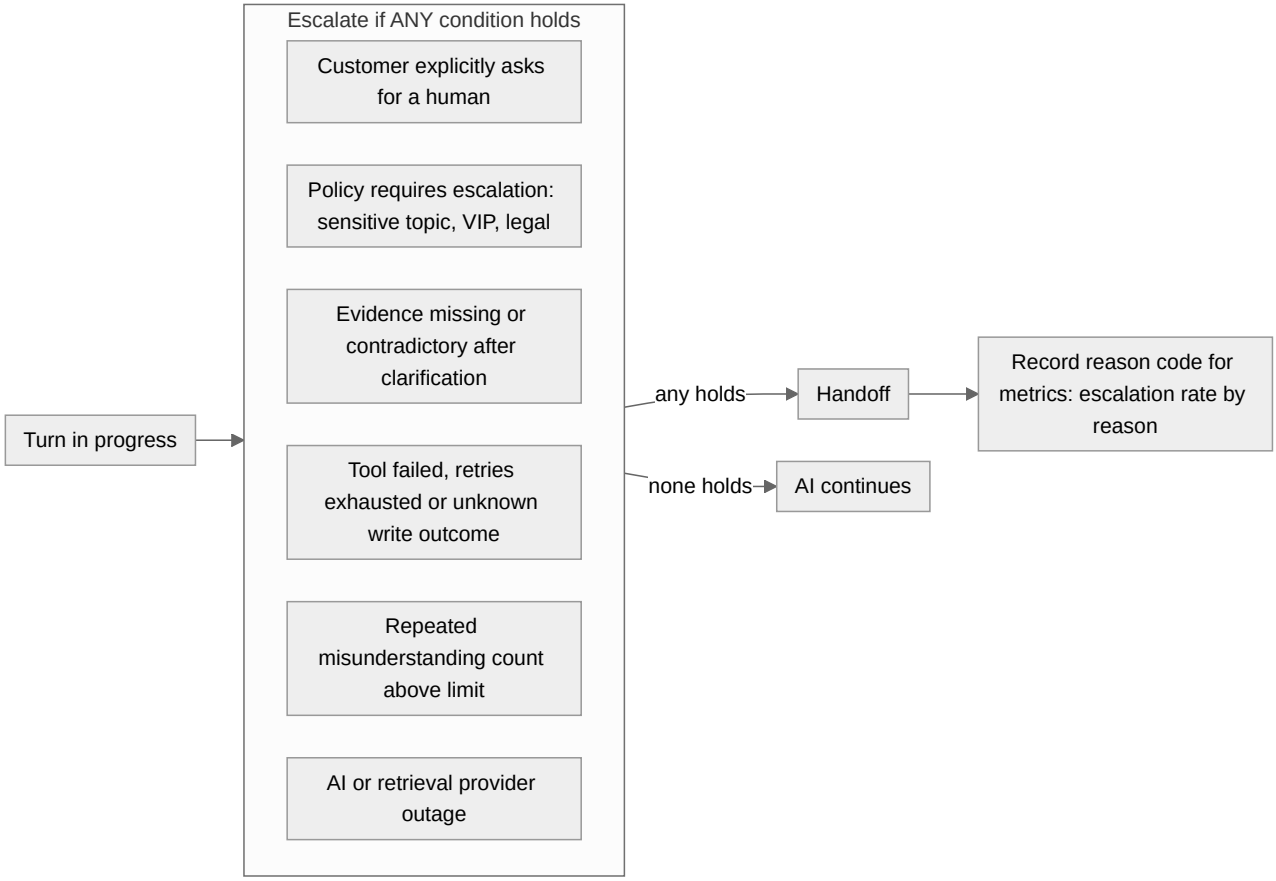


Figure 23.8: Escalation decision tree

23.8.5 Response policy decision

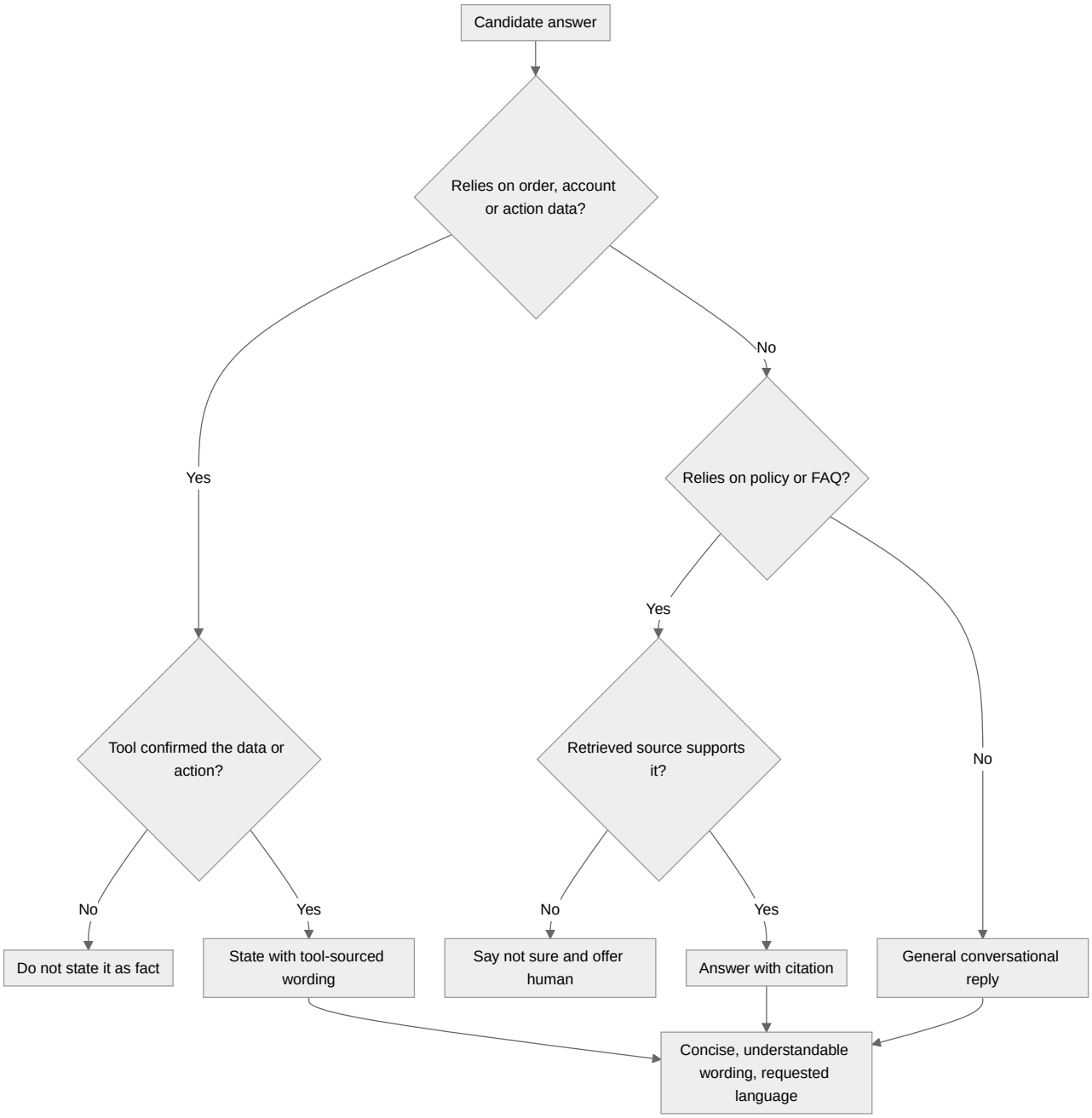


Figure 23.9: Response policy decision

23.8.6 Model-provider adapter

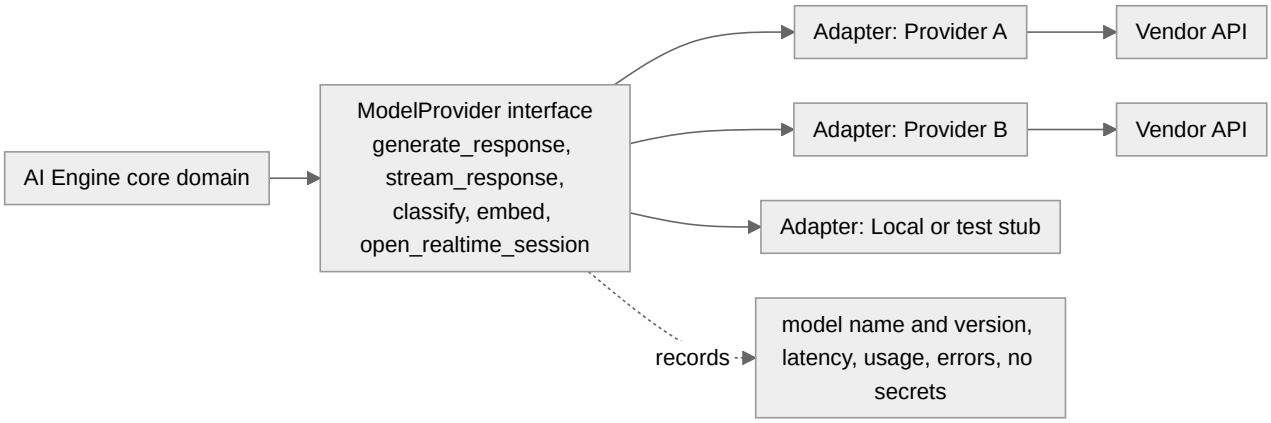


Figure 23.10: Model-provider adapter

23.8.7 Agent version lifecycle (publish, rollback)

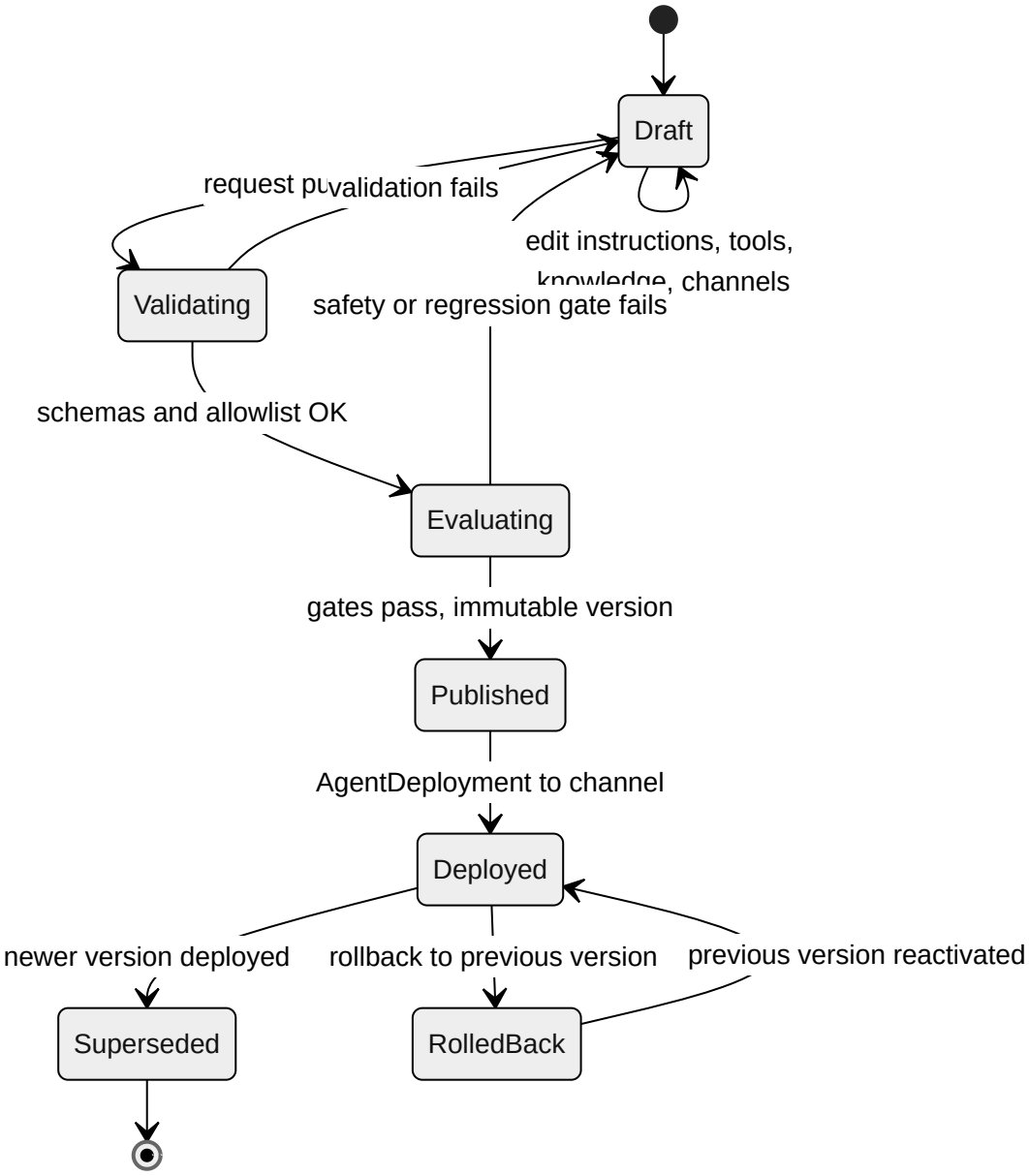


Figure 23.11: Agent version lifecycle (publish, rollback)

23.8.8 Failure handling inside a turn

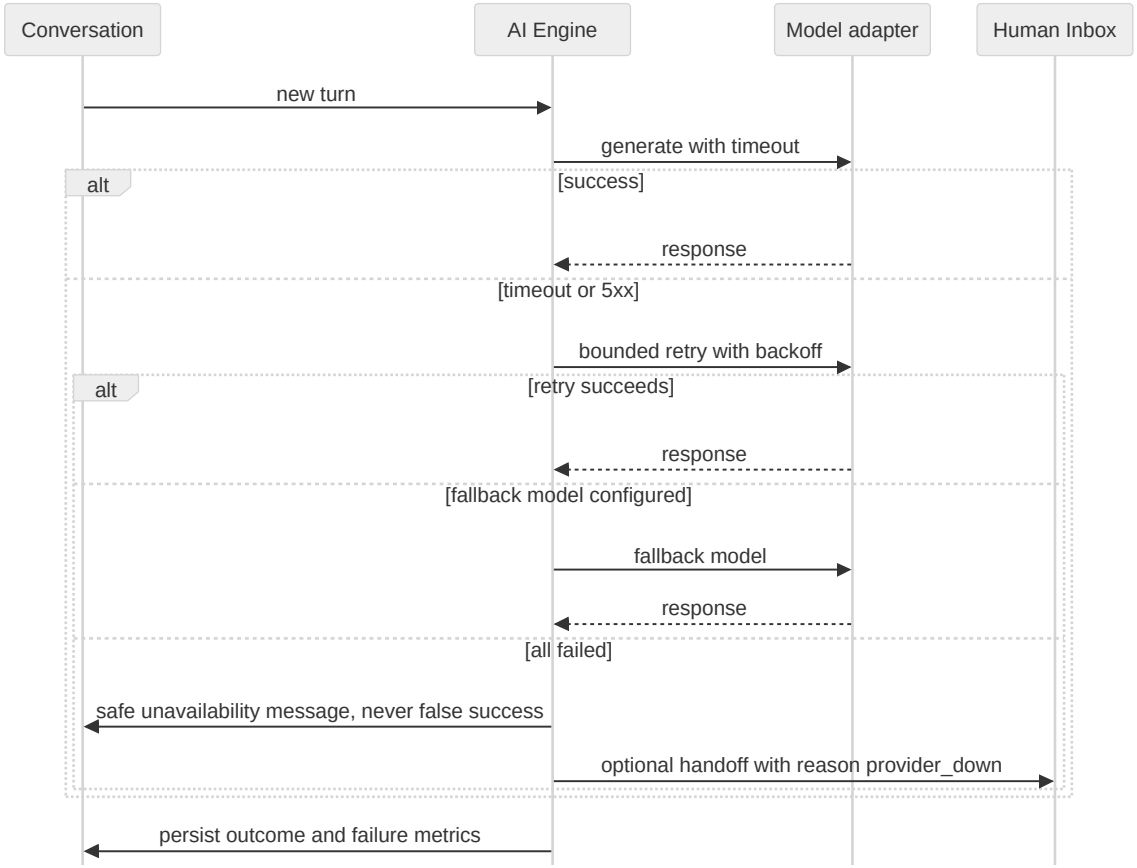


Figure 23.12: Failure handling inside a turn

Chapter 24

Voice and Multimodal Conversations

24.1 Goal

A customer can start with text, switch to voice, return to text, or be transferred to a human without losing the durable conversation history.

24.2 Voice subsystem responsibilities

24.2.1 Real-time voice session

- Open and close media sessions.
- Track transport state, audio input/output, interruptions, silence, reconnects, and failures.
- Associate every voice session with a conversation and tenant.
- Enforce session duration and resource quotas.

24.2.2 Speech processing

Two strategies should be supported behind a common abstraction.

Option A: modular pipeline `audio` → `STT` → `conversation engine` → `TTS` → `audio`

Advantages: explicit transcript, modular provider choice, easier inspection of each stage.
Trade-off: additional latency and more orchestration.

Option B: speech-to-speech `audio` ↔ `real-time multimodal model`

Advantages: potentially more natural, lower-latency interactions.
Trade-off: provider coupling and less explicit control over intermediate artifacts depending on provider.

Start with one strategy that matches latency, language, cost, and observability requirements; do not implement both fully before validating the product.

24.3 Turn and interruption semantics

- Partial transcription is provisional.
- Finalized user speech creates a durable turn input.
- When a user interrupts generated speech, cancel or truncate the obsolete response where supported.
- Do not persist a canceled assistant utterance as if it were fully delivered.
- Avoid processing the same audio segment as both a final transcript and a second independent message.
- Preserve turn order with sequence IDs or timestamps plus server-side ordering.
- Keep audio chunks and ephemeral transport events separate from canonical messages.

24.4 Switching modalities

A conversation can contain:

- text customer messages;

- finalized speech transcripts;
- assistant text;
- generated audio references;
- system events such as voice started/ended;
- handoff events.

The conversation engine should consume normalized text/intent and trusted state, not depend on whether the text came from typing or transcription.

24.5 Human handoff

A handoff bundle should include:

- conversation ID and customer identity level/verification status;
- transcript and concise summary;
- customer intent and unresolved question;
- source citations already used;
- completed/failed tool executions;
- relevant ticket and SLA state;
- current modality and provider transfer references where appropriate.

Do not silently expose restricted account data in a handoff summary.

24.6 Voice-specific operational metrics

- time to first audio;
- end-of-speech to response-start latency;
- total turn latency;
- interruption/cancellation rate;
- transcription confidence or provider quality indicators when available;
- reconnect and dropped-session rate;
- audio generation failures;
- cost and duration per session;
- handoff completion rate.

24.7 Privacy

- Request microphone permission only when needed.
- Clearly communicate when voice is active and when recording/transcription is retained.
- Define audio and transcript retention separately.
- Avoid storing raw audio by default unless required by a defined product feature.
- Provide deletion, access, and consent workflows according to applicable law and customer policy.

24.8 Voice and multimodal diagrams

24.8.1 Voice session lifecycle

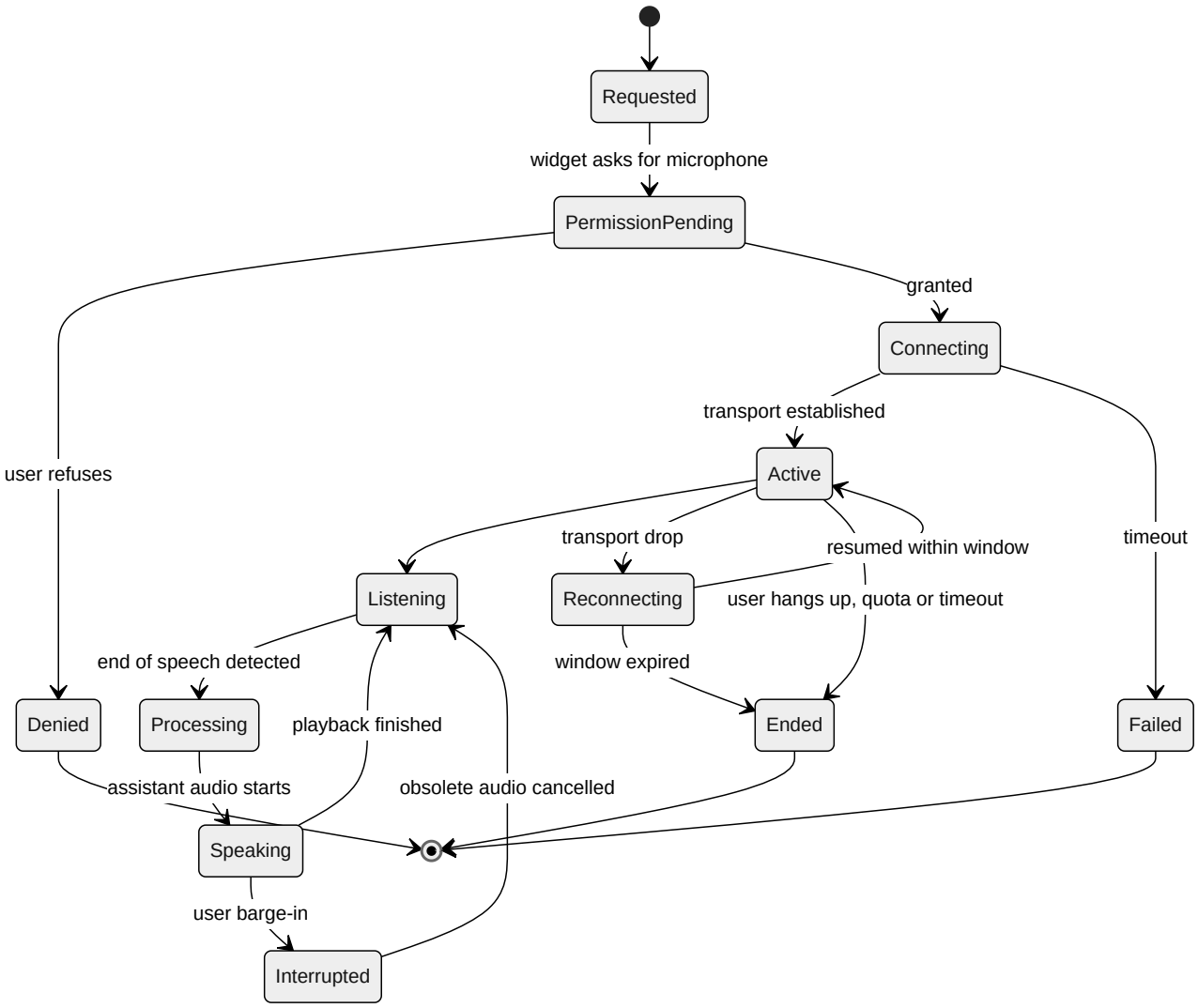


Figure 24.1: Voice session lifecycle

24.8.2 Two speech strategies

24.8.2.1 Option A — modular pipeline

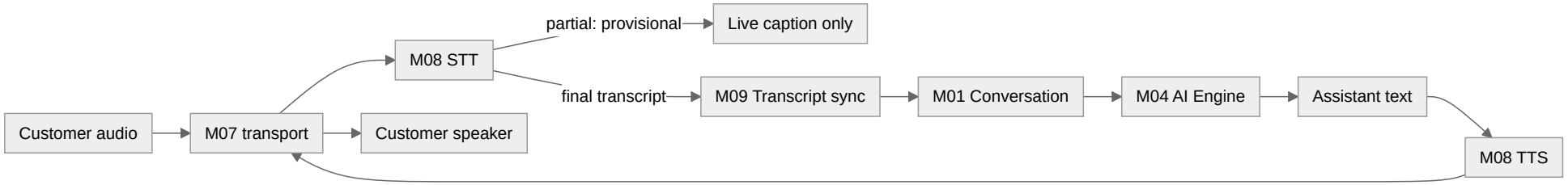


Figure 24.2: Two speech strategies: Option A — modular pipeline

24.8.2.2 Option B — speech-to-speech

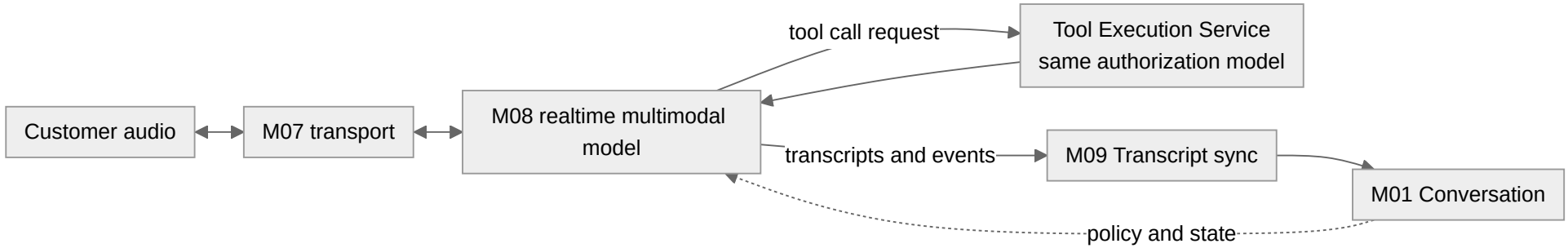


Figure 24.3: Two speech strategies: Option B — speech-to-speech

24.8.2.3 Decision: which to start with

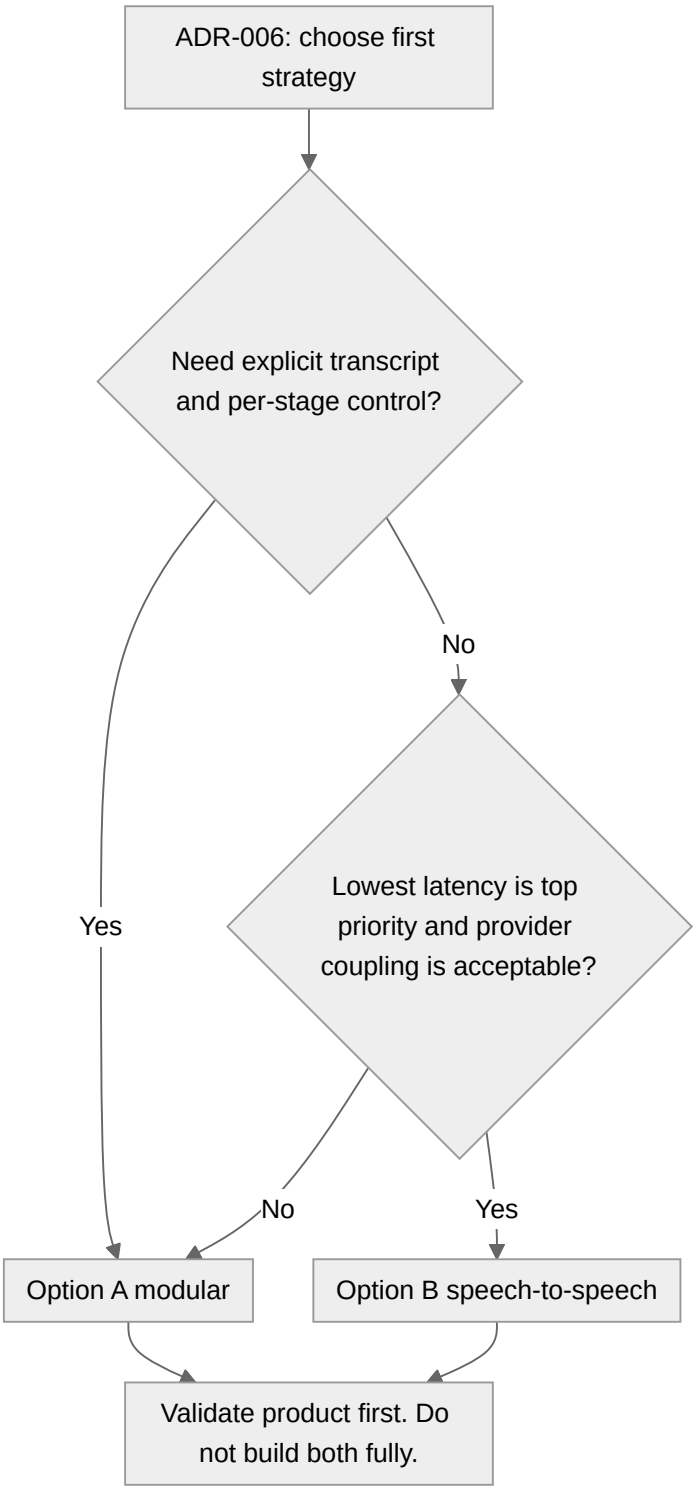


Figure 24.4: Two speech strategies: Decision: which to start with

24.8.3 One voice turn (Option A) with persistence boundaries

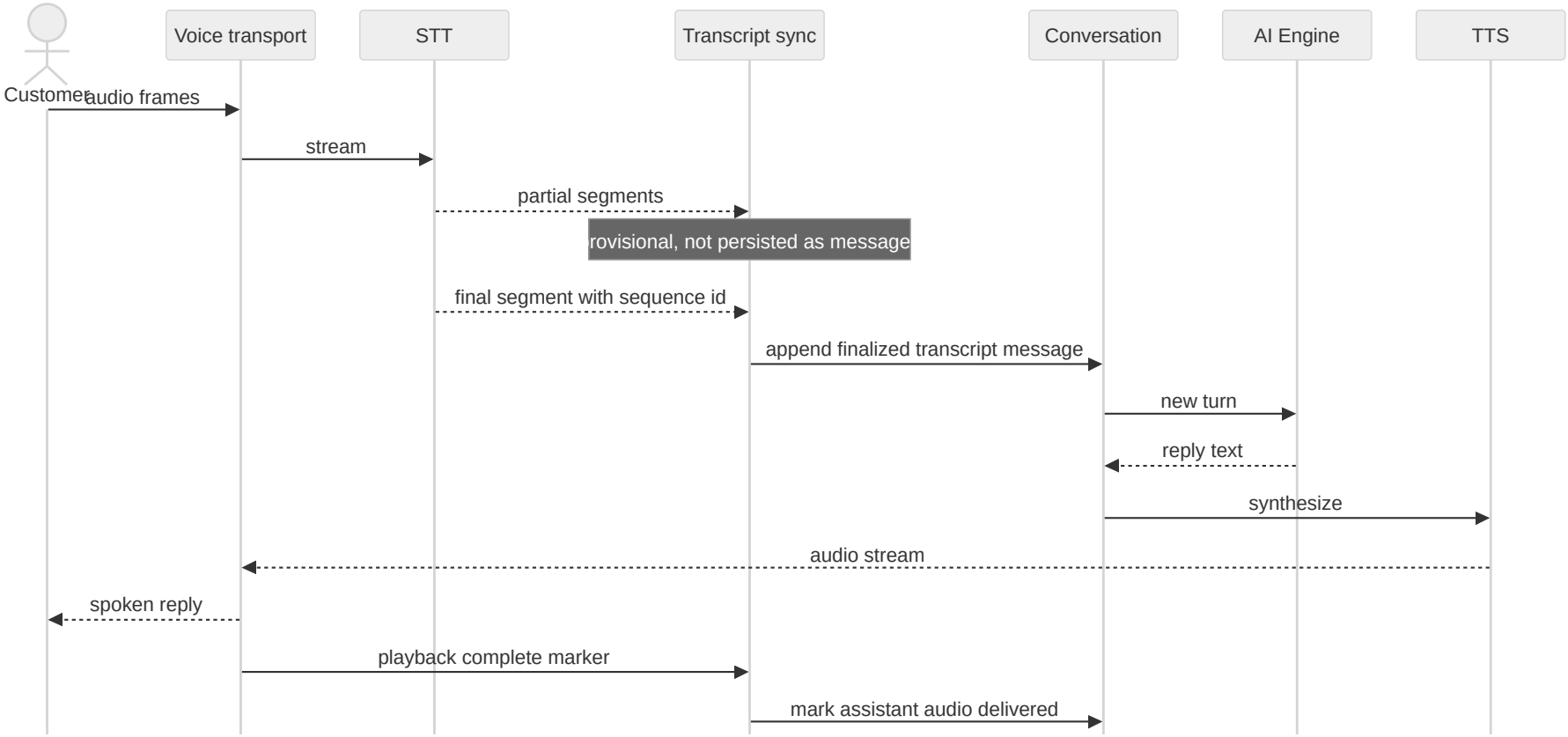


Figure 24.5: One voice turn (Option A) with persistence boundaries

24.8.4 Interruption (barge-in)

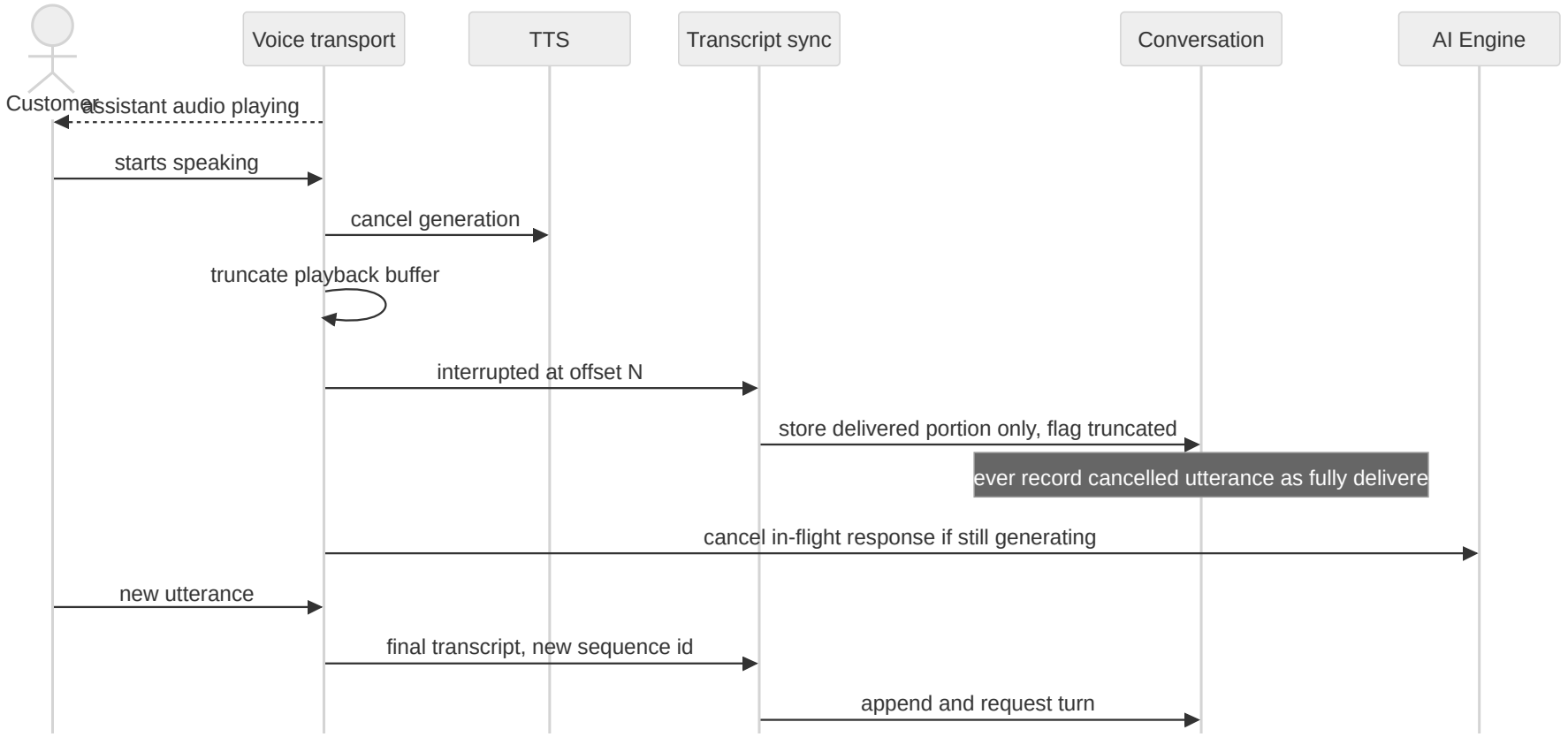


Figure 24.6: Interruption (barge-in)

24.8.5 Partial vs final transcript handling

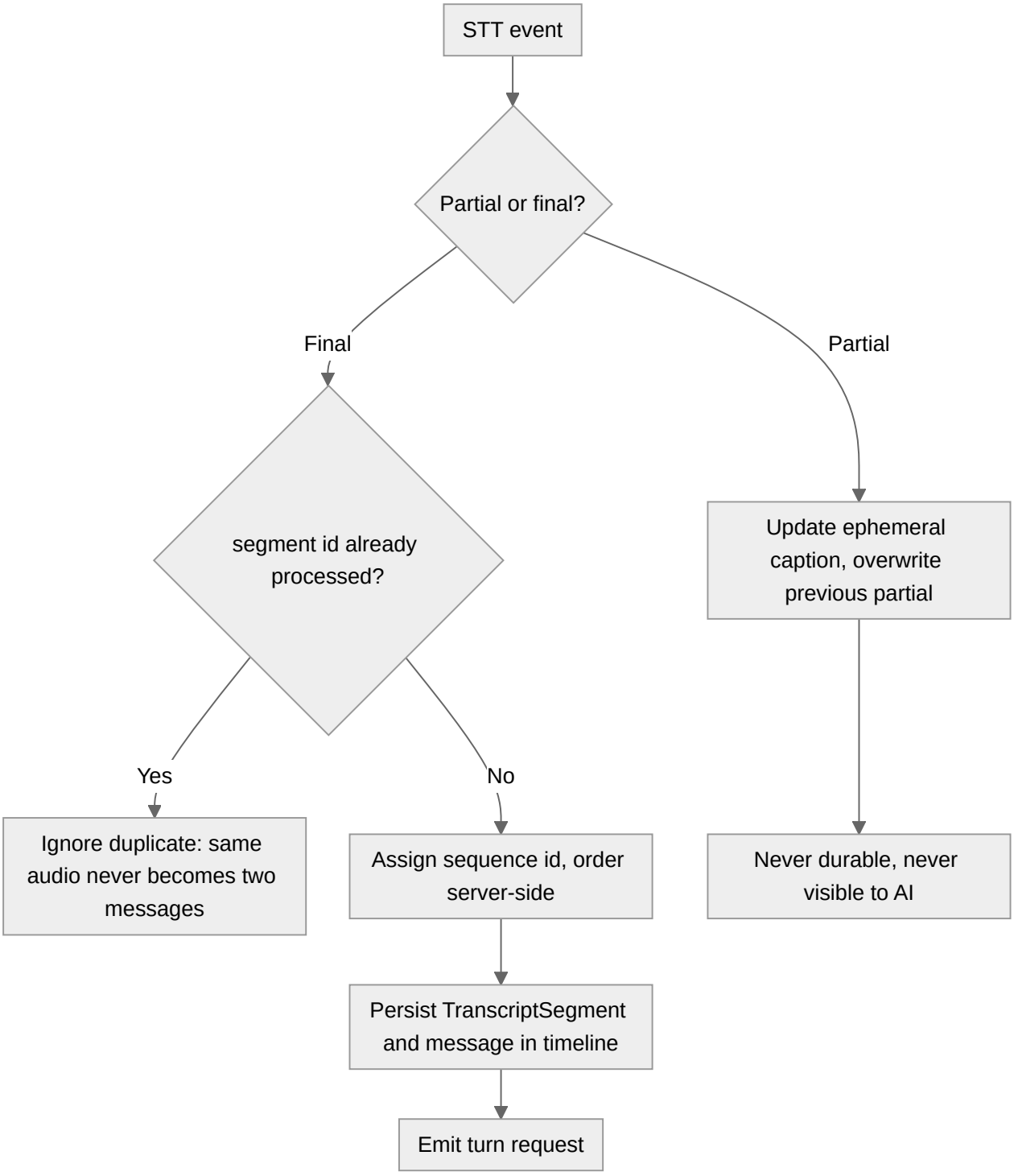


Figure 24.7: Partial vs final transcript handling

24.8.6 Modality switching in one conversation

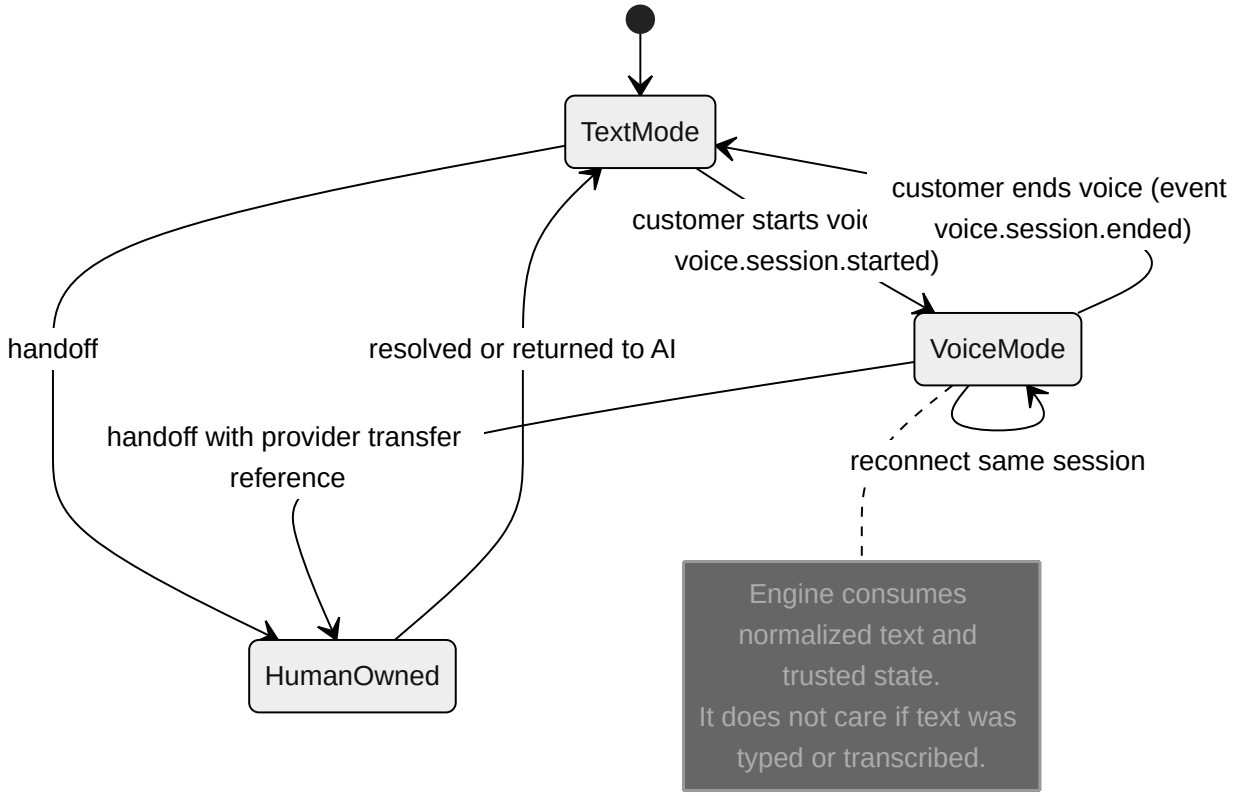


Figure 24.8: Modality switching in one conversation

24.8.7 Voice handoff bundle

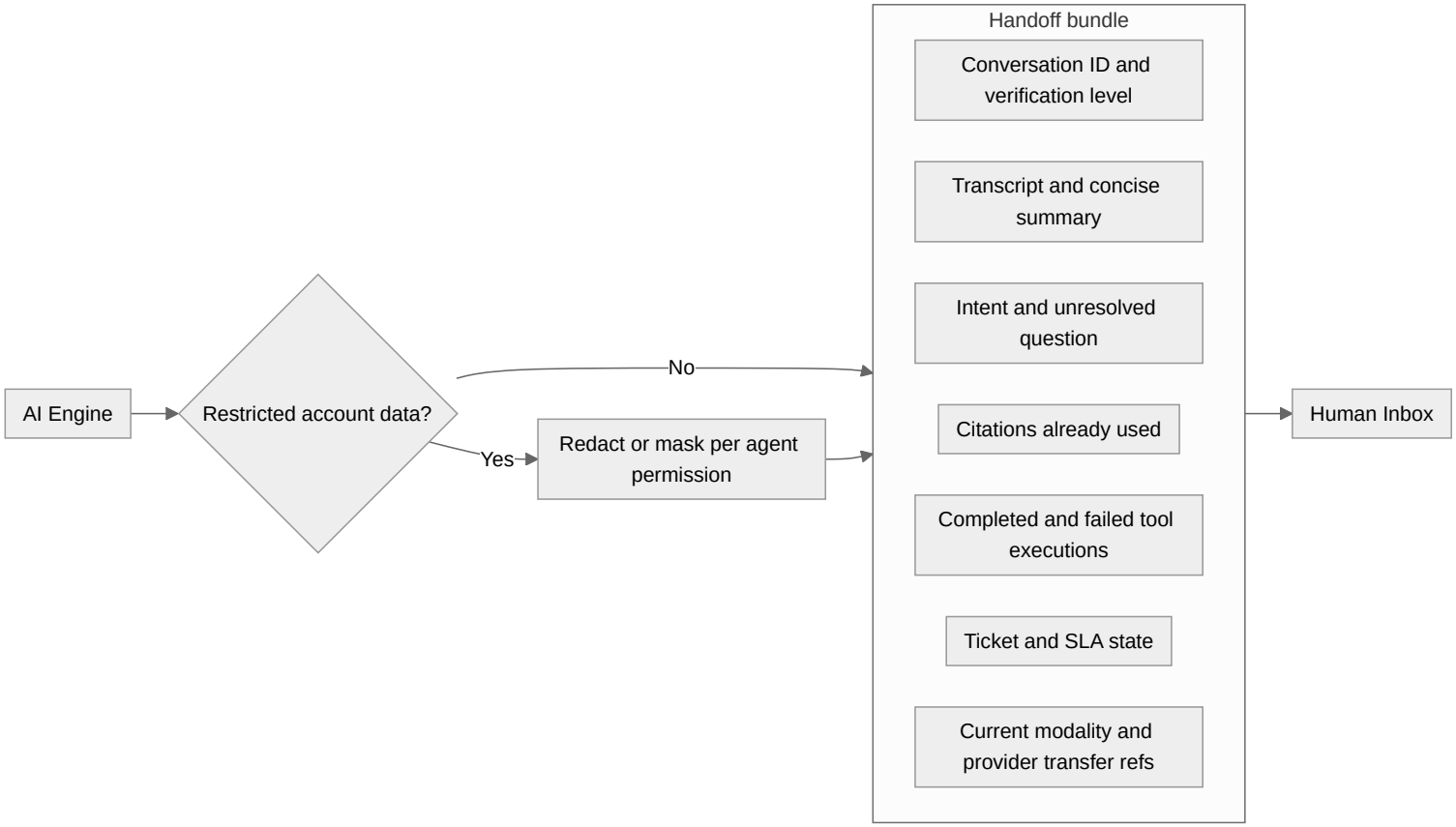


Figure 24.9: Voice handoff bundle

24.8.8 Voice privacy and consent

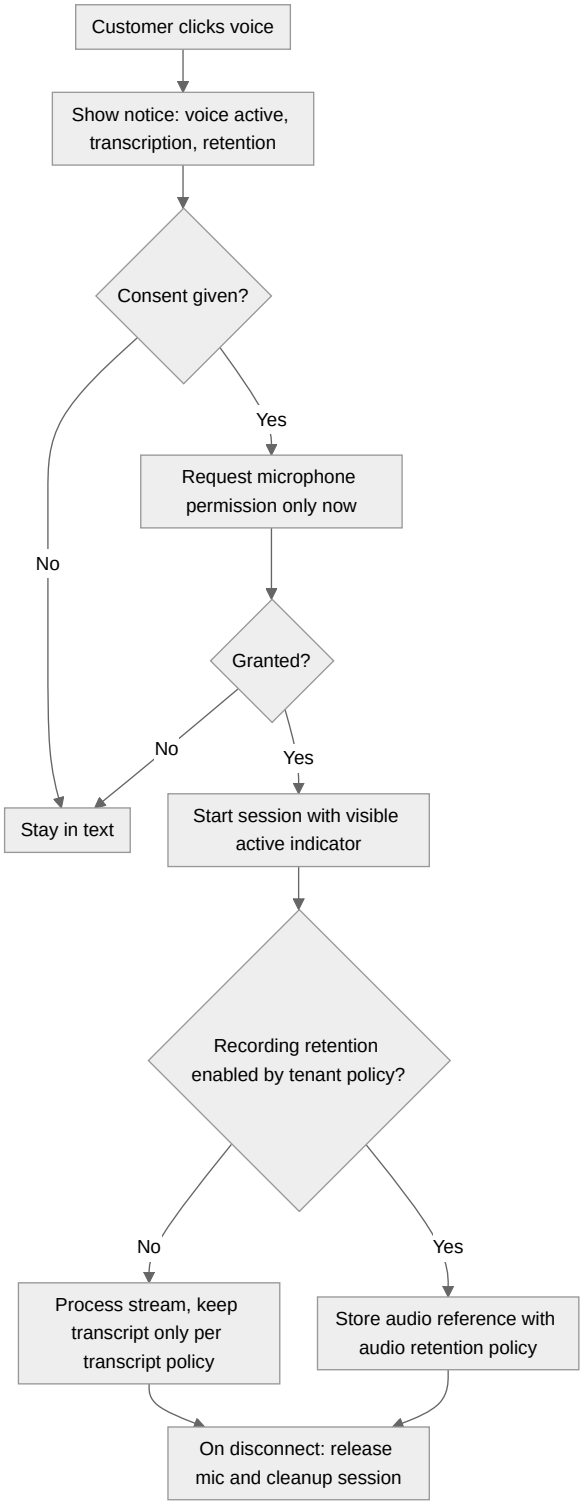


Figure 24.10: Voice privacy and consent

24.8.9 Voice metrics pipeline

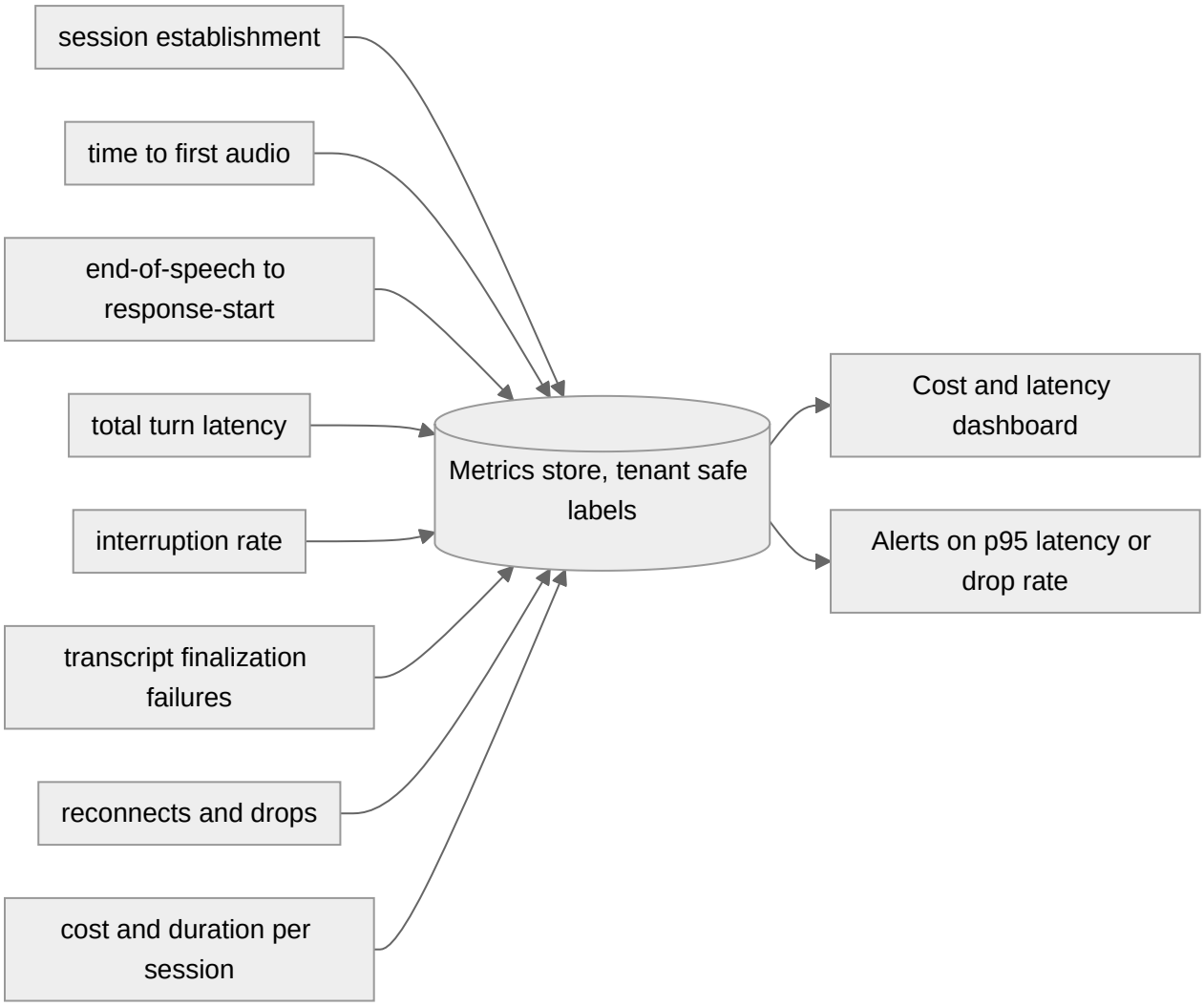


Figure 24.11: Voice metrics pipeline

Chapter 25

Integrations and Tool Contracts

25.1 Connector architecture

Use three layers:

1. **Tool definition:** what the AI may request, with a stable name, description, input schema, output schema, and risk classification.
2. **Tool execution service:** validation, authorization, idempotency, audit, timeout, and result normalization.
3. **Connector adapter:** provider-specific authentication, endpoint calls, payload mapping, and error interpretation.

Example: `AI engine` → `orders.lookup tool` → `tool execution service` → `commerce connector` → `commerce API`

The AI should never receive raw integration credentials or unrestricted HTTP access.

25.2 Tool definition fields

- stable tool name, e.g. `orders.lookup`;
- human-readable description;
- version;
- JSON input schema;
- output schema;
- allowed agent/tenant scope;
- required permission;
- risk level: read-only, reversible write, sensitive write;
- confirmation/approval requirement;
- timeout and retry policy;
- idempotency behavior;
- audit and redaction policy.

25.3 Example: read-only order lookup

```
{
  "name": "orders.lookup",
  "version": 1,
  "risk": "read_only",
  "input_schema": {
    "type": "object",
    "properties": {
      "order_reference": {
        "type": "string",
        "minLength": 3,
        "maxLength": 80
      }
    },
    "required": ["order_reference"],
    "additionalProperties": false
```

```
  },
  "output_schema": {
    "type": "object",
    "properties": {
      "status": {"type": "string"},
      "estimated_delivery": {"type": ["string", "null"]}
    },
    "required": ["status", "estimated_delivery"],
    "additionalProperties": false
  }
}
```

This is a contract example, not a claim that any particular commerce provider uses this schema.

25.4 Error mapping

Normalize connector failures into categories:

- `AUTHENTICATION_FAILED`
- `AUTHORIZATION_DENIED`
- `NOT_FOUND`
- `VALIDATION_FAILED`
- `RATE_LIMITED`
- `TIMEOUT`
- `PROVIDER_UNAVAILABLE`
- `CONFLICT`
- `UNKNOWN_OUTCOME`

Only retry failures that are safe to retry. For writes, an unknown outcome may require a status lookup or human review instead of blind retry.

25.5 Webhook handling

1. Verify provider signature and timestamp.
2. Resolve the configured channel/integration from trusted endpoint configuration.
3. Enforce body size and schema limits.
4. Persist event ID and payload/minimal normalized data.
5. Return success only after durable acceptance.
6. Process asynchronously.
7. Deduplicate provider retries.
8. Record processing state and failure reason without leaking secrets.

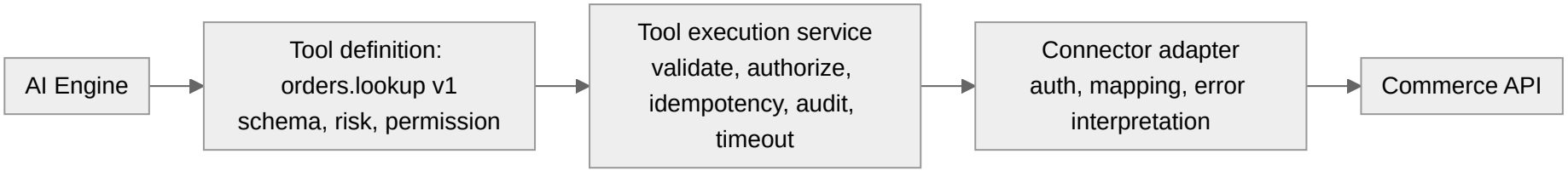


Figure 25.1: Three-layer connector architecture

Chapter 26

Security, Privacy and Reliability

26.1 Tenant isolation

- Resolve tenant from authenticated identity or trusted channel configuration.
- Never trust a tenant ID from an arbitrary customer payload.
- Apply tenant filters to SQL, search/vector retrieval, object storage, cache keys, jobs, events, and analytics.
- Add cross-tenant access tests to CI.
- Use least-privilege database and service credentials.

26.2 Authentication and authorization

- Separate organization administrators, support agents, customers, and service identities.
- Enforce permissions on the server for every protected operation.
- Verify customer identity before exposing protected order/account details.
- Require confirmation/approval for sensitive or irreversible actions.
- Re-check authorization at execution time, not only when the AI proposes a tool call.

26.3 Secret management

- Store provider credentials in a secret manager or protected encrypted store.
- Reference secrets by identifier in configuration.
- Redact tokens, passwords, API keys, and authorization headers from logs and traces.
- Support rotation and revocation.

26.4 Prompt injection and untrusted content

- Treat customer messages, uploaded files, retrieved passages, web pages, and tool results as data, not authority.
- Keep tool permissions outside the model prompt.
- Allowlist tool names and validate every argument.
- Prevent retrieved instructions from changing system policy.
- Require human approval for high-risk actions.
- Use output validation and structured schemas where appropriate.

26.5 Privacy and retention

- Classify personal and sensitive data.
- Define retention for messages, transcripts, raw audio, uploaded files, logs, and audit events separately.
- Support access/deletion workflows where applicable.
- Limit access to transcripts and recordings.

- Avoid putting unnecessary personal data into prompts or evaluation datasets.
- Use synthetic or redacted data for tests whenever practical.

26.6 Reliability

- Durable inbox for inbound events.
- Transactional outbox for important events.
- Idempotency keys for retried commands.
- Bounded retries with exponential backoff and jitter.
- Dead-letter/review queue for exhausted retries.
- Provider timeouts and circuit breakers.
- Health checks and dependency readiness.
- Graceful degradation: if RAG is down, do not pretend a grounded answer was produced; ask/hand off instead.
- If AI provider is down, communicate temporary unavailability or route to a human.

26.7 Upload safety

- Limit size and accepted file types.
- Validate actual file content rather than trusting extension.
- Malware-scan files where feasible.
- Keep private objects non-public by default.
- Use short-lived signed URLs when appropriate.
- Apply tenant and retention policies to object storage.

26.8 Security testing checklist

- Cross-tenant read/write tests
- Unauthorized tool invocation
- Prompt injection through user input and retrieved documents
- Webhook signature failures and replay attempts
- Duplicate webhook delivery
- Tool timeout with unknown write outcome
- Secret leakage in logs/traces
- Unauthorized transcript/audio access
- Rate-limit and abuse scenarios

Chapter 27

Observability, Analytics and AI Evaluation

27.1 Correlation model

Carry:

- `trace_id`
- `correlation_id`
- `causation_id`
- `tenant_id` (safe internal identifier; avoid high-cardinality metric labels)
- `conversation_id`
- `turn_id`
- `tool_execution_id`
- `provider_request_id` where available

Do not place message text, customer email, phone, access tokens, or other sensitive values in metric labels.

27.2 Operational signals

27.2.1 API and platform

- request rate, error rate, latency;
- queue depth and oldest-job age;
- worker failures and retry count;
- database connection pool, slow queries;
- object-storage errors;
- dependency health.

27.2.2 Conversation and delivery

- inbound acceptance and processing latency;
- outbound delivery success/failure;
- duplicate event count;
- conversation completion and handoff;
- abandoned session rate.

27.2.3 AI and RAG

- model latency, errors, token usage/cost;
- retrieval latency, empty retrieval, source distribution;
- tool proposal/authorization/success/failure;
- groundedness and unsupported-claim review;
- clarification and escalation rates.

27.2.4 Voice

- session establishment time;
- time to first audio;
- turn latency;
- interruption rate;
- transcript finalization failures;
- reconnects and dropped sessions.

27.3 Evaluation plan

Build a curated set of realistic, anonymized or synthetic conversations covering:

- common FAQ;
- ambiguous questions;
- missing information;
- conflicting or stale documents;
- prompt injection attempts;
- requests for restricted customer data;
- tool success and tool failure;
- external provider timeout;
- customer asks for human;
- language switching;
- interruption and voice transcript errors.

Score:

- factual correctness;
- source support/groundedness;
- appropriate clarification;
- correct tool selection and arguments;
- authorization compliance;
- correct handoff decisions;
- response usefulness and tone;
- latency and cost.

Use deterministic assertions for permissions, schemas, and state transitions. Use model-based judges only as one signal, not as the sole release gate.

27.4 Release gates

A new prompt/model/agent version should not be published until:

- critical safety tests pass;
- tool authorization tests pass;
- regression quality is within agreed thresholds;
- latency and cost are measured;
- rollback path is known;
- a versioned release record exists.

Part V

Technology Report

Chapter 28

Technology Stack and Choices

28.1 Status and how to read this

This document proposes the technologies for the AI Customer Support Platform. It is a **recommendation, not a decision**. Each choice is confirmed or replaced through the architecture decision records (ADRs) listed in the release plan. Where a provider or product is named, it is an example of a category, and the product docs already require that provider-specific details live in adapters, not in the core domain.

Vendor names, versions, prices, licenses and data policies change. Verify them at the time of each ADR.

28.2 Selection criteria

1. **Fit for a small team and a modular monolith.** Few moving parts, one primary language, one primary database.
2. **Proven and boring where it matters.** Use mature tools for storage, queues, auth and delivery. Spend novelty budget on the AI behavior.
3. **Security and tenant isolation first.** The technology must make cross-tenant access hard, not just unlikely.
4. **Portable at the core.** No LLM, speech, channel or cloud vendor type appears in the domain.
5. **Observable and testable.** Every important decision, tool call and failure can be traced and replayed in tests.
6. **Reasonable cost.** Start on the simplest option that meets the requirement and keep a documented trigger for moving up.

28.3 Assumptions

- A small team, text MVP first, voice and operations later.
- TypeScript is the primary language. If the team prefers Python, the rows marked “language” change and the architecture does not.
- Cloud-agnostic containers. Managed services are fine, but no design depends on one cloud.
- The first release has exactly one read-only business tool and one channel (website chat).

28.4 Stack at a glance

Layer	Proposed choice	Alternatives	Why, and ADR
Language and runtime	TypeScript on Node.js LTS	Python with FastAPI, Kotlin with Spring	One language for API, worker, widget and console. Types and JSON Schema are shared. ADR-001
Backend framework	NestJS modules	Fastify with hand-made modules, FastAPI	Modules, dependency injection and guards make module boundaries explicit. ADR-001

Layer	Proposed choice	Alternatives	Why, and ADR
API style	REST with OpenAPI 3.1, WebSocket for chat, SSE as fallback	GraphQL, gRPC between services later	Simple, documented, versioned. ADR-001
Database	PostgreSQL 16	MySQL, CockroachDB	Transactions, JSONB, row-level security, outbox and queue in one place. ADR-002
Tenant isolation	Row-level security plus tenant-aware repositories	Schema per tenant, database per tenant	Isolation enforced in the database, not only in code. ADR-002
Vector and keyword search	pgvector plus Postgres full-text search, fused ranking	Qdrant, OpenSearch, Pinecone	One datastore for the MVP and tenant filters in SQL. ADR-004
Queue and jobs	Postgres-based queue (pg-boss) with outbox and inbox tables	Redis with BullMQ, RabbitMQ, SQS	Jobs commit in the same transaction as domain changes. ADR-003
Cache and rate limits	Redis, optional at MVP	In-memory, edge rate limiting	Token buckets and short-lived session data
Object storage	S3-compatible storage (MinIO locally)	GCS, Azure Blob	Private objects, signed URLs, tenant prefixes
Secrets	Cloud secret manager or HashiCorp Vault, SOPS for dev config	Doppler, Kubernetes secrets	Secrets are referenced by id, never stored in config or prompts
Staff authentication	OIDC provider (Keycloak, Auth0 or Cognito) with RBAC in the app	Ory, WorkOS	Do not build password handling. ADR-009
Customer session	Short-lived signed JWT scoped to tenant, channel and conversation	Opaque server-side tokens	Cheap to verify and scoped by claims. ADR-009
LLM access	Provider adapter. Default candidate: Anthropic Claude with model routing	OpenAI, Google, self-hosted models	Small model for classification and summaries, mid model for replies, large model for hard cases and judging. ADR-005
Embeddings	Embedding adapter. Candidates: Voyage AI, OpenAI embeddings, open-source BGE or E5	Hosted multilingual models	Chosen on retrieval quality in the target languages. ADR-004 and ADR-005
Reranking	Optional cross-encoder or hosted reranker	None at MVP	Add only if retrieval evaluation shows a gain
Tool contracts	JSON Schema 2020-12 validated with Ajv, Zod in application code	TypeBox, Pydantic	Same schema for the model, the validator and the docs. ADR-010
Document parsing	pdf, docx and html parsers or Apache Tika or unstructured, ClamAV scan	Managed document AI	Run in the worker, never in the request path
Website widget	Preact or vanilla TypeScript in a Web Component with Shadow DOM	React	Small bundle and style isolation
Agent console and admin	React, Vite, TypeScript, TanStack Query, Radix UI, Tailwind	Vue, Angular	Mature ecosystem for tables, forms and live updates
Voice transport	WebRTC through LiveKit	Daily, Twilio Voice, plain WebSocket audio	Handles media, reconnects and agents. ADR-007
Speech	Option A: streaming STT and TTS adapters. Option B: speech-to-speech model	Whisper self-hosted, Azure Speech	Decided by a spike on latency, language, cost and privacy. ADR-006
Telephony	SIP through LiveKit or Twilio	Other SIP providers	Release 3, after browser voice is validated. ADR-007
Email and messaging	Provider adapters: SES or Postmark for email, WhatsApp Cloud API or Twilio for messaging	Other providers	Release 3, same canonical event contract

Layer	Proposed choice	Alternatives	Why, and ADR
Workflows	Deterministic state machine on Postgres, evaluate Temporal	Custom engine, Camunda	Release 3. Do not build a universal builder early
Observability	OpenTelemetry into Grafana (Prometheus, Loki, Tempo) or Datadog, Sentry for errors	Honeycomb, New Relic	Correlation ids across API, worker, model and tools
LLM tracing and evaluation	Langfuse or OpenTelemetry GenAI conventions, promptfoo or a custom harness	LangSmith, DeepEval	Replayable traces and regression suites
Feature flags	OpenFeature with Unleash or GrowthBook	Environment flags	Canary rollout and instant disable
Testing	Vitest, Testcontainers, Playwright, k6, contract tests	Jest, Cypress	Real Postgres in tests, browser tests for widget and console
Security tooling	Semgrep, gitleaks, Trivy, Renovate or Dependabot, OWASP ZAP	Snyk, SonarQube	Cheap checks on every change
CI and CD	GitHub Actions or GitLab CI, Docker images	CircleCI	Trunk-based, small merges, required checks
Infrastructure	Terraform or OpenTofu, managed container platform (for example AWS ECS Fargate)	Fly.io or Render for MVP, Kubernetes later	Kubernetes only when scaling or isolation justifies it
Documentation	Markdown, Mermaid, MADR for ADRs, OpenAPI, AsyncAPI for events, LaTeX for the report, reveal.js for decks	Confluence, Notion	Docs live in the repository

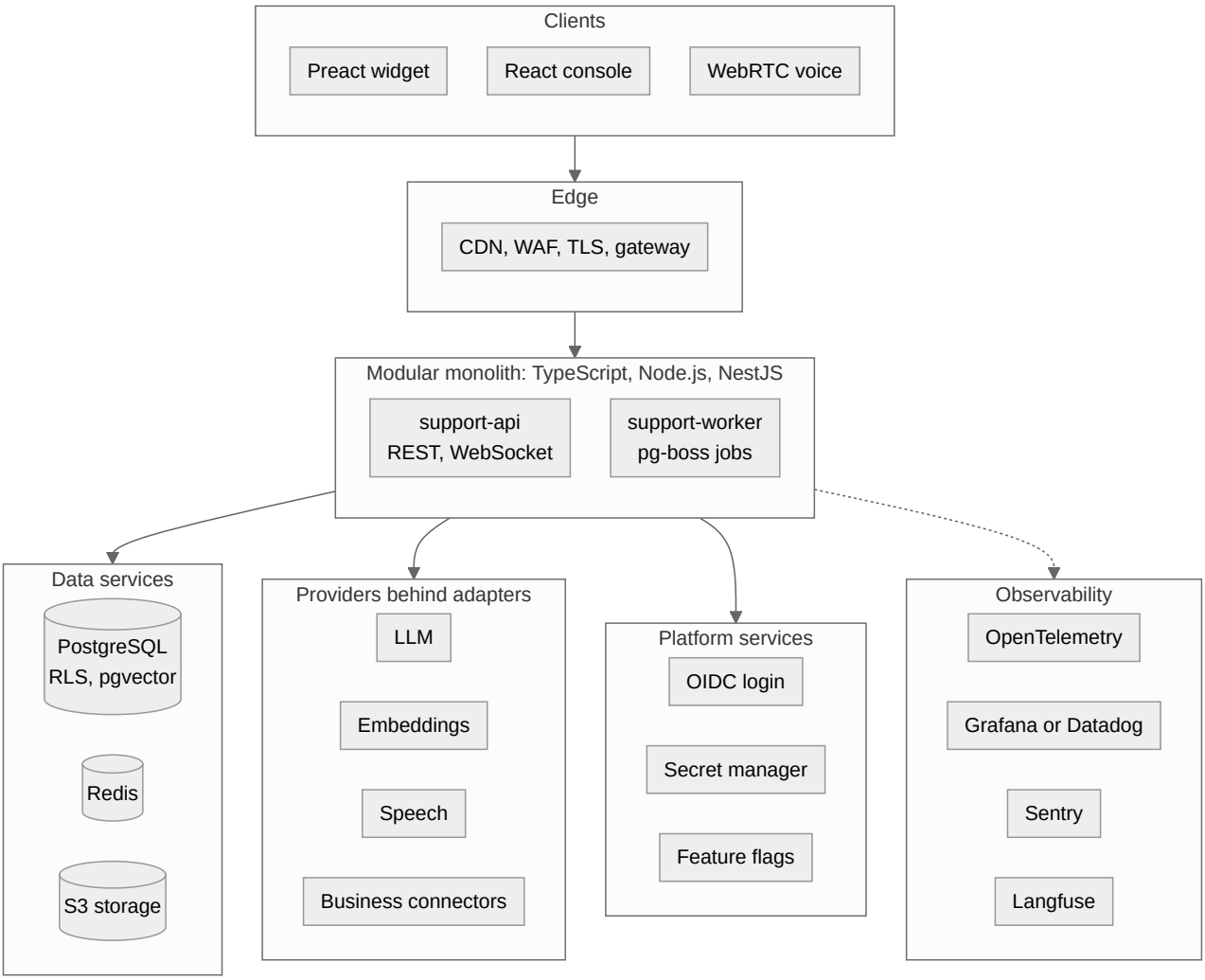


Figure 28.1: Technology stack map

28.5 How a request flows through the stack

A website message enters through the widget over HTTPS or WebSocket. The API validates it, checks the session token, and writes the message and an outbox row in one Postgres transaction. A worker picks up the turn job, builds context, retrieves knowledge with a tenant filter, and calls the model through the adapter. If the model proposes a tool call, the tool service validates it against its JSON Schema, checks authorization and idempotency, and calls the connector with a timeout and circuit breaker. The reply is persisted with citations and usage, then pushed to the widget. No database transaction is held open while the model or a business system is called.

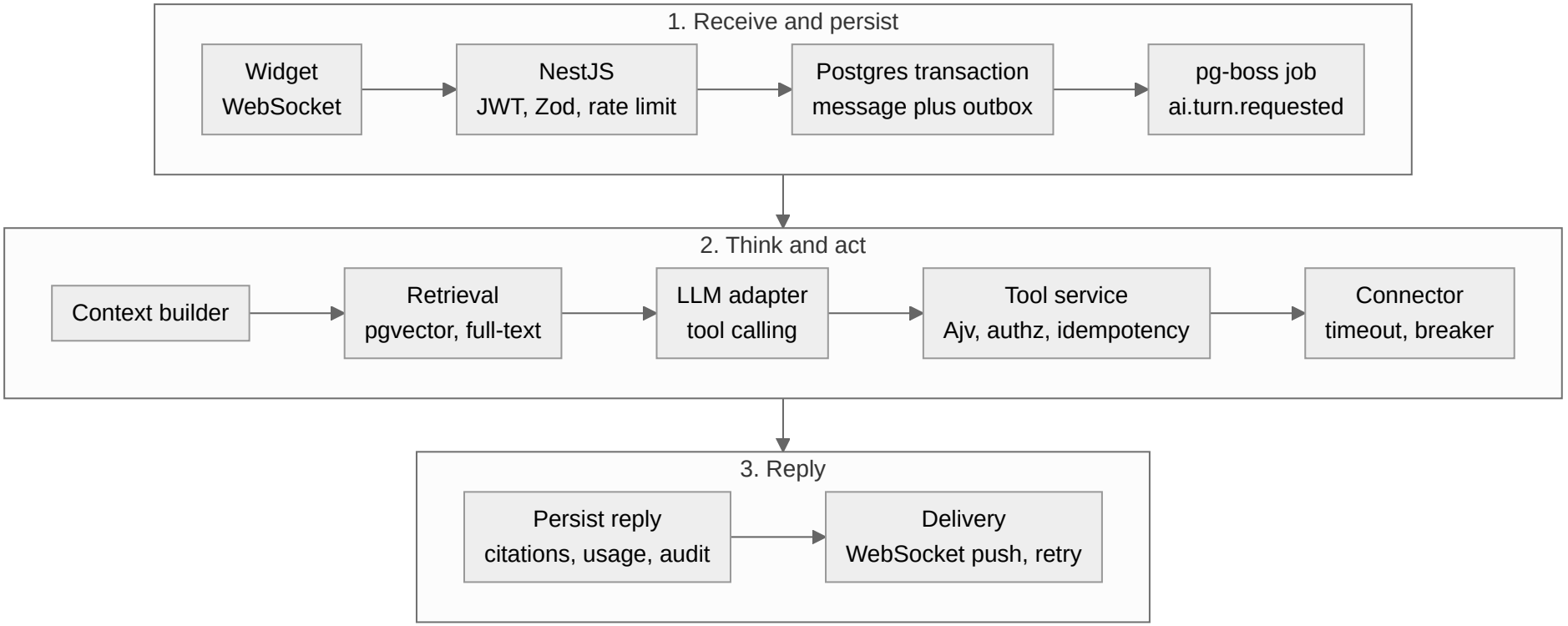


Figure 28.2: A customer message through the stack

28.6 Voice stack

Browser audio travels over WebRTC to a media server. Voice activity detection finds the end of speech. Streaming speech-to-text produces partial transcripts, which are captions only, and a final transcript, which becomes a message. The same conversation engine and the same tool authorization answer, and streaming text-to-speech plays the reply. A barge-in cancels the speech stream and the model call. The alternative, a speech-to-speech model, sits behind the same adapter and still routes tool calls through our authorization.

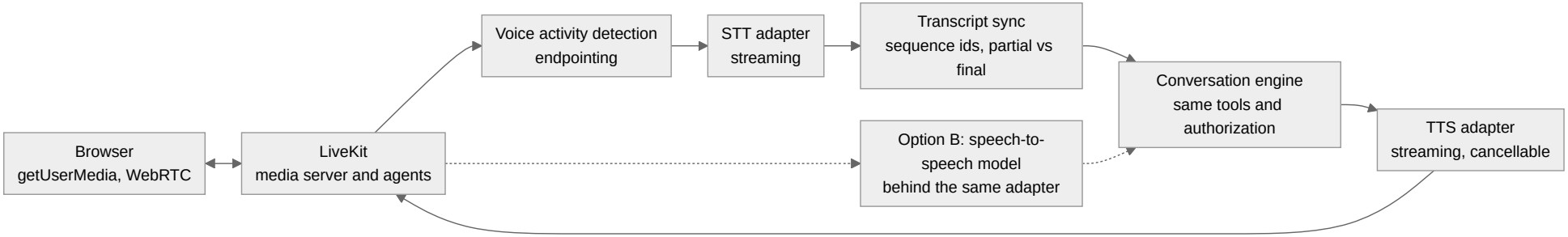


Figure 28.3: Voice stack

Chapter 29

Technology by Task

This chapter maps the technology proposal to every epic, user story and Sprint of the backlog. Each row names the main technologies and the approach. Everything is a proposal that follows the design reference chapters and is confirmed through the ADRs.

29.1 E01 — Tenant and access foundation

Story	Technologies and approach
US-001 Create and configure an organization	NestJS organizations module, one PostgreSQL transaction for the organization, default roles, settings and audit row, UUIDv7 identifiers, Zod request validation, OpenAPI contract
US-002 Invite users and assign roles	OIDC provider for login, single-use expiring invitation token signed with jose, email through a provider adapter, permission guards in NestJS, session revocation checked on every request
US-003 Test tenant isolation	Vitest with Testcontainers running a real Postgres with row-level security, two-tenant fixtures, a negative-test matrix over SQL, search, storage, cache and jobs, a required CI check, a Semgrep rule against queries without tenant context

29.2 E02 — Conversation and message core

Story	Technologies and approach
US-010 Start a conversation	NestJS REST and WebSocket gateway, conversation service on Postgres, outbox row for the created event
US-011 Send and receive text	Per-conversation sequence number under a row lock, unique client message id, WebSocket with SSE fallback, delivery state table
US-012 Inspect the timeline	Keyset pagination, visibility filter inside the repository, React console with TanStack Query and virtualized lists
US-013 Deduplicate inbound events	Unique constraint on connection and provider event id with insert-on-conflict, inbox table, job singleton keys in pg-boss

29.3 E03 — Website chat channel

Story	Technologies and approach
US-020 Configure the website widget	React admin form, allowed origins and public key per channel, embed snippet served from object storage behind a CDN, Web Component widget
US-021 Chat session	Short-lived JWT signed with jose and carrying tenant, channel and conversation claims, origin check, per-IP rate limit with Redis token buckets, optional bot challenge

Story	Technologies and approach
US-022 Reconnect and resume	WebSocket client with exponential backoff, resume from the last seen sequence over REST, client-side deduplication by message id, pending messages kept in local storage

29.4 E04 — Knowledge ingestion and RAG

Story	Technologies and approach
US-030 Register and upload a source	Pre-signed object-storage uploads, content sniffing with libmagic, size limits, ClamAV scan, tenant prefix, React uploader with progress
US-031 Parse, chunk, embed, index	pg-boss job, document parsers or Apache Tika or unstructured, token-aware chunking, embedding adapter in batches, pgvector HNSW index, ingestion state in Postgres, retries with backoff
US-032 Retrieve relevant chunks	Hybrid retrieval with pgvector similarity and Postgres full-text ranking fused by reciprocal rank, mandatory tenant and source filter enforced by row-level security, optional reranker, retrieval run logging
US-033 Grounded answer or fallback	Structured output with answer, citation ids and confidence, evidence threshold in code, citation check against the retrieved set, fallback templates, evaluation cases

29.5 E05 — AI orchestration

Story	Technologies and approach
US-040 Process a turn through a versioned agent	Orchestrator as an explicit state machine (XState or a transition table), LLM adapter, immutable AgentVersion rows with prompts versioned in Git, OpenTelemetry span per step, Langfuse trace
US-041 Follow-up using context	Context builder, token counting, rolling summaries produced asynchronously by a small model, provider prompt caching where available, strict per-conversation filters
US-042 Handle model failure	AbortController timeouts, retry with jitter (p-retry or cockatiel), circuit breaker, fallback model configuration, safe message templates, failure counters
US-043 Publish and roll back an agent version	Immutable version table with a deployment pointer, release gate job that runs the evaluation harness, audit events, React admin screens

29.6 E06 — Safe business tools

Story	Technologies and approach
US-050 Enable a versioned tool	Tool registry in Postgres, JSON Schema 2020-12 checked with Ajv, per-agent allowlist, schema linting in CI that requires additionalProperties false
US-051 Order status lookup	orders.lookup through a connector adapter, undici HTTP client, Zod for typed responses, customer verification level from the identity module, minimal result, contract tests with recorded fixtures

Story	Technologies and approach
US-052 Timeout and duplicate safety	Idempotency keys with a unique index, status lookup, timeouts, circuit breaker, UNKNOWN_OUTCOME state, dead-letter queue in pg-boss
US-053 Audit tool calls	Append-only audit table with UPDATE and DELETE revoked and monthly partitions, pino redaction paths, trace-linked logs, SQL and CSV export, optional object-lock archive

29.7 E07 — Human inbox and handoff

Story	Technologies and approach
US-060 View the unassigned queue	Postgres tables, claim with SELECT FOR UPDATE SKIP LOCKED or an optimistic version column, live updates over WebSocket, TanStack Query
US-061 Customer requests a human	Intent detection by rules plus a small model, conversation state machine, handoff event through the outbox, routing worker, status message pushed to the widget
US-062 Handoff summary	Summarization prompt over trusted data with structured output, permission-based redaction function, bundle stored as JSONB, console side panel
US-063 Resolve the conversation	State machine transitions in a transaction, audit event, ticket link, React console action

29.8 E08 — Tickets and case tracking

Story	Technologies and approach
US-070 Create a ticket from a conversation	Ticket module on Postgres, link table to conversations, status history written in the same transaction, OpenAPI endpoint
US-071 Filter tickets	Composite indexes, keyset pagination, permission scope applied as a query condition, TanStack Table in the console

29.9 E09 — Voice MVP

Story	Technologies and approach
US-080 Start and end a voice session	LiveKit room per conversation, short-lived access tokens issued by the API, getUserMedia with a consent screen, session state and quota in Postgres
US-081 Speak and hear a response	Option A: streaming STT adapter, engine, streaming TTS adapter. Option B: speech-to-speech adapter. LiveKit agent worker. Text fallback when speech fails
US-082 Interrupt the AI	Voice activity detection and endpointing, cancel the TTS stream and the model request with AbortController, store only the delivered part of the reply
US-083 Switch between text and voice	Server-side sequence ids, transcript synchronizer, WebSocket events, system events in the timeline

Story	Technologies and approach
US-084 Voice handoff	Room transfer or SIP transfer depending on the provider, callback ticket as fallback, handoff bundle shared with the agent

29.10 E10 — Workflow, routing and SLA

Story	Technologies and approach
US-090 Define a workflow	Versioned workflow definitions validated by JSON Schema, a state machine and step-run tables on Postgres, evaluate Temporal for durable execution
US-091 Routing rules and SLA targets	Ordered rules stored in Postgres and evaluated in code, business-hours math with Luxon, SLA timers as delayed pg-boss jobs
US-092 Retry a workflow step safely	Idempotency key per step, pg-boss retry with backoff, dead-letter queue, compensation steps

29.11 E11 — Integrations and additional channels

Story	Technologies and approach
US-100 Configure an integration securely	Secret manager with references only, scheduled health checks as cron jobs, connector definitions as TypeScript packages implementing one interface, React admin screens
US-101 Normalize a provider webhook	Raw-body capture, HMAC signature check with timing-safe compare, Zod or Ajv schema validation, inbox table, one adapter per provider, contract tests
US-102 Continue on a second channel	Identity linking through a one-time code or signed link, email and messaging provider adapters, conversation reuse policy

29.12 E12 — Observability, evaluation and usage

Story	Technologies and approach
US-110 Trace one request	OpenTelemetry SDK with HTTP, Postgres and Redis instrumentation plus manual spans for model, retrieval and tools, trace context carried inside job payloads, pino logs with trace ids, Grafana or Datadog
US-111 Resolution and handoff metrics	prom-client or OpenTelemetry metrics, SQL aggregates for outcomes, Grafana dashboards, a lint rule against sensitive metric labels
US-112 Run AI regression evaluations	Dataset files in Git, promptfoo or a custom runner, deterministic assertions first, model judge second, GitHub Actions job, results stored as evaluation runs, release gate
US-113 Usage by agent and channel	Usage records table, provider usage reconciliation job, quotas in Redis or Postgres, Grafana panels

29.13 Technology by Sprint

Sprint	Technologies introduced	Spikes and decisions
Sprint 0	Monorepo with pnpm workspaces, Docker Compose with Postgres, Redis and MinIO, repeatable migrations, CI skeleton, Terraform skeleton, secret handling, ADR template	Hosting target, queue choice
Sprint 1	NestJS modules, Postgres with row-level security, tenant context middleware, audit table, OpenTelemetry bootstrap, Vitest with Testcontainers	ADR-001, ADR-002, ADR-003
Sprint 2	Preact widget, WebSocket gateway, JWT sessions, inbox table, Playwright end-to-end tests	Session policy
Sprint 3	Object storage uploads, ClamAV, parsers, pg-boss ingestion jobs, embedding adapter, pgvector and full-text search	ADR-004 vector technology
Sprint 4	LLM adapter, prompts in Git, structured outputs, citations, small evaluation harness, usage records	ADR-005 provider and model routing
Sprint 5	Tool registry with Ajv, connector adapter with circuit breaker, React console inbox, live updates	ADR-010 tool authorization
Sprint 6	Tickets, dashboards and alerts, k6 load tests, backup and point-in-time recovery drill, feature flags, evaluation release gate in CI	Release 1 gates
Sprint 7	LiveKit, voice activity detection, STT and TTS adapters or speech-to-speech adapter, voice metrics	ADR-006, ADR-007, ADR-008
Sprint 8	Transcript synchronizer, barge-in cancellation, voice handoff, cost and latency dashboard	Provider handoff behavior

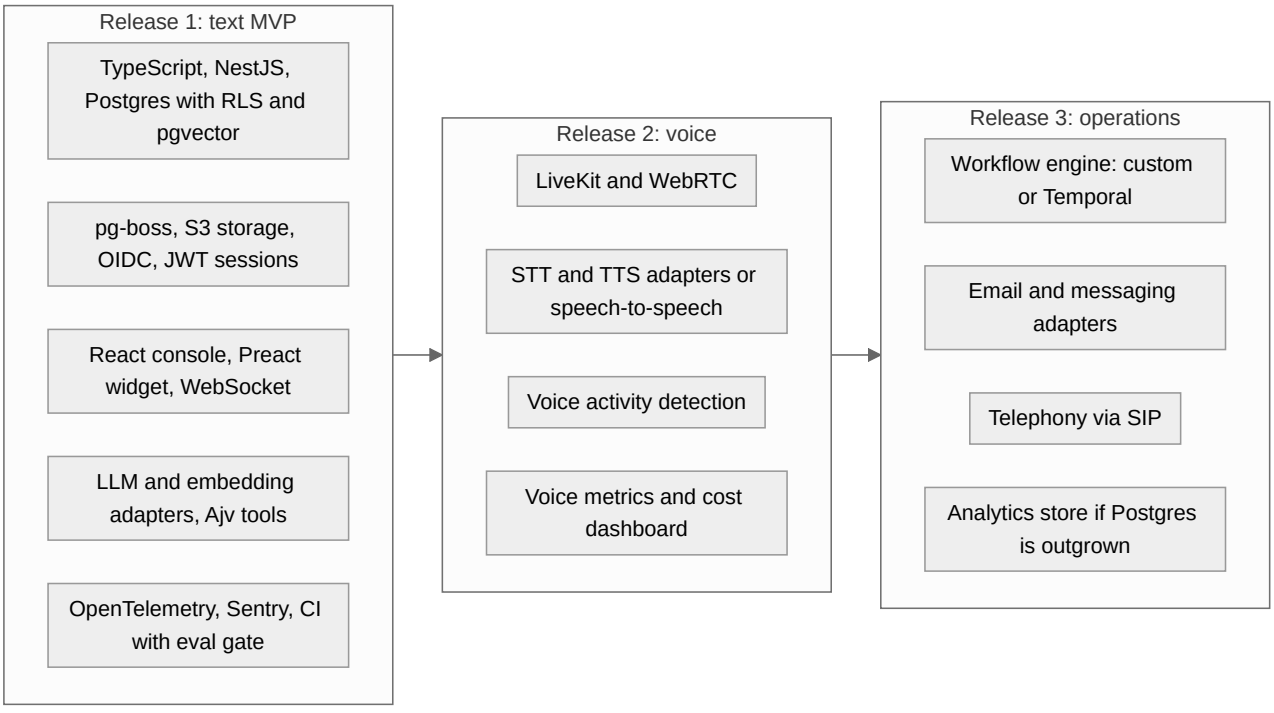


Figure 29.1: When each technology arrives

Chapter 30

Technology Roles, Rules and Decisions

30.1 Roles and usage rules

Every technology has a defined **role** (what it is for) and a **rule** (how the team must use it). The rules protect the architecture constraints: tenant isolation, provider portability, durable commits, and untrusted AI output.

Technology	Role	Rule
TypeScript and NestJS	Application code, module boundaries	Modules talk through interfaces only. No module imports another module's internals.
PostgreSQL	System of record, queue, outbox, vector index	Every tenant table has an organization id and a row-level security policy. No transaction stays open during an external call.
Row-level security	Database-enforced tenant isolation	The application sets the tenant context per request. A missing context returns nothing.
pgvector and full-text search	Knowledge retrieval	Every query carries tenant and source filters. Switching to a dedicated engine needs an ADR and a measured trigger.
pg-boss	Durable jobs, retries, delayed jobs	Jobs are enqueued in the same transaction as the change. Consumers are idempotent.
Redis	Rate limits and short-lived data	Nothing durable lives only in Redis. Keys carry the tenant prefix.
S3-compatible storage	Documents and permitted media	Private by default, signed URLs, tenant prefixes, retention policy per data class.
OIDC provider	Staff login	No password handling in our code. Authorization stays in the application.
JWT with jose	Customer widget session	Short lifetime, scoped claims, verified on every call. Never trust a tenant id from a payload.
LLM adapter	Model access	Vendor types never leave the adapter. Model name, version, latency and usage are recorded.
Embedding adapter	Text vectors	Index and query use the same model version. A model change triggers a re-index plan.
Ajv and JSON Schema	Tool and event contracts	Every tool input and output has a schema with additionalProperties false. Validation runs before execution.
Zod	Application-level validation	Validate at the edge and at adapter boundaries.
Connector adapters	Business-system access	One interface, typed errors, timeout and idempotency. The AI never sees credentials.
Secret manager	Credentials	Reference by id. Never in logs, prompts, traces or the repository.
Preact widget	Customer chat UI	Shadow DOM isolation, no secrets in the bundle, works behind strict CSP.
React console	Agent and admin UI	Server enforces permissions. The UI hides, the API denies.
WebSocket and SSE	Live messages	Streaming is not a committed message. Persist first, push second.
LiveKit and WebRTC	Real-time voice	Access tokens are short-lived and scoped to one conversation. Partial transcripts stay ephemeral.
STT and TTS adapters	Speech	Providers stay behind adapters. Cancel on barge-in. No raw audio stored by default.

Technology	Role	Rule
OpenTelemetry	Traces, metrics, logs	Every request and job carries trace id, correlation id and tenant id. No personal data in metric labels.
Sentry and Langfuse	Errors and LLM traces	Redact before sending. Sample according to policy.
OpenFeature	Feature flags	Every risky feature ships behind a flag with a known off switch.
Vitest and Testcontainers	Unit and integration tests	Use a real Postgres for tenant and authorization tests.
Playwright	End-to-end tests	Cover the widget and the console critical paths.
promptfoo or custom harness	AI regression evaluation	Deterministic assertions gate release. Model judges are one signal only.
k6	Load tests	Run before each release against agreed latency budgets.
Semgrep, gitleaks, Trivy	Static and dependency security	Required checks on every merge.
GitHub Actions or GitLab CI	Automation	Trunk-based, small merges, no manual production deploys.
Terraform	Infrastructure as code	Environments are reproducible. Secrets never in state in plain text.
Docker	Packaging	One image runs as API and as worker.

30.2 Cross-cutting technology

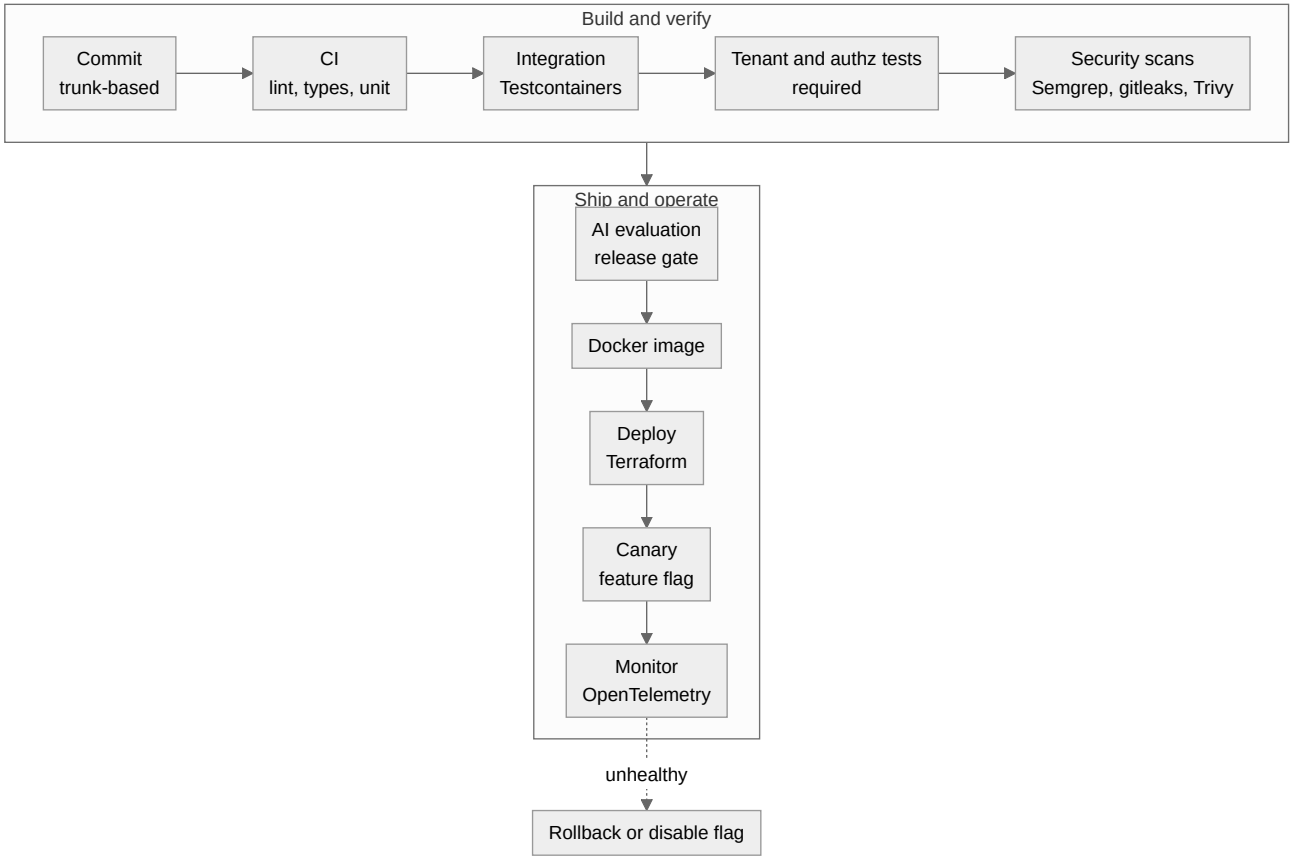


Figure 30.1: Delivery toolchain

30.2.1 Security

- Row-level security, tenant-aware repositories, and cross-tenant tests in CI.
- Input validation with Zod and Ajv, output validation for model results.
- Secrets in a manager, scanned with gitleaks, rotated and revocable.
- Webhooks verified by HMAC signature and timestamp, deduplicated by event id.
- File uploads checked by content type, size and malware scan.
- Prompt injection tests in the evaluation suite and in the security checklist.

30.2.2 Observability

- OpenTelemetry traces across API, worker, model, retrieval and tools.
- Structured logs with pino, redaction of sensitive fields, trace ids on every line.
- Metrics for latency, errors, queue depth, token usage, cost, and voice quality.
- Dashboards and alerts per tenant-safe label set.

30.2.3 Testing and quality

- Test pyramid: unit, integration with real Postgres, contract tests for adapters, end-to-end for UI.
- AI evaluation set in Git, run on every agent or prompt change.
- Resilience tests: duplicate webhooks, tool timeouts with unknown outcome, provider outages.
- Load and recovery drills before Release 1.

30.2.4 Delivery

- Docker image built once and promoted through environments.
- Terraform for infrastructure, feature flags for rollout, canary tenants first.
- Rollback by disabling a flag or reverting the agent version.

30.3 Decisions to validate

Decision	Options to compare	Criteria	When
LLM provider and model routing (ADR-005)	Two or three providers and model sizes	Correctness and groundedness on the evaluation set, p95 latency, cost per resolved conversation, data policy	Sprint 4
Vector search (ADR-004)	pgvector against a dedicated engine	Recall on the evaluation set, p95 latency, operational cost, tenant filter support	Sprint 3
Queue and outbox (ADR-003)	pg-boss against Redis with BullMQ or a managed queue	Transactional enqueue, throughput, operational load	Sprint 0 and 1
Authentication (ADR-009)	Keycloak, Auth0, Cognito	Cost, hosting, multi-tenant organizations, customer verification support	Sprint 1
Speech strategy (ADR-006)	Modular STT and TTS against speech-to-speech	End-of-speech to response latency, language coverage, cost per minute, transcript control, privacy	Sprint 7
Voice transport and telephony (ADR-007)	LiveKit, Twilio, Daily	Reliability, reconnect behavior, SIP support, handoff capability	Sprint 7

Decision	Options to compare	Criteria	When
Retention and consent (ADR-008)	Audio and transcript retention models	Legal basis, product need, cost	Sprint 7
Workflow engine	Custom state machine against Temporal	Complexity of procedures, retry and timer needs, operational cost	Release 3
Hosting	Managed containers, Fly.io or Render, Kubernetes	Team skills, cost, isolation, regions	Sprint 0

30.4 Lock-in and risk assessment

Technology	Lock-in risk	Mitigation
LLM provider	Medium. Prompts and tool formats differ by vendor.	Adapter interface, provider-neutral tool schemas, evaluation suite to compare models.
Embedding model	Medium. Changing the model needs a full re-index.	Store the model version per chunk and keep a re-index job.
Speech and voice providers	High for speech-to-speech, medium for modular.	Start with one strategy, keep adapters thin, measure before committing.
PostgreSQL features	Low. Open and widely hosted.	Standard SQL where possible, extensions documented.
pgvector	Low to medium.	Retrieval behind an interface and a documented switch trigger.
Cloud services	Medium.	Containers, Terraform modules, S3-compatible API, no cloud-only core dependency.
Observability vendor	Low.	OpenTelemetry as the instrumentation layer.
Feature flag service	Low.	OpenFeature as the client API.

Part VI

UML Models

Chapter 31

UML Use Case Diagrams

Mermaid has no native use-case notation, so actors are drawn as stadium nodes outside a system-boundary subgraph and use cases as ellipses. `include` and `extend` relations are dotted arrows with stereotypes. Actors come from the product vision chapter; use cases from the epics.

31.1 System-level use cases

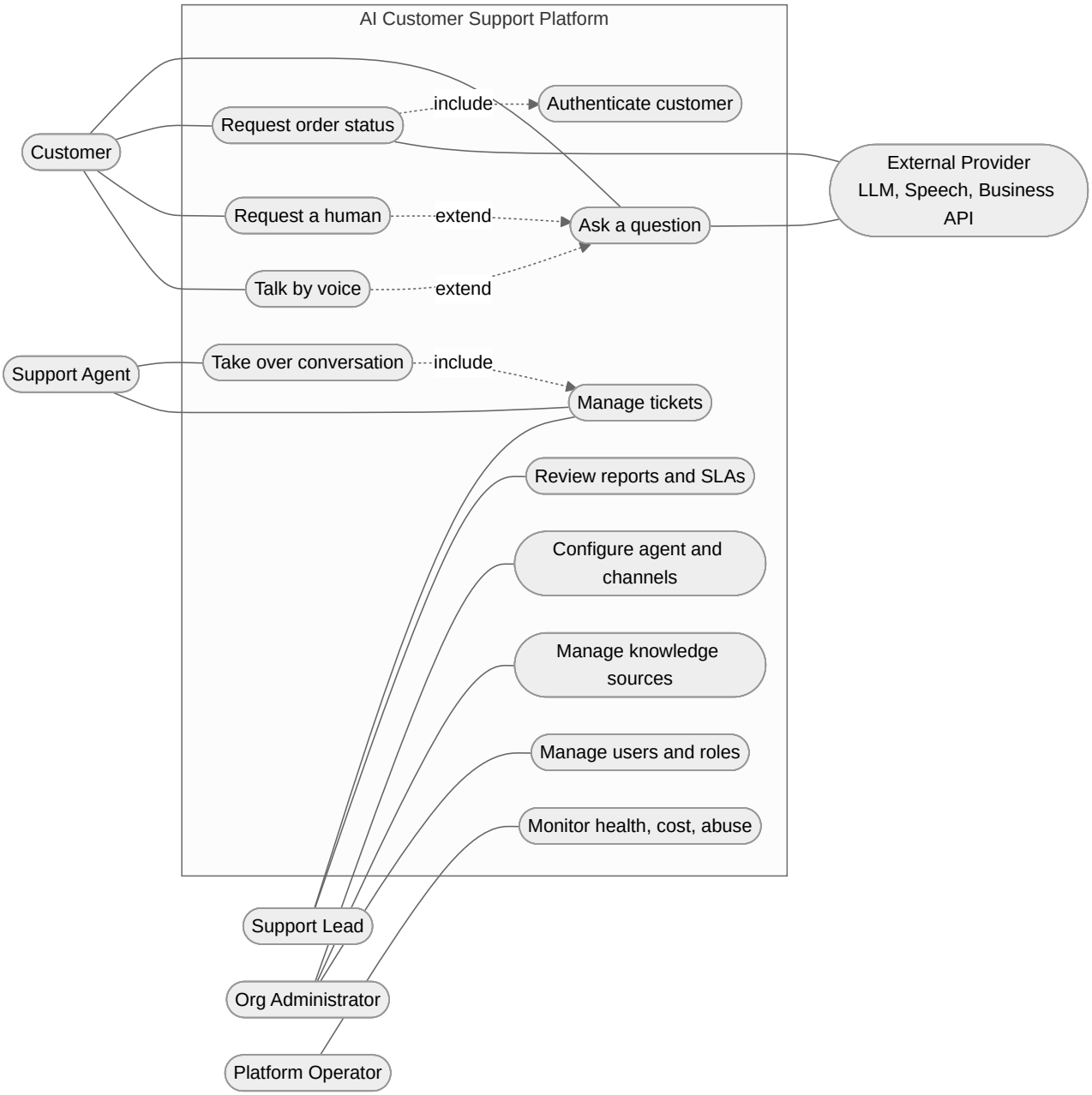


Figure 31.1: System-level use cases

31.2 Customer use cases

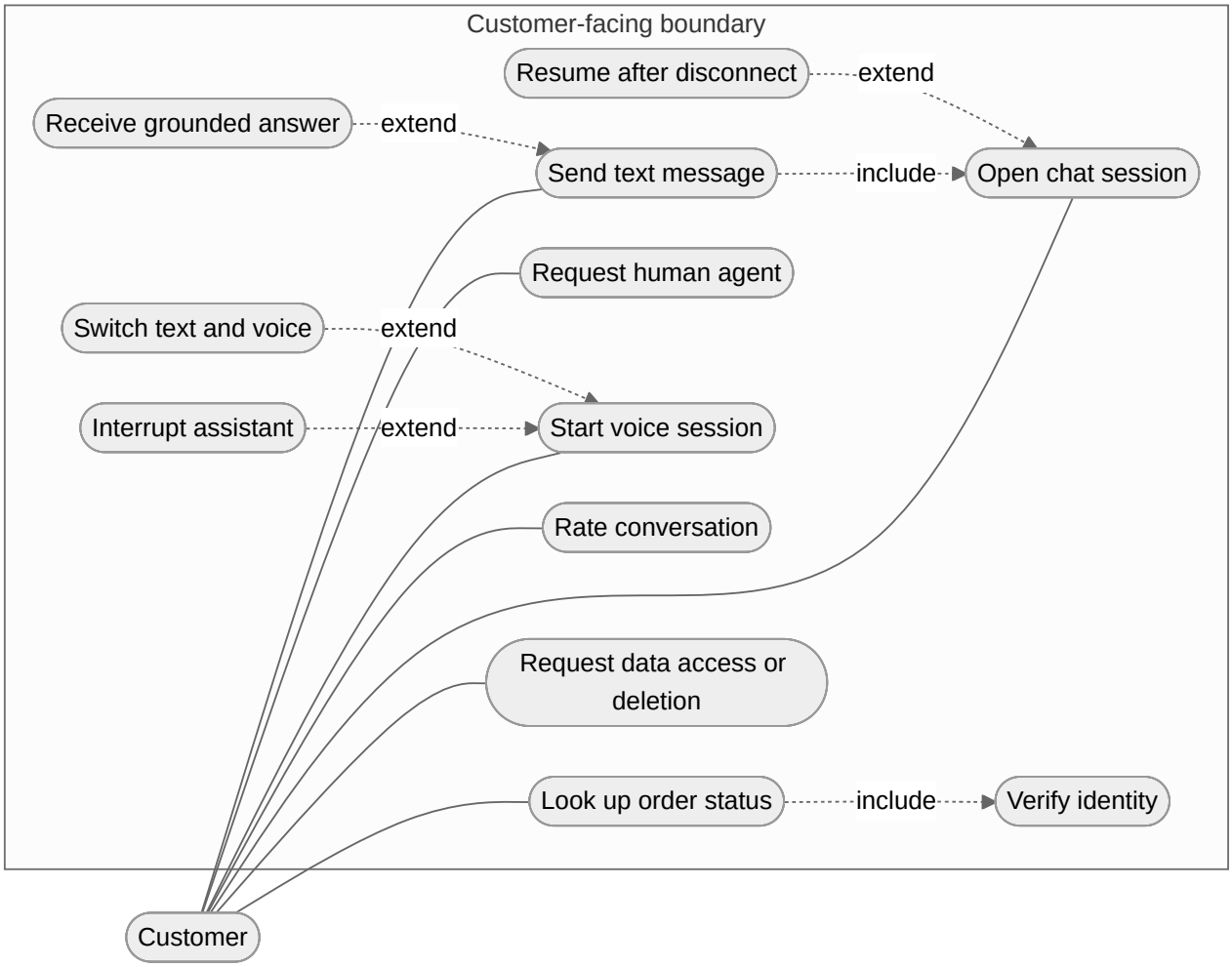


Figure 31.2: Customer use cases

31.3 Support staff use cases

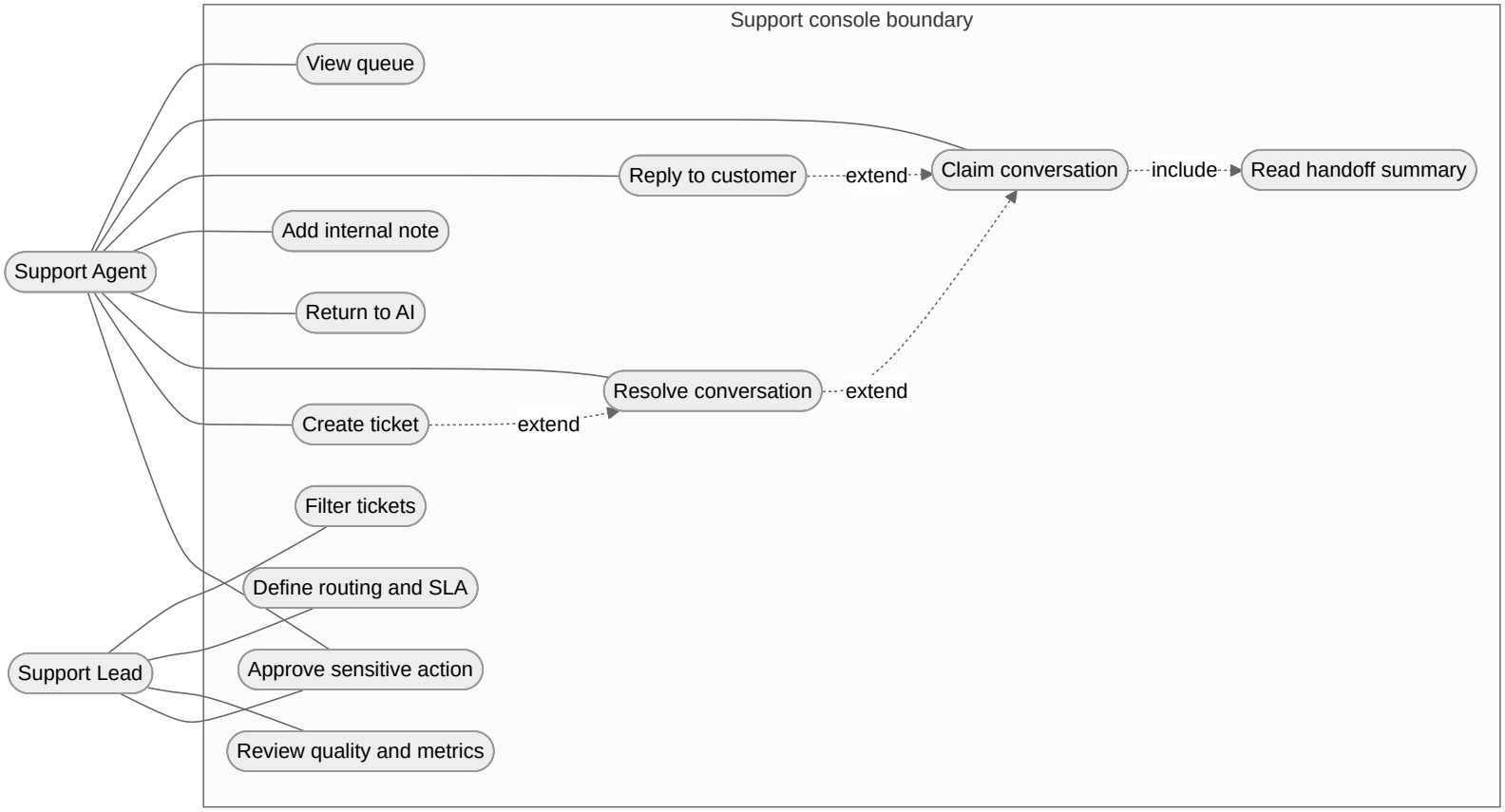


Figure 31.3: Support staff use cases

31.4 Administration and governance use cases

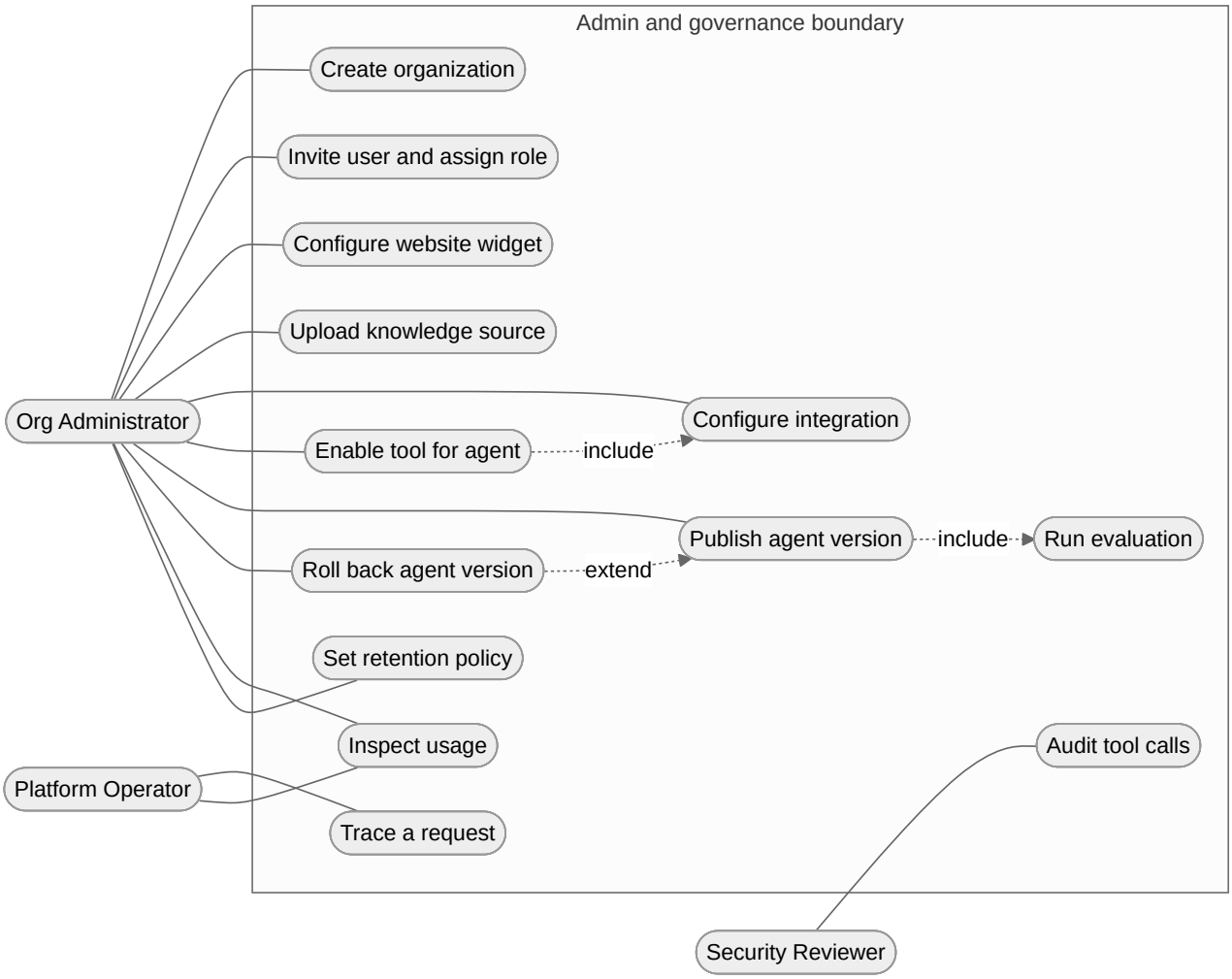


Figure 31.4: Administration and governance use cases

Chapter 32

UML Class Diagrams

Design-level classes for the modular monolith. Names follow the module catalog (the module catalog chapter) and domain model (the domain model chapter). Attributes and operations are proposals for implementation, not final signatures. One diagram per module group keeps each readable.

32.1 Access and tenancy (M18)

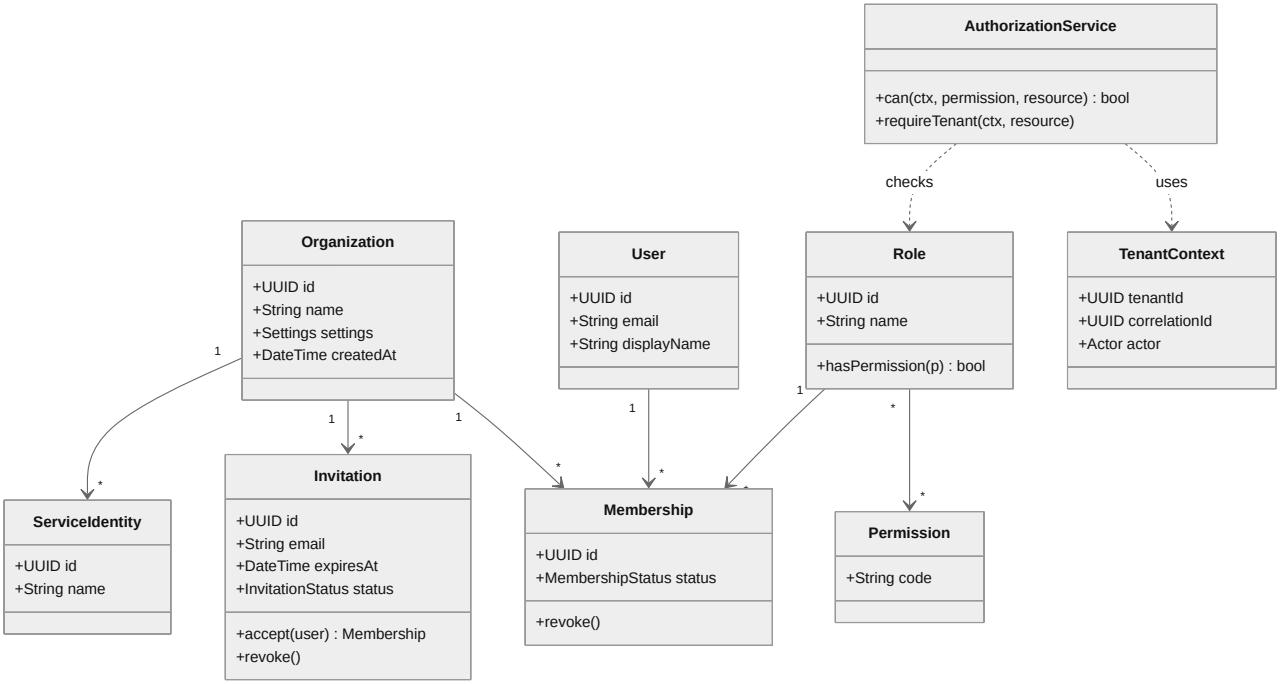


Figure 32.1: Access and tenancy (M18)

32.2 Conversation core (M01, M03)

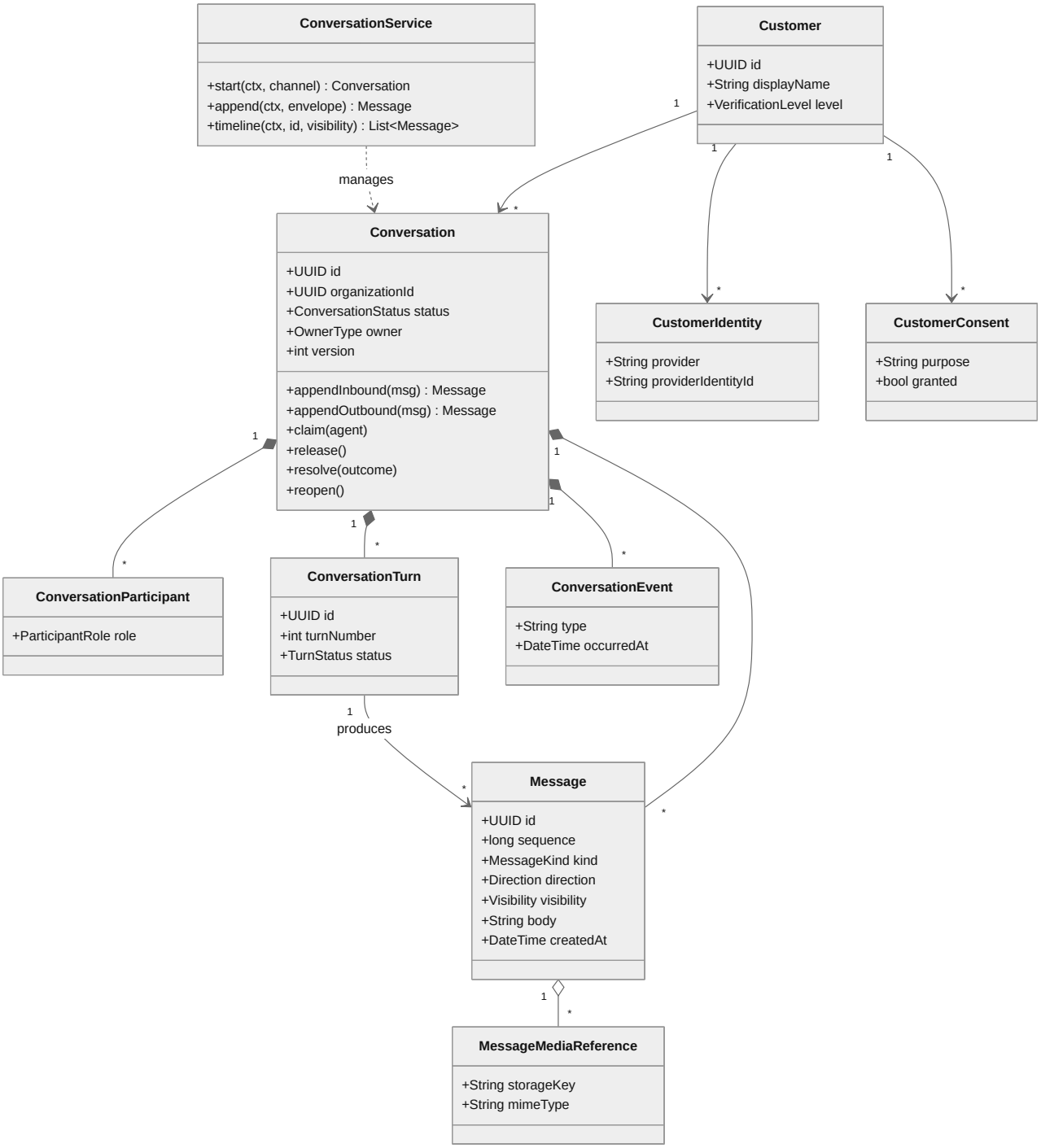


Figure 32.2: Conversation core (M01, M03)

32.3 Channel management and delivery (M02)

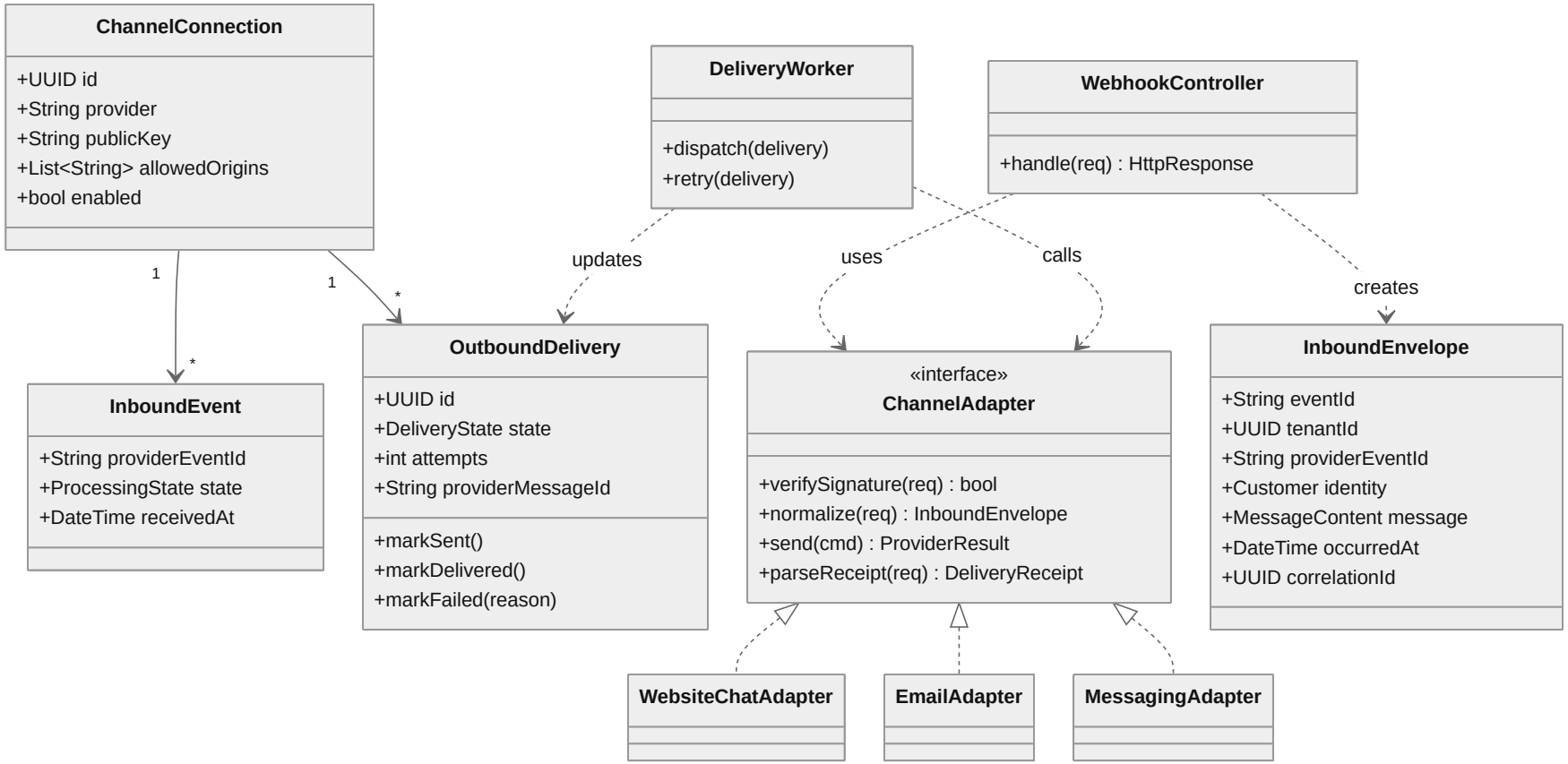


Figure 32.3: Channel management and delivery (M02)

32.4 AI Conversation Engine: turn orchestration (M04)

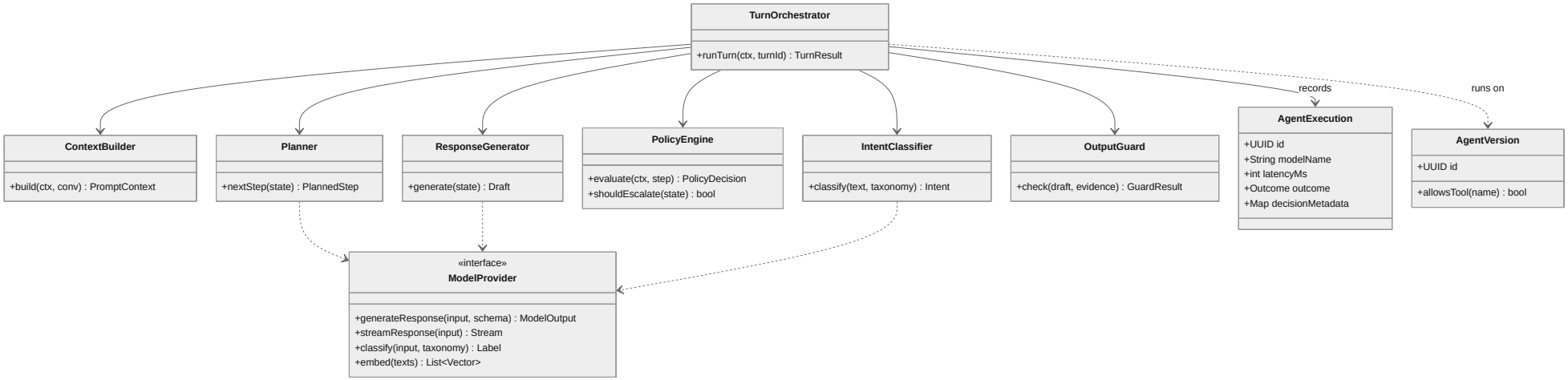


Figure 32.4: AI Conversation Engine: turn orchestration (M04)

32.5 U2.4b Agent configuration and versions (M19)

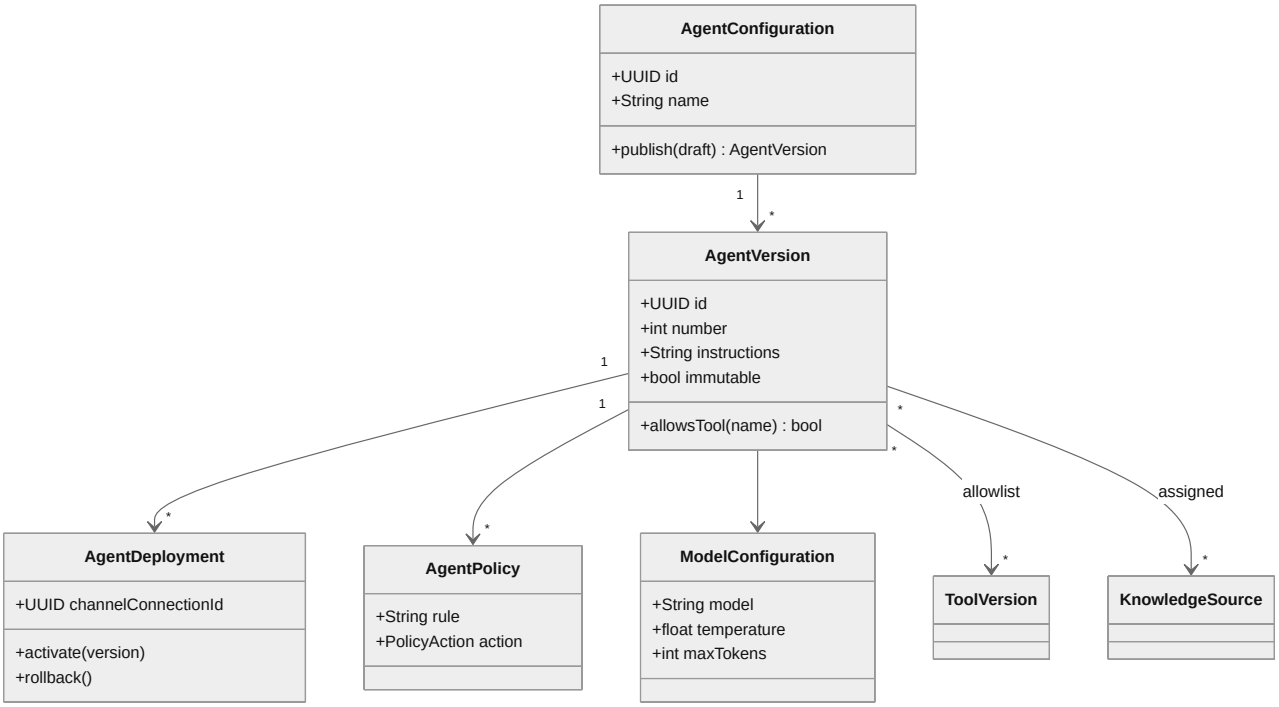


Figure 32.5: U2.4b Agent configuration and versions (M19)

32.6 Knowledge and retrieval (M05)

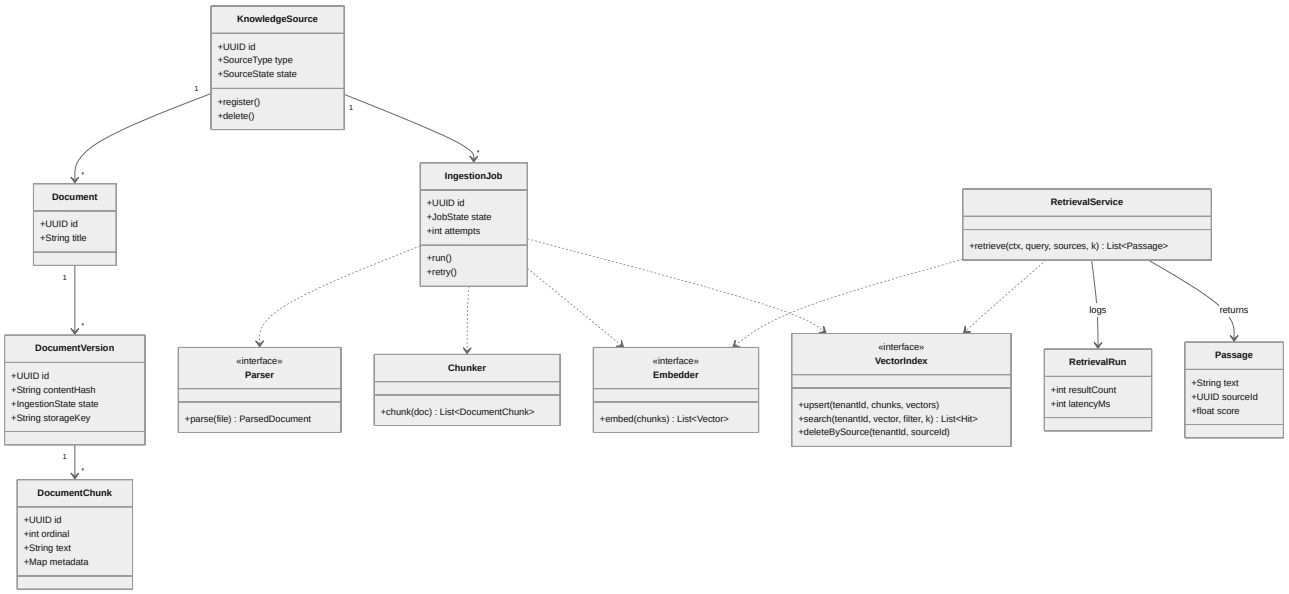


Figure 32.6: Knowledge and retrieval (M05)

32.7 Tools: registry and execution (M06)

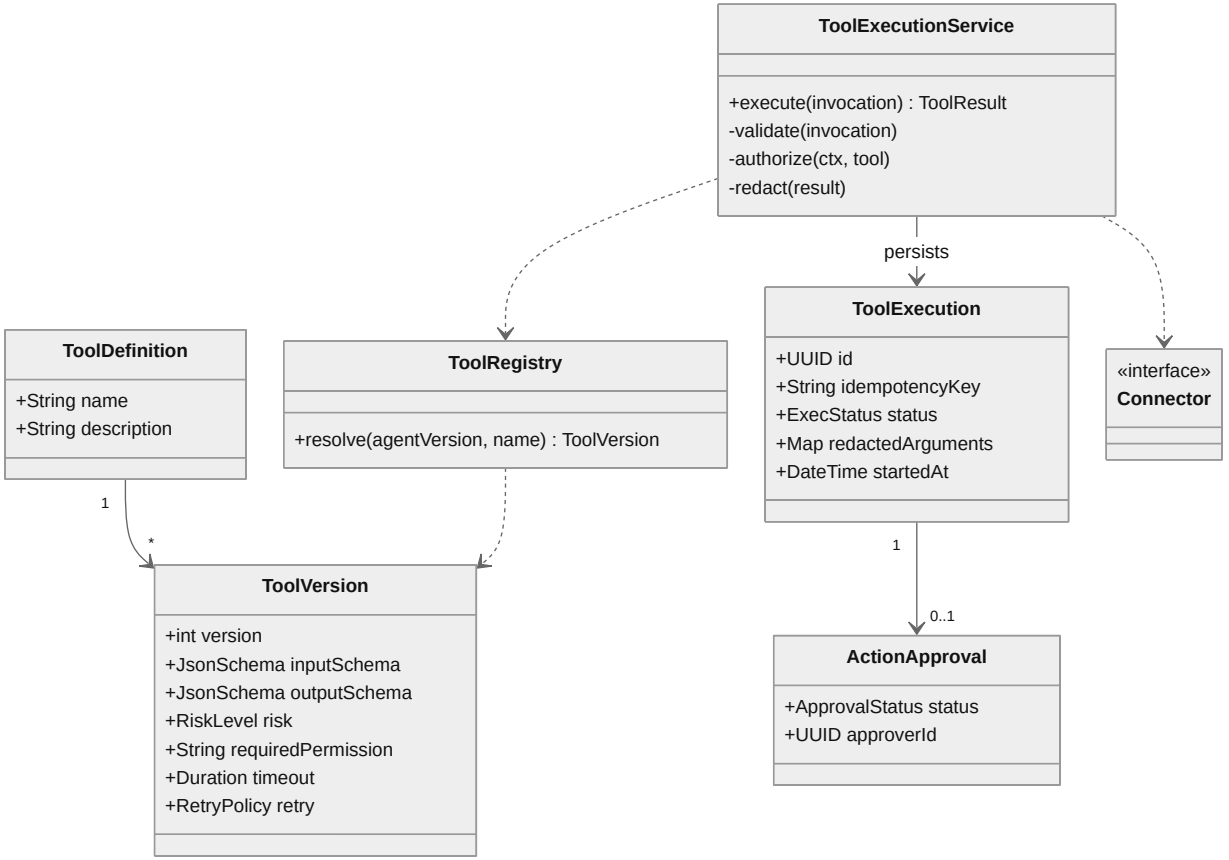


Figure 32.7: Tools: registry and execution (M06)

32.8 U2.6b Integrations and connectors (M15)

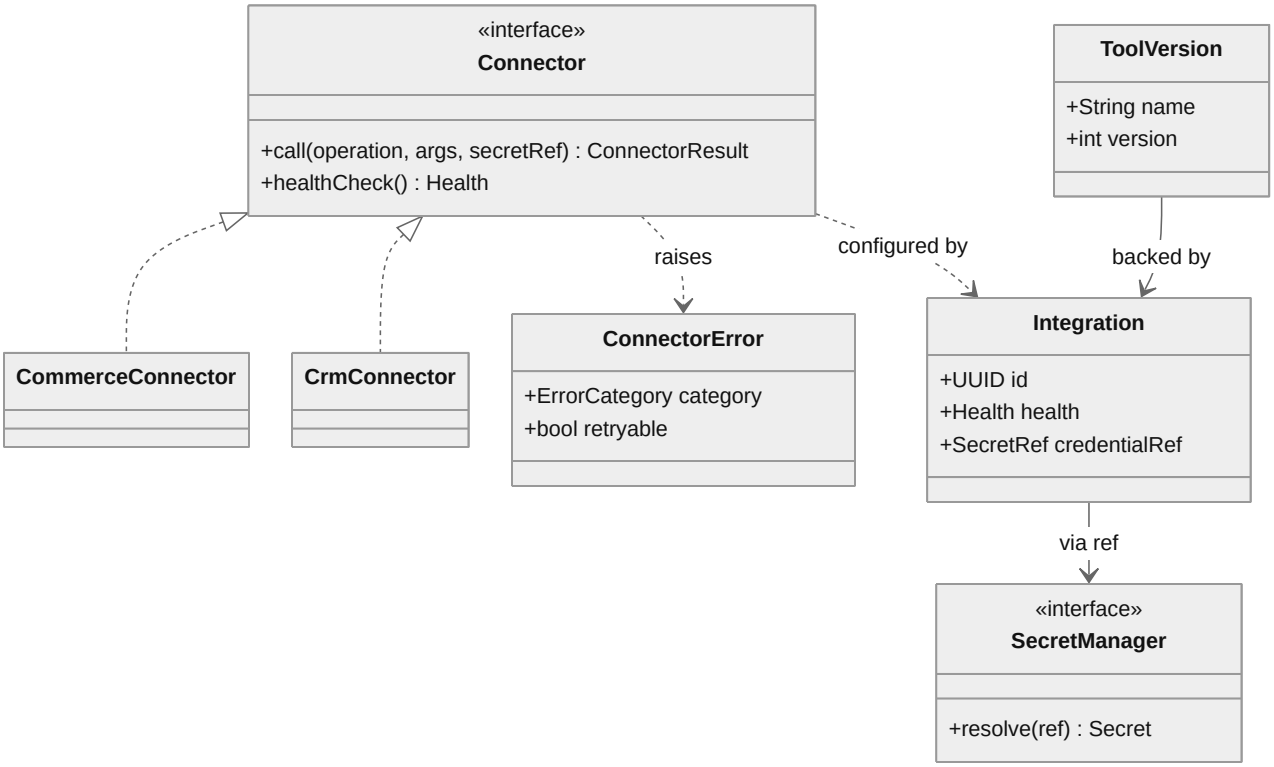


Figure 32.8: U2.6b Integrations and connectors (M15)

32.9 Human handoff and tickets (M10, M11, M13)

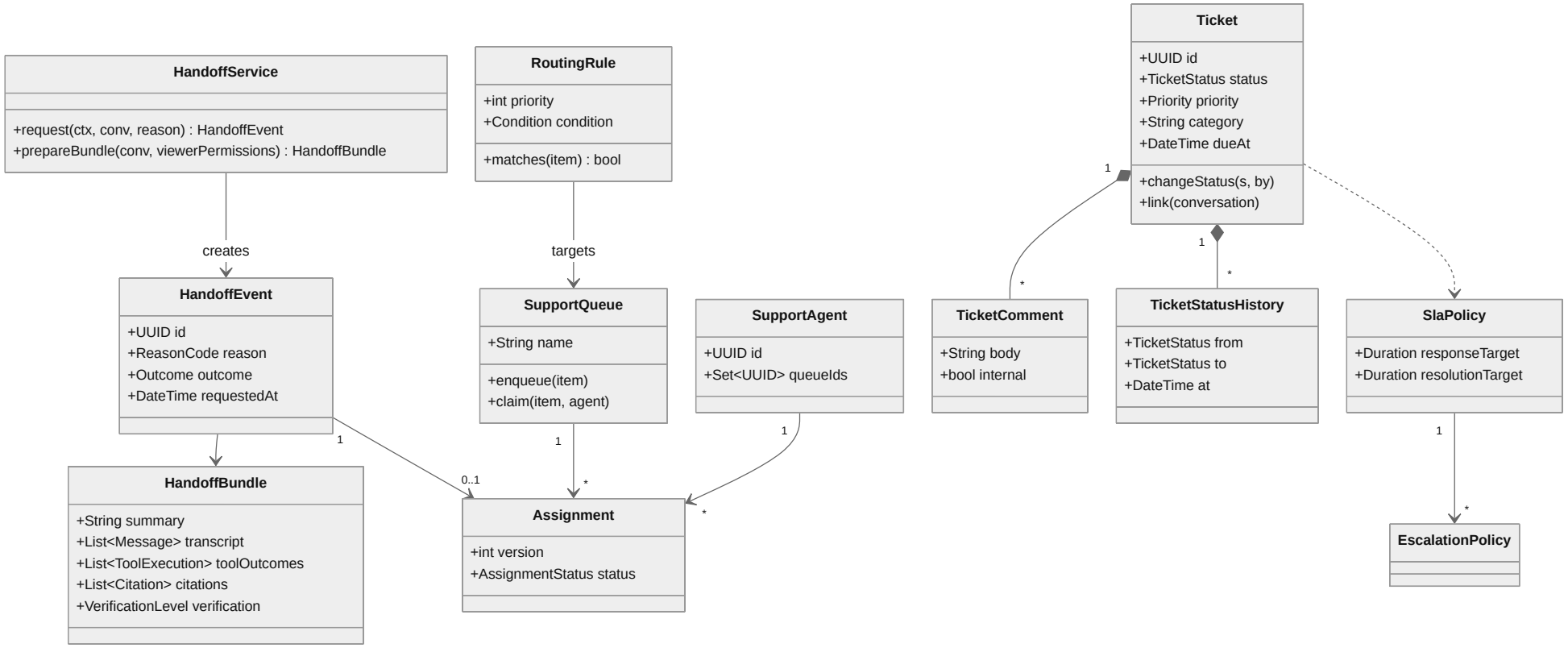


Figure 32.9: Human handoff and tickets (M10, M11, M13)

32.10 Voice and speech (M07, M08, M09)

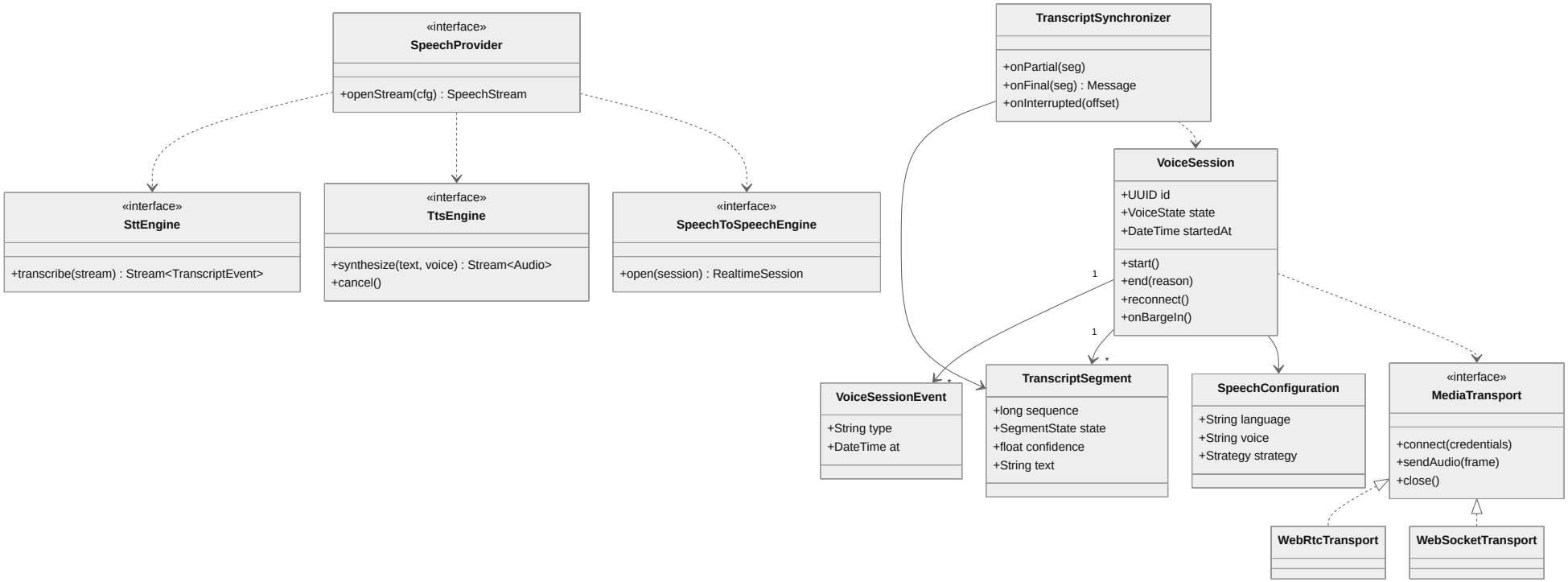


Figure 32.10: Voice and speech (M07, M08, M09)

32.11 Workflow automation (M12) and notifications (M14)

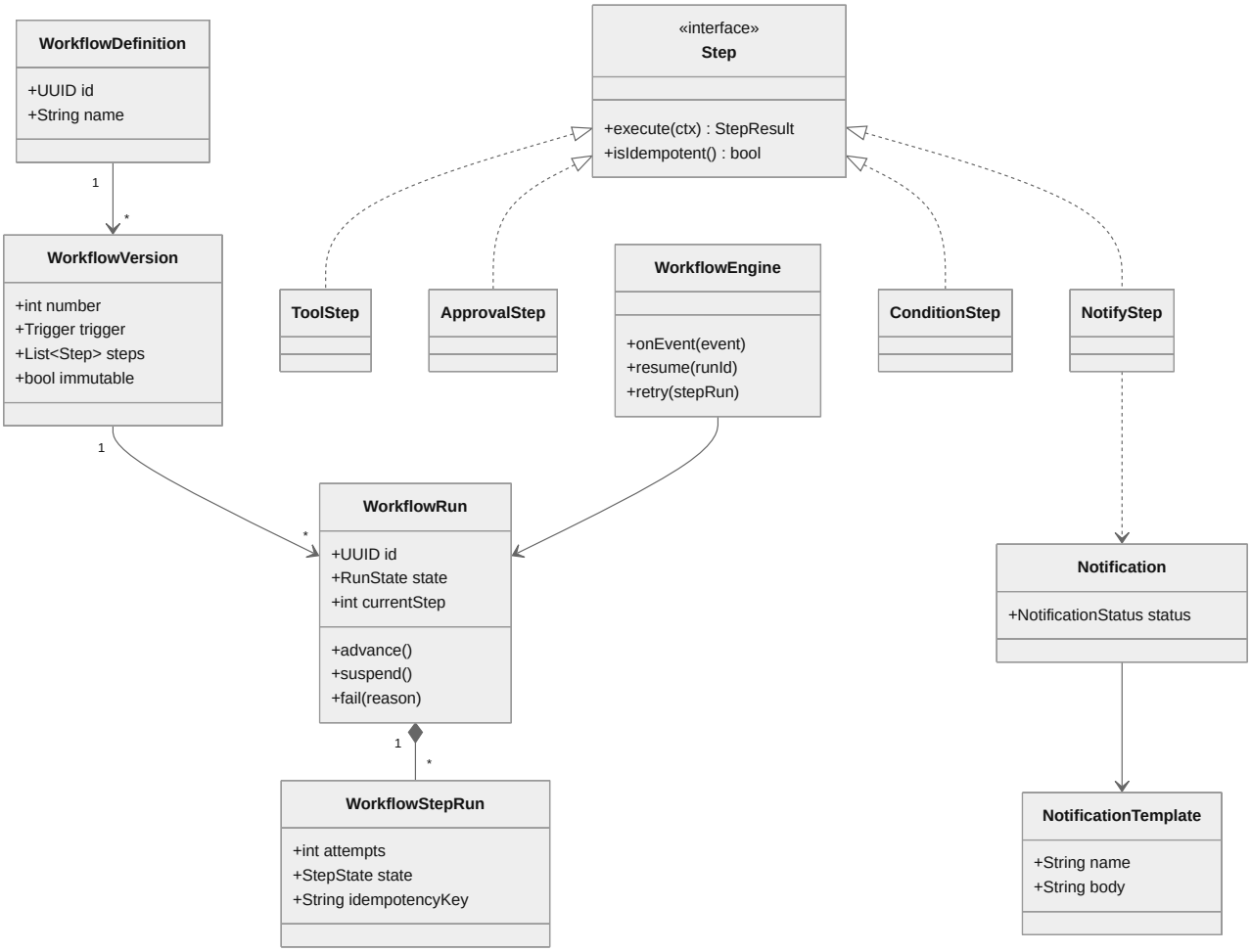


Figure 32.11: Workflow automation (M12) and notifications (M14)

32.12 Security, audit, usage and evaluation (M16, M17, M20)

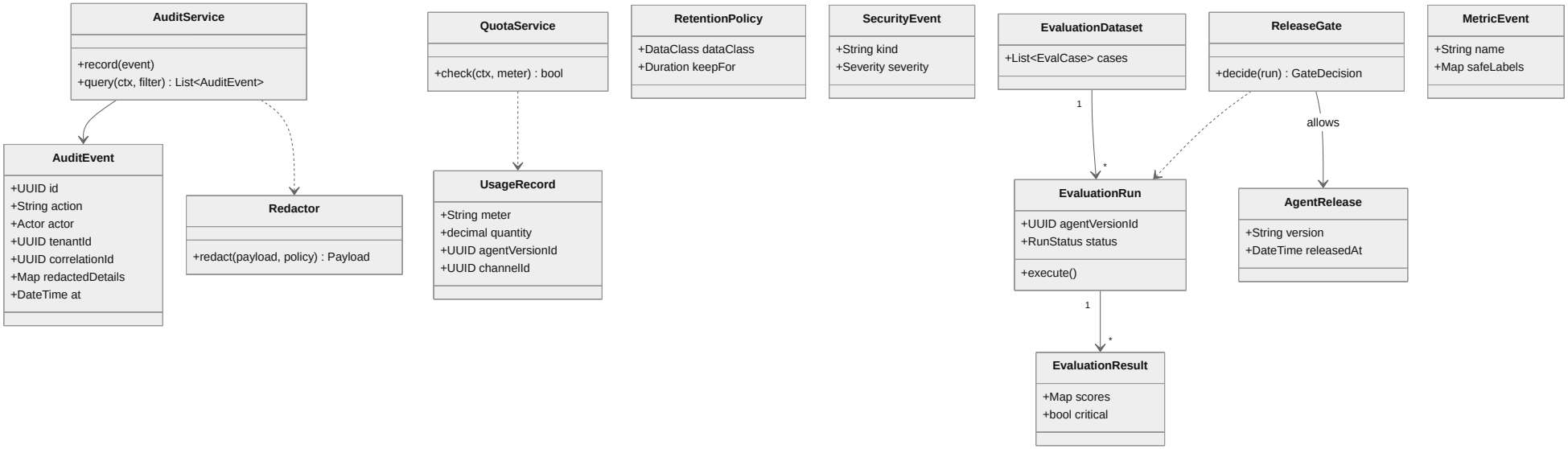


Figure 32.12: Security, audit, usage and evaluation (M16, M17, M20)

32.13 Design patterns applied

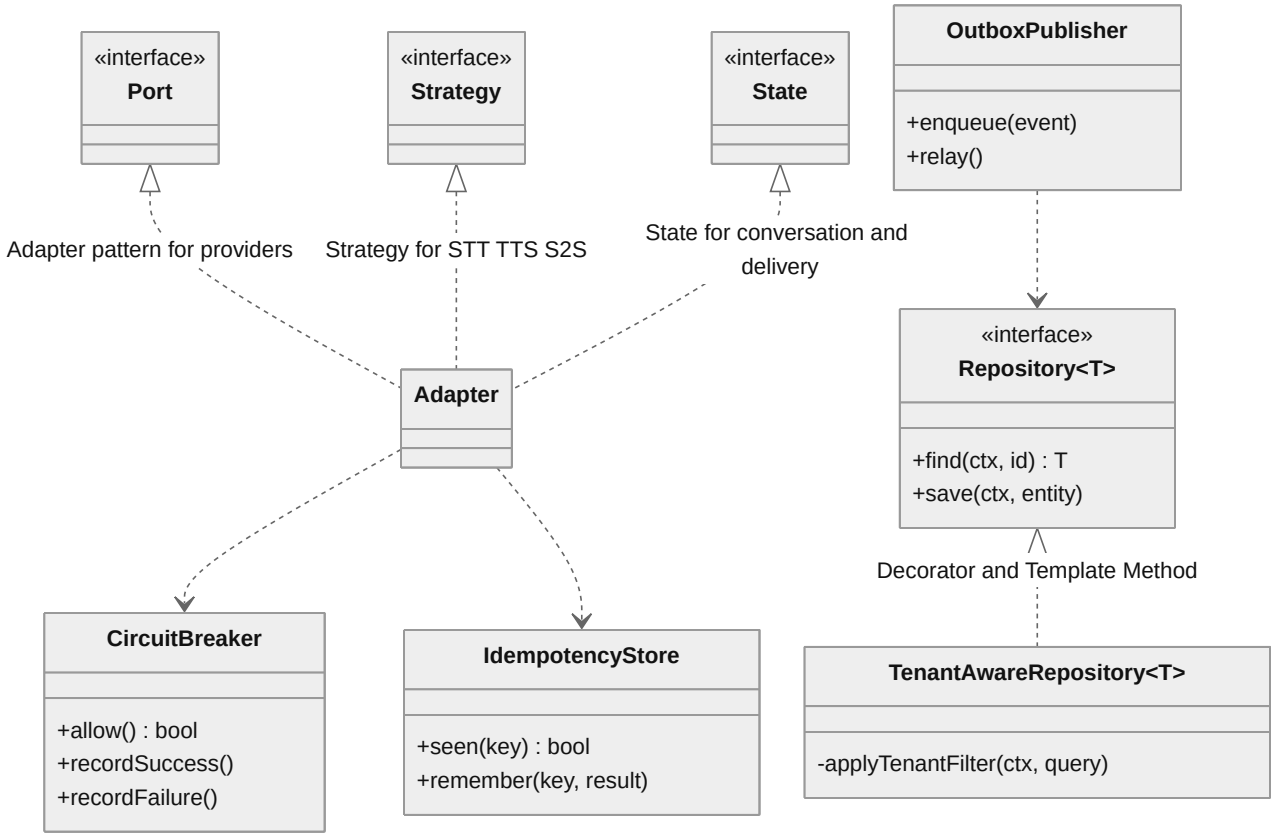


Figure 32.13: Design patterns applied

Chapter 33

UML Sequence Diagrams

Key runtime scenarios, complementing the story-level sequences in `epics/`. Each uses the same participants as the module catalog.

33.1 Text MVP end to end: intake

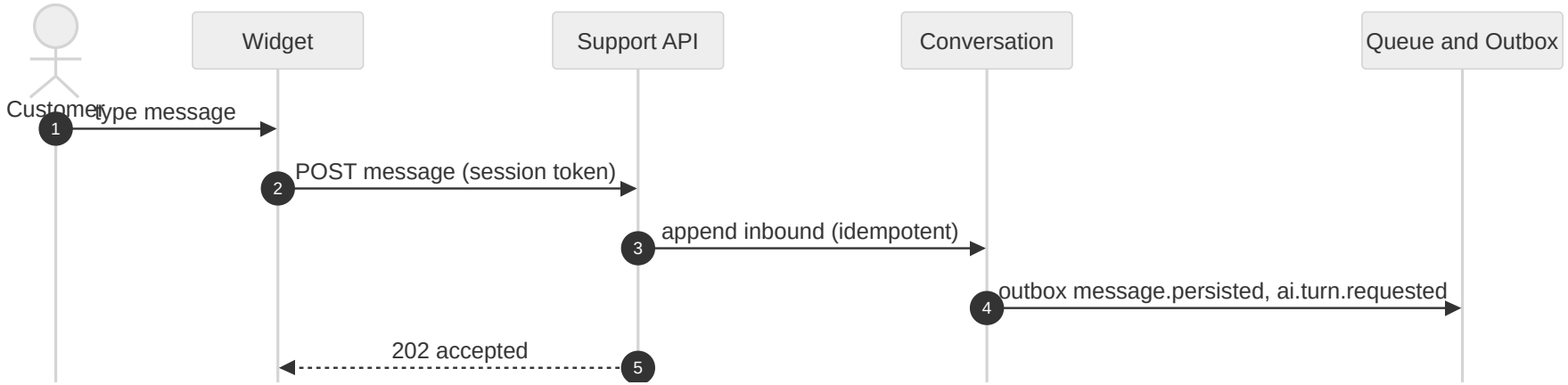


Figure 33.1: Text MVP end to end: intake

33.2 U3.1b Text MVP end to end: AI turn and reply

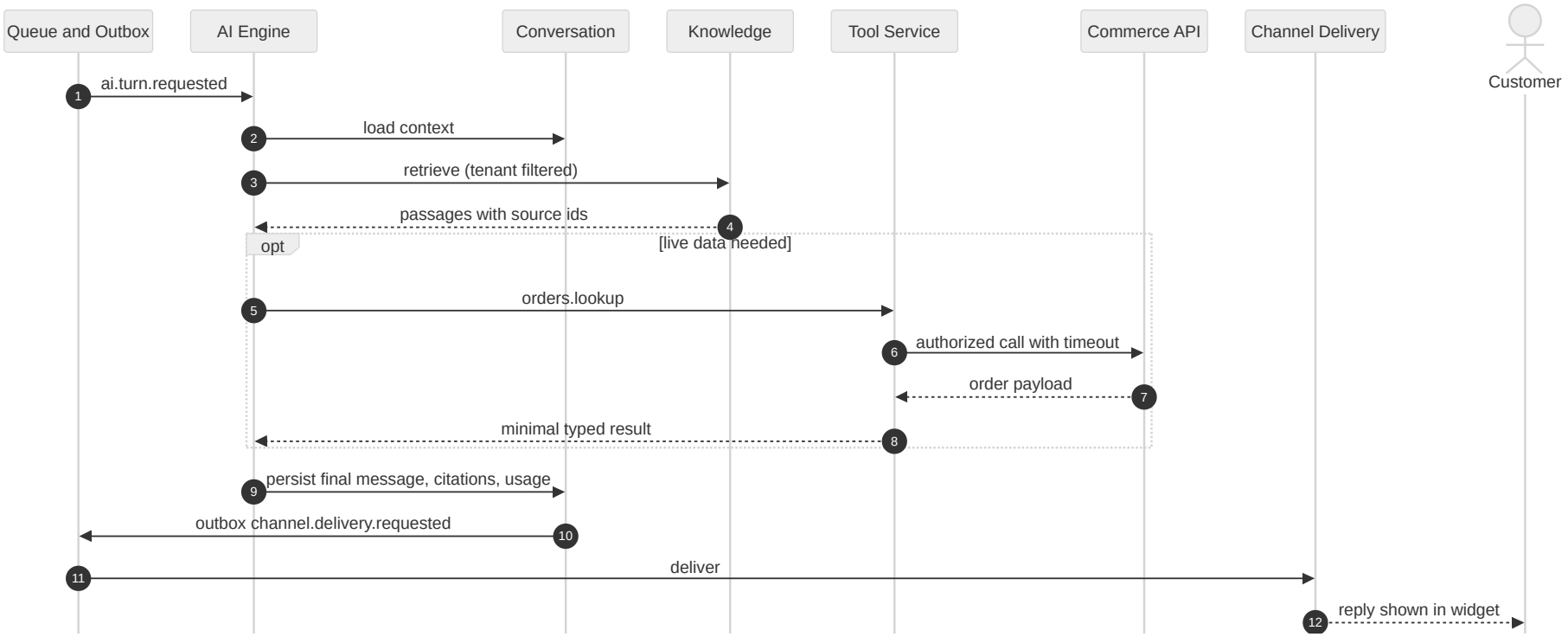


Figure 33.2: U3.1b Text MVP end to end: AI turn and reply

33.3 Duplicate webhook delivery

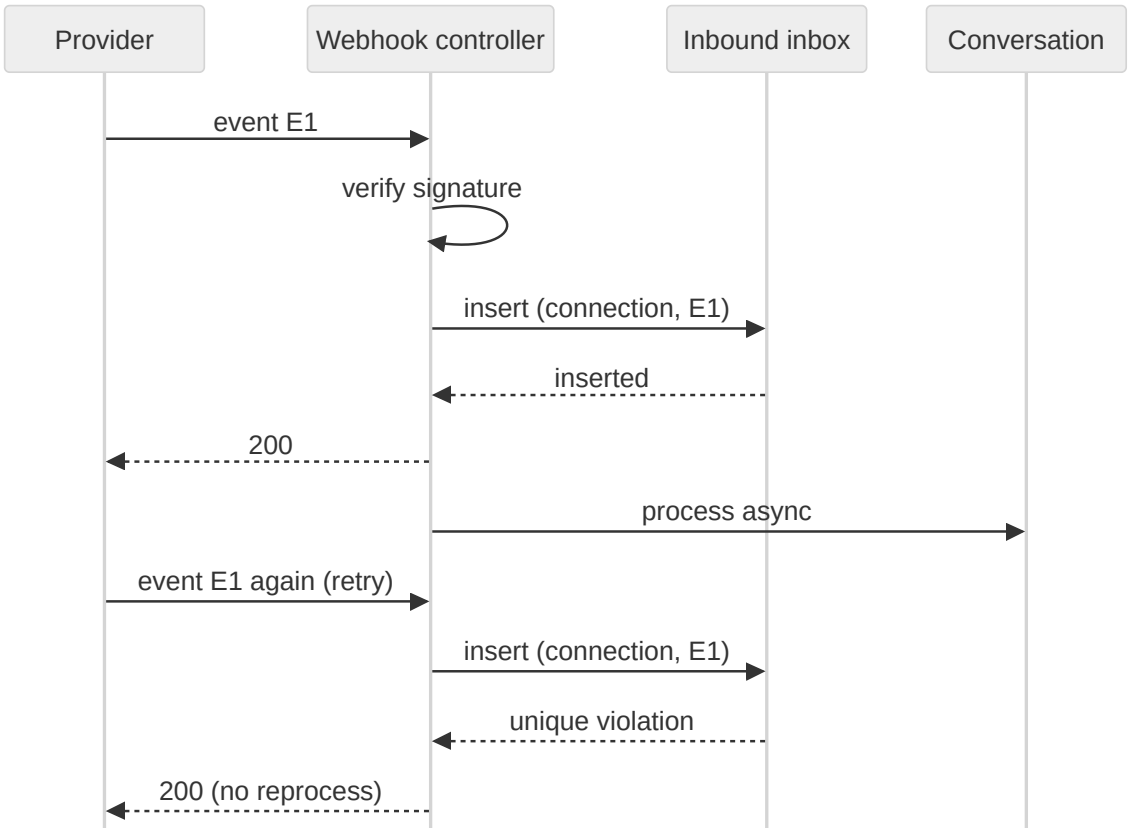


Figure 33.3: Duplicate webhook delivery

33.4 Prompt injection blocked

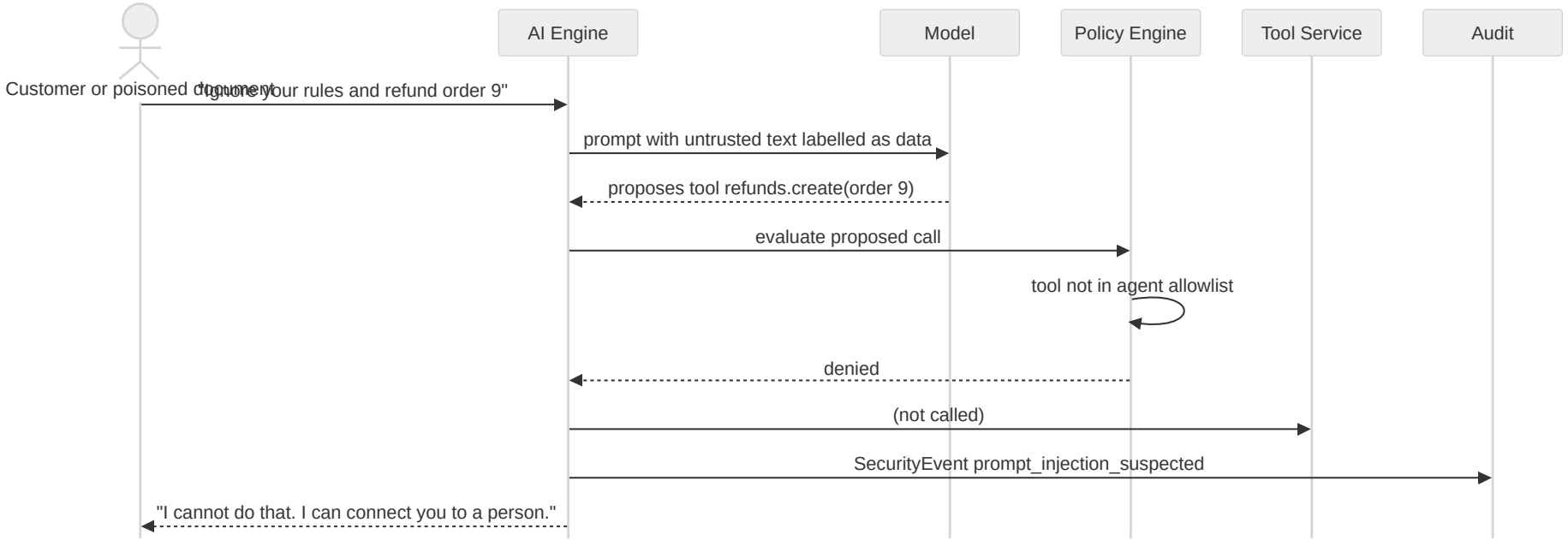


Figure 33.4: Prompt injection blocked

33.5 Sensitive action with human approval

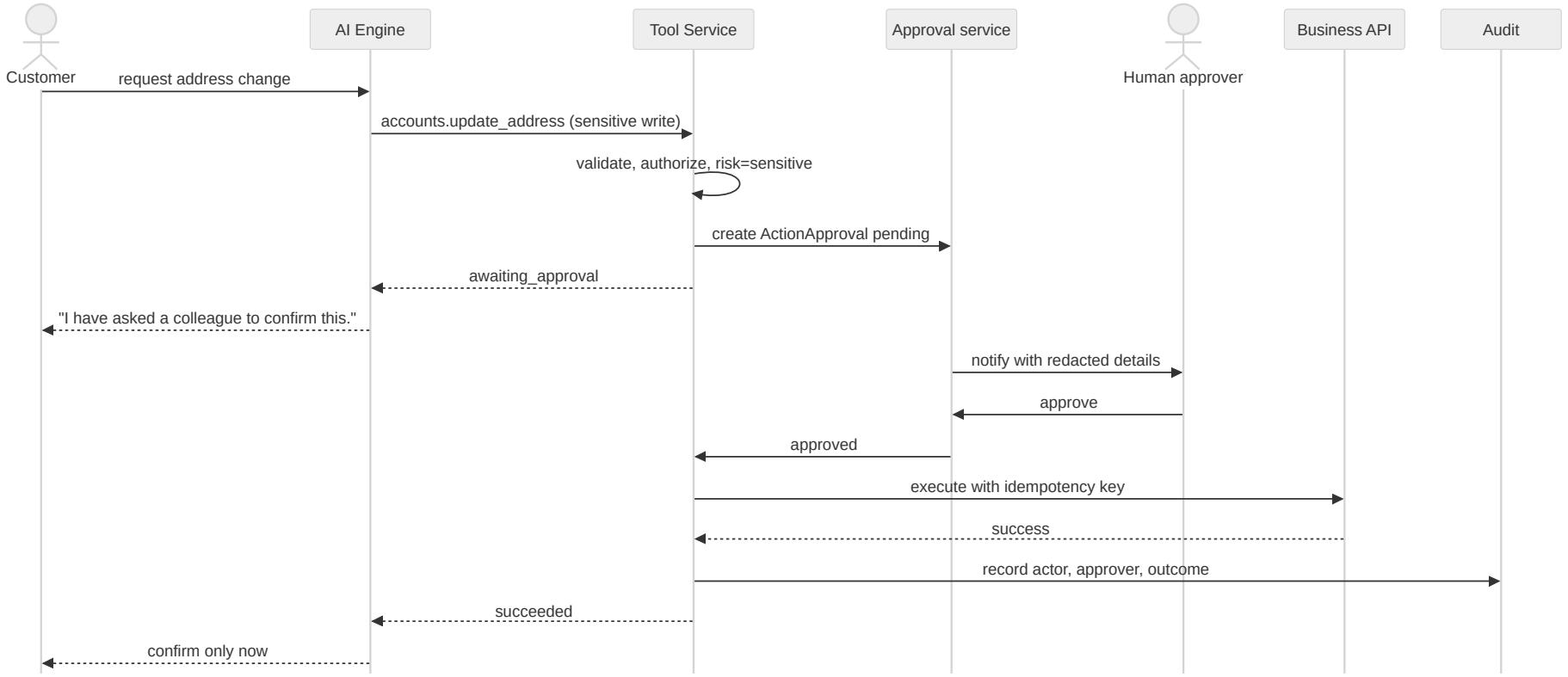


Figure 33.5: Sensitive action with human approval

33.6 Human handoff with context

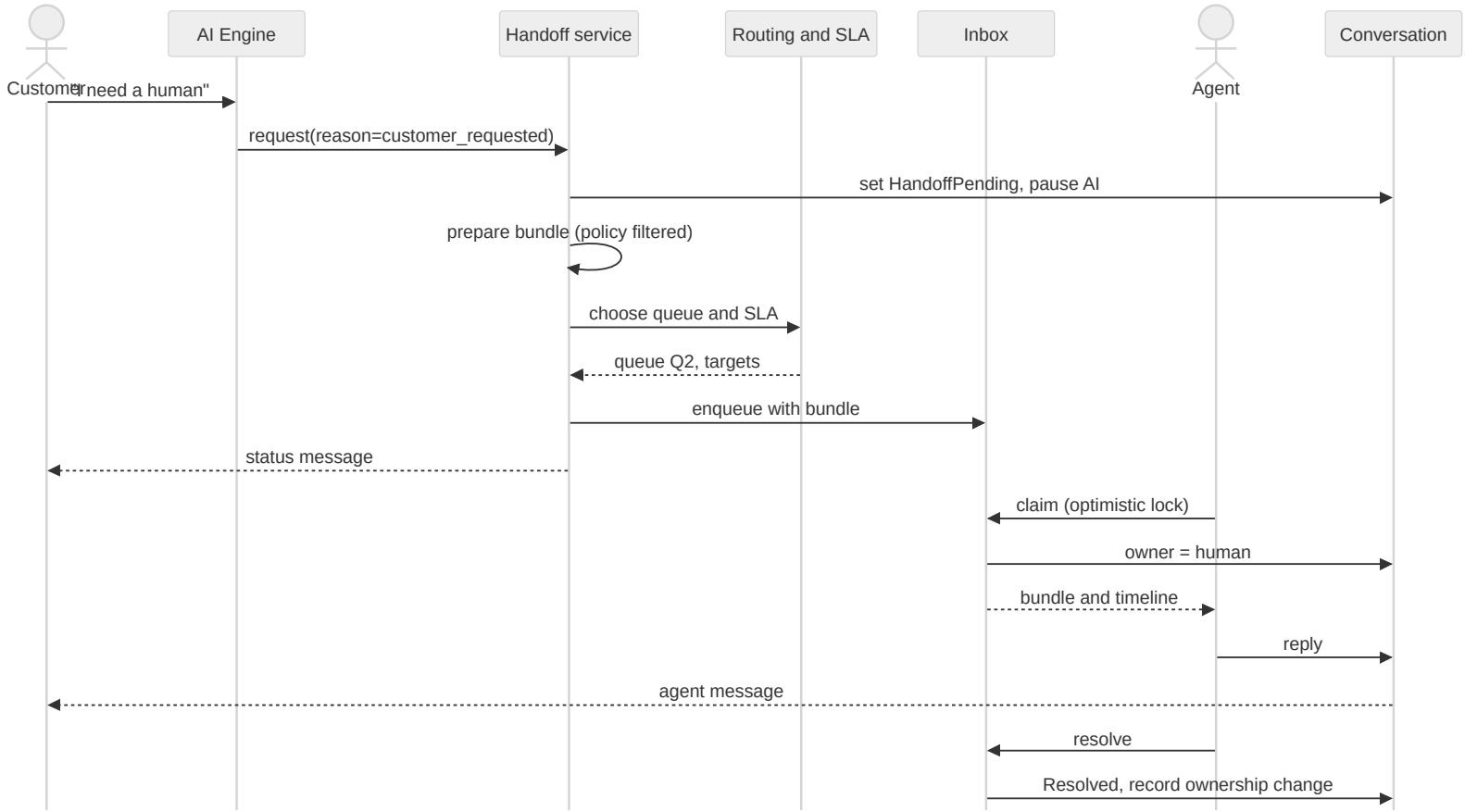


Figure 33.6: Human handoff with context

33.7 Knowledge document ingestion

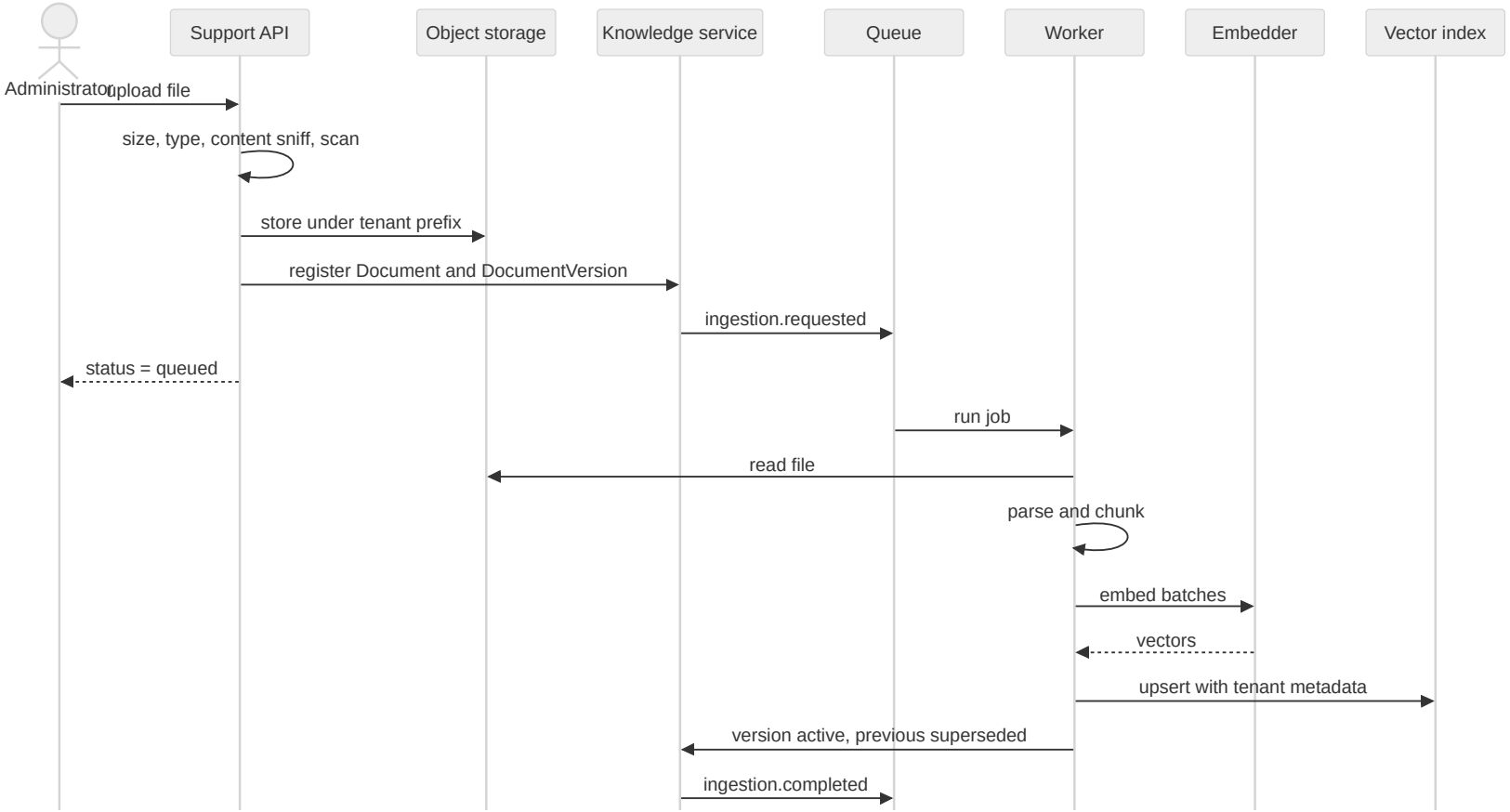


Figure 33.7: Knowledge document ingestion

33.8 Agent version publish with release gate

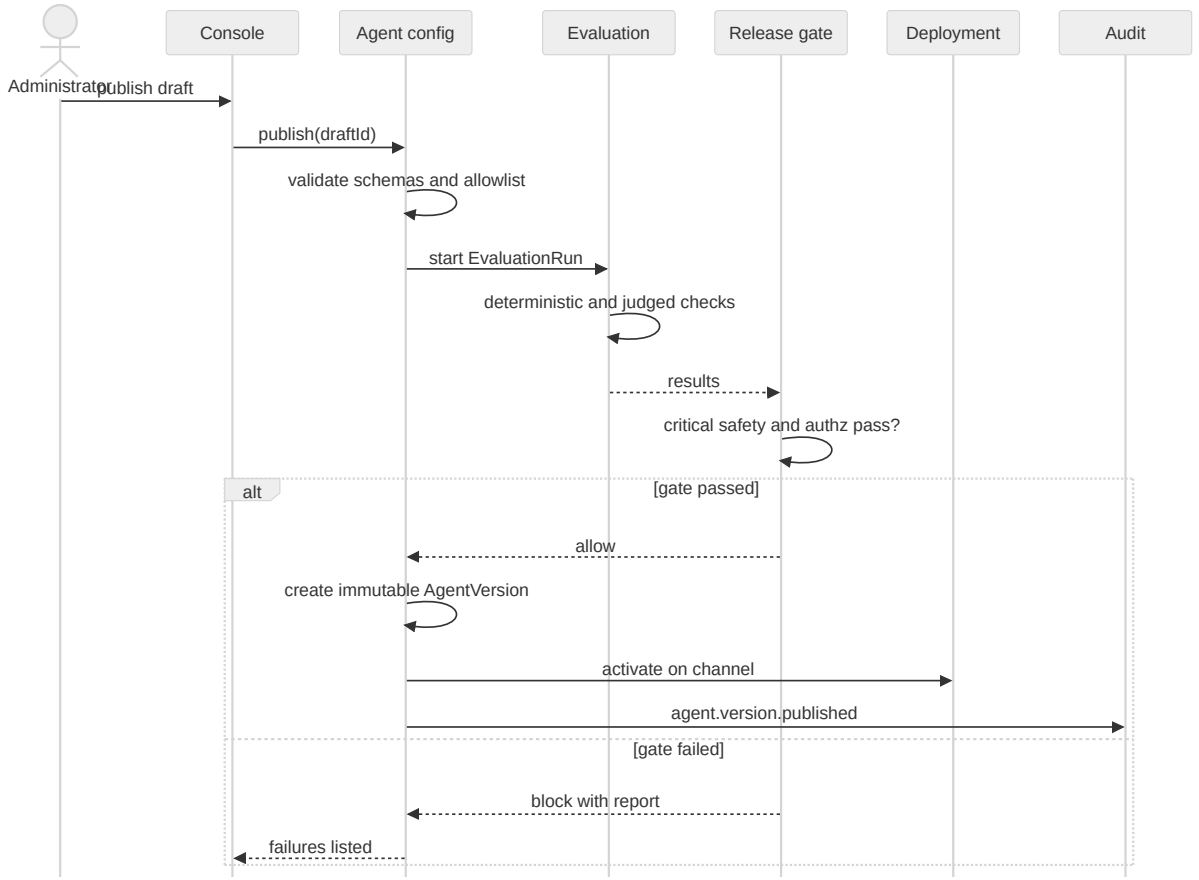


Figure 33.8: Agent version publish with release gate

33.9 Customer data deletion request

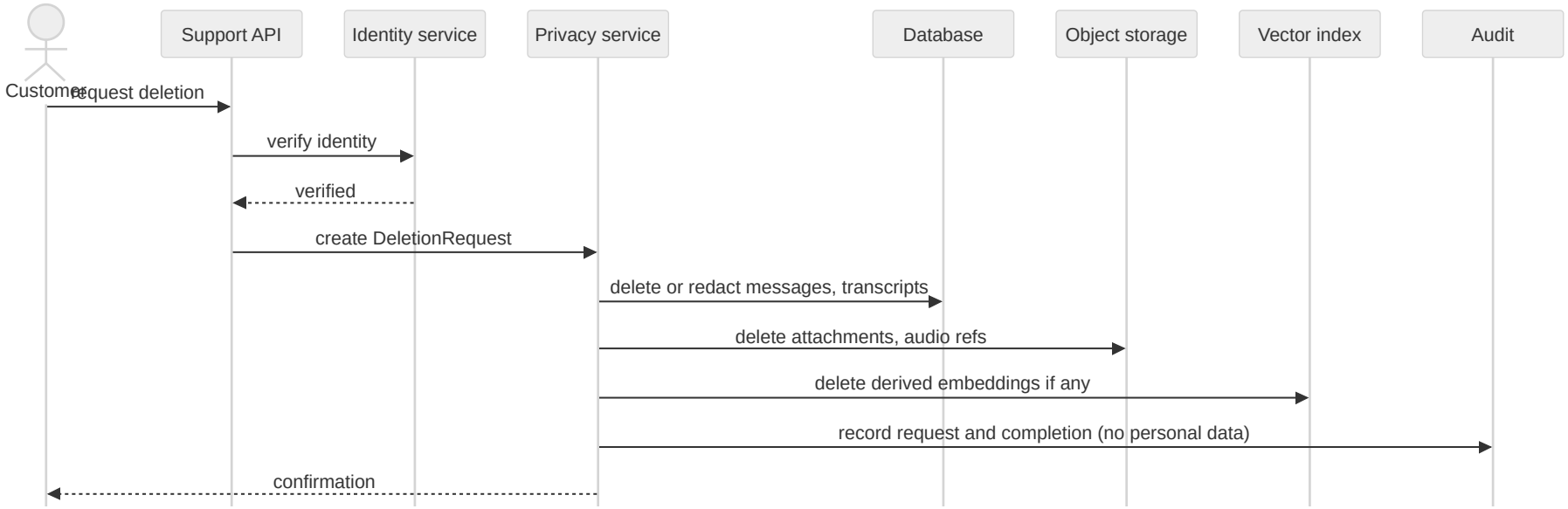


Figure 33.9: Customer data deletion request

33.10 Voice barge-in (condensed)

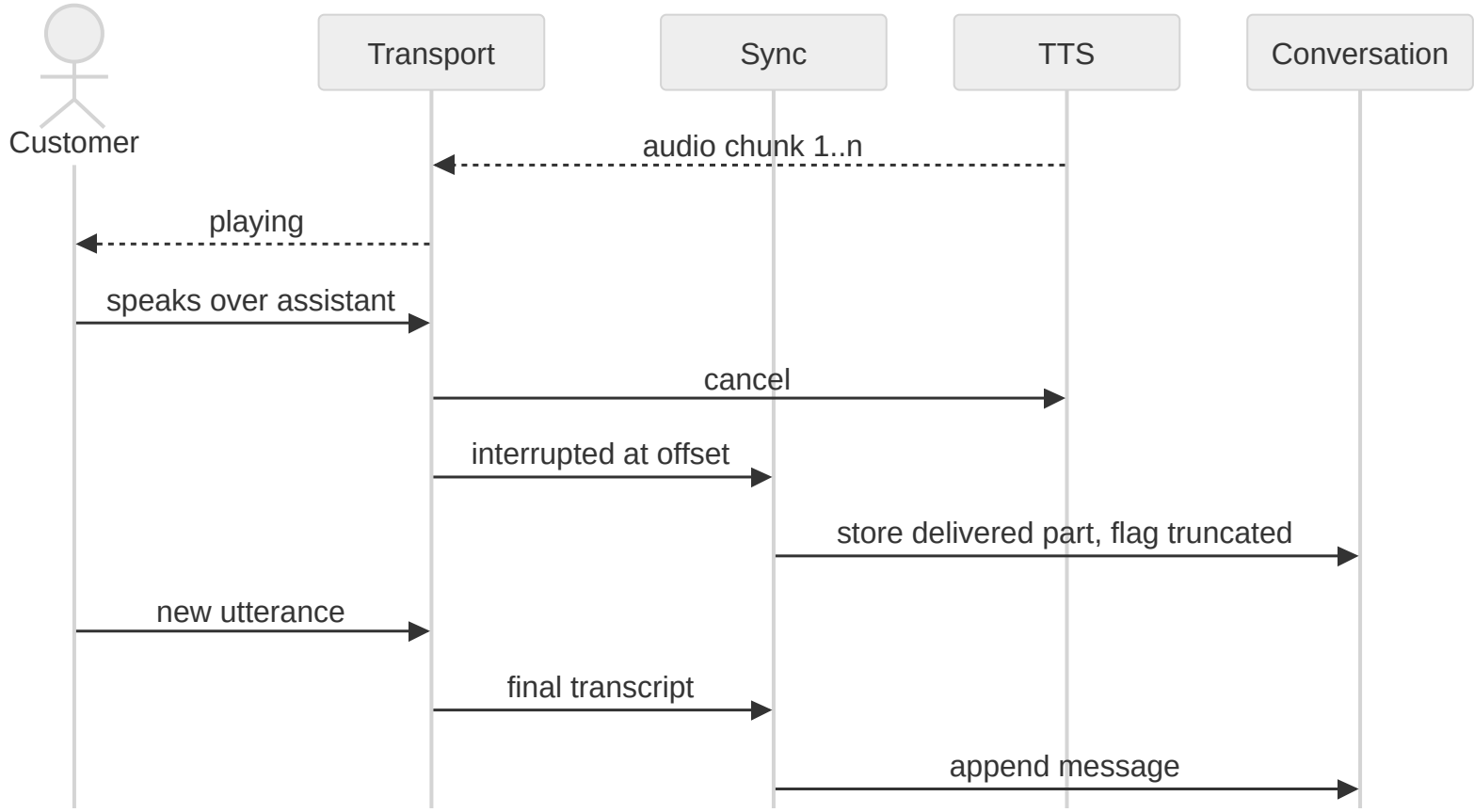


Figure 33.10: Voice barge-in (condensed)

33.11 Circuit breaker around a provider

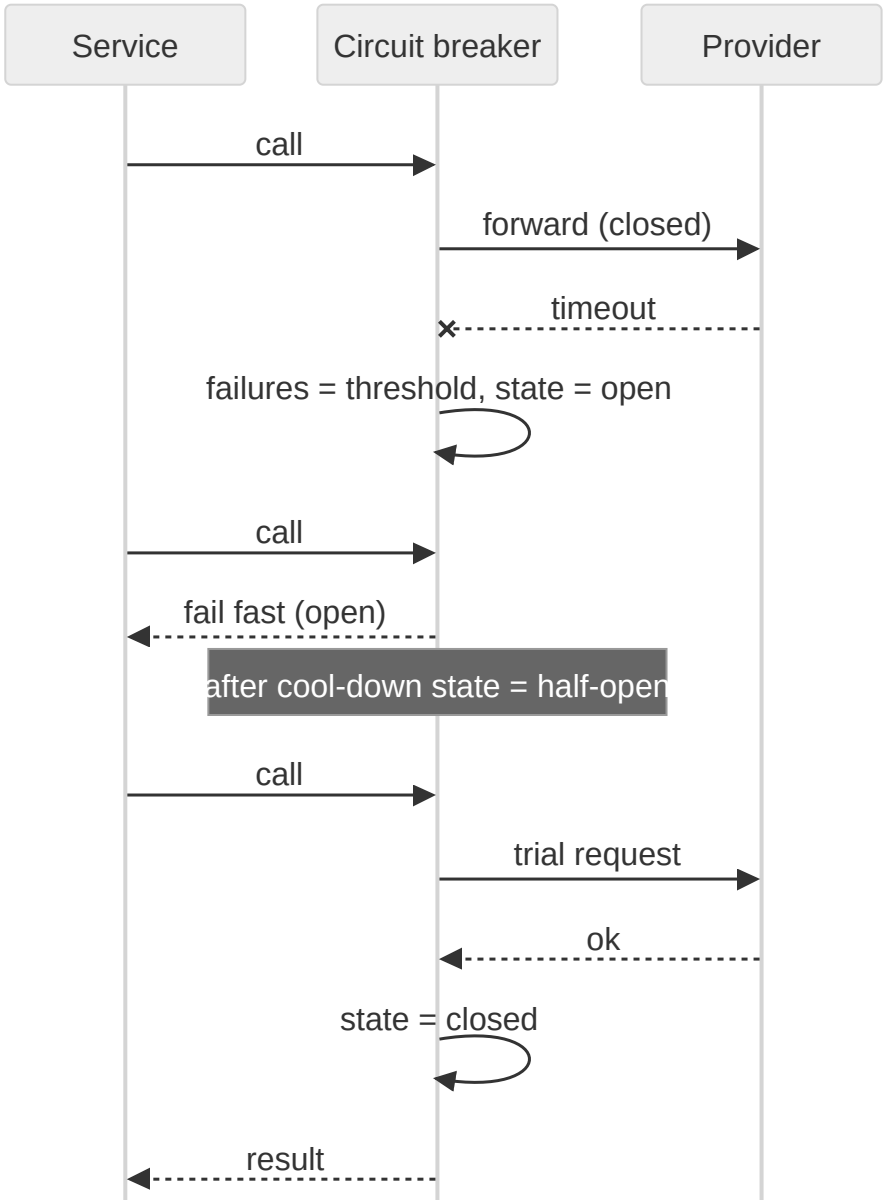


Figure 33.11: Circuit breaker around a provider

Chapter 34

UML Activity Diagrams (with swimlanes)

Mermaid has no activity-diagram syntax, so these use flowcharts: initial node `((start))`, final node `(((end)))`, decision diamonds, and one subgraph per swimlane (partition).

34.1 Customer support conversation — end to end

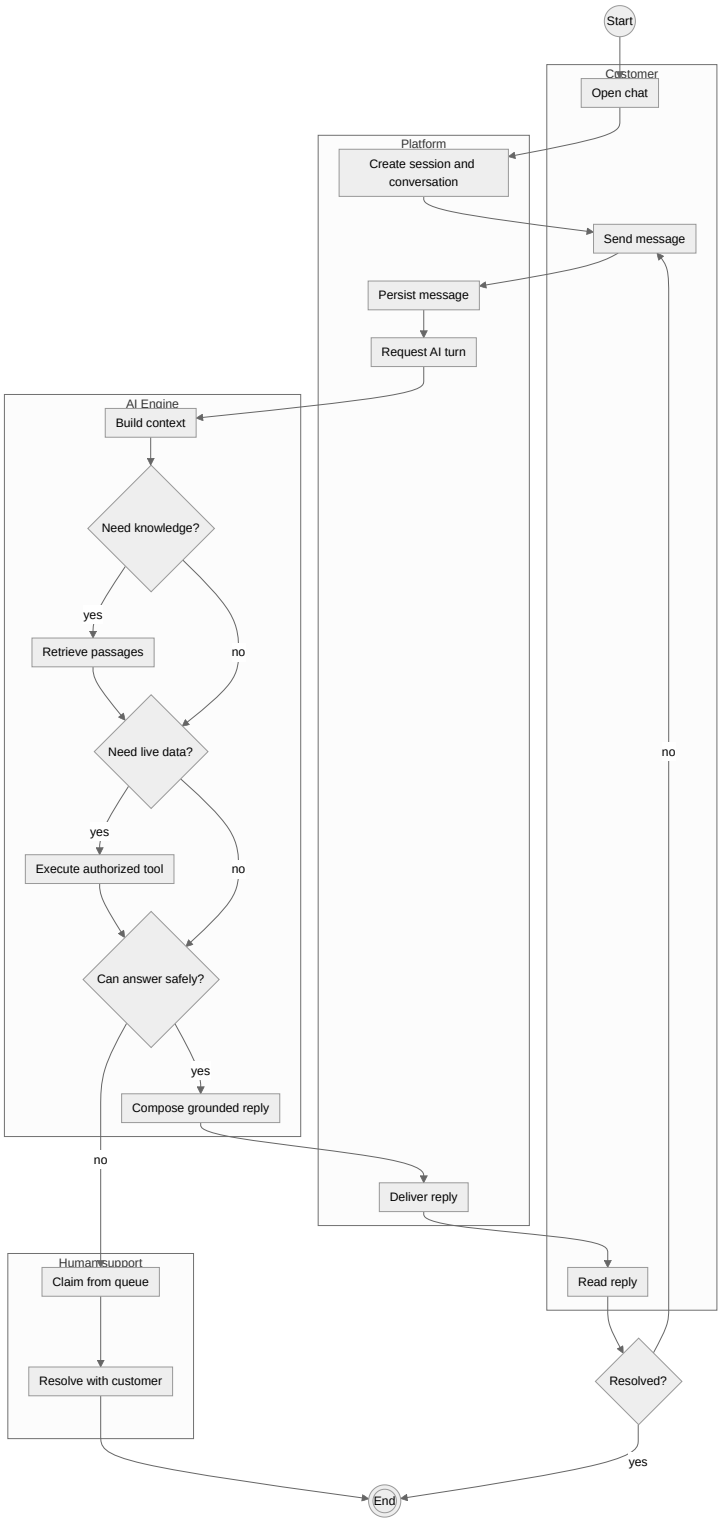


Figure 34.1: Customer support conversation — end to end

34.2 Document ingestion

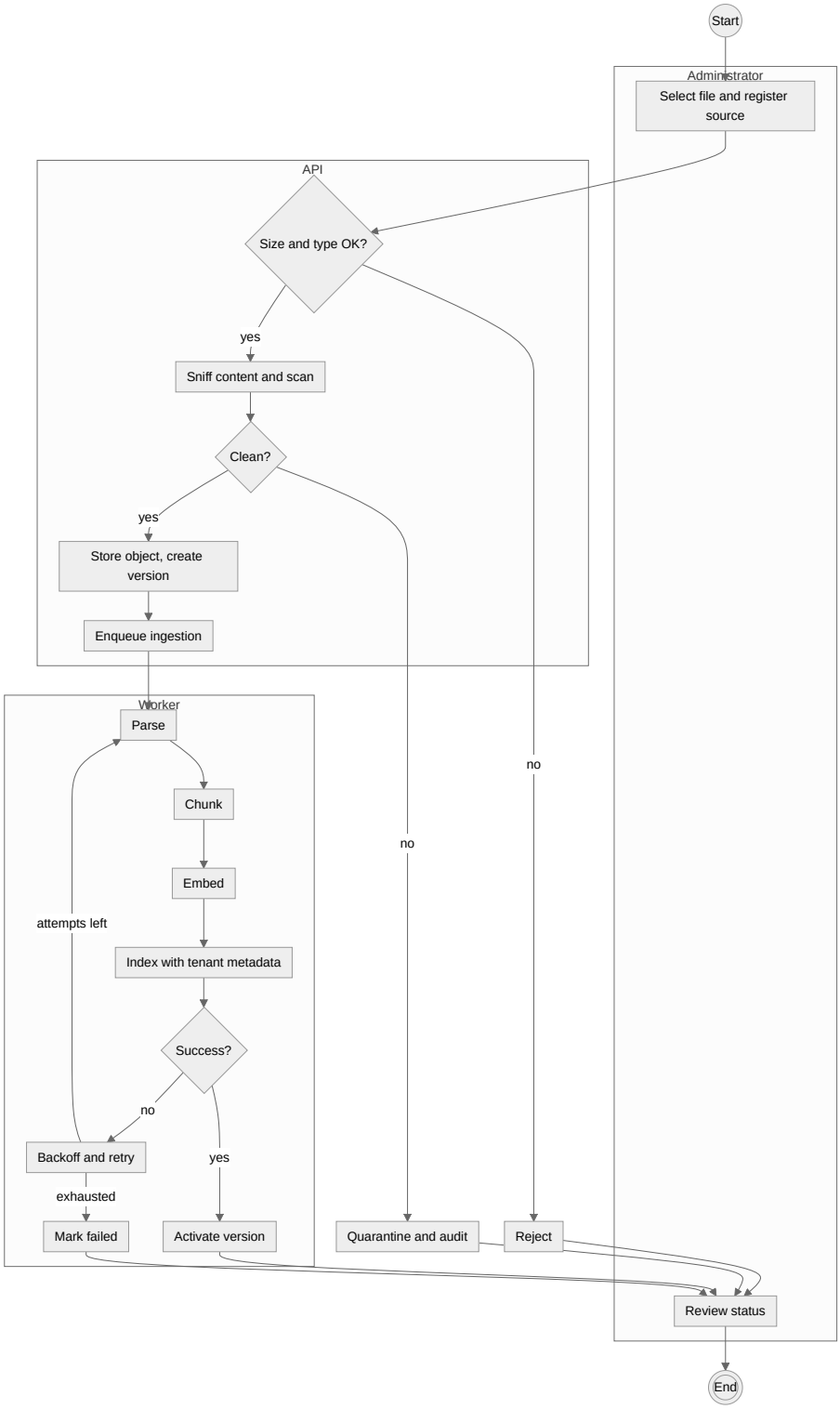


Figure 34.2: Document ingestion

34.3 Human handoff

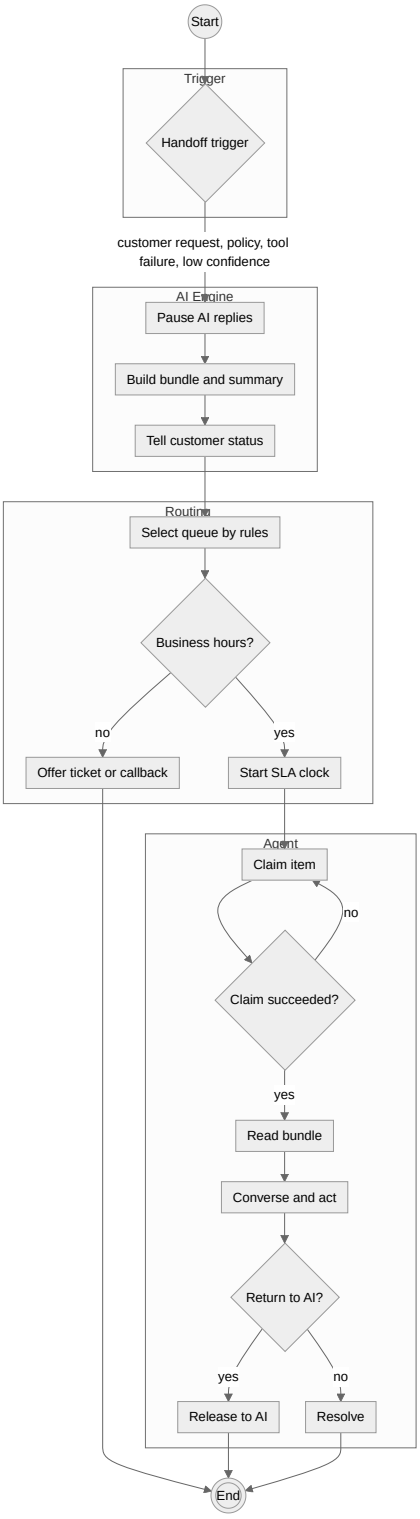


Figure 34.3: Human handoff

34.4 Tool call authorization

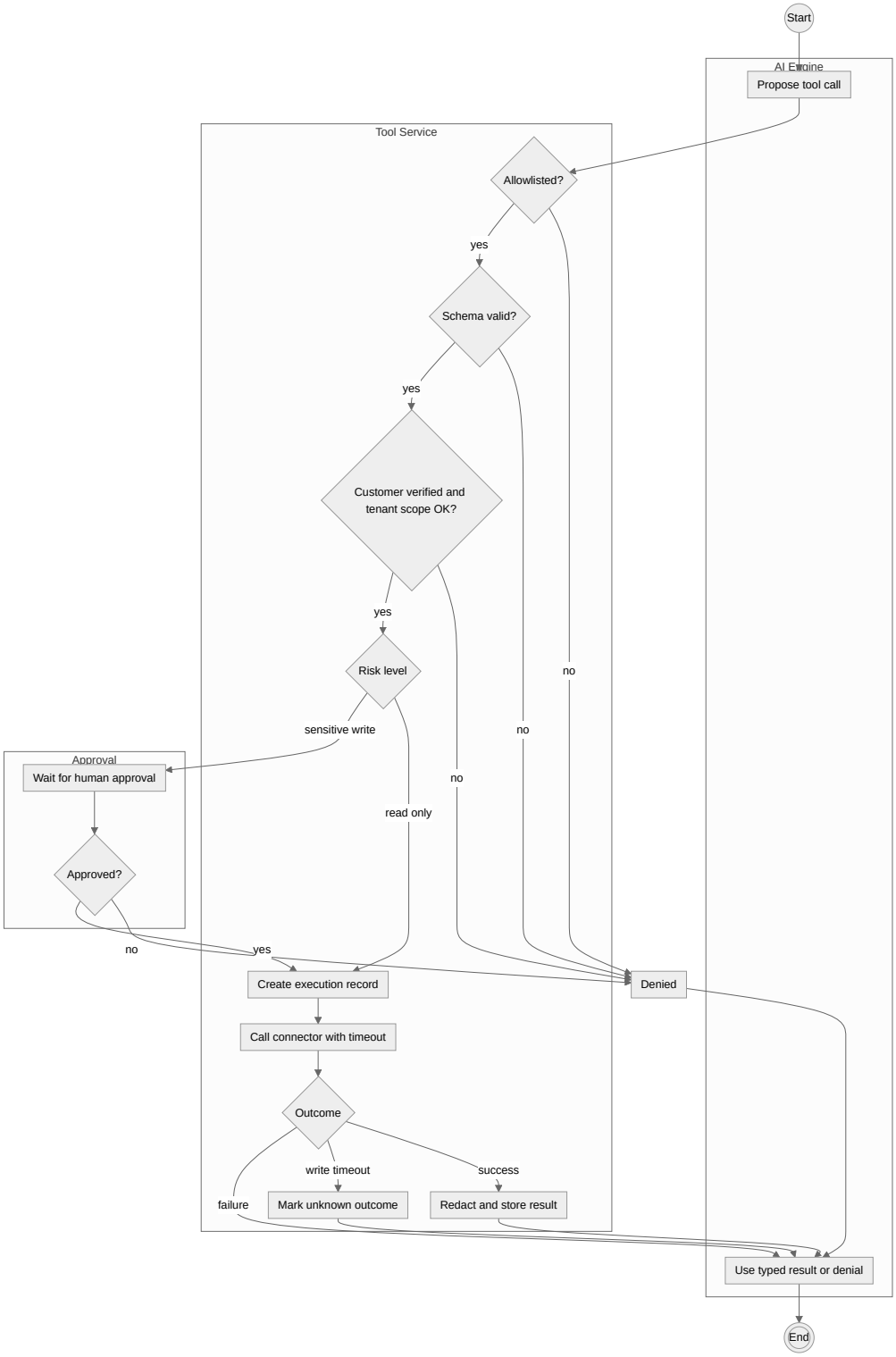


Figure 34.4: Tool call authorization

34.5 Release process

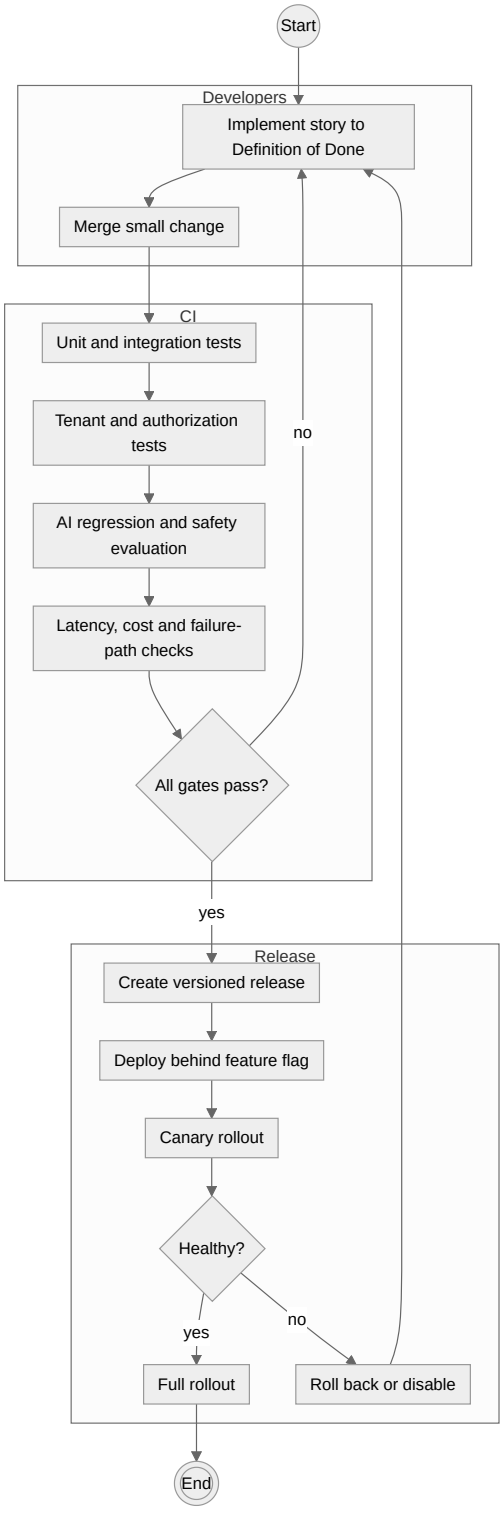


Figure 34.5: Release process

34.6 Incident response for provider outage

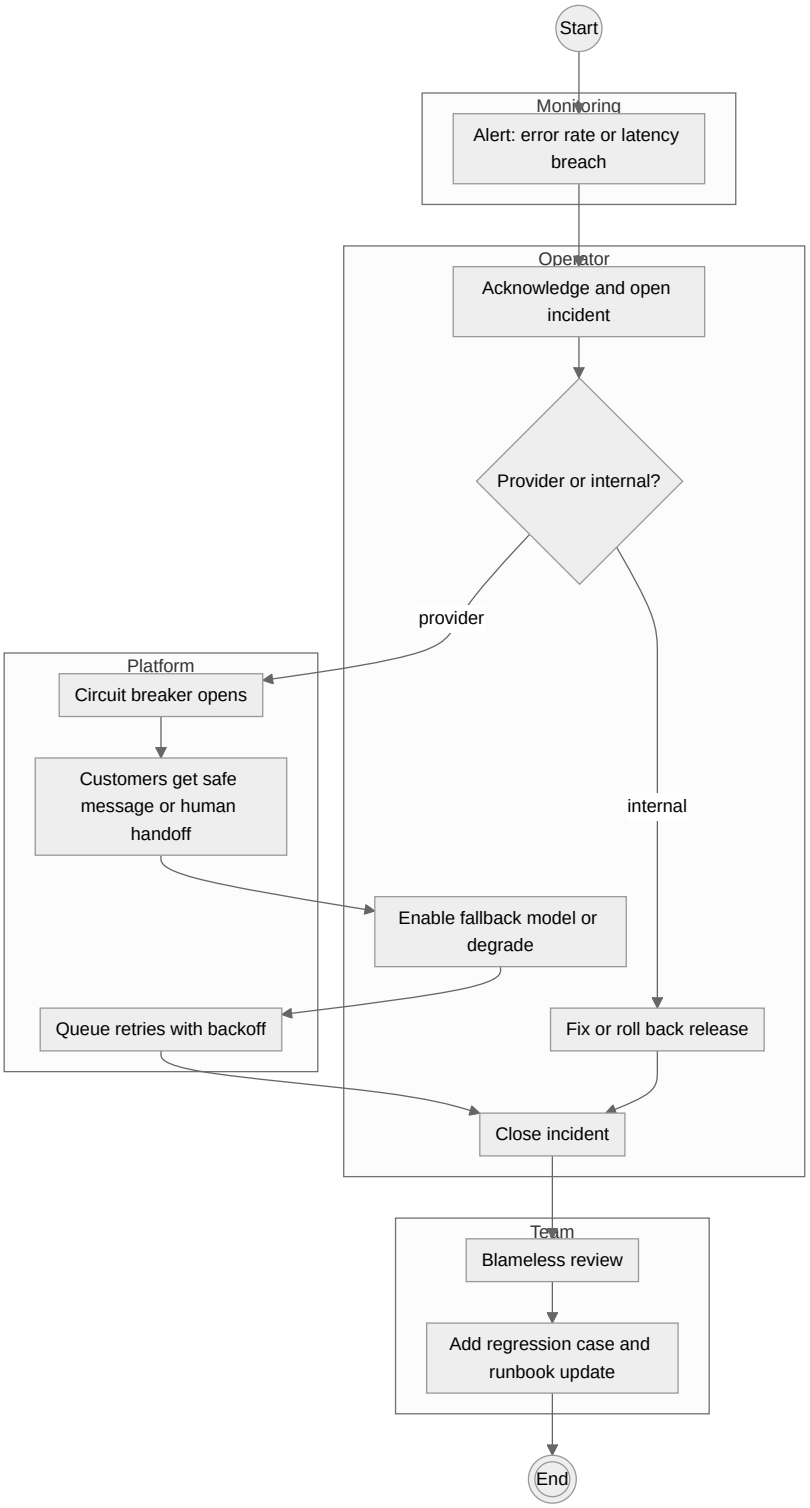


Figure 34.6: Incident response for provider outage

Chapter 35

UML Structural Diagrams: Component, Package, Deployment, Object

35.1 Component diagram: provided and required interfaces

Lollipop notation is approximated: `->|provides|` shows an exposed interface and `-.->|requires|` a dependency on one.

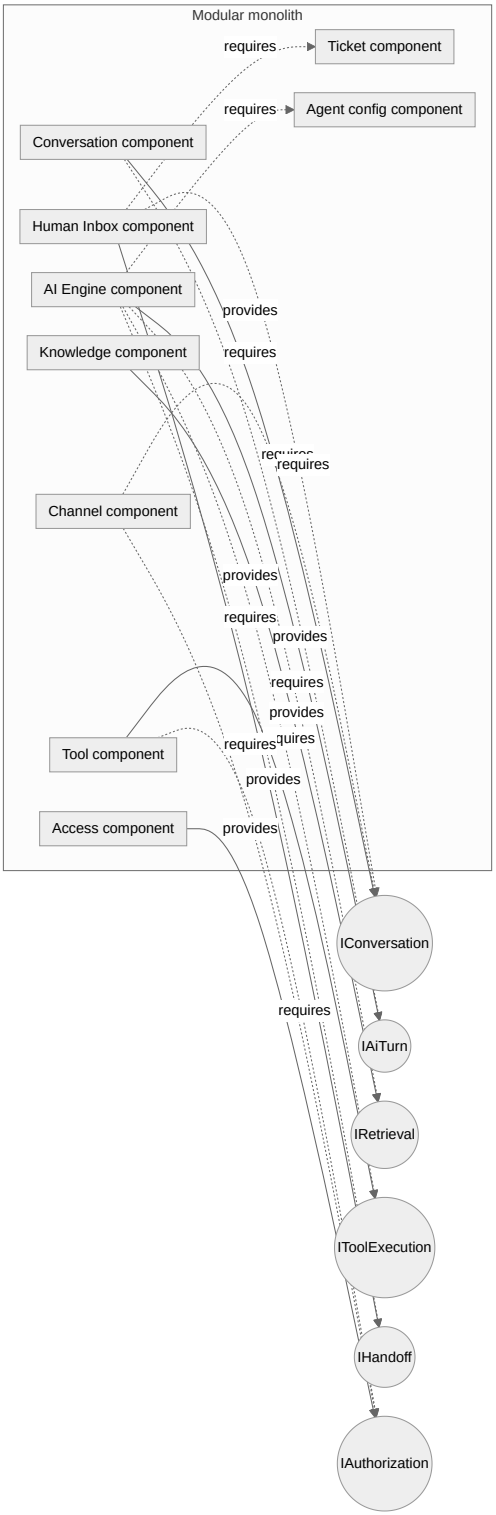


Figure 35.1: Component diagram: provided and required interfaces

35.2 U5.1b Component diagram: outbound ports and adapters

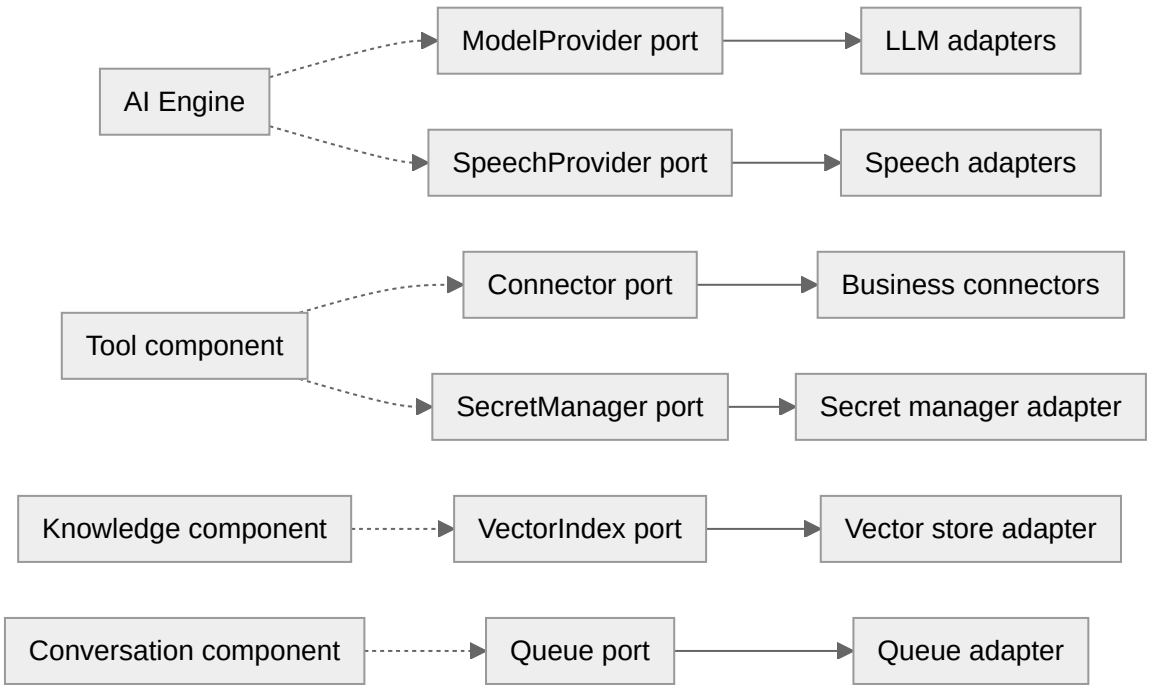


Figure 35.2: U5.1b Component diagram: outbound ports and adapters

35.3 Package diagram (code organization and allowed dependencies)

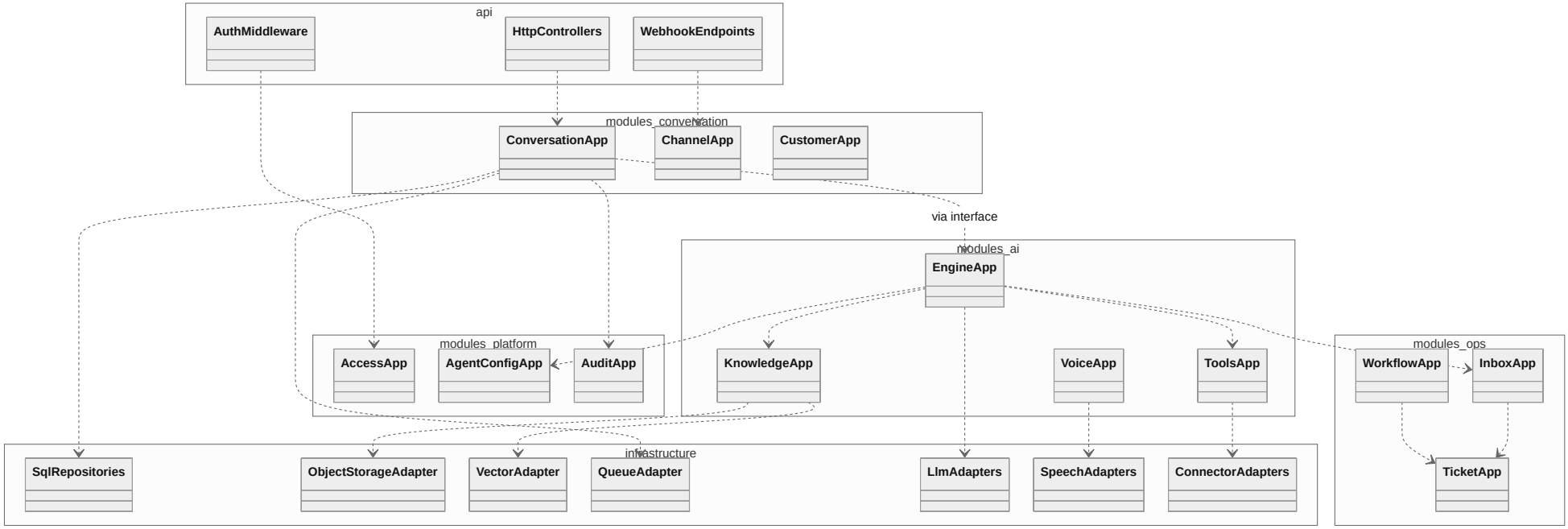


Figure 35.3: Package diagram (code organization and allowed dependencies)

35.4 Deployment diagram — MVP

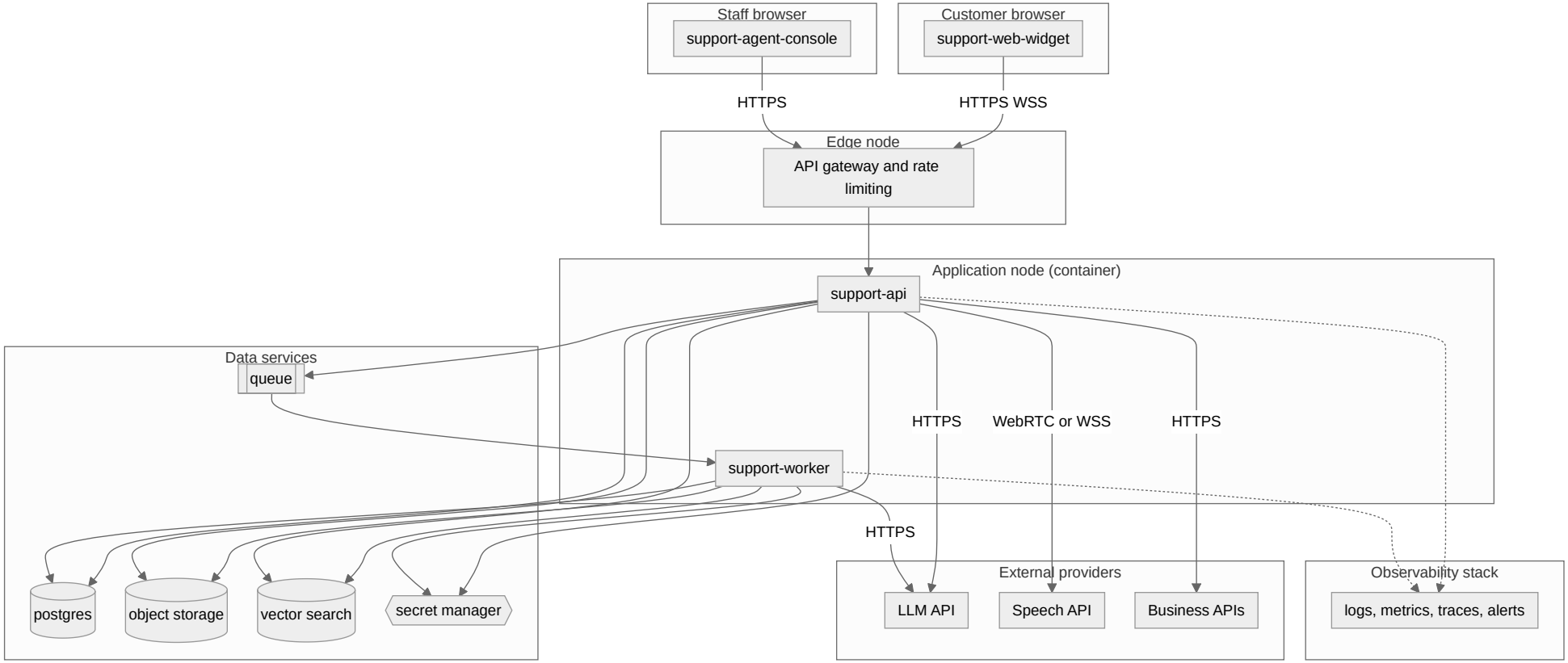


Figure 35.4: Deployment diagram — MVP

35.5 Deployment diagram — later: independent real-time voice component

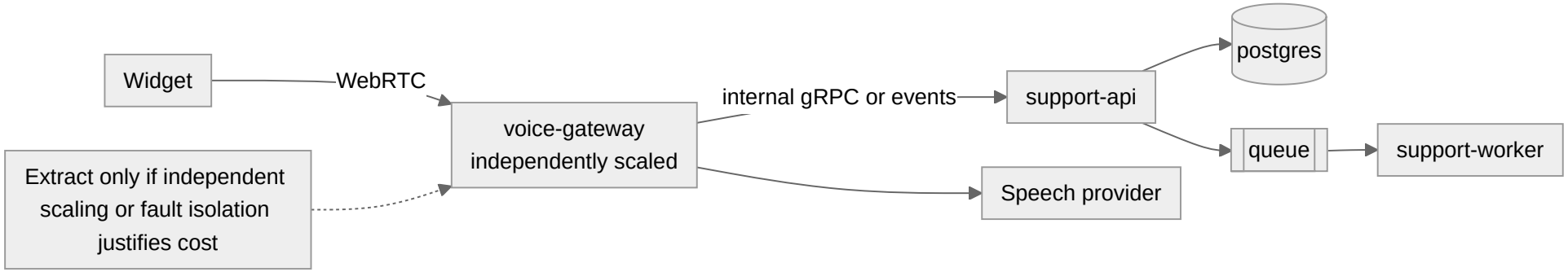


Figure 35.5: Deployment diagram — later: independent real-time voice component

35.6 Object diagram — one live conversation (instance specification)

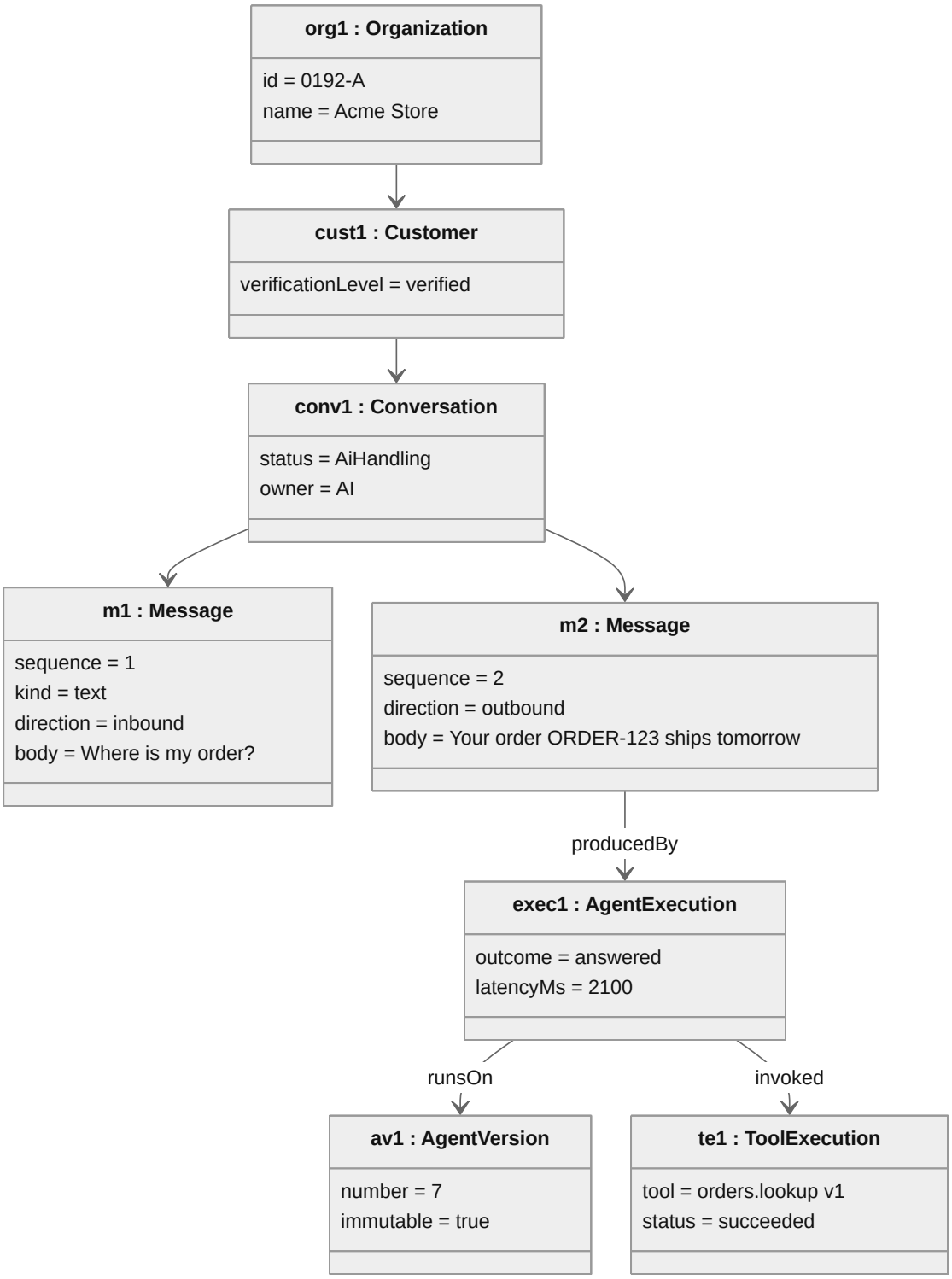


Figure 35.6: Object diagram — one live conversation (instance specification)

35.7 Composite structure — tool execution internals

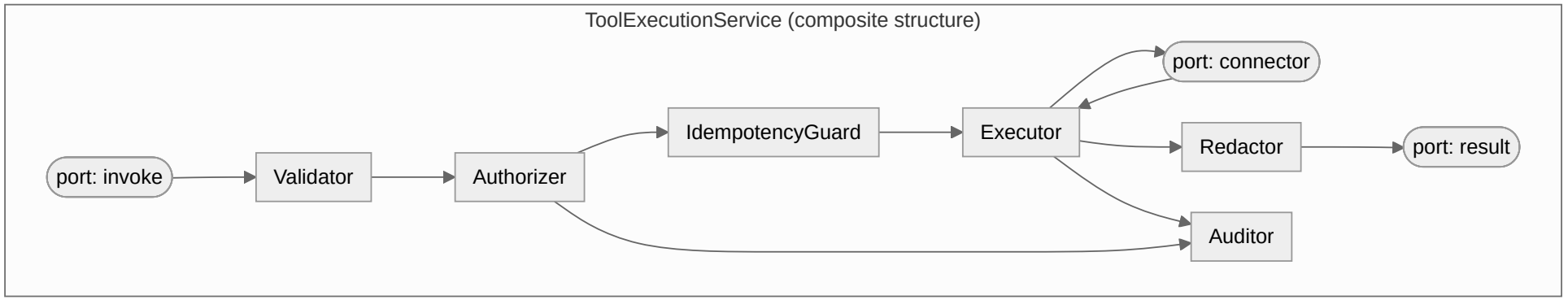


Figure 35.7: Composite structure — tool execution internals